

一, JavaScript详细笔记

1, 基础知识

1-1 基础知识

```
// 1, 标识符命名规则: 第一个字母必须是字母, 下划线或一个美元符号。不能数字开头, 通常不用汉字;  
// 2, 空格忽略, 所以可以在代码中随意使用空格和换行;  
// 3, 注释 // 单行注释; /* 注释内容 */ 多行注释;  
// 4, 直接量 (literal) 12 1.2 "hello world" true 等等; (一般赋给变量或者用在表达式里面);  
// 5, 严格模式 顶部添加 "use strict";  
// 6, 语句以一个分号结束, 如果省略分号, 则由解析器确定语句结尾;
```

1-2 变量

```
function show(){  
    var msg="aini";  
}  
show();  
console.log(msg)
```

// 函数内用 var 定义的变量是局部变量, 只能在函数内使用;

```
function show(){  
    msg="aini";  
}  
show();  
console.log(msg)  
// 不用 var, 而直接赋值就是全局变量, 可以在函数外边使用;
```

注: 虽然省略了 var 可以定义全局变量, 但不推荐使用这种做法, 因为正在局部作用域中的全局变量很难维护。

1-3 数据类型

// 1, js 不区分浮点数和整数;

// 2, 除了十进制, 整数还可以用八进制和十六进制来表示; 16 进制前面加 0x; 8 进制前面必须有 0 (或者 0o); 计算或输出的时候会换算成 10 进制输出; (严格模式下 8 进制无效, ES6 页不能使用的)

// 3, 极大数或极小数可以用科学计数法 e 如 3e+10=3^10/3e-10=3^(-10) 加号可省略; 默认情况下, 会将那些小数点后面带有 6 个零以上的浮点数转换为以 e 表示的数值;

```
console.log(0.1 + 0.2); 不等于 0.3, 而是等于 0.30000000000000004; // 所以浮点数加减不准确;
```

4, 数据范围, 由于内存限制, ESmascrip 并不能保存任意的数值; 其能够表示的最小值保存在 Number.MIN_Value 中, 在绝大多数浏览器中, 这个值是5e-324;表示最大值保存在 Number.MAX_Value 中, 值为1.7976931348623157e+308

```
console.log(Number.MAX_VALUE); 1.7976931348623157e+308
console.log(Number.MIN_VALUE); 5e-324
console.log(Number.MAX_SAFE_INTEGER); 9007199254740991 // 安全整数;
console.log(Number.MIN_SAFE_INTEGER); -9007199254740991
```

5, 想确定一个数是不是有穷的, 可以使用 isFinite()函数, 这个函数会判断参数是否位于最小值与最大值之间;

```
var a=10; // 返回为 TRUE
console.log(isFinite(a));
```

6, 访问 Number.POSITIVE_INFINITY 和 Number.NEGATIVE_INFINITY 也可以得到正负infinity 值; 这两个属性中分别保存着 infinity 和-infinity;

```
console.log(Number.POSITIVE_INFINITY); infinity
console.log(Number.NEGATIVE_INFINITY); -infinity
```

7,NaN 非数值 (not a number) 用于返回一个本来要返回数值却未返回数值的情况;

注: 除以 0 的结果是 infinite, 不是 NaN; NaN 与任何值不相等, 包括其本身; 任何涉及 NaN 的计算都会返回 NaN;

```
console.log(3/"a"); NaN
console.log(NaN==NaN); flase
```

8, isNaN()函数 接受一个参数; 判断这个参数可以是任何类型, 其判断改参数是否“不是数值”, 这个函数首先尝试把这个参数转换成数值; 某些不是数值的值会直接转换成数值, 比如字符串“10”, 或者 Boolean, 而任何不能被转换为数值的值都会导致这个函数返回 TRUE;

```
console.log(isNaN("number")); // 是非数字 所以 TRUE
console.log(isNaN(10)); // 是数字 所以 false
console.log(isNaN("10")); // 能转换成数字 所以 false
console.log(isNaN(NaN)); // 不是数字 所以 true
console.log(isNaN(false)); // 能转换成数字 所以 false
```

9, string 类型 一般使用单引号 如 userName='aini'; 空格也是字符串;

10, 转义字符 \n 换行 \t 制表符 \b 退格 \r 回车 \f 进纸 \斜杠 \ 单引号 \' 双引号 \" 以 16 进制代码表示的一个字符 \unnnn 以 16 进制代码 nnnn 表示的一个 Unicode;

```
var src = "zero\\network " // 输出到页面时, 不会换行; 因为 HTML 代码换行时页面中不
document.write(src); // 换行, 只是多了空格;

// 书写字符串直接量可以拆分多行, 每行必须加上反斜杠\结束;
```

11, 布尔类型

```
var a = 10, b = 15;
console.log(a == b);
if(a == b){
    console.log("相等");
}else{
    console.log("不相等");
}
```

12, Undefined 未定义的：此类型只有一个值，既特殊的 undefined；在使用 var 声明变量，但未对其初始化；

13, Null：此类型也只有一个值，既特殊的 null 值（空值）；从逻辑上看。Null 值表示一个空对象指针，而这也正是使用 typeof 操作检测 null 时会返回 object 的原因；

```
var car = null;
console.log(car); // null
console.log(typeof car); // object
```

// 注：实际上，undefined 值是派生自 null 值得，因此 ECMA-262 规定对他们相等性测试要返回 TRUE；如

```
console.log(null == undefined); // TRUE
```

1-4 复杂数据类型

1, 也称为复杂的数据类型；就是一组数据和功能的集合；

```
var person = {};
person.name="aini";
person.age=18;
person.say = function(){
    console.log("my name is aini");
}

console.log(person); // {name: 'aini', age: 18, say: f}
```

2, 常见的引用数据类型；

```
var arr = [1, 3, 4, 5];
console.log(arr); // (4) [1, 3, 4, 5]
function show(){};
console.log(show); // f show(){}
var date = new Date();// Mon Sep 12 2022 14:04:42 GMT+0800 (中国标准时间)
console.log(date);
```

3, Object 对象

1, Object 对象时多有对象的基础，因此多有对象都具有这些基本的属性和方法；

2, toString(): 返回对象的字符串表示；valueOf():返回对象的字符串，数值或布尔值表示，通常与 toString()返回值相同；

```
var o = new Object();
var arr = new Array(1,2,3,4);
var fun = new Function("a","b");
console.log(o); // {}
console.log(arr); // (4) [1, 2, 3, 4]
console.log(fun); // f anonymous(a) {b}
console.log(o.toString()); // [object Object]
console.log(arr.toString()); // 1,2,3,4
```

```
console.log(fun.toString()); // function anonymous(a) {b}
console.log(o.valueOf()); // {}
console.log(arr.valueOf()); // (4) [1, 2, 3, 4]
console.log(fun.valueOf()); // f anonymous(a) {b}
```

通常这两个方法是可以改写的，如：

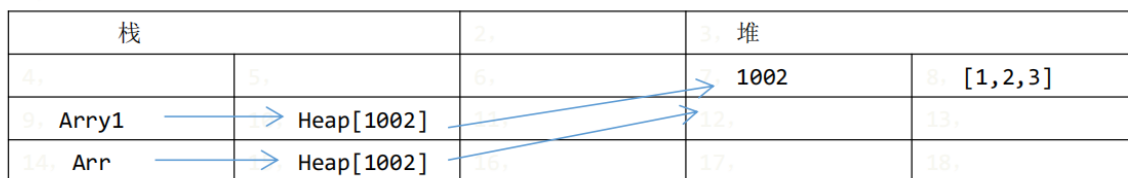
```
var o = new Object();
o.valueOf = function(){
    return "this is a object";
}
o.toString = function(){
    return "this is my name";
}
console.log(o.valueOf()); // this is a object
console.log(o.toString()); // this is my name
```

4. 可变类型和不可变类型；

1, 原始值是不可变类型，引用数据（复杂数据）类型属于可变类型；

```
var arr = [1,2];
var arr1 = arr;
arr1.push(3);
console.log(arr);
```

// Arr 和 arr1 指向同一个堆，所以改变一个的值，另一个也随之改变；



```
var arr = [1,2];
var arr1 = arr;
arr1 = [1,2,3] // (重新创建了 arr1, 并覆盖了之前的数据)
console.log(arr); // [1,2]
```


1-5 数据类型转换

- 1, 因为数据之间是同类型的运算, 如果是不同类型的运算, 需要将其转换为同一类型后再进行运算;
- 2, 类型分两类: 强制类型转换, 自动类型转换;
- 3, 自动类型转换: 根据运算的语义环境, 会自动进行类型转换;

```
var a = "2", b = "1";  
console.log(a-b); // 1;  
b = true;  
console.log(a-b); // 1;
```

// 注: 最经典的用法在 `if` 语句条件中;

```
var car = new Object();  
if(car){ //加入为真(存在), 执行花括号里的语句  
    console.log("汽车"); //汽车  
}
```

// 自动转换注意点:

- 1, " " 空字符串转成数字 0;
- 2, [] 空数组转换数字为 0;
- 3, `Function()` 能转换成字符串"function" 转换为数字为 `NaN` ;

1-5-1 强制类型转换

// 1, 尽管 `JavaScript` 可以自动进行类型转换, 但有时仍需要显式转换, 或者有时为了使代码变得更加清晰易读; 使用 `string()`, `number()`, `Boolean()` ;

// 2, 转换为数值: `Number()`; `parseInt()`; `parseFloat()`; `number()` 可以用于任何类型, 另外两个则专门用于把字符串转成数值, 这三个函数对于同样的输入有不同的返回结果;

// 3, `number()` 函数, 转换规则如下;

- 1, `Boolean` 1, 0;
- 2, 数字 直接返回该数字;
- 3, `undefined` `NaN`;
- 4, `null` 0;

// 如果是字符串则遵循以下规则;

- 1, "123" 123;
- 2, "01235" 1235; // (去掉最前面的 0)
- 3, "1.1" 1.1 // (浮点型);
- 4, "01.1" 1.1 // (浮点型);
- 5, "0xff" 255; // (`number()`能识别出 16 进制);
- 6, "" 0;
- 7, "123abc" // `NaN`;

```
var value = {};  
console.log(Number(value)); // NaN  
  
var value = {};  
value = {valueOf:function(){return 18}};  
console.log(Number(value)); // 18  
  
var value = {};  
value = {valueOf:function(){return true}};  
console.log(Number(value));
```

```
// 4, 有一些操作符 (+, -) 的操作与 Number() 函数相同, 会将操作数转换为数字;  
var x = "w";  
console.log(typeof +x); // number (加了正负号以后把数据类型自动改了)  
  
var x = "w";  
console.log(typeof +x); // number (加了正负号以后把数据类型自动改了)  
console.log(x); // "w"  
console.log(+x); // NaN
```

1-5-2 parseInt() 函数;

// 1, number 函数在转换字符串时比较复杂而且不够合理; 并且只能基于十进制进行转换, 且不能出现非法的尾随字符, 因此在处理整数的时候通常用的是 `parseInt()` 和 `parseFloat()`;

// 2, 二者都是全局函数, 不属于任何对象的方法;

```
console.log(parseInt(" 8a:")); // 8;  
console.log(parseInt(22.5)); // 22;  
console.log(parseInt("wang1")); // NaN;  
console.log(parseInt("")); //NaN
```

```
var value = "";  
console.log(Number(value)); //0  
console.log(parseInt(value)); //NaN
```

// 3, `parseInt()` 能识别 16 进制, 识别不了字符串的 8 进制;

```
console.log(parseInt("0xff")); //255;  
console.log(parseInt("0xafwangwei")); // 175(自动忽略除了 16 进制以外的部分)  
console.log(parseInt(070)); //56;(不用字符串表示, 可以识别出 8 进制并转化)  
console.log(parseInt("075")); //75 (字符串里的八进制无法识别, 会忽略 0 在转换为整数)
```

// 4, `parseInt()` 识别八进制时, 第 3 版可以识别, 但第 5 版识别不了, 为解决这个问题, 可以为 `parseInt()` 指定第二个参数: 转换基数;

```
console.log(parseInt("0xff",16)); // 255 把 16 进制转换为 10 进制
console.log(parseInt("077",8)); // 63 把 8 进制转换为 10 进制
console.log(parseInt("100101",2)); // 37 把 2 进制转换为 10 进制
console.log(parseInt("054",6)); // 34 把 6 进制转换为 10 进制
```

// 在指定了进制以后可以把前面的 0x 或者 0 去掉了;

```
console.log(parseInt("ff",16));
console.log(parseInt("77",8));
console.log(parseInt("100101",2));
console.log(parseInt("54",6));
```

1-5-3 `parseFloat()` 函数;

// 1, `parseFloat()` 只能识别十进制, 16 进制会始终转换成 0;

```
console.log(parseFloat("123abc")); // 123;
console.log(parseFloat("123.56abc")); // 123.56;
console.log(parseFloat("0xaa")); // 0;
console.log(parseFloat("22.5")); // 22.5;
console.log(parseFloat("0909.5")); // 909.5;
console.log(parseFloat("3.125e6")); // 3125000;
console.log(parseFloat("0700")); // 700;
```

7, 转换为字符串

// 1, 要把值转换成字符串有两种方式: 第一种是使用几乎每个值都有的 `toString()` 方法; 该方法会返回相应值得字符串;

```
var age = 18;
var ageString = age.toString();
var found = true;
console.log(found); // "True" 实际上调用了 found.toString()或 found.valueOf()
```

// 注: 数值, 布尔值, 对象和字符串值都有 `toString()` 方法, 但 `null` 和 `undefined` 值没有这个方法;

// 2. 可以为 `toString()` 制定一个可选的基数 (`radix`) 的参数 (前提是对象是数字类型), 默认情况下, `toString()` 方法以十进制格式返回数值的字符串表示; 通过设置基数, 可以输出其他进制的字符串;

```
var age = 18;
console.log(age.toString(2)); // "10010";
console.log(age.toString(8)); // "22"
console.log(age.toString(16)); // "12"
```

// 3, 第二种方法是 `String()` 函数, 这个函数可以把任意类型的数值转换为字符串, 包括 `null` 和 `undefined`;

```
console.log(String(null)); // "null"
console.log(String(undefined)); // "undefined"
```

```
// 4, 如果要把某个值转换为字符串, 可以使用加号操作符把他与一个空字符串拼接;  
var str = 10 + "";  
console.log(typeof str)
```

1-5-4 Boolean()类型;

```
// 1, 使用 Boolean(), 会返回一个 Boolean 值, 但返回 TRUE 或者 false, 取决于要转换值得数据类型及其实际值;  
// 2, 一元"! "运算符也将其操作数转换为布尔值并取反, 这是在代码中进行这种类型转换的常用办法;  
  
var str = "w";  
console.log(!!str); //相当于 boolean(str)
```

1-6 检测类型

1-6-1 typeof

```
// 1, 由于松散的数据类型, 可以用 typeof 方法来检测数据类型 (返回结果都是字符串); 会返回一下值:  
undefined; boolean; string; number; object; null;  
  
console.log(typeof null); // object 比较特殊, 把他看成是空对象
```

1-6-2 instanceof

```
// typeof 在检测引用类型的数据时用处不大; 想要确认一个对象具体是什么类型的对象, 就需要 instanceof 操作符;  
  
var str = "w";  
var o = new Object();  
console.log(o instanceof Object); // true  
console.log(o instanceof String); // false  
console.log(str instanceof Number); // false  
  
// 相当于判断, 返回值布尔类型的;
```

```
console.log(typeof null); // object;  
console.log(null instanceof Object); //false 注意区分
```

说明: 根据规定所有引用类型的值都是 object 的实例; 因此, 在检测一个引用类型值和 Object 构造函数时, instanceof 操作符始终返回 TRUE;

```
var color = new Array();  
console.log(typeof color); // object;  
console.log(color instanceof Object); //true;  
console.log(color instanceof Array); //true;  
  
// 使用 instanceof 操作符在检测基本类型的值, 会返回 false, 因为基本类型不是对象
```

1-7 表达式

```
// 1, "wangwei"; "name"; 1.23; var; name; name; 都算一个表达式;
// 2, 简单表达式 (原始表达式, 比如一些常量, 变量) 复杂表达式 (有简单表达式组合一起的表达式)
```

1-8 运算操作符

```
// 1, 算数运算符: + - * / % (取余数) ++ --;
// 2, 比较运算符: == != (不等) > >= < <= === (全等) !== (不全等);
// 3, 条件运算符: ? : ;
// 4, 逻辑运算符: && (和) || (或) ! (非);
// 5, 位操作符: ~ <<左移位 >>右移位 >>>无符号右移 &按位与 ^按位异或|按位或
// 6, 字符元算符: + ;
// 7, 赋值运算符: = += -= *= /= %= &= ^= |= <=<= >=>= >>=>;
// 8, 其他操作符: delete typeof void instanceof in (测试属性是否存在), 逗号, 忽略第一个操作数, 返回第二个操作数。
```

1-8-1 运算符优先级

运算符	说明	越往上优先级越高
其他操作符	()、[]	
单目操作符	!, ~, ++, --	
算术操作符	*, /, %	
算术与字符操作符	+, -, +(字符加)	
关系 (比较) 操作符	<, <=, >, >=	
关系 (比较) 操作符	==, !=, ===, !==	
按位与	&	
按位异或		
条件运算符	?:	
赋值运算符	=, +=, -=, *=, /=, %=, >>=, <<=, &=, ^=, =	

1-8-2 操作数及个数

```
// 1, 运算符所操作的数据就是操作数, 可以位于运算的左右两边或者一遍;
// 2, 运算符可以根据其操作数的个数进行分类: 一元操作数, 如: "+", 二元操作数, 如: "*", 三元操作数, 如: "?";
```

1-8-3 操作数类型和结果类型

有一些运算符可以作用于任何数据类型, 但在真是场景中, 一般都是有确定的数据类型, 并且大部分运算符都会计算并返回一个特定类型的值;

1-8-4 算数运算符

```

console.log("1"+2); //12 (加法当中字符串中的数字不会自动类型转换)
console.log(1+{}); //10object
var sum = "1" + 2;
console.log(sum); // 12
console.log(+ "2" + 3); // 5
console.log(Number("3")+2) // 5
console.log(true + true); // 2
console.log(1 + true); // 2
console.log(1 + null); // 1
console.log(1 + undefined); // NaN

```

1, 考虑到加法的结合性, 既运算结果依赖于运算的顺序;

```

console.log(1 + 2 + "wang"); //"3wang"
console.log(1 + "wang" + 2); //"1wang2"
console.log("wang" + 2 + 1); //"wang21" (都是从左右的顺序计算);

```

2, 连接字符串, 对于 null 和 undefined, 则分别调用 String()函数并取得字符串"null"和"undefined";

```

console.log("wang" + null); //"wangnull"
console.log("wang" + undefined); // "wangweiundefined"
console.log("wang" + String(null)); //"wangnull"
console.log("wang" + String(undefined)); // "wangweiundefined"

```

3, 减法: 如果一个操作符是字符串, 布尔值, null 或 undefined, 则先在后台调用 Number()进行转换, 在进行转换运算;

```

console.log(8 - true); // 7
console.log(8 - "true"); // NaN (注意"TRUE"引号引起来就是字符串)
console.log(8 - Number(true));
console.log(8 - null); // 8
console.log(8 - {}); // NaN
console.log(8 - "3"); // 5
console.log(8 - undefined); // NaN
console.log(Number("true")); // NaN

```

4, 如果一个操作数是对象, 数值, 布尔值, 则调用他们的 toString()或 valueOf()方法取得相应的字符串值, 在进行计算;

```

console.log(8 - {toString(){return 2}}); // 6

```

5, 其他运算

```

console.log(5 - true); // 4
console.log(NaN - 1); // NaN
console.log((5 - "")); // 5
console.log(5 - "2"); // 3
console.log(5 - null); // 5
console.log(-0 + 0); // 0
console.log(-0 - 0); // -0
console.log(Infinity-Infinity); //NaN
console.log(-Infinity - Infinity); //-Infinity

```

6, 乘除操作符: 如果操作数是非数值, 会执行自动类型转换 (既后台用 Number()函数转换), 空字符串被当做 0; 其余遵循以下规则

```
// 1, 如果计算结果超出范围 返回 Infinity -Infinity;  
// 2, 有一个操作数是 NaN 返回 NaN;  
// 3, Infinity 与 0 相乘 返回 NaN;  
// 4, Infinity 与非零相乘 返回 Infinity -Infinity;  
// 5, Infinity 与 Infinity 相乘 返回 Infinity;  
// 6, 0 被 0 除 返回 NaN;  
// 7, 非零的有限数被 0 除 返回 Infinity -Infinity;  
// 8, 如果 Infinity 被任何非 0 除 返回 Infinity -Infinity;
```

```
console.log(5 / 0); //Infinity  
console.log(-5 / 0); // -Infinity  
console.log(0 / 0); //NaN
```

7, 取模

```
console.log(10 % 3); // 1  
console.log(6.3 % 2.1); //2.0999999999999996  
// 注:小数取模会出问题;
```

8, 一元操作符 ++ 自增 -- 自减

```
// 1, 分为前置 ++a 后置 a++  
// 2, 前置操作时, 变量的值都是语句被求值之前完成的 (先完成变量本身自增, 自减, 在计算整个语句);  
  
var age = 18;  
var anotherage = --age + 2;  
console.log(age); //先算 --age 再算 anotherage 17;  
console.log(anotherage); // 19
```

```
// 3, 后置操作时, 先算整个语句再考虑变量本身的自增, 自减  
  
var age = 18;  
var anotherage = age-- + 2;  
console.log(age); //先算 another 再算 age-- 17;  
console.log(anotherage); // 20
```

```
var num1 = 2;  
var num2 = 20;  
var num3 = num1-- + num2;  
console.log(num3) //先算 num3 再算 num1-- 22;  
console.log(num1); // 1
```

```
// 4, 包含数字的字符串会进行自动转换; ++不会进行字符串的拼接;  
  
var x = "1";  
console.log(++x); //2 也把 x 的数据类型改变了 x=2  
console.log(x + 5); // x=x+5 7
```

```
var x = "1";
console.log(x + 5); // 进行字符串拼接 "15";
console.log(++x); //2 也把 x 的数据类型改变了 x=2
```

5, 在应用于对象时, 先调用对象的 `valueOf` 方法; 如果为 `NaN`, 则再调用 `toString` 方法再应用;

```
var s1 = "2", s2 = "z" , b = false , f = 1.1 ;
o = {
  valueOf: function(){
    return -1;
  }
}
console.log(s1++); // 2 同时 s1 变成 3;
console.log(s1); // 3
console.log(s2++); //NaN 同时把数据类型也改了;
console.log(s2); //NaN
console.log(b++); //0 同时 b 变成 1;
console.log(b);
console.log(f--); //1.1 同时 f = f-1;
console.log(f);
console.log(o--); // -1 同时 o 变成 -2;
console.log(o); //-2
```

6, 一元加和减操作符; (可以把操作数转换成数字或 `NaN`) 而对象先调用 `valueOf` 和 `toString` 方法

```
var s1 = "2", s2 = "z" , b = false , f = 1.1 ;
o = {
  valueOf: function(){
    return -1;
  }
}
console.log(+s1); // 2
console.log(+s2); //NaN
console.log(+b); //0
console.log(+f); //1.1
console.log(+o); -1
```

1-9 关系比较运算符

1-9-1 > < <= >=

```
// 1, 关系运算符用于测试两个值的关系, 根据他们的关系返回 TRUE 或 false; 通常应用在 if, while
等控制语句中;
// 2, 如果两个操作数都是字符串, 则比较他们的大小;
// 3, 如果一个是数值, 则将另一个转换为数值, 再比较;
// 4, 如果一个操作数是对象, 则调用 valueOf()或 toString()方法, 得到结果再进行比较;
// 5, 如果是布尔值, 将其转换为数值再进行比较;
// 6, Infinity 比任何数字都大 (除了其本身); -Infinity 比任何数都小
// 7, 任何操作数与 NaN 进行比较, 结果都是 false;
// 8, 被认定为 false 的值: undefined, null, NaN, "",0,false
```



```

var o = {};
console.log(o);
o = {valueOf:function(){return 3}};
console.log(3 > 2);
console.log(3 >= 5);
console.log("a" > "b");
console.log("3" > 2);
console.log(o > 2);
console.log("a" > 5);
console.log("a" > "A");
console.log(NaN != "a"); //true
console.log(NaN > 3); //false
console.log( NaN == NaN); //false

```

```

// 9, 大小写转换的方法;
console.log("abcdef".toUpperCase());
console.log("ABCDEF".toLowerCase());

```

1-9-2 相等和不相等 == !=

1, 这两个操作数都会先转换操作数（通常称为强制转换），然后再比较；

```

console.log(2 == 2);
console.log(2 == 3);
console.log("w" == "w");
console.log("2" == 2); // true
console.log(true == 1);
console.log(2 != 3);

```

Null 和 undefined 是相等的；

```

console.log(null == undefined); //true

```

1-9-3 全等和不全等 === !==

(不进行类型转换)

```

console.log( "19" === 19); //false
console.log(+0 === -0); //true
var o1 = {};
var o2 = o1;
console.log(o1 === o2); //true

var x = NaN;
console.log(x !== x); //true 只有 x 等于 NaN 时才成立;

```

Null 和 undefined 是相等，但不是全等

```

console.log(null === undefined); //false
console.log(null == undefined); //true

```

1-9-4 条件运算符

```
var num1 = 10, num2 = 20
var max = num1 > num2 ? num1 : num2;
console.log(max); // 求绝对值;
```

基本语法:

```
// 表达式? 返回值 1: 返回值 2;
// 如果表达式为真则返回 返回值 1 , 若表达式为假。则返回 返回值 2;

var x = -5;
console.log(x > 0 ? x : -x); //求绝对值;

var a = 10, b = 15;
console.log(a == b ? true : false);
```

判断某个变量有没有定义

```
var a,b;
a = "aini";
console.log(a? a:"未定义"); // "aini"
console.log(b? b:"未定义"); //"未定义"
```

1-10 位操作符

```
// 1, 计算值负数: 正数绝对值二进制取反 加一 如求 -18 先求 18 的二进制 10010取反以后是 01101
加一以后 01110 所以-18 二进制就等于 01110; 再加一个符号位就是 101110
// 2, 按位非 ~ 按位与& 按位或| 按位异或^
// 3, 按位非 返回数字的反码;
```

```
var num = 25; // 25 = 11001 -25 = 00110 +00001 = 00111
var num1 = ~num; //反码 00110 所以反码是 00110 = 00111 - 00001 = -25 - 1
console.log(num1); // -26 所以反码是 00110 = 00111 - 00001 = -25 - 1 = -26
```

4, 按位与, 有两个操作符, 相同位数对齐比较 全 1 出 1, 有 0 出 0;

```
var sum = 25 & 3; // 11001
console.log(sum); // 00011 00001 = 1
```

5, 按位或, 有两个操作符, 相同位数对齐比较 有 1 出 1, 全 0 出 0;

```
var sum = 25 | 15; // 11001
console.log(sum); // 01111 11111 = 31
```

6, 按位异或, 相同返回 0, 不同返回 1;

```
var sum = 25 ^ 15; // 11001
console.log(sum); // 01111 10110 = 22
```

7, $c = a \ll b$ 左移; $c = a$ 的二进制数左移 b 位后的结果, 也等于 $c = a * 2^b$;

```
var sum = 3 << 3; // 11 左移 3 位就是 11000 = 24
console.log(sum); // 24
```

8, $c = a >> b$ 右移(有符号的右移 $>>$, 正数补 0, 可以忽略, 负数补 1, 需要注意, 无符号右移 $>>>$, 只会用 0 来填补); $c = a$ 的二进制数右移 b 位后的结果, 也等于 $c = a / 2^b$;

```
var sum = 64 >> 3; // 1000000 右移 3 位就是 0001000 = 8
console.log(sum); // 8
```

1-11 赋值运算符

1, 赋值运算符, 有很低的优先级;

```
var a, b = 0;
console.log((a = b) == 10); // 因为 = 优先级很低, 一般需要用括号括起来, 来提高计算的优先级

// 赋值运算符的结合性是从右到左;
var j = k = i = 0; // i=0, k=i, j=k 由于 k, i 不是用 var 声明, 所以属于全局白变量;
```

2, 如果等号前面添加了其他操作符就可以完成复合赋值操作;

$+=$, $-=$, $*=$, $/+=$, $\%+=$, $>+=$, $<+=$, $>>+=$, $\&+=$, $\^+=$, $|+=$

```
num = 10;
num+=10; // num = num + 10
console.log(num);

// += 可用于字符串
var str = "aini";
str += "&AINI";
console.log(str);

// 有时候两者有区别, 指向的不是同一个元素; 会出现问题
var date = [1, 2, 3, 4];
var i = 1;
// date[i++]*=2; // date[1] = date[1]*2=4 [1, 4, 3, 4]
date[i++] = date[i++] * 2 // [1, 6, 3, 4] date[1] -> i=2 -> date[2] 3*2=6 最终
date[1]=date[2]*2=6
console.log(date);
```

1-12 其他运算符

1-12-1 typeof

两种语法 `typeof value` / `typeof (value)` ;

```
console.log(typeof 1);
console.log(typeof true);
console.log(typeof undefined);
console.log(typeof NaN);
console.log(typeof null);
console.log(typeof "aini");
console.log(typeof function() {});
```

1-12-2 instanceof

不能判断基本数据类型;

```
var num = 1;
console.log(num instanceof Number); // false 不能判断基本数据类型;
var num1 = new Number(1); // 基本包装类型
console.log(typeof num1); // object
console.log(num1 instanceof Number); //true
console.log(num1 instanceof Object);

var str = new String("aini");
console.log(typeof str); // object
console.log(str instanceof String); // true
console.log(str instanceof Object); // true
var colors = new Array();
console.log(typeof colors);
console.log(colors instanceof Array);
console.log(colors instanceof Object);
```

1-12-3 in 运算符

in 运算符希望它的操作数是一个字符或者可以转换为字符，右操作数是一个对象；如果该对象拥有一个名为左操作符的属性名，则表示返回 TRUE;

```
var point = {x:1,y:1};
console.log(1 in point); // false
console.log("x" in point); // true
console.log("z" in point); // false
console.log("toString" in point); //True 所有对象都继承了 Object 的 toString 方法
var date =[1,2,3];
console.log("0" in date); // 能自动转换
console.log(1 in date); // 判断索引
console.log(3 in date); // false
```

1-12-4 delete 运算符

能删除对象的方法和属性;

```
var point = {x:1,y:1};
delete point.x;
console.log(point); // {y: 1}
var date =[1,2,3];
delete date[1];
console.log(date); // (3) [1, 空, 3]
```

1-12-5 逗号运算符

1, 一次性声明多个变量需要逗号隔开;

2, 逗号运算符在一条语句中执行多个操作; 在用于赋值的时候, 逗号操作符总会返回表达式中的最后一项;

```
var a = (1,3,5,7,9);
console.log(a); // 9
```

3, 其首先计算左操作数, 再计算右操作数, 最后返回右操作数的值;

```
var num = (i =2 , j =3, k =i+j);  
console.log(num); //5
```

2, 流程控制

2-1 前言

1, 表达式语句

```
// 1, 具有副作用的表达式是js中最简单的语句: 如赋值语句str = "zero" + "network";  
// 2, 递增递减运算也是一种表达式;  
// 3, Delete 运算符非作用是删除一个对象的属性, 所以他也是语句, 而不是表达式;  
// 4, 函数调用是表达式语句中的另一个大类;
```

```
3 + 2; //表达式  
a = 2; // 语句  
alert("wangwei"); //语句  
console.log(x = Math.cos(60)); //语句
```

2 复合语句和空语句

复合语句和空语句, 可以把多条语句联合在一起, 形成一条复合语句; 使用花括号将多条语句括起来, 也称为代码块;

```
{  
var x = Math.PI;  
var cs = Math.cos(x);  
console.log("cos(PI) = " + cs);  
}
```

// 注: 语句块结尾不需要分号;

空语句: 允许包含 0 条语句的语句, 其实用分号; 表示; Js 解释器在执行空语句时不会执行任何动作, 但某些情况下, 空语句还是很有用的;

```
var a =[];  
for(var i = 0 ;i < 10;a[i++]=i);  
console.log(a);
```

3, 声明语句 var function

4, 顺序结构 ES 程序就是一些列可执行语句的集合; 默认情况下, Js 解析器依照语句的编写顺序依次执行, 这是程序最基本的结构;

5, 流程控制结构: 流程就是代码执行顺序, 流程控制就是改变语句的默认执行顺序, 使其按一定的方式进行;

- 1, 条件(condition) 语句
- 2, 循环语句 (loop)
- 3, 跳转语句 (jump) 如 break, return, throw 语句

2-2 条件语句

2-2-1 if 条件

2-1-1 形式一

if(condition) statement;

```
var a = 10;
if(a == 10) console.log("a is 10");
if("wangwei") console.log("aini is a good person"); //只要是个真值就行
```

ES 语法规则规定，if 关键字和带圆括号的表达式之后必须跟随一条语句，但只能是一条语句，可以分行写，也可以写在一起；如果有多条，可以使用语句块；

```
var a = 10;
if(a != 10){
    console.log(a);
    console.log(a + 10);
}
```

注：一般使用 X==null 或 X==undefined 或简写！X 来判断一个变量是否有效；

```
var x;
if(!x) x = 12; //如果 x 没有值的话
console.log(x);
```

2-1-2 x形式二

if else 语句 if (condition) statement1; else tatement2;

多条语句加花括号； 同?:一样

```
var x = 10;
var y = 15;
if(x == 10){
    console.log("x = 10")
}else{
    console.log("x != 10")
}

if(y==10) console.log("y = 10")
else console.log("y != 10");
```

2-1-3 形式三

if-else if 语句 多条件语句

```
var grade = 41;
if (grade>=80 & grade<=100)
console.log("良好");
else if(grade>=60)
console.log("及格");
else if(grade>=40)
console.log("不及格");
else
console.log("0");
```

2-1-4 if 语句嵌套

```
var a = 30;
if(a >= 0 & a<=10){
if(a>=0 & a<=5){
    console.log("a 在 5 之内");
}else{
    console.log("a 在 6 到 10 之内");
}
}else if(a >10 & a<=20){
    if(a>10 & a<=15){
        console.log("a 在 11 到 15 之内");
    }else{
        console.log("a 在 16 到 20 之内");
    }
}else{
    console.log("a 大于 20");
}
```

2-2-2 Switch 语句

```
// 语法:
Switch(expression){
    case value 1:statement ;
    break;
    case value 2: statement;
    break;
    default :statement;
}
```

```
var n = 2;
switch(n){
case 1:
console.log(1);
break;
case 2:
console.log(2);
break;
case 3:
console.log(3);
break;
case 4:
console.log(4);
break;
```

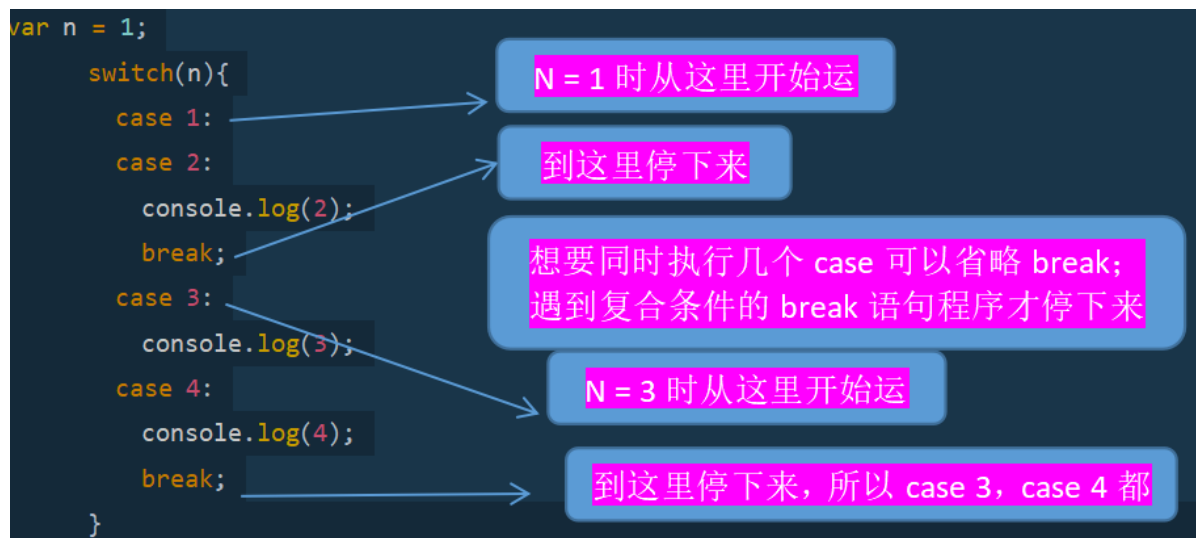
```
default:
console.log("other");
```

// 本质来讲，**case**只是指明了程序运行的起点，只要找到复合的**case**语句，就开始进行，直到程序结束
// 每一个**case**语句里加**break**；不然找到了第一个匹配的值以后就会继续往下算，直到程序结束
// **Default** 是当所有**case**不匹配时才运行，一般放到最后面

```
var n = 2;
switch(true){
  case n > 0:
    console.log("大于0");
    break;
  case n = 0:
    console.log("等于0");
    break;
  case n < 0:
    console.log("小于0");
    break;
}
```

// 如果把所有可能性已经囊括了，就不需要写**default**
// **Case**后：具体的一个值，运算，返回，常量，变量
// **Case**语句在比较值用的是全等操作符，因此不会进行类型转换；所以字符串"10"不等

```
var n = 1;
switch(n){
  case 1:
  case 2:
    console.log(2);
    break;
  case 3:
    console.log(3);
  case 4:
    console.log(4);
    break;
}
```



如果在函数中使用Switch，可以使用return代替break，两者都用于终止Switch语句，也会防止一个case语句块执行完后继续执行下一个case语句块；


```
function convert(x){
  switch(typeof x){
    case 'number':
      return x.toString(16);
    case 'string':
      return '"' + x + '"';
    default:
      return "other";
  }
}
console.log(convert(18));
console.log(convert("aini"));
console.log(convert(true));
```

// Return会返回值并直接退出函数

```
var str = "world";
switch("hello world"){
  case str:
    console.log(str);
    break;
  case "world":
    console.log("world");
    break;
  case "hello " + str:
    console.log("hello " + str);
    break;
  default:
    console.log("other");
}
```

```
var str = "world";
switch("hello world"){
  case str:
    console.log(str);
    break;
  case "world":
    console.log("world");
    break;
  case "hello " + str:
    console.log("hello " + str);
    break;
  default:
    console.log("other");
}
```

注意空格，要不然还是匹配不上

```

switch(true){
    case "wangwei": //这是真值，但不是布尔的TRUE，也不会进行自动转换，所以不成立
        console.log("wangwei");
        break;
    case 1:
        console.log(1);
        break;
case 3 > 2: //返回的结果是 TRUE ，所以会运行case 3 > 2的情况
    console.log(true);
    break;
default:
    console.log("other");
}

```

```

var str = "10";
switch(str){
    case 10:
        console.log("这是数字10");
        break;
    case "10":
        console.log("这是字符串10");
        break;
}

```

// case语句在比较值用的是全等操作符，因此不会进行类型转换；所以字符串"10"不等于数字10

Case 后面建议使用常量，避免使用副作用的表达式

3，循环控制

3-1 while 循环

1，属于前测试循环语句，循环之前判断条件是否成立；

2，语法 while (expression) {statement; }

```

var count = 0;
while(count < 10){
    console.log(count);
    count++;
};

```

```

var count = 0;
var sum = 0;
while(count <= 100){
    sum += count
    count++;
};
console.log(sum);

```

// while语句里的分号建议加上

```

var i = 18;
while(true){
    console.log(i)
    if(i == 18){
        break;
    }
}

```

// **break**语句能结束**while**循环，跳出循环；有时候为了避免死循环，可以使用

3-2 Do while 循环

1, 后测试循环，先执行一遍循环，再判断条件是否满足，所以循环至少执行一次；

```

var i = 0;
do{
    i+=2;
}while(i<=10)
console.log(i);

```

// 也可以这样用，改变条件的表达式直接写在

```

var i = 0;
do{
    console.log(i);
}while((i+=2) < 10);

```

// 这样可以先改变条件，再进行循环；相当于前测试循环

```

function printArray(arr){
    var len = arr.length,i=0;
    if(len == 0){
        console.log("Empty Array")
    }else{
        do{
            console.log(arr[i]);
        }while(++i < len);
    }
}
var arr=["beijing","nanjing","xinjiang","gaungdong"];
printArray(arr);

```

```

<body>
  <div id="mydiv"></div>
  <script>
    var imgArr =
["xm/1.webp","xm/2.webp","xm/3.webp","xm/4.webp","xm/5.webp","xm/6.webp","xm/7.w
ebp","xm/8.webp",];
    var mydiv = document.getElementById("mydiv");
    var i = 0;
    while(i < imgArr.length){
        var img = document.createElement("img");
        img.src = imgArr[i];
        mydiv.appendChild(img);
        i++;
    }
  </script>

```

```
    }  
  </script>  
</body>
```

3-3 for 循环

- 1, 前测试循环语句; 已知循环次数情况下使用for循环比较合适;
- 2, 语法 for(初始循环变量;test条件表达式 ;增量表达式){statement}
- 3, 初始循环变量只是循环开始时执行一次

```
for(var i = 0; i < 10; i++){  
    console.log(i);  
}  
// 跟上面的for循环等价  
var i = 0;  
while(i < 10){  
    console.log(i);  
    i++;  
}
```

```
var count = 0;  
while(count++, count < 10){  
    console.log(count);  
}
```

// 注: 不能把初始化变量放到while括号里面; 但是可以放不影响增量的其他变量;
// 从左往右计算, 返回逗号运算符最右侧的代码

```
var count = 0;  
while(count++, count < 10){  
    console.log(count);  
}
```

由于逗号运算符返回最后面的值, 所以也可以这样用

```
var result = 1, i = 1;  
while(i <= 8){  
    result *= i;  
    console.log(i + "的阶乘是: " + result);  
    i++;  
};
```

简化上面的while循环

```
var result = 1, i = 1;  
while(result *= i, i++, i <= 8){  
    console.log(i + "的阶乘是: " + result);  
};
```

```
var result = 1;
for(var i = 1; i <= 8; i++){
    result*=i;
    console.log(i + "的阶乘是: " + result);
}
```

```
var count = 10,i;
for(i=0;i<count;i++)
    console.log(i);
// i可以在外部定义，for循环里面就不用定
```

```
var count = 10,i=0;
for(;i<count;i++)
    console.log(i);
// For循环中，初始化变量，条件表达式，增量表达式都是可选的，但是分号不能省略
```

```
var count = 10,i=0;
for(;;i++)
    if(i<count){
        console.log(i);
    }else{
        break;
    }
```

```
// 这是一个死循环
var count = 10,i=0;
for(;;)
    console.log(i);
```

把初始化放到外部，把条件表达式以及更新表达式放到内部，条件不满足使用break跳出循环

```
var count = 10,i=0;
for(;i<count;){
    console.log(i);
    i++;
}

// 所以可以把初始化和增量放到外部
```

可以有多个初始化变量，条件表达式，增量表达式，需要用逗号运算符

```
var sum = 0;
for(var i=0,j=1;i<10;i++,j++){
    sum+= i*j;
    console.log(i + "," + j);
}
console.log(sum);
```

For循环的执行顺序

```

for(var i=0,j=console.log("A");console.log("B"),i<10;console.log("C"), i++){
    console.log("aini");
}

// 顺序:
i=0,j=console.log("A")           console.log("B"),i<10           console.log("aini")
console.log("C"), i++           console.log("B"),i<10           console.log("aini")
console.log("C"), i++

```

总之：初始变量语句只运行一次，增量表达式在statement以后再执行

```

for(var i=0,j=1;i<10;i++,j++){
    document.write(i);
    if(j%4==0){
        document.write("</br>");
    }
}

```

3-4 for-in 循环

- 1, for in 是精准的迭代语句;
- 2, 语法 for (variable in object) {statement}

```

for(var pronaame in window){
    console.log(pronaame);
}

// window是一个内置对象，for in可以把内置对象的所有属性，方法，事件枚举出来

```

```

var arr=[];
for(var i=0;i<10;i++){
    arr[i]=i*10;
}
for(var j=0;j<arr.length;j++){
    console.log(arr[j]);
}
for(var k in arr){
    console.log("数组中的第" + k + "个元素是: " + arr[k]);
}

```

// 把数组中的元素枚举出来,可以用for循环，也可以用for-in语句
 // For-in来枚举数组索引

```

var str ="hello world";
for(var a in str){
    console.log(a + "字符: " + str[a]);
}

```

// 字符串也可以像数组一样枚举出来

```
var person = {
  name:"aini",
  age:18,
  walk:function(){
    console.log("aini can walk")
  }
}
for(var p in person){
  console.log(p + ": " + person[p]);
}

// 可以把对象的属性和属性值枚举出来
// 注: P是属性, person[p]才是属性值数组里面也是同理, 索引和值
```

ESMAScript对象的属性没有顺序, 因此用for-in循环出来的属性名的顺序不可预测; 如果迭代的对象时null或undefined, ES5之前会抛出错误, 在ES5里面不会报错误, 也不执行如果object表达式等于一个原始值, 这个原始值会转换为与之对应的包装对象;

```
var num = 18;
for(var n in num){
  console.log(n);
}
```

```
var str = "hello world";
for(var a in str){ //这里使用了new String(str) 把原始类型转换为基本包装类型;
  console.log(a);
}
```

```
var person = {
  name:"aini",
  age:18,
  walk:function(){
    console.log("aini can walk")
  }
}
var a = [], i=0;
for(a[i++] in person);
console.log(a);

// A[i++]在这里既可以当变量, 也可以自增, 同时对自己赋值
```

```
▼ (3) ['name', 'age', 'walk'] ⓘ
  0: "name"
  1: "age"
  2: "walk"
  length: 3
  ► [[Prototype]]: Array(0)
```

For-in并不会遍历所有的属性和方法, 只会枚举可枚举的属性和方法, 由js核心语言内置的属性和方法就是不可枚举的, 比如toString(), valueOf();

各种循环可以任意嵌套

3-5 跳转

1, 可以从一个位置跳转到另一个位置;

2, 标签语句: 任何语句如果在前面加上标识符和冒号都可以构成标签语句, 以便将来使用;

语法: label: statement

```
var flag = true;
one:
if(flag){
  console.log("flag");
}
start:
for(var i=0; i<10;i++){
  console.log(i);
}
two:
{
  var num =18;
  num++;
  console.log(num);
}
```

// 一般与for, while语句配合使用; 既在循环体内使用break和continue退出或跳转到标签外, 二者是唯一可以使用标签的语句;

// 标签可以跟变量函数名同名;

// 标签可以嵌套

// 嵌套时标签名不允许跟标签名重复

```
one: for(var i = 0; i<5; i++){
  two: for(var j = 0; j<5; j++){
    console.log(i+j);
  }
}
```

// 嵌套里的标签不允许

// 单独使用break语句可以立即退出最内层的循环或者Switch语句 (break语句只能用在循环或者Switch中)

// break与标签语句配合使用, 退出这个循环的同时跳转到标签处结尾处, 可以用于跳出多层循环的

```
var num = 0;
outermost:
for(var i = 0; i < 10; i++){
  for(var j = 0; j < 10; j++){
    if(j==5){
      break outermost;
    }
    console.log(i + ", " + j);
    num++;
  }
}
console.log(num);
```



```

var num = 0;
outermost:
for(var i = 0; i < 10; i++){
  for(var j = 0; j < 10; j++){
    if(j==5){
      break outermost;
    }
    console.log(i + "," + j);
    num++;
  }
}
console.log(num);

```

跳出循环, 直接跳出标签所在循环

不在嵌套语句里的平行标签可以重名

0,0
0,1
0,2
0,3
0,4
5

```

var num = 0;
outermost:
for(var i = 0; i < 10; i++){
  inner: for(var j = 0; j < 10; j++){
    if(j==5){
      break;
    }
    console.log(i + "," + j);
    num++;
  }
  for(var k = 0; k < 5; k++){
    if(k==2){
      break inner;
    }
  }
}
console.log(num);

```

```

var num = 0;
outermost:
for(var i = 0; i < 10; i++){
  inner: for(var j = 0; j < 10; j++){
    if(j==5){
      break;
    }
    console.log(i + "," + j);
    num++;
  }
  for(var k = 0; k < 5; k++){
    if(k==2){
      break inner;
    }
  }
}
console.log(num);

```

Break 标签不能用在兄弟循环之间, 比如左边例子

Break inner 无法跳出这个循环上面的兄弟循环

```

var sumarr = [[1,2,3],[2,"a",4],[4,5,6]], sum = 0;

```

```

compute_sum:
if(sumarr){
  for(var i=0;i<sumarr.length;i++){
    var row = sumarr[i];
    if(!Array.isArray(row)){
      break compute_sum;
    }
    for(var y = 0; y < row.length; y++){
      var cell = row[y]
      if(isNaN(cell)){
        break compute_sum;
      }
      sum+=cell;
    }
  }
  console.log("全部执行完毕");
}
console.log(sum);

```

3-6 continue 语句

// Continue语句与break语句非常相似，区别在于他不在退出循环，而是结束本次循环，继续下次循环，其与break用法基本一致
 // Continue只能用在循环体当中

```

for(var i = 0;i<5; i++){
  if(i == 3) continue;
  console.log(i);
}
// 遇到3结束本次循环，开始下次

```

```

var date = [1,2,undefined,4,NaN,8,9,null]; sum = 0,error = 0;
for(var i = 0; i<date.length;i++){
  if(!date[i]){
    error+=1;
    continue;
  }
  sum+=date[i];
}
console.log(sum);
console.log(error);

```

// Continue可以与标签配合使用，就是跳出外侧循环，继续进行外侧循环的下一循环

3-7 return 语句

// Return语句只能在函数中使用，既函数调用后的返回值；
 // 如果没有return语句，则程序返回的结果就是undefined
 // Return默认返回值也是undefined

```
function display_object(o){
    if(!o) return ;
    for(p in o){
        console.log(p);
    }
    return;
}
var person ={
    name:"aini",
    age:18,
    city:"xinjiang"
}
console.log( display_object(person)) ;
```

3-8 with 语句

```
// 为了简写代码
with (object) {statement}
```

```
var person = {
    name:"aini",
    age:18,
    city:"xinjiagn"
}
console.log(person.name);
console.log(person.age);
console.log(person.city);

// 可以这样简写代码
with(person){
    console.log(name);
    console.log(age);
    console.log(city);
}
```

3-9 Debugger语句

```
// 什么也不做，用于调试，不怎么重要，会在debugger语句的地方产生一个断点
```

3-10 “use strict” 语句

```
// 让js代码在严格模式下执行
```

4， 函数

4-1 定义函数

4-1-1 第一种方式

// 第一种方法：函数声明式，关键字function定义：

```
function showMsg(str){  
    console.log("东华大学"+ str);  
}  
showMsg("kaikele");
```

```
function square(a,b){  
    return Math.sqrt(a*a + b*b);  
}  
console.log(square(6,8));
```

4-1-2 第二种方式

// 第二种方法：function实际上也是对象，所以可以使用function对象的构造函数来创建一个函数：
// var 变量名 = new Function(形参1, 形参2, 代码体);

```
<script>  
    var square = new Function("x","y","return Math.sqrt(x*x + y*y)");  
    console.log(square(4,5));  
</script>
```

```
var date ="var num =prompt('input');console.log(num);"  
var date1 ="alert('aini');"  
var date2 ="var p=fname:'aini',age:19};console.log(p.name)";  
var fun =new Function(date2);  
fun();
```

```
var scope ="global";  
function func(){  
    var scope ="local";  
    var myFun =new Function("return scope;");  
    console.log(myFun());  
}  
func();
```

```
// 注：只能认全局变量，在函数中的局部变量识别不了；所以上述例子返回“global”；内部的“local”识别不了；  
// 注：1，参数，代码体必须使用引号引起来，代码体放在最后一个引号里，语句和语句之间分号引起来；  
// 2，如果没有参数则省略，只有代码体的引号；
```

4-1-3 第三种方式

可以直接把定义函数保存在变量中，该变量名实际上就是函数

```
// var 变量名 = function (参数1, 参数2) {表达式语句}
```

```
var mysum =function(num1,num2){return num1+num2;  
console.log(mysum(10,20));
```

由于函数名仅仅是函数的指针，所以可以把函数名赋值给多个变量，这么说一个函数可以有多个名字

```
<script>  
function sum(num1,num2){  
    return num1 +num2;  
}  
mysum =sum;  
anotherSum =sum;  
console.log(mysum(10,20));  
console.log(anotherSum(30,50));  
</script>
```

把函数名赋值的时候不能带括号，带括号的话立马就执行了

```
<script>  
var sayworld;  
var condition =false;  
if(condition){  
    sayworld =function(){console.log("aini")};  
}else{  
    sayworld =Function()fconsole.log("AINI");  
}  
sayworld();  
</script>
```

函数声明和函数表达式的区别

// 解释器在执行环境中加载数据时，会率先读取函数声明，并使其在执行任何代码之前可用（可以访问）；至于表达式，则必须等解析器执行到它所在的代码行，才会真正被解析；

```
sayHi();  
function sayHi(){  
    console.log("hi");  
}
```

```
// Function函数声明的时候，访问可以放在函数之前，也可以放在函数后面
```

```
sayHi();  
var sayhi =function(){console.log("Hi");
```

```
✖ ▶ Uncaught TypeError: sayHi is not a function HanShu.html?_ijt=qe3...=RELOAD_ON_CHANGE:9  
    at HanShu.html?_ijt=qe3...ELOAD_ON_CHANGE:9:5
```

```
// 通过表达式定义的函数，访问必须在定义之后，不然会出错
```

4-1-4 定义函数的三种方式

```
// 1，函数声明；  
// 2，利用Function类构造函数；  
// 3，通过函数表达式构造函数
```

```
// 函数的调用：可以在代码中调用，也可以在事件中调用，还可以通过链接调用；
```

```
<script>  
function sum(a,b){  
    return a+b;  
}  
console.log(sum(10,20));  
var sum1 =function(a,b){  
    return a +b;  
}  
console.log(sum1(20,30));  
var fun =new Function("a","b","return a+b");  
console.log(fun(50,100));  
</script>
```

```
// 1，如果调用函数时输入的实参的个数大于形参的个数，则忽略多余的实参；  
// 2，实参的个数小于形参，则没有接受传递值得参数赋予undefined；
```

```
// Return返回值：只要执行带return就会返回，并退出，后面的代码永远不会执行；return只能用在函数  
当中
```

```
// Return 也可以不带任何返回值，在这种情况下函数就会停止运行并返回undefined
```

```
// 函数中不用return也会返回undefined
```

4-1-5 一些例子

```

<script>
    function printPro(o){
        for(var p in o){
            console.log(p + ":" + o[p] + "\n");
        }
    }
    var person = {
        name:"aini",
        age:18,
        color:"yellow",
        say:function(){
            "aini can say with other people";
        }
    };
    printPro(person);

```

```

<script>
    function distance(x1,y1,x2,y2){
        dx = x2 - x1;
        dy = y2 - y1;
        return Math.sqrt(dx*dx + dy*dy);
    }
    console.log(distance(2,2,6,8));
</script>

```

5, JavaScript 引用类型

// 本地对象：独立于宿主环境的由ECMAScript实现提供的对象；如object, Function, Array等；
 // 内置对象：由ECMAScript实现提供的，独立于宿主环境的所有对象，如Global和Math（本质上是静态的，就不需要创建对象）；
 // 宿主对象：所有非本地对象都是宿主对象，既由scmascript实现的宿主黄江提供的对象，如多有BOM和DOM对象丢失宿主对象；

5-1 object 类型

5-1-1 创建

// 创建object实例：两种方法；

- 1, var person = new Object();
- 2, var person = {}; （属性名可以是字符串，用引号引起来，属性名中的空格无法识别）；
- 3, 数值属性名会自动转换成字符串，但是用方法不同；

```
<script>
var person = {
  "my name" : "aini",
  age : 18,
  sex : "男",
  18 : true,
  3 : "女朋友"
};
console.log(person);
console.log(person[3]);
console.log(person[18]);
</script>
```

当属性名是数字的时候，访问时用对象名[属性名]，这样访问

```
var myo = {};
```

// 是一个空对象，只包含默认的属性和方法；

```
var myo = new Object();
```

// 两者可以等同；


```
function print(obj){
    var output = "";
    if(typeof obj.name == "string"){
        output += "name:" + obj.name + "\n";
    }
    if(typeof obj.age == "number"){
        output += "age:" + obj.age + "\n";
    }
    alert(output);
}

var o = {
    name:"aini",
    age : 18
};

print(o);
```

为对象添加方法

```
function smoke(){
    console.log("i can smoke");
};

function mydrink(){
    console.log("i can drink");
};

var aini = {
    name : "aini",
    age : 18,
    smoke : smoke,
    drink : mydrink
};

aini.smoke();
aini.drink();
```

```
var aini = {
    name : "aini",
    age : 18,
    smoke : function(){
        console.log(this.name + " can smoke")
    },
    drink : function(){
        console.log(this.name + " can drink")
    }
};

aini.smoke();
aini.drink();
```

// 注：字面量对象花括号里面用逗号
// 类创建对象里面的类里面使用分号

5-1-2 对象属性

// Object对象属性是动态的

// 两种访问方式:

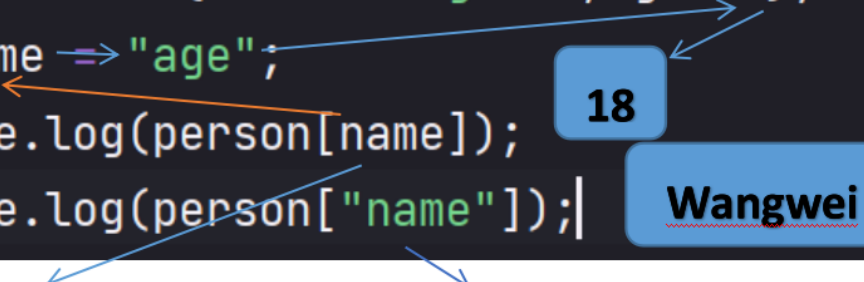
1. 使用点访问如`person.name`;
2. 使用方括号表示法, 属性应以字符串的方式放在括号中;

```
<script>
    alert(person.name);
    alert(person["name"]);
</script>
```



方括号里面的可以是变量名, 所以说可以通过变量来访问属性

```
var person = {name:"wangwei",age:18};
var name => "age";
console.log(person[name]);
console.log(person["name"]);
```



指向变量

指向对象里面的属性

// 注: 如果属性名中包含会导致语法错误的字符, 或者属性名使用的是关键字或者保留字, 就可以使用方括号访问法;

```
<script>
    var person = {
        "first name" : "wangwei",
        8             : true
    };
    console.log(person[8]);
    console.log(person["first name"]);
```

5-1-3 动态添加属性和方法

```
var person = {};  
person.name = "wangwei";  
person.age = 18;  
person.say = function(){  
    console.log("this person can say with other people");  
}  
function drink(){  
    console.log("this people can drink ");  
}  
person.drink = drink;  
person.say();  
person.drink();  
console.log(person.name);  
console.log(person.age);
```

5-1-4 Object类及对象的属性和方法

4-1 constructor 属性

对创建对象构造函数的引用；如：

```
<script>  
    var obj = new Object();  
    console.log(obj.constructor);  
    var obj1 = new obj.constructor;  
    console.log(typeof obj1);  
    var obj2 = new obj1.constructor;  
    console.log(typeof obj2);  
</script>
```

```
f Object() { [native code] }  
object  
object  
>
```

4-2 prototype 属性

对对象的原型的引用

```
<script>  
    console.log(Object.prototype); //原型对象  
    function Person(){};  
    console.log(Person.prototype);  
</script>
```

4-3 obj.hasOwnProperty(property)

判断该对象是否具有某个属性

```
var obj = new Object();  
obj.name = "aini";  
console.log(obj.hasOwnProperty("name")); //true  
console.log(obj.hasOwnProperty("age")); //false
```

4-4 Object.prototype.isPrototypeOf(object)

判断该对象是否为另一个对象的原型;

```
var obj = new Object();  
obj.name = "aini";  
console.log(Object.prototype.isPrototypeOf(obj)); //true
```

// 就是判断obj是不是由object创建出来的

```
function Person(){};  
var p = new Person();  
console.log(Person.prototype.isPrototypeOf(p)); //true
```

// 结果是TRUE，因为p就是由person创建出来的

4-5 Obj.propertyIsEnumerable(property)

判断给定的属性是否可以用for in 枚举;

```
var obj = {age:18};  
console.log(obj.propertyIsEnumerable("age")); //true
```

4-6 obj.toString()

返回对象的原始字符串表示

4-7 obj.valueOf() :

返回最适合该对象的原始值

5-2 其他引用类型

```
1, 数组 var arr = new Array(1,2,3,4,5);  
2, 字符串 var str = new String("aini");  
3, 正则表达式 var pattern = new RegExp("w", "i");
```

5-3 对象的废除

```
// 一个对象不使用，应该废除，释放内存；JS自带垃圾回收程序；  
// 强制销毁：object = null；这个对象就不存在了；  
// 必须把所有对象的所有引用都设为null，对象才会被清除；  
// Delete操作符删除不掉对象，但是可以删除对象的属性，对象的引用
```

```
var o = new Object();  
var obj = o;  
o = null;  
obj = null;
```

5-4 基本引用类型和引用类型的值

```
// 对象的比较并非值得比较，即使两个对象包含相同的属性和相同的值也并不相等
```

```
var a = {x:1}, b = {x:1};
console.log(a == b);    //false
var c = [], d = [];
console.log(c == d);    //false
```

// 对象的比较是引用的比较，当且仅当他们引用同一个对象时，他们才相等；

```
var a = {x:1}, b = {x:1};
var o1 = a;
console.log(o1 == a);    //true
```

5-5 引用类型的复制

```
var a = [1,2,3,8,9,7,45,12,56,89];
var b = [];
for(var i=0 ; i<a.length;i++) b[i] = a[i];
console.log(b);
```

判断两个对象是否有相同的属性值

```
function equalArrays(arr_a,arr_b){
    if(arr_a.length != arr_b.length) return false;
    for(var i=0 ; i<arr_a.length; i++){
        if(arr_a[i] != arr_b[i]) return false;
    }
    return true;
};
a = [1,2,3,4,6];
b = [1,2,3,4,5,6];
console.log(equalArrays(a,b));
```

```

var o = {
  name : "aini",
  age : 18,
  sex : "nan",
  say : function(){
    console.log(this.name + " can say with other");
  }
};
var p = {};
for(k in o){
  p[k] = o[k];
};
console.log(p);

```

6, JS基本包装类型

// ES提供了三个基本的引用类型: Boolean, Number, String;
 // 但是同时还具有与各自的基本类型相应的特殊行为;
 // 实际上, 每当读取一个基本类型的值得时候, 后台就会创建一个对应的基本包装类型的对象, 让我们用一些方法来操作这些数据

```

var s1 = "aini like reading book";
var s2 = s1.substring(4); //截取字符串的一个方法
console.log(s2);

```

// 后台默认给s1创建了一个String对象, 是临时的一个对象, 我们是看不见的再调用了他的substring()方法, 调用完了以后再把这个临时对象销毁
 // 后台默认给s1创建了一个引用类型, 并调用了他的substring()方法赋给s2;

基本包装类型的实例调用typeof 会返回object, 而且所有包装类型的对象转换为布尔值类型都是TRUE

```

var s1 = new Number(0);
alert(s1);
console.log(typeof s1);
console.log(Boolean(s1));

```

Object构造函数, 仅接受一个参数; 其会根据传入的值得类型返回相应包装类型的实例;

```
var obj = new Object("zero net week");
console.log(obj instanceof String);
console.log(obj instanceof Object);
var n = new Object(5);
console.log(n instanceof Number);
```

ES在必要时会将包装类型转换成原数值；如：

```
var s1 = "aini is a good boy";
var s2 = new String("aini is a good boy");
console.log(s1 == s2);    //true
console.log(s1 === s2);  //false
var a = 18;
var b = new Number(18);
console.log(a == b);      //true
console.log(a === b);     //false
```

实际上比较的是他们的值，基本包装类型用valueOf方法返回他的值再进行比较，所以两者相等；

注：null和undefined没有包装对象，访问他们的属性会造成一个类型错误；

6-1 Boolean 类型

// 1, Boolean类型是与布尔值对应的引用类型，Boolean 对象表示两个值："true" 或 "false"。
 // 2, 创建Boolean对象: var oBool=new Boolean(true); // 传入true或false值
 // 注：如果逻辑对象无初始值或者其值为 0、-0、null、""、false、undefined 或者 NaN，那么对象的值为 false，否则，其值为 true；

```
var oBoolean = new Boolean();
console.log(oBoolean);
console.log(Boolean(oBoolean));
```

// 不判断对象的值，而是判断这个对象存不存在，所以返回TRUE；


```
var oBoolean = new Boolean();
console.log(oBoolean);
console.log(Boolean(oBoolean));
if(Boolean){
    console.log("这个对象存在");
}
```

► *Boolean {false}*

true

这个对象存在

// 总结：理解基本类型的布尔值与Boolean对象之间的区别非常重要，建议不要使用，最好使用Boolean原始值；
// 可以使用Boolean(参数)进行数据类型转换；

6-2 Number 类型

// 1, Number 对象，是原始数值的包装对象。在必要时，JavaScript 会自动地在原始数据和对象之间转换；
// 2, 可以用构造函数 Number() 明确地创建一个 Number 对象：

```
var num = new Number(10);
```

6-2-1 对象属性

// 对象属性：

- 1, MAX_VALUE: // 表示最大的数，静态属性；
- 2, MIN_VALUE: // 表示最小的数，静态属性；
- 3, NaN: // 非数字值；静态属性；
- 4, NEGATIVE_INFINITY: // 负无穷大，溢出时返回该值，静态属性；
- 5, POSITIVE_INFINITY: // 正无穷大，溢出时返回该值，静态属性；

```
console.log(Number.MAX_VALUE);
console.log(Number.MIN_VALUE);
console.log(Number.POSITIVE_INFINITY);
console.log(Number.NEFATIVE_INFINITY);
console.log(Number.MAX_SAFE_INTEGER);
console.log(Number.MIN_SAFE_INTEGER);
```

一般用于判断表达式

// 注: `Number.NaN` 是一个特殊值, 说明某些算术运算(如求负数的平方根)的结果不是数字。

// 方法 `parseInt()` 和 `parseFloat()` 在不能解析指定的字符串时就返回这个值。

6-2-2 对象方法

`Number`类或对象方法:

1, `parseInt()` 和 `parseFloat()`方法, 静态方法;

2, `isNaN()`// 来判断是否为数字, 静态方法;

3, `isFinite`: // 是否为有限, 静态方法;

4, `isInteger`: // 是否为整形, 静态方法;

5, `toString()`方法: // 如:`NumberObject.toString(radix)`, `radix`可选, 规定表示数字的基数, 使 2 ~ 36 之间的整数, 若省略该参数, 则使用基数 10。

```
var number = new Number(1337);
console.log(number.toString(16));
console.log(number.toString(32));
console.log(number.toString(2));
```

6, `toLocaleString()` :// 把一个`Number`对象转换为本地格式的字符串

7, `toFixed()` 方法: // 把 `Number` 四舍五入为指定小数位数的数字, 类型是字符串; 参数规定了小数的位数, 是 0 ~ 20 之间的值; 有些实现可以支持更大的数值范围。如果省略了该参数, 将用 0 代替

```
var number = new Number(13.2567845);
console.log(number.toFixed(3));
```

8, `toExponential()`方法 // 可把对象的值转换成指数计数法。

如:`NumberObject.toExponential(num)` , `num`规定指数计数法中的小数位数, 是 0 ~ 20 之间的值;

```
var number = new Number(1200);  
console.log(number.toExponential(2));
```

1.20e+3

// 用指数计数法表示，而且还要保留指定的小数位数

9, toPrecision() // 可在对象的值超出指定位数时将其转换为指数计数法；其可能会返回固定大小格式，也可能返回指数格式，具体规则是看哪种格式最合适；如: `NumberObject.toPrecision(num)`，`num` 规定必须被转换为指数计数法的最小位数，该参数是 1 ~ 21；

```
var num = 99;  
console.log(num.toPrecision(1)); //1e+2  
console.log(num.toPrecision(2)); //99  
console.log(num.toPrecision(3)); //99.0  
  
var num = 10000;  
console.log(num.toPrecision(1)); //1e+3  
console.log(num.toPrecision(4)); //1.000e+4
```

// 就是小数点后或者整数位加起来就等于指定的位数，若不满足可以用科学计数法来凑

6-3 String 对象

String类型是字符串对象包装类型，用于处理文本（字符串）。语法：`var str = new String(str)`；如

```
var str = new String();  
var str = new String("aini");  
  
var str = String("525");
```

// 上面两个是对象
// 下面是强制类型转换的字符串；

6-3-1 字符串属性

1-1 length属性

返回字符串中的字符个数 如: str.length (汉字也是一个字符)

6-3-2 字符串操作方法

2-1 charAt() charCodeAt()

都接受一个参数，是基于0的字符位置；其中charAt() 方法以单字符串的形式返回给定位置的那个字符；

```
var str = "aini is a good boy";  
console.log(str.charAt(2));    // n
```

如果想得到字符编码而不是字符时，使用charCodeAt()，如：

```
var a = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";  
console.log(a.charCodeAt(1));  // 66
```

也可以用方括号来访问字符串，向数组一样

```
var a = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";  
console.log(a[2]);    // c
```

2-2 concat()

用于将一或多个字符串拼接起来，返回拼接得到的新字符串，如：

```
var str = "zero";  
var result = str.concat(" network");  
console.log(result);
```

concat()方法可以接受多个任意参数，即可以通过它可以拼接任意多个字符串，如：

```
var str = "zero";  
var result = str.concat(" network", "!", "aini");  
console.log(result);
```

zero network!aini

2-3 slice(x,y)

// x位置开始截取，到y结束（不包含y位置）

slice()会将传入的负值与字符串的长度相加

```
var str = "zero network";  
console.log(str.slice(-4));  
console.log(str.slice(-4,-1));  
console.log(str.slice(-1,-4));  
console.log(str.slice(3,-5));
```

work

wor

o ne

倒数第四个位置取到尾

倒数第四个位置取到倒数第一个位置（不包含倒数第一个）

从倒数第一个位置开始取到倒数第四个，不能方法向取，所以无返回值

第三个位置取到倒数第五个位置（不包含倒数第五个）

2-4 substr(x,y)

// x是截取开始的位置，y是截取长度

```
var str = "zero network";  
console.log(str.substr(-8,2));  
console.log(str.substr(3,-4));
```

从倒数第八个开始取，取2个长度；

第二个参数表示长度，所以不能为负数

2-5 substring(x,y)

// x位置开始截取，到y结束（不包含y位置）

// 如果只设置一个参数，则从开始位置截取到末尾

substring()方法会把所有负值参数都转换为0

```
var str = "zero network";  
console.log(str.substring(-3));  
console.log(str.substring(3,-4));  
console.log(str.substring(4,1));
```

zero network

zer

ero

负数转换为0，所以从第0位开始截取；

第二个转换为0，然后从第0位截到第3位（不包含3）

从第1位截到第4位，不包含4

把两个参数进行比较，小的当做第一个参数，大的当做第二个参数

```
var str = "zero network";
console.log(str.slice(3,10));
console.log(str.substr(3,10));
console.log(str.substring(3,10));
```

o netwo

o network

o netwo

// 同时这些方法的值可以是负值；其中，`slice()`会将传入的负值与字符串的长度相加，`substr()`将负的第一个参数加上字符串的长度，而将负数的第二个参数转换为0，`substring()`方法会把所有负值参数都转换为0，如：

2-5-1 文字滚动效果

```
<body>
  <div id="mydiv"></div>
</script>
var msg = "我叫艾尼，来自新疆，现在就读于东华大学松江校区";
var mydiv = document.getElementById("mydiv");
var pos = 0;
function scrollSmg(){
  mydiv.innerHTML = msg.substr(pos,msg.length) + "..." + msg.substr(0,pos);
  pos++;
  if(pos > msg.length) pos = 0;
  setTimeout("scrollSmg()",200);
}
scrollSmg();
```

6-3-3 字符串位置方法

3-1 indexOf() 和 lastIndexOf()

// 这两个方法都是从一个字符串中搜索给定的子字符串，然后返回子字符串的位置，如果没有找到子字符串，则返回-1

```
var str = "zero network";
console.log(str.indexOf("o")); // 3
console.log(str.lastIndexOf("o")); // 9
```

// `lastIndexOf`从后面往前找，两个方法只返回第一个出现的位置

// 可以设置第二个参数，表示从什么位置开始查找；

```
var str = "zero network";
console.log(str.indexOf("o",6)); // 9
console.log(str.lastIndexOf("o",6)); // 3
```

在使用第二个参数的情况下，可以循环调用两个办法找到所有匹配的字符串

```
var str = "Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laborum omnis quos similique!";
var position = new Array();
var pos = str.indexOf("e");
while(pos > -1){
    position.push(pos);
    pos = str.indexOf("e",pos+1);
}
console.log(position);
```

3-2 trim()

trim()方法：.// 删除前置及后缀的所有空格，然后返回结果；

还有两个非标准的trimLeft()和trimRight()方法， // 分别用于删除字符串开头和末尾的空格

```
var str = "    aini    ai    dilnur    ";
console.log(str.trim());
console.log(str.trimLeft());
console.log(str.trimRight());
script>
body>
html>
```


3-3 大小写转换

```
toLocaleLowerCase() // 把字符串转换为小写；
toLocaleUpperCase() // 把字符串转换为大写；
toLowerCase() // 把字符串转换为小写；
toUpperCase() // 把字符串转换为大写；

// 其中toLowerCase()和toUpperCase()是最常用的方法
```

```
var str = "aini ai dilnur";
console.log(str.toLowerCase());
console.log(str.toUpperCase());
console.log(str.toLocaleUpperCase());
console.log(str.toLocaleLowerCase());
```

6-3-4 字符串匹配方法

4-1 match()

本质上与RegExp的exec()方法相同；其只接受一个参数，要么是一个正则表达式，要么是一个RegExp对象，如

```
var text = "cat,bat,sat,fat";
var pattern = /.at/;
var matches = text.match(pattern);
console.log(matches.index);
console.log(matches);
console.log(matches[0]);
console.log(matches[1])
```

// 说明：match()返回了一个数组，其第一项是与整个模式匹配的字符串，之后的每一项（如果有）保存着与正则表达式中的捕获组匹配的字符串

4-2 search()

接受的参数与match()一样；该方法返回字符串中第一个匹配项的索引；如果没有找到匹配项，则返回-1；如


```
var text = "cat,bat,sat,fat";
console.log(text.search(/at/)); // 1
console.log(text.search("bat")); // 4
```

4-3 replace()

replace()方法：// 替换字符串，接受两个参数，第一个参数是一个查找的字符串或者是正则，第二个参数是要替换的字符串或者函数；
// 注意：如果第一个参数是字符串，则只会替换第一个找到的字符串，想替换所有的话第一个参数可以写个正则；

```
var text = "cat,bat,sat,fat";
var result = text.replace("at","ond");
console.log(result); // cond,bat,sat,fat
var result = text.replace(/at/g,'aini');
console.log(result);
```

// 想要替换全部就加上g(全局标志)，不然只会替换第一个如果没匹配上替换不了，就会输出原有的值

```
var text = "cat,bat,sat,fat";
var result = text.replace("at12","ond");
```

cat,bat,sat,fat st

4-4 split

split()方法：// 基于指定的分隔符将一个字符串分割成多个字符串，并把结果放在一个数组中；分隔符可以是字符串，也可以是正则；其可以接受可选的第二个参数，用于指定数组的大小；

```
var colorText = "red,blue,green,yellow";
var namebox = "aini-dilnur-xiaoai-aili";
var colors1 = colorText.split(",");
var name1 = namebox.split("-");
console.log(colors1);
console.log(name1);
```

scripts.html?_ijt=6dj...RELO/

► (4) ['red', 'blue', 'green', 'yellow']

scripts.html?_ijt=6dj...RELO/

► (4) ['aini', 'dilnur', 'xiaoi', 'aili']

```
var colorText = "red,blue,green,yellow";
var namebox = "aini-dilnur-xiaoai-aili";
var colors2 = colorText.split(", ", 2);
var name2 = namebox.split("-", 3);
console.log(colors2);
console.log(name2);|
```

► (2) ['red', 'blue'] scripts.html?_ijt=6

scripts.html?_ijt=6

► (3) ['aini', 'dilnur', 'xiaoi']

第二个参数可以指定长度

```
var colorText = "red,blue,green,yellow";
var colors2 = colorText.split(/[^\,]+/);
console.log(colors2);
```

也可以是正则表达式

6-2-5 其他方法

5-1 localeCompare()

localeCompare() // 用本地特定的顺序来比较两个字符串，默认返回下列值中的一个：-1、0、1，如

```
var str = "yellow";
console.log(str.localeCompare("aini"));
console.log(str.localeCompare("yellow"));
console.log(str.localeCompare("zero"));
```

1

0

-1

5-2 fromCharCode()

fromCharCode()方法： // String构造函数本身还有一个静态方法：fromCharCode()，这个方法的任务是接受一或多个字符编码，然后将它们转换成一个字符串；从本质上看，这个方法与实例方法charCodeAt()执行的是相反的操作

```
console.log(String.fromCharCode(65,44,88,125,69));
```

A,X}E

把ascii码转换成对应的字母或数字

5-3 HTML方法

// 早期的web浏览器可以使用JavaScript动态格式化HTML，其扩展了字符串的标准，实现了一些专门用于简化常见HTML格式化任务的方法；但是，尽量不要使用这些方法，因为它们创建的标记通常无法表达语义

```
anchor(name) // 创建 HTML 锚，输出如：<a name="name">string</a>
big() // 用大号字体显示字符串，如：<big>string</big>
small() // 用小字号来显示字符串，如：<small>string</small>
blink() // 显示闪动字符串；
bold() // 使用粗体显示字符串，如：<b>string</b>
fontcolor(color) // 使用指定的颜色来显示字符串，如：<font color="color">string</font>
fontsize(size) // 使用指定的尺寸来显示字符串，如：<font size="size">string</font>
italics() // 使用斜体显示字符串，如：<i>string</i>
link(url) // 将字符串显示为链接，如：<a href="url">string</a>
fixed() // 以打字机文本显示字符串，如：<tt>string</tt>
sup() // 把字符串显示为上标，如：<sup>string</sup>
sub() // 把字符串显示为下标，如：<sub>string</sub>
strike() // 使用删除线来显示字符串，如：<strike>string</strike>
```

7, Global 对象

// 全局对象及其属性在任何地方都可以直接使用；
// Global对象是对象，不是类，所以没有构造函数，不能new实例化
// 注：所有全局变量也都是全局对象的属性；

这个对象是不存在的，换句话说，不属于任何其他对象的属性和方法，最终都是它的属性和方法；因此它没有名字（Global不是全局对象的名字，只是我们这样称呼它）

```
console.log(isNaN("a"));
```

// isNaN就是全局对象的函数

7-1 全局属性

// 如undefined、NaN以及Infinity都是Global属性；
// ECMAScript5禁止给undefined、NaN和Infinity赋值，这样做即使在非严格模式下也会导致错误。

7-2 全局对象

Math, json

```
console.log(Math.sqrt(16));
```

7-3 全局函数

```
// isNaN(), isFinite(), parseInt(), parseFloat()等
// isNaN()用来确定一个值是否为NaN，而Number.isNaN()确定传递的值是否为NaN和其类型是Number；
它是原始的全局isNaN的强大的版本
```

```
console.log(Number.isNaN("a")); // false
console.log(isNaN("a")); // true
```

```
console.log(Number.isNaN(undefined));
console.log(isNaN(undefined));
```

// Number.isFinite()和全局的isFinite()相比，不会强制将一个非数值的参数转换成数值，这意味着，只有数值类型的值，且是有穷的，才返回true

// 只有为Number的值且是有限的时候Number.isFinite()会返回true。而isFinite()在参数经过Number转换后再判断是否是有限的。正无穷、负无穷和NaN都不是有限数字。

// Number.isInteger()用来判断给定的参数是否为整数；只有是Number类型的值并且是整数才会返回true，Infinity和NaN都不是整数。

7-4 全局构造函数

// 全局对象还定义了一些属性，用来引用JavaScript预定义的所有其他对象，这些对象大部分是原生引用类型的构造函数：

Object、Number、String、Boolean、Array、Date、Function、RegExp、Error、EvalError、RangeError、ReferenceError、SyntaxError、TypeError、URIError

// 根据JavaScript的运行环境，在JS中存在两种全局对象：JS全局对象和window全局对象

// 当JavaScript放在特定宿主环境时，全局对象通常具有与该特定环境相关的额外属性；这些额外属性并不是ES标准规定的，而是由宿主环境实现的；如在客户端JavaScript中，全局对象是window对象，表示运行JS的web浏览器窗口；或在nodejs中，Global指的就是global对象；

```
var str="aini is a good boy";
console.log(window.str);
```

// 全局中的this：由于全局对象位于作用域链的最顶部，所以在全局环境中，关键字this指的就是全局对象；如果在浏览器中，this指的就是window对象；

```
▶ window {window: Window, self: Window, document: document, name: '', location: Location, ...}
```

This 就相当于全局对象的代理人，在浏览器环境中就是window；

在ES中，全局对象的预定义属性都是不可枚举的，因此使用for/in来只能列出所有隐式或显式声明的全局变量，如

```
for(name in this){
    console.log(name);
}
```

webkitStorageInfo
speechSynthesis
openDatabase
webkitRequestFileSystem
webkitResolveLocalFileSystemURL
aini
TEMPORARY
PERSISTENT
addEventListener

7-5 全局变量

所有的全局变量都全局对象的属性

声明的4种方法：

// 1.直接在全局作用域中用var 声明的变量就是全局变量，此种方式声明的变量具有不可配置的属性，不能使用delete操作符把变量删除。

// 2.window.变量,这种声明的变量也是全局变量，但这种变量跟上面用var 声明的变量有点不一样，这种方式声明的全局变量是可配置的，因此能用delete操作符把变量删除。

// 3.隐式声明全局变量，就是不使用var声明，直接进行赋值的变量，在不严格模式中，相当于window.变量这种方式，但在严格模式下，会报错。

// 4.在html中给标签指定一个id属性，也相当于给window对象添加了一个id的属性，在javascript中可直接通过标签的id访问该标签（或者window['id']）。

7-6 Global对象的方法

7-6-1 字符编码方法

1-1 Escape() 函数

// escape() 函数可对字符串进行编码，这样就可以在所有的计算机上读取该字符串（其中某些字符被替换成了十六进制的转义序列 ）

// 注:该方法不会对 ASCII 字母和数字进行编码，也不会对下面这些 ASCII 标点符号进行编码： - + _ . *

var str ="aini is a good boy"
var a="_,.艾尼!-+";
console.log(escape(str));
console.log(escape(a));

// 转换成以%开头的16进制码
// 如 空格→%20 艾尼 → %u827e %u5c3c

aini%20is%20a%20good%20boy

_%2C.%u827E%u5C3C%21-+

1-2 unescape() 函数

unescape() 函数

// **unescape()** 函数可对通过 **escape()** 编码的字符串进行解码。

// 注:通过找到形式为 **%xx** 和 **%uxxxx** 的字符序列 (**x** 表示十六进制的数字), 用 **Unicode** 字符 **\u00xx** 和 **\uxxxx** 替换这样的字符序列进行解码。

```
console.log(unescape("a%20i%20s%20a%20good%20boy"));
```

解码方式 :

// 把% 抓换成 \ ,这样以后浏览器就能识别\uxxxx的Unicode编码了

```
console.log("\u827e"+" \u5c3c"); // 艾尼
```

7-6-2 URL编码方法

// **Global**对象的**encodeURIComponent()**和**encodeURIComponent()**可把字符串作为 **URI (Uniform Resource Identifiers, 通用资源标识符)** 进行编码, 以便发送给浏览器;

// 有效的**URI**中不能包含某些字符, 如空格; 这两个编码方法, 用特殊的**UTF-8**编码替换所有无效的字符, 从而让浏览器能够接受和理解。

// 注: 两个方法不会对 **ASCII** 字母和数字进行编码, 也不会对这些 **ASCII** 标点符号进行编码: **- _ . ! ~ * ' ()**

encodeURIComponent() 也不会对在**URI**中具有特殊含义的符号;**/?:@&=+\$, #**进行转义, 但**encodeURIComponent()** 会

// 其中, **encodeURIComponent()** 主要用于整个**URI**, 如:

http://www.zeronetwork.cn/my case.html, 而**encodeURIComponent()** 主要用于对**URI**中的某一段, 如**my case.html**进行编码;

// 它们的主要区别在于, **encodeURIComponent()** 不会对本身属于**URI**的特殊字符进行编码, 如: 冒号、正斜杠、问号 and 井号; 而**encodeURIComponent()** 则会对它发现的任何非标准字符进行编码

2-1 encodeURIComponent() 和 encodeURIComponent()

encodeURIComponent() 函数 和 **encodeURIComponent()** 函数

```
var str="zero!ne work零-!~'(*&#?";
console.log(encodeURIComponent(str));
```

zero!ne%20work%E9%9B%B6-!~'(*&#?

```
var str="zero!ne work零-!~'(*&#?";
console.log(encodeURIComponent(str));
```

zero!ne%20work%E9%9B%B6-!~'(*%26%23%3F

// **encodeURIComponent()** 函数 编码范围广一点 (用的时候仅仅对**URL**里面某一部分进行编码)


```
var url1="https://www.zeronetwork.cn/case.html?user%20name&age=18#one";
console.log(encodeURIComponent(url1));
console.log(encodeURIComponent(url1));
```

<https://www.zeronetwork.cn/case.html?user%2520name&age=18#one>

<https%3A%2F%2Fwww.zeronetwork.cn%2Fcase.html%3Fuser%2520name%26age%3D18%23one>

// 所以encodeURIComponent不适合对整个URL进行编码
// 注：一般来说，使用encodeURIComponent()的场景比使用encodeURIComponent()要多，因为在实践中更常见的是对查询字符串参数而不是对基础URI进行编码

2-2 decodeURI() 和 decodeURIComponent()

decodeURI() 和 decodeURIComponent()

// 其中decodeURI() 只能对 encodeURIComponent() 编码过的 URI 进行解码；
// 如：可将%20替换成一个空格，但不会对%23作任何处理，因为%23代表井号，而井号不是使用encodeURIComponent()替换的；
// 同样的，decodeURIComponent()能够解码encodeURIComponent()编码的所有字符，即它可以解

```
var url1="https://www.zeronetwork.cn/case.html?user name&age=18#one";
console.log(encodeURIComponent(url1));
console.log(encodeURIComponent(url1));

console.log(decodeURI("https://www.zeronetwork.cn/case.html?
user%20name&age=18#one"));
console.log(decodeURIComponent("https%3A%2F%2Fwww.zeronetwork.cn%2Fcase.html%3Fu
ser%20name%26age%3D18%23one"));
```

<https://www.zeronetwork.cn/case.html?user%20name&age=18#one>

<https%3A%2F%2Fwww.zeronetwork.cn%2Fcase.html%3Fuser%20name%26age%3D18%23one>

<https://www.zeronetwork.cn/case.html?user name&age=18#one>

<https://www.zeronetwork.cn/case.html?user name&age=18#one>

// 注：URI方法代替了已经被ECMA-262第3版废弃的escape和unescape()方法，因为URI方法能对所有Unicode编码，而原来的方法只能对ASCII符号正确编码；因此在实际场景中，一定要使用URI方便，不要再使用escape()和unescape()方法；

```
onsole.log(escape("http://www.zeronetwork.cn/index.html?id=101&username=艾尼"));
console.log(escape("?!=()#%&"));
```

```
console.log(encodeURIComponent("http://www.zeronetwork.cn/index.html?id=101&username=艾尼"));
console.log(encodeURIComponent("?!=()#%&"));
```

```
console.log(encodeURIComponent("http://www.zeronetwork.cn/index.html?
id=101&username=艾尼"));
console.log(encodeURIComponent("?!=()#%&"));
```

```
http%3A//www.zeronetwork.cn/index.html%3Fid%3D101%26username%3D%u827E%u5C3C%3F%21%3D%28%29%23%25%26
http://www.zeronetwork.cn/index.html?id=101&username=%E8%89%BE%E5%B0%BC?!=())#%25&
http%3A%2F%2Fwww.zeronetwork.cn%2Findex.html%3Fid%3D101%26username%3D%E8%89%BE%E5%B0%BC%3F!%3D()%23%25%26
```

7-7 eval 函数

```
//只接受一个字符串参数，是正常的JS或者ES代码
// 可以动态执行JS代码
```

```
function swapImg(){
    var img = eval("document.getElementById(myimg)")
    myimg.src="images/02.jpg";
}
var myImg=document.getElementById("myimg");
myImg.onmouseover=swapImg;
```

同样的，也可以在eval()中定义一个函数，在该调用的外部代码中引用这个函数，如：

```
eval("var msg='hello world';");alert(msg);
```

// 注：在eval()中创建的任何变量或函数都不会被提升，因为在解析代码的时候，它们被包含在一个字符串中，它们只在eval()执行的时候被创建；

```
console.log(str);    // undefined
var str = "aini like dilnur";
```

// 虽然变量声明在后，访问在前；但是str会被提升，但是他的值不会被提升，所以str存在，返回undefined；

```
show();

function show(){
    console.log("show");
}
```

// 这里函数也提升了

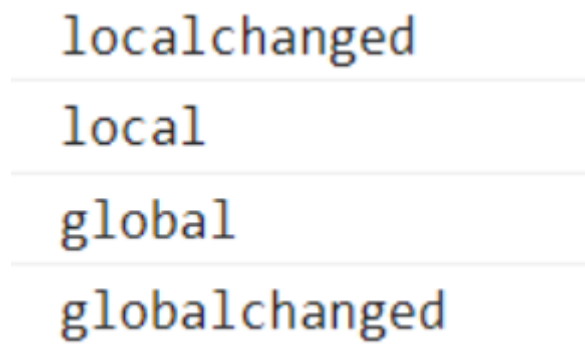
```
eval("console.log(str)");
str="aini like dilnur";
```

// 会报错，str未定义，说明str没有被提升


```
var s="global";
function func(){
    eval("var s = 'local--';") // 局部的 在外边不能够访问
    window.eval("var s = 'window.local';") //全局
}
func();
console.log(s); // window.local
```

```
var x = y ="global";
function f(){
    var x = "local";
    eval("x+='changed';") //改变的是局部x的值
    console.log(x);
}

function g(){
    var y = "local";
    window.eval("y+='changed';"); //改变的是全局x的值
    console.log(y)
}
f(); //返回函数局部变量 x 的值
g(); //返回局部变量 y 的值，由于没改变 所以返回 local
console.log(x); //返回全局变量 x 的值
console.log(y); //返回全局变量 y 的值
```



所以说window.eval在函数内能改变全局变量的值;

```
// 函数内可以访问到全局变量，函数外不能访问函数内的局部变量；当函数内，函数外声明同一个变量时，
函数内优先访问函数内的局部变量；
```

Window.eval()在函数内优先访问全局变量

eval() 当在函数内部作用域中，eval执行的是局部，如果调用window.eval则在全局空间执行

```
// 注：在严格模式下，ES5对eval()的行为施加了更多的限制，在外部访问不到eval()中创建的任何变量
或函数，同时，为eval()赋值也会导致错误
```

不是严格模式下:

```
var x = 18;
eval("var y=20;console.log(y)");
console.log(y); // 外部也能访问
```

在严格模式下:

```
"use strict"
var x = 18;
eval("var y=20;console.log(y)");
console.log(y);    // 外部不能访问
```

7-8 window对象

// ECMAScript虽然没有指出如何直接访问Global对象，但web浏览器都是将这个全局对象作为window对象的一部分加以实现的；因此，在全局作用域中声明的所有变量和函数，就都成为了window对象的属性

```
var color="red";
function sayColor(){
    console.log(window.color);    //就可以当window的属性用
}
sayColor();
```

// 注：JavaScript中的window对象除了扮演ECMAScript规定的Global对象的角色外，还承担了很多别的任务（即window对象下的一些方法和属性都是js原生提供的，但在浏览器环境中，大量的功能都是由宿主对象完成的）；另外一种取得Global对象的方法

```
var global=function(){
    return this;
}();    // 相当于global();

// 定义的同时函数也运行了
```

// 说明分析：以上代码创建了一个立即调用的函数表达式，返回this；在没有给函数明确指定this值的情况下，this值等于Global对象

```
var global=function(){
    return this;
}();    // 相当于global();

var str="aini like dilnur";
console.log(str);
console.log(window.str);
console.log(this.str);
console.log(global.str);
```

aini like dilnur

aini like dilnur

aini like dilnur

aini like dilnur

在函数内访问外部变量的方法

```
var global=function(){
    return this;
}();    // 相当于global();

var str="aini like dilnur";
function show(){
    var str="aini";
    console.log(window.str);
    window.eval("str");
    console.log(global.str);
    console.log(this.str);    //容易产生歧义
}
show();
```

7-9 Math 对象

// Math对象是一个静态对象，或者称为工具类，不能使用new关键字创建对象，应直接使用“对象名.成员”的格式来访问其属性和方法，如：
var num=Math.random();

7-9-1 math对象的属性

E: // 自然对数的底数，即常数e.约等于2.718;
LN10: // 代表10的自然对数,约等于2.302;
LN2: // 代表2的自然对象,约等于0.693;
LOG2E: // 代表以2为底E的对数;
LOG10E: // 代表以10为底E的对数;
PI: // 代表π的值,约等于3.1415926;
SQRT1_2: // 代表2的平方根分之一，约等于0.707;
SQRT2: // 代表2的平方根,约等于1.414;

```
console.log(Math.PI);
console.log(Math.E);
console.log(Math.SQRT1_2);
console.log(Math.abs(-5));
```

7-9-2 math对象的方法

2-1 min()和max()

// 用于确定一组数值中的最小值和最大值；这两个方法都可以接收任意多个数值参数

```
var max =Math.max(3,25,45,78,96,54,100);
var min =Math.min(87,45,12,59,87);
console.log(max);
console.log(min);
```

注：数组中不能用，所以需要借助apply()方法，如：

```
var arr=[1,25,84,75,95,42,35,68];
var max=Math.max.apply(Math,arr);
console.log(max);
```

2-2 ceil()

```
// 向上取整

console.log(Math.ceil(1.1)); // 2
console.log(Math.ceil(-1.5)); // -1
```

2-3 floor()

```
// 向下取整
console.log(Math.floor(1.1)); // 1
```

2-4 round()

```
// .5的时候round返回比它大的整数，所以负数有点不一样
// 注：在round里面使用负数时注意：
```

```
console.log(Math.round(-1.1));
console.log(Math.round(-1.5)); //注意中间的
console.log(Math.round(-1.6));
```

-1

-1

-2

2-5 Random()

```
Random()方法：// 随机数；
// 产生[0,1)之间的随机数
```

```
console.log(Math.random());
```

0.937220219428909

般与其他方法配合使用，以求得一个在一个范围内的随机数

```
console.log(Math.floor(Math.random()*10+1)); // [1,10]
```

总结公式：
 $\text{Math.floor}(\text{Math.random()} * (\text{max} - \text{min} + 1) + \text{min})$

得到一个区间的整数随机数

```
function intRandom(min,max){
    return Math.floor(Math.random()*(max-min+1)+min);
}
console.log(intRandom(10,100));
```

数组当中随机拿出一项

```
var colors=["red","blue","green","black","purple"];
var color=colors[Math.floor(Math.random()*(colors.length))];
console.log(color);
```

随机返回字符

```
var str="ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789";
var a="";
function getRandomStr(x){
    for(var i=0;i<x;i++){
        var random=Math.floor(Math.random()*(str.length-1));
        a+= str.substring(random,random+1);
    }
    return a;
}
console.log(getRandomStr(4));
```

从中文里面随机取x个字符

```
// 首先确定汉字编码的范围
4E00 -----9FA5
// 16进制转换成10进制
19968 -----40869
```

字数	Unicode 编码
20902字	4E00-9FA5
38字	9FA6-9FCB
6582字	3400-4DB5
42711字	20000-2A6D6
4149字	2A700-2B734
222字	2B740-2B81D
214字	2F00-2FD5
115字	2E80-2EF3
477字	F900-FAD9
542字	2F800-2FA1D
81字	E815-E86F
452字	E400-E5E8
207字	E600-E6CF
36字	31C0-31E3
12字	2FF0-2FFB
22字	3105-3120
22字	31A0-31BA
1字	3007

```

function getRandomHan(len){
var str="";
for(var i=0;i<len;i++){
    c=(Math.floor(Math.random()*(40869-19968+1)+19968)).toString(16);
    c = "\\u" +c;
    c=c.replace(/\\/g, "%");
    c= unescape(c);
    c.replace(/%/g, "\\") //还存在%的用\代替回去;
    str+=c;
}
return str;
}
console.log(getRandomHan(4))

```

继续封装，继续简化

```

function getRandomHan(len){
var str="";
for(var i=0;i<len;i++){
    c=getRandom(19968,40869).toString(16);
    c = decodeUnicode(c);
    str+=c;
}
return str;
}
console.log(getRandomHan(4))

function getRandom(min,max){
    return Math.floor(Math.random()*(max-min+1)+min);
};
function decodeUnicode(str){
    str="\\u"+str;
    str=str.replace(/\\/g, "%");
    str=unescape(str);
    str=str.replace(/%/g, "\\");
    return str
}

```

老师课件上的封装

```

function getRandomHan(len){
    var str = '';
    for(var i=0;i<len; i++){
        var c = (Math.floor(Math.random() * (40869 - 19968) + 19968)).toString(16);
        c = "\\u" + c;
        c = c.replace(/\\/g, "%");
        c = unescape(c);
        c = c.replace(/%/g, "\\");
        str += c;
    }
    return str;
}
console.log(getRandomHan(6));
// 拆分整合

```

```

function getRandom(min,max){ // 获取指定范围内的随机数
    return Math.floor(Math.random() * (min - max) + max);
}
function decodeUnicode(str) { // 解码
    str = "\\u"+str; //Unicode显示方式是\u4e00
    str = str.replace(/\\/g, "%");
    str = unescape(str); //转换中文
    str = str.replace(/%/g, "\\"); //将其他受影响的转换回原来
    return str;
}
function getRandomChinese(len){
    var str = "";
    for(var i = 0;i<len;i++){
        str += decodeUnicode(getRandom(0x4e00,0x9fa5).toString(16));
    }
    return str;
}
console.log(getRandomChinese(2));

```

7-9-3 其他函数

```

abs(num): // 绝对值、
asin(x): // 反正弦值、
exp(num): // Math.E的num次幂、
atan(x): // 反正切值、
log(num): // num的自然对数、
atan2(y,x): // y/x的反正切值、
pow(num,power): // num的power次幂、
cos(x): // 余弦值、
sqrt(num): //平方根、
sin(x): // 正弦值、
acos(x): // 反余弦值、
tan(x): // 正切值

```

8，函数的形参与实参

(形参 parameter 实参 argument)

8-1 形参

8-1-1 可选参数

可选参数：

// 当实参比形参的个数要少时，剩下的形参都将设置为undefined值；为了让实参与形参保持较好的适应性，有必要为可选形参设置一个合理的默认值；如：

```

function show(name,msg){
    if(name==undefined) name="aini";
    if(msg==undefined) msg="18 years old";
    console.log("hello,"+name+",message:"+msg);
}
show();
show("aili");
show("dilnur","22 years old");

```

不用 if 可以使用||，也是惯用手法

```
function show(name,msg){
    name=name || "aini";
    msg= msg || "18 years old";
    console.log("hello,"+name+",message:"+msg);
}
show();
show("aili");
show("dilnur","22 years old");
```

// 将对象o中可枚举的属性追加到数组a中，并返回数组a，如果省略a，则创建一个新数组并返回这个新数组；

► (4) ['name', 'age', 'sex', 'friend']

► (7) [1, 2, 3, 'name', 'age', 'sex', 'friend']

```
function getProprtyNames(o,/* optional */a){ // 一般可以加个注释来指定一下不定参数
    if(a == undefined) a=[];
    // a= a || [];
    for(var property in o){
        a.push(property);
    }
    return a;
}
var o={name:"aini",age:18,sex:"nan",friend:"dilnur"};
var result=getProprtyNames(o);
console.log(result);
var arr=[1,2,3];
console.log(getProprtyNames(o,arr));
```

// 说明：当设置可选参数时，一定要把可选参数放到形参列表的最后，因为不能省略第一个参数（定参）并传入第二个实参；如果第一个参数（定参）不存在或不使用，也需要显式的传入undefined，真实的场景是传入一个无意义的占位符null

```
function show(a,b,/* optional */c,d){
}
show(1,2,3,4); //正常使用
show(1,null,2,3); // 使用null占位
```

8-1-2 形参个数

//函数本身的length属性，只读，它代表函数的形参个数，即函数期望传入的实参个数，如：
// 函数名.length 可以返回传入的形参的个数


```
function show(a,b,/* optional*/c,d){
    console.log(show.length);
}
show(1,2,3,4);
show(1,null,2,3);
```

8-2 实参

8-2-1 实参对象 (arguments)

```
// 传入的实参个数超过函数定义的形参个数时，没有办法直接获得超出的实参的值，但可以使用实参对象
arguments来获取
// 在函数体内，arguments是指向实参对象有的引用，其是一个类数组对象；
// 函数接收的始终都是这个数组，而不关心数组中包含哪些参数；
// 即使没有传递参数，这个数组也存在，只不过是包含0个元素的数组；
```

```
function show(a,b){
    console.log(arguments);
}
show();
show(1,2,3,4);
```

► Arguments [callee: f, Symbol(Symbol.iterator): f]

► Arguments(4) [1, 2, 3, 4, callee: f, Symbol(Symbol.iterator): f]

Arguments.length 可以返回实参个数

```
function sayHi(){
    console.log("传入了， "+arguments.length+"个实参");
    console.log("hello,"+arguments[0]+","+"arguments[1]");
}
sayHi();
sayHi("aini","huan ying ni lai xue xi");
```

可以使用arguments对象来验证所传递的参数是否符合函数要求；如

```
//实际场景中，如果对参数个数有要求需要验证；
function func(x,y,z){
    //首先验证传入的个数是否正确
    if(arguments.length!== func.length) throw new Error("需要"+func.length+"三个函数"); //抛出异常错误；
    //在判断类型；
    if(typeof arguments[0] !== "number")
        console.log("实参类型是数字");
    z+=arguments[2]?y:0;
    return x+y+z;
}
console.log(func(1,2,"w"));
```

```
// 在某些时候，没有必要检查实参的个数，因为JS的默认行为是可以满足需要的，如省略的实参都是
undefined，多出的参数自动省略
```

```
function check(args){
    var actual=args.length; //实参的真实个数;
    var expected=args.callee.length; //形参个数;
    if(actual !== expected) throw new Error("参数个数不对");
}
function f(x,y,z){
    try{
        check(arguments);
        return x+y+z;
    }
    catch(e){
        console.log(e);
    }
};
console.log(f(1,2,3));
console.log(f(2,3));
```

```
function check(args){
    var actual=args.length; //实参的真实个数;
    var expected=args.callee.length; //形参个数;
    if(actual !== expected) throw new Error("参数个数不对");
}
function f(x,y,z){
    try{
        check(arguments);
        return x+y+z;
    }
    catch(e){
        console.log(e);
    }
};
console.log(f(1,2,3));
console.log(f(2,3));
```

Arguments.callee 指向调用者这里指向的就是 f(x,y,z)这个

6

```
Error: 参数个数不对
    at check (biaogeaini.html:19:32)
    at f (biaogeaini.html:23:5)
    at biaogeaini.html:31:13
```

undefined

可以利用arguments对象，让函数能够接收任意个数并分别实现适当的功能，如

```
function doAdd(){
    if(arguments.length==1) console.log(arguments[0]);
    else if(arguments.length==2) console.log(arguments[0]+arguments[1]);
    else if(arguments.length==3) console.log(arguments[0]+
arguments[1]+arguments[2]);
}
doAdd(10,20);
doAdd(5,10,20);
doAdd(10,35,12,45);

// 根据实参的个数进行处理
```

优化代码（利用循环）

```
function doAdd(){
    var sum = 0;
    for(var i=0;i<arguments.length;i++){
        sum+=arguments[i];
    }
    console.log(sum);
}
doAdd(10,20);
doAdd(1,2,3,4,5,6,7,8,9);
```

继续优化

```
function doAdd(){
    var sum = 0;
    for(var i=0;i<arguments.length;i++){
        sum+= (+arguments[i]?arguments[i]:0);
    }
    console.log(sum);
}
doAdd(10,20);
doAdd(1,2,3,4,5,"aini",NaN,8,9);
```

根据实参返回最大值

```
function max(){
    var max=Number.NEGATIVE_INFINITY;
    for(var i=0;i<arguments.length;i++){
        (arguments[i]>max)?(max=arguments[i]):max;
    }
    return max;
}
var largest=max(1,45,32,189,245,12,36,985,421,65);
console.log(largest);
```

// 类似于这种可以接收任意个数的实参，这种函数也被称为“不定实参函数”，这个术语来自C语言

// 但真正的不定实参的个数不能为零，其应用的场景是：该函数包含固定个数的命名和必须的参数，以及随后个数不定的可选实参，即arguments对象可以与命名参数一起使用

```
function doAdd(num1,num2){
    var result=parseInt(num1)+parseInt(num2);
    if(arguments.length > 2){
        for(var i=2;i<arguments.length;i++){
            result+= parseInt(arguments[i]);
        }
    }
    return result;
}
console.log(doAdd(20,30,40,50,60,30));
```

arguments并不是真正的数组，它是一个对象，其数组元素是函数实参的别名；arguments的值永远与对应命名参数的值保持同步

```
function func(num1){
    console.log(num1);    // 10
    console.log(arguments[0]); //10
    arguments[0]=1;
    console.log(num1);    // 1
    console.log(arguments[0]) //1 ;
}
func(10);
```

```
function func(num1){
    console.log(num1);    // 10
    console.log(arguments[0]); //10
    arguments[0]=1;
    console.log(num1);    // 1
    console.log(arguments[0]) //1 ;
}
func(10);
```

这并不是说读取这两个值会访问相同的内存空间，
它们的内存空间是独立的，但它们的值会同步；即
两者本质上是不同的，其使用的是映射的关系

// 注：在严格模式下，重写arguments的值会失效，但不会导致语法错误
// Javascript是一门弱类型语言，所以函数中的形参并没有类型声明，并且在传入参数前也未做任何类型检查，虽然JS可以进行数据类型的自动转换，但在某些时候（类型不同，不会导致语法错误，但在程序运行时有可能导致错误），函数还是希望得到一个类型明确的参数值，此时，应当对所传入的参数进行类型的检查，如：

```
function sum(arr){
    if(Array.isArray(arr)){
        var total=0;
        for(var i=0;i<arr.length;i++){
            var element=arr[i]
            if(element == null) continue;
            if(isFinite(element)) total+=element;
            else throw new Error("数组元素必须是有限数");
        }
        return total;
    }else throw new Error("函数的参数必须是数组");
}
console.log(sum([1,2,3,4,5,6]));
```

扩大累加的数据类型

```
function flexsum(a){
    var total=0;
    for(var i=0;i<arguments.length;i++){
        var element=arguments[i],n;
        if(element == null) continue;
        if(Array.isArray(element)){
            n=flexsum.apply(this.element);
        }else if(typeof element ==="function"){
            n=Number(element());
        }else n=Number(element);
        if(isNaN(n)) throw new Error("无法把: "+element+"转换为数字");
        total+=n;
    }
    return total;
}
console.log(flexsum(1,2,3));
console.log(flexsum(1,"",3));
console.log(flexsum(1,[7,8],3));
console.log(flexsum(function(){return 18;}));
```

没有重载

```
unction func(){
    console.log("func函数");
}
function func(num){
    console.log("年龄为: "+num);
}
function func(name,age){
    console.log("名字为: "+name+", 年龄为: "+age);
}
func();
func(18);
func("aini",18);
```

名字为: undefined, 年龄为: undefined

名字为: 18, 年龄为: undefined

名字为: aini, 年龄为: 18

// 虽然定义了三个一样的函数，但是最后定义的把前面定义的两个覆盖掉

// ECMAScript函数不能像传统意义上那样实现重载；而在其他语言中，可以为一个函数编写两个定义，只要这两个定义的签名（参数的类型和数量）不同即可；ECMAScript函数没有签名，因为其参数是由包含零或多个值的数组来表示的；而没有函数签名，真正的重载是不可能做到的；

```
var addNum = function(num){return num+100};
var addNum = function(num){return num+300};
```

函数表达式定义同名的函数时，下面的变量就把前面的变量覆盖了，所以也不存在重载；

8-2-2 模拟重载

可以通过检查传入函数中参数的类型和数量并作出不同的反应，可以模拟方法的重载；

```
function overrideFunc(){
    var arglen=arguments.length;
    if(arglen==0) console.log("wu canshu de fangfa");
    else if(arglen==1) console.log("传入的参数为: "+arguments[0]);
    else if(arglen==2) {console.log("传入了2个参数");
        if(!isNaN(arguments[0]&& !isNaN(arguments[1]))){
            console.log("和为: "+(arguments[0]+arguments[1]));
        }else console.log("不是数字:"+arguments[0]+arguments[1]);
    }else console.log("未知，其他处理");
}
overrideFunc();
overrideFunc("零点程序员");
overrideFunc(3,4);
overrideFunc(3,"a",33);
```

wu canshu de fangfa

传入的参数为: 零点程序员

传入了2个参数

和为: 7

未知，其他处理

8-2-3 callee和caller属性

// **callee**属性指向当前正在执行的函数，在某些时候非常有用，如在一个匿名函数中使用递归；但在严格模式下会抛出错误，所以，尽量避免使用**callee**属性，可以为函数明确定义一个名称；

```
//数的阶乘
function factorial(x){
    if(x<=1) return 1;
    return x* factorial(x-1);
}
console.log(factorial(10));
```

```
//数的阶乘
function factorial(x){
    if(x<=1) return 1;
    // return x* factorial(x-1);
    return x*arguments.callee(x-1) //这样就不会出错了;
}
var myFactorial = factorial;
factorial = null;
console.log(myFactorial(10));
```

8-2-4 匿名函数中使用

```
//数的阶乘
function create(){
  return function(n){
    if(n<=1) return 1;
    return n*arguments.callee(n-1);
  };
}
console.log(create()(5));
// 由于内函数没有名字，所以只能用arguments.callee来指向他；
```

caller是非标准的，但大多数浏览器都实现了这个属性，它指向调用当前函数的函数，在ES5中被移除了，其始终会返回undefined，不会报错

8-3 参数的传递方式

```
// 基本数据类型：值传递
// 引用数据类型：引用传递，地址传递（本质上还是按值传递）
```

```
function setname(obj){
  obj.name="aini";
}
var person = new Object();
setname(person);
console.log(person.name); //aini

// obj和person内存对应同一个地址
```

```
function setname(obj){
  obj.name="aini";
  obj = new Object();
  obj.name = "dilnur";
  console.log(obj.name);
}
var person = new Object();
setname(person); //dilnur
console.log(person.name); //aini
```

8-4 函数回调和匿名函数

8-4-1 函数回调

```
// 回调：将一个函数对象a 传递给另一个函数对象 b，让后者在适当的时候执行 a。这就是所谓的“回调”

// 适当的时候:当这个外部函数在一定条件下就会调用参数指定的函数，此函数就是回调函数。如：
```

```
function funcA(){
    console.log("this is func A");
}
function funcB(func){
    func();
}
funcB(funcA); // this is a func A
```

```
<input type="text" id="score">
<input type="button" value="检测" onclick="test()"><br>
<p id="myp"></p>
<script>
function test(){
    var myp=document.getElementById("myp");
    var str;
    var score = document.getElementById("score").value;
    if(score<0) str="分数不能为负"
    else if(score == 0) str="此考生没有参加考试"
    else if(score>0 && score<60) str="不及格"
    else if(score>=60 && score <70) str="及格"
    else if(score>=70 && score<90) str="良好"
    else if(score>=90 && score<=100) str="优秀"
    else str="无效分数";
    myp.innerHTML=str;
}
```

也可以这么样写

```
var str;
function fun(score){
    if(score<0) str="分数不能为负"
    else if(score == 0) str="此考生没有参加考试"
    else if(score>0 && score<60) str="不及格"
    else if(score>=60 && score <70) str="及格"
    else if(score>=70 && score<90) str="良好"
    else if(score>=90 && score<=100) str="优秀"
    else str="无效分数";
}
function test(){
    var myp=document.getElementById("myp");
    var score = document.getElementById("score").value;
    fun(score);
    myp.innerHTML=str;}
```

继续改进，用回调函数

```
<body>
<input type="text" id="score">
<input type="button" value="检测" onclick="test()"><br>
<p id="myp"></p>
<script>
var str;
function fun(score,callback){
    if(score<0 || score>100) str="分数无效"
    else if(score == 0) str="此考生没有参加考试"
```



```

    else callback();
  }
function test(){
  var myp=document.getElementById("myp");
  var score = document.getElementById("score").value;
  fun(score,function(){ //匿名函数
    if(score>0 && score<60) str="不及格"
    else if(score>=60 && score <70) str="及格"
    else if(score>=70 && score<90) str="良好"
    else if(score>=90 && score<=100) str="优秀"
  });
  myp.innerHTML=str;
}

```

数组排序

```

function sortArr(arr,fun){
  if(!Array.isArray(arr) || !(fun instanceof Function)){
    throw new Error("参数类型不准确");
  }else{
    for(n in arr){
      for(m in arr){
        if(fun(arr[n],arr[m])){
          var tmp=arr[n];
          arr[n]=arr[m];
          arr[m]=tmp;
        }
      }
    }
  }
}
function compare(num1,num2){
  return num1>num2;
}
try{
  var arr=[45,12,68,95,115,65,32,25,12,78,35];
  sortArr(arr,compare);
  console.log(arr)
}catch(e){
  console.log(e);
}

```

8-4-2 匿名函数

// 匿名函数即为没有名字的函数，也称为拉姆达函数；匿名函数功能强大，用途很广

// 一般将匿名函数作为参数传入另一个函数，或者从一个函数中返回另一个函数，这是一种极为有用的技术

// 函数也可以作为值来使用；也就是说，可以将函数赋值给变量、存储在对象的属性或数组元素中，也可以当作一个参数进行传递

```

function func(x){return x*x}
var s = func;
console.log(func(4));
console.log(s(4));

```

同样可以将函数赋值给对象的属性，此时应该称为方法

```
var o = {  
  name: "aini",  
  say: function(x) {  
    return x*x;  
  }  
}  
console.log(o.say(5));
```

或者

```
var o = {}  
function say(x) {  
  return x*x;  
};  
o.say = say;  
console.log(o.say(5));
```

或者

这里用的就是匿名函数

```
var o = {}  
o.say = function(x) { return x*x; };  
console.log(o.say(5));
```

赋值给数组元素，此时的函数可以不需要名字，即是匿名函数

```
var arr = [function(x) { return x*x; }, 18];  
console.log(arr[0](arr[1]));
```

函数作为参数传递

```
function myFun(someFun, someArg) {  
  return someFun(someArg)  
}  
function add(num) {  
  return num += 10;  
}  
var result = myFun(add, 20);  
console.log(result);
```

还可以这样用

```
//定义一些简单函数
function add(x,y){return x+y};
function subtract(x,y){return x-y};
function multiply(x,y){return x*y};
function devide(x,y){return x/y};
function operate(operator,num1,num2){
    return operator(num1,num2);
}
var result = operate(add,operate(multiply,4,5),operate(add,2,3));
console.log(result);
```

第二种用法

```
// 使用函数直接量，定义在一个对象直接量中
var operators = {
    add: function(x,y){return x + y;},
    subtract: function(x,y){return x - y;},
    multiply: function(x,y){return x * y;},
    divide: function(x,y){return x / y;},
    pow: Math.pow // 使用内置的Math对象的pow方法
};
// operate函数接受一个名字作为运算符，在operators对象中查找这个运算符
// 然后将它作用于所提供的操作数
function operate(operation, num1, num2){
    if(typeof operators[operation] === "function")
        return operators[operation](num1, num2);
    else throw "unknown operator";
}
// 调用并计算
var result = operate("add", "hello", operate("add", " ", "wangwei"));
console.log(result); // hello wangwei
result = operate("pow", 10,2);
console.log(result); // 100
```

// 当函数作为值的一个典型的应用，就是在Array.sort()方法中使用，该方法用来对数组元素进行排序；因为排序的规则有很多，如：基于数值大小、字母表顺序、日期大小、从小到大等，所以sort()方法接受一个自定义函数作为参数，该自定义函数用来处理具体的排序操作，其原理是：对于任意两个值都返回一个值，以指定它们在排序后的数组中的先后顺序；如：

```
function compare(x,y){
    return x - y;
}
var arr = [1,88,3,5,12,18,67];
console.log(arr.sort(compare));
```

9，数组

// 数组是值的有序集合；每个值叫做一个元素，而每个元素在数组中都有一个位置，以数字表示，称为索引或下标；

// JavaScript数组是无类型的，数组元素可以是任意类型；即使同一个数组内，不同的元素也可以有不同的类型

// JS数组的索引是基于零的32位整数，第一个元素的索引为0；

// ES数组是动态的，即它的大小是可以动态调整的，可以随着数据的添加与删除会自动增长或缩减

9-1 创建数组

9-1-1 第一种

使用构造函数Array()创建

```
var arr = new Array(); // 空数组
var arr = new Array(3); // 指定长度
var arr = new Array("wangwei","wujing","jingguo"); //显示指定元素
```

// 1.如果预先知道数组要保存的项目数量，可以为构造函数传递该数量，这个数量实际上就是该数组length属性（实际上是预分配了一个数组空间，但数组中没有存储值，甚至数组的索引都未定义）
// 2.直接传递项，即参数就是数组的元素
// 3.如果需要创建只有一个值的数组，并且这个值是数字，只能把该数字使用字符串的形式；当这一个值如果是非整数时，会抛出异常：

```
var arr = new Array(18.5); //抛出异常
```

也可以省略new操作符，实际上是进行了数据类型转换，如

```
var arr = Array(); // 空数组
console.log(arr);
var colors = Array(3); // 指定长度
console.log(colors);
var names = Array("wangwei","wujing","jingguo"); //显示指定元素
console.log(names);
```

9-1-2 第二种

使用数组字面量（直接量）创建

```
var empty = [];
var num = [2,3,4,5];
```

// 这是创建数组最简单的方法，即使用方括号[]将数组元素用逗号分隔开
// 当为[]空数组时，与new Array()等价
// 数组直接量中的值不一定是常量，也可以是任意的表达式

```
var year = 2022;
var years = [year,year+1,year+2];
console.log(years);
```

还可以包含对象或其他数组

```
var objArr = [{name:"wangwei",age:18},[1,2],[3,{x:4,y:5}]];
```

如果省略数组中某个值，省略的元素的值都是undefined

```
var count = [1,,3];
console.log(count[1]); // undefined;
```

允许有可选的结尾逗号，但会省略最后的元素

```
count = [1,2,]; // 2
console.log(count);
count = [, ,]; // 3
console.log(count);
```

► (2) [1, 2]

► (3) [empty × 3]

// 不建议使用，某些浏览器会解析为3项数组，因为它们对数组的实现不一致
// 总结：在实际场景中，使用数组字面量要比构造函数简单、灵活

9-2 数组的内存分布

// 数组在内存中用一串连续的区域来存放一些值；在 JavaScript 中，数组是哈希映射，在内存中不是连续的，它可以通过多种数据结构实现，其中一种是链表，因此，如果使用大数组，需要的时间很长；

// 在ES6中引入了类型化数组，JavaScript 引擎已经在为同种数据类型的数组分配连续的存储空间了，如 ArrayBuffer，其使用的就是一块连续的内存空间



9-3 数组的读写

要读取或设置数组元素的值时，使用“[]”运算符，并提供相应的基于0的数字索引下标

```
var citys = new Array("蚌埠", "宿州", "南京");
var c1 = citys[0]; // 读取第一个元素
citys[0] = "怀远"; // 写第一个元素
```

在越界访问不存在的数据元素时，会返回undefined

如果设置某个值的索引超过了数组现有的项数，数组会自动增加到该索引值加1的长度

```
var colors=["red","blue"];
colors[2]="green";    // 明确添加了一个新元素
var i = 3;
colors[i] = "purple";
colors[i+1] = "yellow";
```

注意：跳过某个索引添加值时，跳过的自动变成empty

```
var colors=["red","blue"];
colors[3] = "purple";
console.log(colors);
console.log(colors.length);    // 4
```

```
► (4) ['red', 'blue', empty, 'purple']
```

```
4
```

// 数组的最大索引为4294967294(2的32次方-2)

// 数组是对象的特殊形式；使用方括号访问数组元素就像用方括号访问对象的属性一样；
// 其会将指定的数字索引值转换成字符串，并将其作为属性名来使用

```
var n = [];  
n[0] = "one";   n["0"]="one"  
n[1] = "two";   n["1"]="two"    等同  
console.log(n[1]);  
var p = {};  
p[0] = "one";   // 将0转换为"0"  
p[1] = "two";  
console.log(p[1]);  
var o = {0:"wangwei",1:"wujing",2:"huiyuan"};  
o[1] = "lili";  
console.log(o[1]);    // lili
```

// 所有的数组都是对象，所以可为其创建任意名字的属性，此时这个任意名字如果不是非负整数，它就只能当作常规的对象属性，而非数组的索引；

```
var arr = [1,2];  
arr[-2] = "-2的值";  
arr["name"] = "aini";  
console.log(arr);
```

```
▼ (2) [1, 2, -2: '-2的值', name: 'aini'] ⓘ  
  0: 1  
  1: 2  
 -2: "-2的值"  
 name: "aini"  
 length: 2  
 ► [[Prototype]]: Array(0)
```

// 如果凑巧使用了是非负整数的字符串，它就当作数组索引，而非对象属性；当使用一个浮点数和一个整数相等时情况也是一样的

```
var arr = [1,2];  
arr["2"] = "333";    把二当做数组的索引，因为字符串2能转换成数字
```

```
var arr = [1,2];  
arr[-1.23] = true;    // 创建了一个名为“-1.23”的属性  
arr["1000"] = 18;     // 数组的第1001个元素  
arr[1.000] = 10;      // 等同 arr[1]  
console.log(arr)
```

// 事实上，数组索引仅仅是对象属性名的一个特殊类型，这意味着ES数组没有“越界”错误的概念，即试图查询任何对象中不存在的属性时，不会报错，只是得到undefined值

// 所有的索引都是属性名，但只有在2的32次方之内的整数属性名才是索引；超出的索引只能是属性名（所以超出4294967295的都属属性，数组索引最大长度为4294967295）

```
arr[4294967294] = "a";  
arr[4294967294] = "a";  
console.log(arr);  
console.log(arr.length);  
arr[4294967295] = "b";  
console.log(arr);  
console.log(arr.length);  
arr[4294967296] = "c";  
console.log(arr);  
console.log(arr.length);
```



通常，数组的实现是经过优化的，用数字索引来访问数组元素一般来说比访问常规的对象属性要快很多，所以尽量使用数组传递数据

9-4 length 属性

数组对象的length（长度）属性返回或指定数组元素的个数

```
var colors=["red","blue","green","skey","gray"];
console.log(colors.length);
colors.length = 3;
console.log(colors.length);
console.log(colors);
```

// 说明：即多于length的元素被删除了

```
var names=["wangwei"];
names.length = 4;
console.log(names);    // 添加了三个空的向;
```

// length属性设置为大于数组项数的值，则新增的每一项都会取得undefined值；（从本质上讲，并不会向数组中添加新的元素，它只是在数组尾部创建一个空的区域，其实就是后面要讲的稀疏数组）

利用length属性可以方便的在数组末尾添加新项

```
var colors=["red","blue","green"];
colors[colors.length]="black";
colors[colors.length]="brown";
console.log(colors);
```

// 数组的最大索引为4294967294(2的32次方-2)，即可以包含4292967295个元素，当索引小于此值并且不为非负时，数组会自动更新其length属性；超出这个范围，length不会更新

在ES中，可以用Object.defineProperty()让数组的length属性变成只读的

9-5 稀疏数组

Sparse arrays 稀疏数组：ES中的数组是稀疏的，即数组的元素的索引不一定要连续，它们之间可以有空缺

```
var arr = new Array(5);    // 数组没有元素，但length是5
arr = [];    // 空数组，length为0
arr = [1,,4];    //
arr[1000] = 0;    // 赋值添加了一个元素，length为1001
arr = [1,2,3,4,5];
delete arr[2];    // 使用delete也能生成稀疏数组
console.log(arr);
```

如果数组是稀疏数组，length值大于元素的个数，否则，该属性就是数组元素的个数；（即稀疏数组的length长度与个数是不一致的）

遍历时会跳过这些空隙

```
var a1 = [1,,4];
for(i in a1)
    console.log(i);
```


循环遍历时都能打印出来，空数组就打印undefined；

9-6 稠密（密集）数组

与稀疏相对应，则为密集数组，即元素中不存在空隙，其实密集数组基本就是平时常见的正常数组

```
var spare = new Array(3);
var dense= Array.apply(null,spare);  可以产生值为undefined
console.log(spare);                  的密集数组，跟稀疏数组
console.log(dense);                  有区别
console.log(spare[0]);
console.log(dense[0]);
```

区别

```
var arr = new Array(3);
arr[2] = "aini";
for (index in arr){
    console.log(index,arr[index]);  // 2 aini
}

var dense = Array.apply(null,Array(3));
dense[2] = "dilnur";
for (index in dense){
    console.log(index,dense[index]);  //都能打印
}
```

2 aini

0 undefined

1 undefined

2 dilnur

// 说明稀疏数字里面的undefined和密集数组里面的undefined是两回事儿

```
// 密集数组
console.time('one');
Array.apply(null,Array(1e5)).forEach(function(){});
console.timeEnd('one');
// 稀疏数组
console.time('two');
Array(1e5).forEach(function(){});
console.timeEnd('two');
// one: 8.589111328125ms
// two: 2.122314453125ms
```

// 稀疏数组速度比密集数组处理速度快

// 在实际应用中，绝大部分的es数组都不是稀疏数组；如果使用了稀疏数组，可以像处理非稀疏数组一样处理，只不过它们包含一些undefined值

// 稀疏的数组通常在实现上比稠密的数组更慢、内存利用率更高，在这样的数组中查找元素的时间与常规元素属性的查找时间一样长；另外，在通常的情况下，我们想要直接声明一个数组并赋予其一些特定的初始值，并且为了避免问题，通常是希望申明为密集数组的

压缩稀疏数组：可以通过filter()方法压缩其中的空隙，因为filter会跳过空隙，返回密集的数组

```
var a1 = [1,,,4];
var a2 = a1.filter(function(x){return true;});
for(i in a2)
    console.log(i);
```

9-7 数组方法

ES在Array.prototype中定义了很多操作数组的方法

```
console.log(Array.prototype);
```

```
► [constructor: f, at: f, concat: f, copyWithin: f, fill: f, ...]
```

9-7-1 数组转字符串

1-1 toString(), valueOf()

所有对象都具有toLocaleString()、toString()及valueOf()方法

1. valueOf() // 返回的是数组本身
2. toString() // 方法将数组表示为字符串，各个元素按顺序排列组合以逗号分隔的字符串返回。（通过对每项调用toString()方法,然后用逗号把它们连接在一起构成的）

```
var arr = [1,2,3,5];
console.log(arr);
console.log(arr.valueOf());
console.log(arr.toString());
```

```
► (4) [1, 2, 3, 5]
```

```
► (4) [1, 2, 3, 5]
```

```
1,2,3,5
```

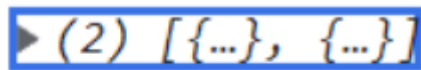
```
var colors=["red","blue",["green","black"]];
console.log(colors.valueOf()); // ["red", "blue", Array(2)]
console.log(colors.toString()); // red,blue,green,blank
console.log(colors);           // ["red", "blue", Array(2)]
```

// 以上`console.log()`换成`alert`，由于`alert`要接收字符串参数，所以它会在后台调用`toString()`，由此会得到与直接调用 `toString()` 方法相同的结果

// 这里的`toString()`方法与不使用参数的`join()`方法返回的字符串是一样的

// `toLocaleString()`是`toString()`方法的本地化版本，一般情况下会返回与`toString()`和`valueOf()`方法相同的值，但也不总是如此（其会使用本地化的分隔符）；当调用数组的`toLocaleString()`方法时，后台会为每一项调用 `toLocaleString()`方法，而不是`toString()`方法

```
var p1 = {
  toLocaleString:function(){return "aini"},
  toString:function(){return "ai"}
}
p2 = {
  toLocaleString:function(){return "dilnur"},
  toString:function(){return "norah"}
}
```



ai,norah

aini,dilnur

```
var person = [p1,p2];
console.log(person);
console.log(person.toString());
console.log(person.toLocaleString());
```

1-2 join()

2, `join()`方法

// 将数组中所有元素使用不同的分隔符转化为字符串，分隔符号由用户指定

```
var colors = ["red","green","blue","balck"];
console.log(colors.join());
console.log(colors.join(","));
console.log(colors.join("-"));
console.log(colors.join("|"));
console.log(colors.join(" "));
var arrStr = new Array(10);
console.log(arrStr.join("-"));
```

```
red,green,blue,balck
red,green,blue,balck
red-green-blue-balck
red|green|blue|balck
red green blue balck
```

```
-----
```

如果不指定分隔符，或者传入`undefined`，则使用默认的逗号分隔

```
var colors = ["red","green","blue","balck"];
console.log(colors.join(undefined));
console.log(colors.join("undefined"));
console.log(colors.join(null));
console.log(colors.join("null"));
```

```
red,green,blue,balck
```

```
redundefinedgreenundefinedblueunde
```

```
rednullgreennullbluenullbalck
```

```
rednullgreennullbluenullbalck
```

// 注：“`undefined`”，“`null`”是字符串；如果直接是`undefined`则会用逗号分割；`null`的话转换成“`null`”，再用“`null`”分割

// `join()`方法是`split()`方法的逆向操作，后者是将字符串分割成若干块来创建一个数组。

// 注：如果数组中的某一项的值是`null`或者`undefined`，那么该值在以上所有方法中返回空字符串。

字符串`split()`方法：把字符串转换为数组

```
var str = "red,blue,green";    // 字符串
var aStr=str.split(",");      // 数组
var aStr=str.split("");       // 单个字符数组
```

关于字符串`split`方法的几个注意点：

// （1）如果指定错误的分隔符，则把整个字符串当做数组的一个 元素返回

```
var str = "red,blue,green";
var aStr=str.split("-");
console.log(aStr);
```

► ['red,blue,green']

// (2) 如果什么也不传，就会把字符串里面的每一个字符拿出来，当做数组的一个元素

```
var str = "red,blue,green";
var aStr=str.split("");
console.log(aStr);
```

► (14) ['r', 'e', 'd', ',', 'b', 'l', 'u', 'e', ',', 'g', 'r', 'e', 'e', 'n']

// (3) 如果把空字符串声明为分隔符，split会返回字符串的每个字条符；一般用于需要逐个处理字符。

9-7-2 队列方法

// (1) 栈和队列是一种限制了插入和删除数据项操作的数据结构；
// (2) 栈数据结构的访问规则是LIFO（先进后出）
// (3) 而队列数据结构的访问规则是FIFO（First-In-First-Out，先进先出）
// (4) 栈最近添加的项是最先删除的项；栈中的插入和删除操作都只发生在一个位置，即栈顶部；把一个项添加到栈中，叫做推入栈；从栈中删除一项，叫做从栈中弹出；
// (5) 队列在列表的末端添加项，从列表的前端移除项；可以使用push()方法向数组末尾添加项，使用shift()方法从数组前端移除项，这两个方法模拟队列；

2-1 push()

在数组尾部添加一个或多个元素；并返回更新后数组长度会修改原数组

```
var colors = new Array();
var count = colors.push("red","green");
console.log(colors);
console.log(count); // 2
count = colors.push("black","blue");
console.log(colors);
console.log(count); //4

// push()方法实际上同手工添加数据一样。
// 注：pop()不带值也不会报错，只返回数组长度
```

2-2 pop()

与push()方法相反，其会删除数组末尾的一个元素，并将其返回；会修改原数组

```
var colors=new Array();
colors.push("red","green","blue");
var item = colors.pop();
console.log(item); // blue
console.log(colors);
```

2-3 unshift()

// (1) 在数组顶端插入元素，一次可以插入单个或多个元素，所有元素按顺序插入，操作完成后返回新数组的长度；会修改原数组；其与push()类似
// (2) unshift()方法将引发所有下标的改动
// (3) 在一次性插入多个参数时，插入的元素的顺序和它们在参数列表中的顺序一致
// (4) 如果不计较元素插入的位置，则推荐使用push方法，因为，unshift()方法可能会影响依靠下标才能准确进行的计算。

```
var colors = new Array();
colors.push("red", "green", "blue");
var newLen = colors.unshift("black", "gray");
newLen = colors.unshift("r", ["g", "b"]);
console.log(newLen);
console.log(colors);
```

7

► (7) ['r', Array(2), 'black', 'gray', 'red', 'green', 'blue']

2-4 shift()

// shift()移除数组的第一个元素并将其返回；该方法执行后数组剩下的元素向前移动，下标索引号重新调整从0开始按顺序赋予所有元素；会修改原数据；其与pop方法类似；

```
var colors = new Array();
colors.push("red", "green", "blue");
var shiftItem = colors.shift();
console.log(shiftItem);
var item = colors.shift();
console.log(item);
console.log(colors);
```

red

green

► ['blue']

调用push()方法和shift(),可以使Array对象具有队列一样的行为（先进先出）

```
var colors=new Array();
var count = colors.push("red", "green", "blue");
count = colors.push("black");
var item = colors.shift();
console.log(colors);
```

使用unshift()和pop()方法，可以从相反的方向来模拟队列，即在数组的前端添加项，在末端移除项；

```
var colors=new Array();
var count = colors.push("red","green","blue");
count = colors.unshift("black");
var item = colors.pop();
console.log(colors);
```

9-7-3 数组排序

排序算法：选择排序和冒泡排序；

3-1 选择排序

```
var arr = [78,15,69,3,45,78,95,62,35,98,789,125,98,5,58];
for (var i=0; i<arr.length;i++){
    var element
    for(var j=i+1; j<arr.length;j++){
        if(arr[j]>arr[i]){
            element = arr[i];
            arr[i] = arr[j];
            arr[j]=element;
        }
    }
};
console.log(arr);
```

3-2 冒泡排序

```
var arr = [78,15,69,3,45,74,785,125,654,12,35,64,05,45,78,126];
function compare(num1,num2){
    return num1<num2?true:false;
}
function sort(arr){
    var len = arr.length;
    for (var i=0; i<len;i++){
        var element
        for(var j=0; j<len-i;j++){
            if(compare(arr[j],arr[j+1])){
                element = arr[j];
                arr[j] = arr[j+1];
                arr[j+1]=element;
            }
        }
    }
    return arr;
};
console.log(sort(arr));
```

3-3 reverse()

reverse()方法将一个Array对象中所有元素的次序反转，会修改原数组

```
var arr = ["aini","dilnur","xinjiang","shanghai"];
console.log(arr.reverse());
```

► (4) ['shanghai', 'xinjiang', 'dilnur', 'aini']

3-4 sort()

sort()方法:

- (1) sort()方法: // 对数组元素进行排序后并返回; 默认将一个数组中的所有元素进行升序排序: 会改变原数组
- (2) // 如果数组元素中undefined或null, 它们会被排列在数组的尾部;

```
var values = [0,1,5,10,15];
values.sort();
console.log(values);
var arr = new Array("banana",undefined,"cherry",null,"apple");    按照Unicode编码大小
arr.sort(); 进行排序;
console.log(arr);
```

► (5) [0, 1, 10, 15, 5]

► (5) ['apple', 'banana', 'cherry', null, undefined]

实际上, sort()方法是把元素转换成字符串进行排序的

- // (1) sort()方法可以接收一个比较函数, 以确定一个排序的规则
- // (2) 比较函数接收两个参数, 如果第一个参数位于第二个之前则返回一个负数
- // (3) 如果两个参数相等则返回0
- // (4) 如果第一个参数位于第二个之后则返回一个正数

```
function compare(value1,value2){
    if(value1<value2){
        return -1;
    }else if(value1>value2){
        return 1;
    }else{
        return 0;
    }
}
var values = [0,1,5,10,15];
values.sort(compare);
```

► (5) [0, 1, 5, 10, 15]

// 对于数值类型或者valueOf()方法会返回数值类型的对象类型, 可以使用一个更简单的比较函数, 这个函数只要用两个值互减即可

```
function compare(value1,value2){
    return value1 - value2;
}
```

注: 参数必须是数字才能这样用;

// 说明：由于该函数通过返回一个小于零、等于零或大于零的值来影响排序结果，因此减法操作就可以适当地处理所有这些情况。

3-5 随机排序

```
var arr = [1,2,3,4,5,6,8,9];
var randomArr = arr.sort(
    function(){
        return Math.random()-0.5;
    }
);
console.log(arr);
```

对于字符串排序，主要是大小写的问题

```
var str = ["dilnur","AINI","aini","DILNUR"];
console.log(str.sort());
str.sort(function(s,t){
    var s = s.toLowerCase();
    var t = t.toLowerCase();
    if(s < t) return -1;
    if(s > t) return 1;
    return 0;
});
console.log(str);
```

► (4) ['AINI', 'DILNUR', 'aini', 'dilnur']

► (4) ['AINI', 'aini', 'DILNUR', 'dilnur']

另外，有时需要按字符串长度进行排序，此时还应该同时判断字符是否为双字节

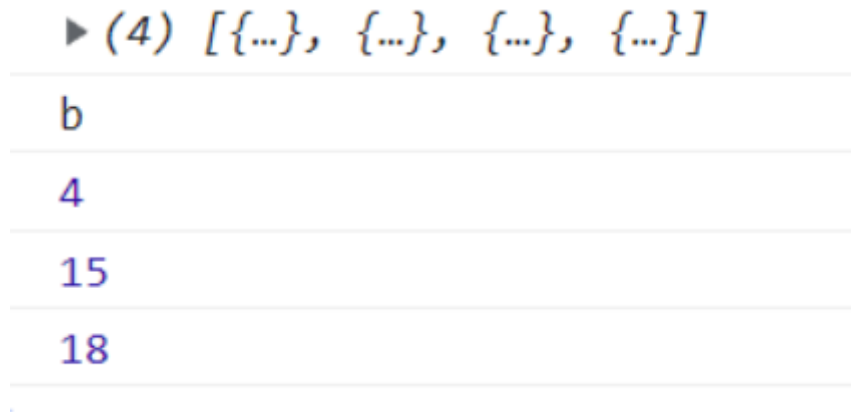
```
var arr = ["阿卜杜艾尼a","aini-艾尼","dilnur_地","aini艾尼","dilnur地理"];
function getDword(str){
    var len = str.length;
    for(var i=0;i<len;i++){
        if(str.charCodeAt(i)>255){
            len++;
        }
    }
    return len;
}
arr.sort(function(s1,s2){
    return getDword(s1)-getDword(s2);
})
console.log(arr);

// 汉字的长度就按双字节算；
```

3-6 对象排序,

比较的主要是对象的valueOf()值:

```
var arr = [o,  
  {valueOf:function(){return 15}},  
  {valueOf:function(){return 18}},  
  {valueOf:function(){return 4}}  
];  
arr.sort(function(o1,o2){return o1 - o2});  
console.log(arr);  
for(var i in arr)  
  console.log(arr[i].valueOf());
```



```
► (4) [{...}, {...}, {...}, {...}]  
b  
4  
15  
18
```

或者按指定的对象属性进行排序

```
var wang = {name:"wangwei",sex:"男",age:18};  
var wu = {name:"wujing",sex:"女",age:28};  
var guo = {name:"jianguo",sex:"男",age: 25};  
var arr = [wang,wu,guo];  
var pArr = arr.sort(function(p1,p2){  
  return p1.age - p2.age;  
});  
console.log(pArr);
```

9-7-4 数组操作

4-1 concat()

concat()方法:

```
// (1) 基于当前数组, 将多个元素连接一起成为新的数组并返回  
// (2) 新数组中的元素按连接时的顺序排列
```

```
// (具体来说, 这个方法会先创建当前数组一个副本, 然后将接收到的参数添加到这个副本的末尾, 最后返回新构建的数组。当需要合并多个数组时, 此方法比较方便)
```

```
var arr = [1,2];
arr1 = arr.concat();
arr2 = arr.concat(3,4);
arr3 = arr.concat([3,4,],[ "aini"]); // 3,4,aini拿出来放到源数组里面; 扁平化
arr4 = arr.concat([3,4,[5,6,7]]);
console.log(arr);
console.log(arr1);
console.log(arr2);
console.log(arr3);
console.log(arr4);
```

// (3) **concat()**不会递归扁平化数组的数组，也不会修改原来的数组
// (4) 当没有给**concat()**方法传递参数的情况下，它只是复制当前数组并返回副本

4-2 slice()

2, **slice()**方法:

// (1) 能够基于当前数组中的一个或多个项创建一个新数组
// (2) 其接受一或两个参数，即要返回项的起始和结束位置;
// (3) 在只有**start**一个参数的情况下，**slice()**方法返回从该位置开始到当前数组末尾的所有项;
// (4) 如果两个参数，该方法返回起始和结束位置之间的项，但不包括结束位置的项;
// (5) 其不会影响原有数组;

```
var colors = ["red","green","blue","yellow","purple"];
var a = colors.slice(2);
var b = colors.slice(1,4);
console.log(a);
console.log(b);
var c = colors.slice(1,-2); //从索引1 取到 倒数第二个;
var d = colors.slice(-4,-2); // 倒四 取到 倒二;
console.log(c);
console.log(d);
```

只能从左往右截，像**colors.slice(3,1)** 就会返回空数组;

4-3 splice()

3, **splice()**方法: (改变原数组)

// (1) 可以删除、替换或插入数组元素，是最强大的数组方法，有很多种用法;
// (2) 具体作用是，从一个数组中移除一个或多个元素; 剩下的元素组成一个数组，移除的元素组成另一个新数组并返回;
// (3) 同时，原数组可以在移除的开始位置处顺带插入一个或多个新元素，达到修改替换数组元素的目的;

start: // 必选项，表示从数组中剪切的起始位置下标索引号。
deleteCount: // 必选项，表示将从数组中切取的元素的个数。
item: 可选项，// 表示切取时插入原数组切入点开始处的一个或多个元素;

3-1 删除

```
var colors = ["red","green","blue","yellow","purple"];
var a = colors.splice(1,3);
console.log(colors); // 返回原数组删除某些元素以后的数组
console.log(a); // 返回删除的项
```

- 1) 删除: `//` 可以删除任意数量的项, 只需要指定2个参数, 如: `splice(0,2);` `//` 删除数组中前两项; 如果只指定1个参数, 则删除从该位置到结尾的元素;
- 2) 插入: `//` 可以向指定位置插入任意数量的项, 需要指定3个参数, 其中第二个参数为0; 如果要插入多个项, 可再传入第4第5等任意多个项, 如: `splice(2,0,"red","green");`
- 3) 替换: `//` 可以向指定位置删除任意数量的项, 同时插入任意数量的项, 需要指定至少3个参数; 插入的项数不必与删除的项数相等, 如: `splice(2,1,"red","green");`

`splice()`方法始终会返回一个数组, 该数组中包含从原始数组中删除的项 (如果没有删除任何项, 则返回一个空数组)

3-2 插入

没必要把返回值赋给新变量

```
var colors = ["red","green","blue","yellow","purple"];
var a = colors.splice(1,0,"grey","skey");
console.log(colors); // 返回插入以后的新的数组;
console.log(a); // 返回空数组
```

3-3 替换

替换的数量跟删除的数量不一定一致

```
var colors = ["red","green","blue","yellow","purple"];
var a = colors.splice(1,2,"grey","skey");
console.log(colors); // 返回替换以后的新的数组;
console.log(a); // 返回数组删除的项
```

注: `splice`会改变源数组, 但是`splice`本身只返回删除的项

```
var colors = ["red","green","blue","yellow","purple"];
var a = colors.splice(1,2,["wangwei"],["aini"]);
console.log(colors);
console.log(a);
```

► (5) `['red', Array(1), Array(1), 'yellow', 'purple']`

► (2) `['green', 'blue']`

不会扁平化, 区别于`concat()`方法

第一个参数可以接受负数, 如果为负数, 则指定的位置是数组长度加上该负值, 或者从后向前定位

```
var colors = ["red","green","blue","yellow","purple"];
var a = colors.splice(-3,2,"aini","dilnur");
console.log(colors);
console.log(a);
```

► (5) `['red', 'green', 'aini', 'dilnur', 'purple']`

► (2) `['blue', 'yellow']`

```
// (倒数第三个位置往后数两个)
// 在实际的场景中，主要还是向数组的中部插入项；
// delete运算符删除数组元素：
// 通常使用delete运算符删除一个指定的元素（与删除对象属性一样）
```

```
var colors=["red","green","blue","yellow","purple"];
console.log(colors.length);
delete colors[1];
console.log(colors);
console.log(colors.length);
```

5

► (5) ['red', empty, 'blue', 'yellow', 'purple']

5

```
// delete删除不会修改数组的length属性；如果从数组中删除一个元素，该数组就变成稀疏数组；
// 此删除并没有真正删除数组元素，仅删除元素内容；（此删除与为其赋undefined值是类似的，但有一些微妙的区别）
// 删除数组元素有多种方式，如设置length属性小于数据长度，会删除数组尾部的元素；
```

9-7-5 数组位置操作

5-1 indexOf(), lastIndexOf()

```
// 1.两者会搜索整个数组中具有给定值的元素，并返回要查找的元素在数组中的第一个索引位置，或者在没找到返回-1；
// 2.这两个方法都是接收两个参数：要查找的项和（可选的）表示查找起点位置的索引；其中，indexOf()方法从数组的开头（位置为0）开始向后查找，lastIndexOf方向则从数组的末尾开始向前查找。
```

```
// 注：在查找时，使用是全等操作符
```

```
var numbers = [1,2,3,4,5,4,3,2,1];
console.log(numbers.indexOf(4)); // 3
console.log(numbers.lastIndexOf(4)); // 5
console.log(numbers.indexOf(4,4)); // 5 从索引4位置往后找“4”这个元素；
console.log(numbers.lastIndexOf(4,4)) // 3 从索引4位置往前找“4”这个元素；
```

```
var p ={name:"wangwei"};
var p1 = p;
var people = [{name:"wangwei"},{name:"aini"}];
var people = [p1,{name:"aini"}];
var morePeople = [p];
console.log(people.indexOf(p)); // 0
console.log(morePeople.indexOf(p)); // 0
```

```
// 4.第二个参数也可以是负数，即为数组长度加上该负数的值为查找的索引开始位置，或者从后往前定位
```

```
var numbers = [1,2,3,4,5,4,3,2,1];
console.log(numbers.indexOf(4,-5)); // 5
```

封装一个函数，查找所有给定值的索引，并返回一个包含匹配索引的数组

```
var numbers = [1,2,3,4,5,4,3,2,1,4,2,7,9,4,2,3,5];
function findIndex(arr,index){
    var len = arr.length;
    var result = [];
    var pos = 0;
    while(pos<len){
        pos = arr.indexOf(index,pos);
        if(pos == -1) break;
        result.push(pos);
        pos++;
    }
    return result;
}
console.log(findIndex(numbers,4));
```

字符串也有indexOf()和lastIndexOf()方法，它们和数组的方法功能类似

对于数组这些方法，也可以重写，本质上就是修改其prototype对象的属性方法，如

```
var arr = [1,2,3];
arr.push(4,5);
console.log(arr);
Array.prototype.push = function(){
    for(var i=0; i<arguments.length; i++)
        this[this.length] = arguments[i] * arguments[i];
}
arr.push(6,7);
console.log(arr);
```

► (5) [1, 2, 3, 4, 5]

► (7) [1, 2, 3, 4, 5, 36, 49]

9-7-6 遍历数组

// 所谓的数组的遍历就是通过索引挨个取出数组元素；遍历目的是为了读取所有元素或者处理所有元素；使用for循环是遍历数组最常见的方法

```
var cities = ["beijing","nanjing","bengbu"];
for(var i=0;i<cities.length;i++) console.log(cities[i]);
for(var a in cities) console.log(a,cities[a]);
var o ={name:"aini",age:18,sex:true,hobby:"reading"};
var keys = Object.keys(o); // 把对象的属性放到数组里返回
console.log(keys);
var values = [];
for(var i=0,len=keys.length;i<len;i++){
    // values[i]=o[keys[i]]
    values.push(o[keys[i]]);
}
console.log(values);
```

```

beijing
nanjing
bengbu
0 beijing
1 nanjing
2 bengbu
▶ (4) ['name', 'age', 'sex', 'hobby']
▶ (4) ['aini', 18, true, 'reading']
.

```

// 在遍历稀疏数组，或者需要排除null、undefined和不存在的元素时，需要先检测：

```

var arr = [1,,3,null,5,undefined,7];
for(var i=0,len=arr.length;i<len;i++){
    console.log(arr[i]); // 可以把所有元素都打印出来;
}
console.log("-----");
for(var i in arr){
    console.log(arr[i]); // 会过滤掉empty元素
};
console.log("-----");
for(var i in arr){
    if(!arr[i]) continue; // 过滤掉null和undefined
    console.log(arr[i]);
};
console.log("-----");
for(var i in arr){
    if(arr[i] === undefined) continue; // 过滤掉undefined
    console.log(arr[i]);
};
console.log("-----");
for(var i in arr){
    if(arr[i] === null) continue; // 过滤掉null
    console.log(arr[i]);
};
console.log("-----");
for(var i=0,len=arr.length;i<len;i++){
    if(!(i in arr)){
        console.log(arr[i]); // 可以用for循环过滤空元素;
    }
} ;

```

// （注：空元素的索引用in操作符判断时返回false；用for循环时可以用来过滤掉稀疏数组里的空元素；）

// 但是，for/in能够枚举继承的属性名，如添加到Array.prototype中的方法；由于这个原因，在数组上不应该使用for/in循环，除非使用额外的检测方法来过滤不用的属性，如：

```

Array.prototype.company = "zeronetwork";
var arr = [1,,3,null,undefined,6];
arr.custom = "wangwei";
arr[-18.1]=18;
for(var i =0;i<arr.length;i++){
    console.log(i,arr[i]);
}

```

```

} //for循环只遍历元素；

//用 for in把所有可枚举的属性枚举出来：
for(var i in arr){
    console.log(i,arr[i]);
} //custom ,company,-18.1都打印出来；
//hasOwnProperty()方法过滤：
for(var i in arr){
    if(!arr.hasOwnProperty(i)){
        console.log(i,arr[i]); //只有 company zeronetwork;
    }
};
//跳过不是非负整数的i；
for(var i in arr){
    if(String(Math.floor(Math.abs(Number(i))))!= i) continue;
    console.log(i,arr[i]);
}

```

// （1）ES允许for/in循环以不同的顺序遍历对象的属性；
 // （2）通常数组元素的遍历实现是升序的，但不能保证一定是这样的；特别地，如果数组同时拥有对象属性和数组元素，返回的属性名很可能是按照创建的顺序而非数值的大小顺序；
 // （3）如何处理这个问题的实现各不相同，如果算法依赖于遍历的顺序，那么最好不要使用for/in而使用常规的for循环。

9-7-7 遍历数组的迭代方法

（1）// ECMAScript5为数组定义了5个迭代方法；
 （2）// 这些方法的第一个参数接收一个函数，并且对数组的每个元素调用一次该函数；
 （3）// 第二个参数是可选的：其是运行该函数的作用域对象（影响this的值）；
 （4）// 或者说：调用的函数被当作是第二个参数的方法；
 （5）// 如果是稀疏数组，对不存在的元素不调用传递的函数
 （6）// 这些方法不会修改原始数组；
 （7）// 参数函数使用三个参数：数组元素、元素索引和数组对象本身；通常，只需要第一个参数，可以忽略后两个参数；
 （7）// 参数函数可以修改原始数组；
 （8）// 根据使用的方法不同，这个函数执行后的返回值可能也会不同；

7-1 forEach()

遍历数组，为每个元素调用指定的函数；该函数使用三个参数：数组元素、元素索引和数组本身；此方法没有返回值，本质上与使用for循环迭代数组一样

```

var numbers = [1,2,3,4,5,4,3,2,1];
numbers.forEach(function(item,index,array){
    console.log(item,index,array[index]);
});

```

第一个参数：// 数组元素；
 第二个参数：// 数组索引；
 第三个元素：// 数组对象本身


```
// 为每个元素加1
var data = [1,2,3,4];
data.forEach(function(v,i,a){
    a[i] = v + 1; //会改变原数组元素
});
console.log(data);
```

如果只关心数组元素的值，可以只使用一个参数，额外的参数将被忽略

```
// 求和
var data = [1,2,3,4];
var sum = 0;
data.forEach(function(value){ //第一个参数数组元素，相当于遍历多有元素；
    sum += value; //不会改变原数组元素；
});
console.log(sum);
```

- (1) // `forEach()` 方法无法在所有元素都被传递给调用的函数之前终止遍历，即没有像 `for` 循环中使用的 `break` 语句；
- (2) // 如果要提前终止，必须把 `forEach()` 方法放在 `try` 块中，并能抛出一个异常：

```
function foreach(a,f,t){
    try{a.forEach(f, t);}
    catch(e){
        if(e === foreach.break) return;
        else throw e;
    }
}
foreach.break = new Error("StopIteration");
```

7-2 every(), some()

- (1) // 最相似的是 `every()` 和 `some()`，它们都是数组的逻辑判定：用于判定数组元素是否满足某个条件；
- (2) // 对 `every()`，传入的函数必须对每一项都返回 `true`，这个方法才返回 `true`；
- (3) // 而 `some()` 是只要传入的函数对数组中的某一项返回 `true`，就会返回 `true`，否则返回 `false`

```
var numbers = [1,2,3,4,5,4,3,2,1];
var everyResult = numbers.every(function(item,index,array){
    return (item > 2);
});
console.log(everyResult); // false
var someResult = numbers.some(function(item,index,array){
    return (item > 2);
});
console.log(someResult); // true
```

- (4) // 一旦 `every()` 和 `some()` 确定该返回什么值时，它们就会停止遍历数组元素；
- (5) // (`some()` 在判定函数第一次返回 `true` 后就返回 `true`，但如果判定函数一直返回 `false`，它将会遍历整个数组；`every()` 恰好相反，它在判定函数第一次返回 `false` 后就返回 `false`，但如果判定函数一直返回 `true`，它将会遍历整个数组)。
- (6) // 根据数学上的惯例，在空数组上调用时，`every()` 返回 `true`，`some()` 返回 `false`；

7-3 filter()方法

- (1) // 返回的是数组元素是调用的数组的一个子集;
- (2) // 回调函数是用来逻辑判定的, 该函数返回true或false; 如果返回的是true或真值, 则该函数处理的数组元素就被添加到返回的子集数组中;
- (3) // filter()方法会跳过稀疏数组中缺少的元素, 它的返回数组总是密集的

```
var numbers = [1,2,3,4,5,4,3,2,1];
var filterResult = numbers.filter(function(item,index,array){
    return item>2;
});
console.log(filterResult);
var evenResult = numbers.filter(function(value,index){
    return index % 2 == 0; // [1, 3, 5, 3, 1] index是索引
});
console.log(evenResult);
// 压缩稀疏数组
var sparse = [1,,,4];
var dense = sparse.filter(function(){
    return true; //过滤稀疏元素
});
console.log(dense);
// 压缩并删除undefined和null元素
var sparse = [1,null,3,undefined,,6];
var dense = sparse.filter(function(v){
    return v !== undefined && v !== null;
});
console.log(dense);
```

► (5) [3, 4, 5, 4, 3]

► (5) [1, 3, 5, 3, 1]

► (2) [1, 4]

► (3) [1, 3, 6]

(注: 该方法返回的是符合条件的数组元素;)

7-4 map()

- (1) // 将调用的数组的每个元素传递给回调函数, 并将调用的结果组成一个新数组返回;
- (2) // 其不会修改原始数组, 如果是稀疏数组, 返回的也是相同方式的稀疏数组, 即具有相同的长度、

相同的缺失元素;

```

var numbers = [1,2,3,4,5,4,3,2,1];
var mapResult = numbers.map(function(item,index,array){
    return item*2;
});
console.log(mapResult);
var a = [1,null,3,undefined,5];
var b = a.map(function(v){
    return v * v;
});
console.log(b);

```

► (9) [2, 4, 6, 8, 10, 8, 6, 4, 2]

► (5) [1, 0, 9, NaN, 25]

不做任何处理

```

var spare = [1,,4,null,undefined,NaN,6];
var dense = spare.map(function(v,i,a){

});
console.log(dense);

```

► (7) [undefined, empty, undefined, undefined, undefined, undefined, undefined]

返回所有数组元素

```

var dense = spare.map(function(v,i,a){
    return v;
});
console.log(dense);

```

► (7) [1, empty, 4, null, undefined, NaN, 6]

能处理的元素进项处理，空元素返回空元素

```

var spare = [1,,4,null,undefined,NaN,6];
var dense = spare.map(function(v,i,a){
    return v*v; //进行数据类型转换，由于null和undefined不能进行数据类型转换});
console.log(dense) //，所以都返回NaN

```

► (7) [1, empty, 16, 0, NaN, NaN, 36]

7-5 reduce()和reduceRight()

- (1) // `reduce()`和`reduceRight()`；这两个方法都会迭代数组的所有项，然后构建一个最终返回的值；
- (2) // 其中，`reduce()`方法从数组的第一项开始，逐个遍历到最后；而`reduceRight`则从数组的最后一项开始，向前遍历到第一项；
- (3) // 这两个方法都接收两个参数：调用的函数`callbackfn`和作为归并基础的初始值`initialValue`（可选的）；
- (4) // 这个函数接收4个参数：前一个值、当前值、项的索引和数组对象；其返回的任何值都会作为第一个参数自动传给下一项；第一次迭代发生在数组的第二项上，因此第一个参数是数组的第一项，第二个参数就是数组的第二项

使用`reduce()`方法可以执行求数组中所有值之和的操作，如

```
// 求和
var values = [1,2,3,4];
var sum = values.reduce(function(prev,cur,index,array){
    return prev + cur;
});
console.log(sum);    // 10
```

第一次遍历: `prev=1` `cur=2`;

第二次遍历: `prev=3` `cur=3`;

第三次遍历: `prev=6` `cur=4`;

第四次遍历: `prev=10`

第一次 `prev`=第一个元素;
第二次迭代开 `prev` 是前一次 `return` 的结果

说明: 第一次执行回调函数, `prev`与`cur`分别为数组的第1和第2项, 即 `prev`是1, `cur`是2; 第二次, `prev`是第一次执行的结果, 即3 (1+2的结果), `cur`是数组的第三项, 即是3, 以此类推;

`reduce()`参数函数的参数也可以只使用两个参数, 如:

```
// 求和
var values = [1,2,3,4];
var sum = values.reduce(function(prev,cur){
    return prev + cur;
});
console.log(sum);    // 10

// 求积
var product = values.reduce(function(prev,cur){
    return prev * cur;
});
console.log(product);    // 24

// 求最大值
var max = values.reduce(function(prev,cur){
    return prev > cur ? prev : cur;
});
console.log(max);    // 4
```

`reduce()`方法的第二个参数`initialValue`是回调函数的第一个参数`previousValue`的初始值; 如:

```
// 把以上所有示例添加第二个参数，如：  
var values = [1,2,3,4];  
var sum = values.reduce(function(prev,cur){  
    return prev + cur;  
},2); // 12
```

意思就是pre不是第一项，而是给pre赋一个值，cur从第一项开始迭代

说明，第一次调用时，prev 为 initialValue 值，即为 2，cur 为数组第一项，
即为 1；第二次调用，prev 为 3，cur 为 2，以此类推；

reduceRight()的作用类似，只不过方向相反，即按照数组索引从高到低（从右到左）处理数组；如：

```
var values = [1,2,3,4,5];  
var sum = values.reduceRight(function(prev,cur,index,array){  
    console.log(prev,cur);  
    return prev + cur; //21  
},6); //pre的初始值为6;  
console.log(sum); // 15  
// 求2^(3^4)，乘方操作的优先顺序是从右到左  
var a = [2,3,4];  
var big = a.reduceRight(function(accu, value){  
    console.log(value);  
    return Math.pow(value, accu);  
});  
console.log(big);
```

// 在空数组上，不带初始值参数的reduce()将导致类型异常；如果数组只有一个元素并且没有指定初始值，或者一个空数组并且指定了一个初始值，该方法只是简单的返回那个值而不会调用回调函数；

```
var a = [];  
// var b1 = a.reduce(function(x,y){return x + y}); //no initial value 会报错  
// var b2 = a.reduce(function(x,y){return x + x}); //也会报错  
var b3 = a.reduce(function(x,y){return x + y},3); //只返回初始值  
  
console.log(b3); // 3
```

(1) // 使用reduce()还是reduceRight()，主要取决于要从哪头开始遍历数组；除此之外，它们完全相同。

(2) // reduce()和reduceRight()是典型的函数式编程；有些地方，会把其回调函数称为化简函数，这个化简函数的作用就是用某种方法把两个值组合或化简为一个值，并返回化简后的值；如以上的示例，化简函数通过各种方法，组合了两个值

9-8 检测数组及类数组

9-8-1 检测数组

1. // 数组是具有自己特殊行为的对象，因此，确定某个对象是不是数组就很重要；
2. // `typeof`只能检测其是`object`的类型，而不能确定是否为`Array`类型；
3. // 对于一个页面或者一个全局作用域而言，可以使用`instanceof`操作就可以得到检测结果，如：

```
var arr = [];  
console.log(arr instanceof Array);  
console.log(arr instanceof Object);  
console.log(arr.constructor);  
console.log(arr.constructor === Array);
```

```
true
```

```
true
```

```
f Array() { [native code] }
```

```
true
```

注：实际上就是判断它的原型；

4. // 但`instanceof`与`constructor`有问题：如果网页中包含多个框架，那实际上就存在两个以上不同的全局执行环境，分别具有自己的全局对象，每个全局对象都有自己的`Array`构造函数，因此一个框架窗口中的对象不可能是另一个框架窗口的实例；
5. // 比如：如果从一个框架向另一个框架传入一个数组，那么传入的数组与在第二个框架中创建的数组分别具有各自不同的构造函数，它们就不是同一类对象

```
var frame = document.createElement('iframe');  
document.body.appendChild(frame);  
var FrameArray = window.frames[0].Array; // 获取框架全局环境中的Array构造函数  
var fa = new FrameArray(1,2,3); // 在框架全局环境中创建数组  
console.log(fa instanceof Array); // false, 在当前环境中判断fa是不是数组  
console.log(Array.isArray(fa)); // true  
console.log(fa.constructor === Array); // false
```

6. // 解决方案：使用`Object.prototype.toString`上的原生`toString()`方法判断；如：
`Object.prototype.toString.call(arr)`;
7. // 该方法返回一个`[object NativeConstructorName]`格式的字符串；但该方法不能检测非原生构造函数的函数名，因此自定义的构造函数都将返回`[object Object]`，如：

```
var frame = document.createElement('iframe');  
document.body.appendChild(frame);  
var FrameArray = window.frames[0].Array; // 获取框架全局环境中的Array构造函数  
var fa = new FrameArray(1,2,3); // 在框架全局环境中创建数组  
console.log(fa instanceof Array); // false, 在当前环境中判断fa是不是数组  
console.log(Array.isArray(fa)); // true  
console.log(fa.constructor === Array); // false  
console.log(Object.prototype.toString.call(fa));  
console.log(Object.prototype.toString.call(fa) === "[object Array]");
```

[object Array]

true

```
var d = new Date();  
console.log(Object.prototype.toString.call(d));
```

[object Date]

```
console.log(Object.prototype.toString.call(p));
```

[object object]

// 所以说如果不是系统自己定义的，而是我们自己定义的独享无法用
Object.prototype.toString.call(); 这个方法来判断，因为都会返回[object object];

```
<body>  
  <iframe src="a.html" name="myFrame"></iframe>  
<script>  
  // 修改上例  
  console.log(Object.prototype.toString.call(fa) === "[object Array]"); // true  
  
  function Person(){}  
  var p = new Person();  
  console.log(Object.prototype.toString.call(p)); // [object Object]  
  // 或者:  
  
  <script>  
  // a.html 中定义了 var aArr = [1,2];  
  window.onload = function(){  
    var myFrame = window.frames["myFrame"];  
    console.log(myFrame.aArr);  
    console.log(myFrame.aArr instanceof Array); // false  
    console.log(Object.prototype.toString.call(myFrame.aArr) === "[object  
Array]"); // true  
  }  
</script>
```

根据封装一个判断类型的函数:

```
function getType(target){  
  var types = {  
    "[object Object]" : "Object",  
    "[object Array]" : "数组",  
    "[object Number]" : "Number",  
    "[object Boolean]" : "Boolean",  
    "[object String]" : "String",  
    "[object Date]" : "日期",  
  };  
  if(target == null) return null;
```

```

if(typeof target == "object"){
    var toStr = Object.prototype.toString.call(target);
    return types[toStr];
}else{
    return typeof target;
}
}

```

```

console.log(getType(18));
console.log(getType({}));
console.log(getType([]));
console.log(getType(new Boolean()));
console.log(getType(new Date()));
console.log(getType([1,2,"aini"]));

```

// 在ES5中，新增了Array.isArray()静态方法，该方法的作用是确定某个值是不是数组，而不管它是在哪个全局环境中创建的：如：

```

var colors = ["red","blue","yellow"];
console.log(Array.isArray(colors));

```

(万能的静态方法，都能判断)

9-8-2 数组去重

- 1, // 使用一个空对象作为临时数组不重复元素值的载体，即把数组不重复元素值保存为空对象属性，再把该属性名添加到一个返回的数组中；
- 2, // 最主要的原理：对象的属性名不可能重复，会覆盖，如

```

var arr = [1,3,2,3,4,2,3,2,1,1,2,3,4];
function getUnique(arr){
    var obj=[];
    var a = [];
    for(var i=0;i<arr.length;i++){
        if(!obj[arr[i]]){
            console.log(arr[i]);
            obj[arr[i]]="aini";
            a.push(arr[i]);
        }
    }
    return a;
}
console.log(getUnique(arr));

```

1

3

2

4

► (4) [1, 3, 2, 4]

9-8-3 类数组对象

ES数组的有一些特性是其他对象所没有的：

- (1) // 当有新的元素添加到列表中时，自动更新length属性；
- (2) // 设置length为一个较小值时将截断数组；
- (3) // 从Array.prototype中继承一些方法；
- (4) // 其类属性为Array；
- (5) // 这些特点让数组和常规的对象有明显的区别；
- (6) // 通常来说，把一个拥有数值length属性和对应非负整数属性的对象看作一种“类数组”对象；
- (7) // 这些“类数组”对象不能直接调用数组方法或者其length属性也没有特殊的行为（如字符串的length属性就是只读的），但可以使用数据遍历的方法来遍历它们，就像操作真正的数组一样；

```
// 字符串是一个类数组对象：  
// Arguments对象是一个类数组对象：
```

```
str = "zeronetwork";  
str.length = 5;  
console.log(str.length); // 11
```

字符串Length属性只读，不可写

```
str = "zeronetwork";  
console.log(str[0],str[1],str[2]); // z e t
```

在客户端JS中，一些DOM方法，如

document.getElementsByTagName()也返回类数组对象，如：

```
var lis = document.getElementsByTagName("li");  
console.log(lis);  
for(var i=0;i<lis.length;i++){  
    var li = lis[i];  
    li.onclick = function(e){  
        alert(this.innerText); //在对应的li标签鼠标单击时弹出里面内容  
    }  
}
```

自定义一个类数组对象

如果让一个对象成为一个类数组对象，主要的做法是：利用属性名模拟数组元素索引；动态的length属性；甚至可以调用push等数组方法；

添加length属性

```
var obj = {0:"aini",1:"dinur",2:"dil"};  
console.log(obj[0],obj[1],obj[2]);  
  
for(p in obj){  
    console.log(p,obj[p]);  
};  
//添加length属性  
var obj = {0:"aini",1:"dinur",2:"dil",length:3};  
console.log(obj[0],obj[1],obj[2]);  
console.log(obj.length);  
//动态添加length属性
```

```

var o = {};
var i = 0;
while(i<10){
    o[i]= i*i;
    i++;
};
o.length=i;
//把o当做数组一样遍历
var sum = 0;
for(var j=0;j<o.length;j++){
    sum+=o[j];
}
console.log(sum);

```

自定义一个类数组对象

```

//自定义类数组类及对象
function MyArray(){
    this.length = arguments.length;
    for(var i=0;i<this.length;i++){
        this[i]=arguments[i]
    }
};
var arr = new MyArray(4,3,5,"wangwei");
for (var i=0; i<arr.length;i++){
    console.log(arr[i]);
};

function YourSize(size){
    this.length=size;
    for(var i=0;i<size;i++){
        this[i]=[i];
    }
};
var a = new YourSize(3);
for(var x in a){
    this[i]=i;
    console.log(x,a[x]);
};
console.log(a);

```

添加push方法

```

//添加push方法，同时更新length属性；
var obj = {'0':'a','1':'b','2':'c',length:3,push:function(){
    this[this.length]=arguments[0];
    this.length+=arguments.length;
    return this.length;
}}
console.log(obj.length);
console.log(obj.push("aini"));

```

添加push方法；添加多个元素

```
//添加push方法，同时更新length属性；
var obj = {'0':'a','1':'b','2':'c',length:3,push:function(){
    for(var i=0;i<arguments.length;i++){
        this[this.length]=arguments[i];
        this.length++
    }
    return this;
}}
console.log(obj.length);
console.log(obj.push("aini","dilnur","diana","norah"));
```

可间接应用Array的方法：

```
//间接使用array的push方法；
var obj = {
    length:0,
    push:Array.prototype.push,
    splice:Array.prototype.splice
};
console.log(obj.push("c","d")); //2
console.log(obj.length); // 2
```

9-8-4 检测类数组对象

封装一个函数，用来检测类数组对象，如

```
var o = new String("aini");
// 判定 o 是否是一个类数组对象
// 字符串和函数都具有length属性，但可以用typeof进行排除
// 客户端，DOM文件节点也具有length属性，需要用额外判断o.nodeType!=3将其排除

function isArrayLike(o){
    if(o && // o 是非null或undefined
        typeof o == "object" && // 是对象
        isFinite(o.length) && // 有限数
        o.length>0 && // 大于0
        o.length == Math.floor(o.length) && //length 属性是整数
        o.length < 2^32 //
    )
        return true
    else
        return false;
};
console.log(isArrayLike(o));
```

ES的数组方法为了能应用在类数组对象上而特意定义为通用的，但是，由于类数组没有继承自Array.prototype，就不能直接调用某些数组方法，只有间接的使用Function.call方法调用，如

```
var s1 = "aini like ";
console.log(s1.concat(" dilnur"));

var o = {0:"a",1:"b",2:"c",length:3};
o.length=3;
console.log(Array.prototype.join.call(o,"+"));
console.log(Array.prototype.slice.call(o,0));
console.log(Array.prototype.map.call(o,function(x){
    return x.toUpperCase();
}));
```

aini like dilnur

a+b+c

► (3) ['a', 'b', 'c']

► (3) ['A', 'B', 'C']

注: concat方法是特例, 可以直接用在字符串上面

作为数组的字符串

// 字符串的行为类似于只读的数组; 除了用charAt()方法来访问单个字符外, 还可以使用方括号; ES为可索引的字符串定义这些特性, 主要为了更加简洁、可读和更高效的操作;

```
var s = "aini";
console.log(s.charAt(0)); // a
console.log(s[0]); // a
```

字符串的这些类数组的行为, 可以使用数组方法

```
var str = "ainilikedilnur";
console.log(Array.prototype.join.call(str,""));
console.log(Array.prototype.filter.call(str,function(s){
    return s
}).join(""));
```

a,i,n,i,l,i,k,e,d,i,l,n,u,r

ainilikedilnur

字符串是不可变值, 所以当把它当作数组看待时, 它们是只读的; 故如push、sort、reverse、splice等数组方法会修改数组, 但在字符串上是无效的;

9-8-5 二维数组和 multidimensional array

// 二维或多维数组就是以数组作为数组元素的数组; 多维数组的应用场景还是比较多的, 比如一个班的学生成绩表;

但从本质上说, ES并不支持真正的二维或多维数组;

```

var a1 = [1,2,3];
var a2 = [4,5,6];
var a3 = [7,8,9];
var arr = [a1,a2,a3];
// 或者
var arr = [
    [1,2,3],
    [4,5,6],
    [7,8,9,10,12]
];
var arr = [
    [1,2,3],
    ['wangwei','wujing'],
    ['零点','网络']
];
var arr = new Array(
    new Array(1,2,3),
    ['wangwei','wujing'],
    new Array('零点','网络')
);

```

如果把二维数组当作另外一个数组的元素，那就是三维数组了，以此类推，就形成多维数组；

访问二维或多维数组，只要多次使用[]操作符即可，如：

```

var arr = [
    [1,2,3],
    ['wangwei','wujing'],
    ['零点','网络']
];
console.log(arr[1][1]); // 访问
arr[3] = new Array('one','two','three'); // 添加
arr[2][2] = "程序员";
console.log(arr);

```

二维或多维数组的遍历同一维数组一致，只不过使用了嵌套遍历

求最大值

```

var arr = [
    [88,67,44,98],
    [56,78,99,56],
    [79,95,78,92]
];
var max = arr[0][0];
for(var i=0;i<arr.length;i++){
    for (var j=0;j<arr[i].length;j++){
        arr[i][j]>max?(max=arr[i][j]):max;
    }
};
console.log(max);

```

九九乘法表

```

var table = new Array(9);

```

```

for(var i=0;i<table.length;i++) table[i]=new Array(9);
for(var row=0;row<table.length;row++){
    for(var col=0;col<table[row].length;col++){
        table[row][col]=row*col;
    }
}
console.log(table[5][9]);

function getTable(arr){
    var table = document.createElement("table");
    for(var i=1; i<arr.length;i++){
        var tr = document.createElement("tr");
        for(var j=1;j<arr[i].length;j++){
            var td = document.createElement("td");
            td.innerText=i+"*"+j+"="+arr[i][j];
            tr.appendChild(td);
        }
        table.appendChild(tr);
    }
    document.body.appendChild(table);
}
getTable(table);

```

10, Date日期对象

GMT: // 格林尼标准时;
 UTC: // 格林尼治标准时间;
 时区: // 共有24区, 东12区, 西12区, 一个时区一小时;
 计算机元年: // 1970年1月1日 0:00:00 用于计时的开始

Date使用的是UTC: // 是所有时区的基准标准时间, 是1970年1月1日 00:00:00 开始经过的毫秒数保存日期;

10-1 Date对象的创建

d = new Date() // 以当前日期和时间创建date对象;
 d = new Date(0) // 以1970-1-1 00:00:00 的毫秒数创建date对象;
 d = new Date(2020,7,18) // 就表示创建了一个2020年8月18号的日期对象;

```

var d = new Date();
var d1 = new Date(0)
var d2 = new Date(2022,12,17);

```

Sat Dec 17 2022 11:50:03 GMT+0800 (中国标准时间)

Thu Jan 01 1970 08:00:00 GMT+0800 (中国标准时间)

Tue Jan 17 2023 00:00:00 GMT+0800 (中国标准时间)

`new Date()` 里面直接传年份注意:

// JS里的月份是 0~11 分别表示1~12月; 所以计算机里 0 表示1月, 1表示2月, 11就是12月;
`d = new Date(2020,7,18)` //得到的是2020年8月18日;

// ECMAScript提供了两个静态方法:
`Date.parse()`和`Date.UTC()`;

10-2 Date.parse()

// 跟时区无关, 月份基于1;

`Date.parse()`方法接受一个表示日期的字符串参数, 返回一个时间戳(毫秒数);

1, // 日期字符串应该符合 RFC 2822 和 ISO 8061 这两个标准:

2, // ISO 8601扩展格式 YYYY-MM-DDTHH:mm:ss:sssZ, 如2020-05-25T00:00:00; (yyyy4位年份、MM月份、DD天、HH时、mm分、ss秒、sss毫秒)

常见的日期格式

- 1, // mm/dd/yyyy 如: 3/21/2009, 即月/日/年
- 2, // yyyy/mm/dd 如: 2009/3/21
- 3, // mmmm dd,yyyy 如: Apr 21,2009, 即英文月名 日, 年, 即January 12,2010

```
console.log(Date.parse("May 25,2020"));
console.log(Date.parse('2018-07-22'))
console.log(Date.parse('2018-07'))
console.log(Date.parse('2018'))
console.log(Date.parse('07/22/2018'))
console.log(Date.parse('2018/07/22'))
console.log(Date.parse('2018/7/22'))
console.log(Date.parse('July 22, 2018'))
console.log(Date.parse('July 22, 2018 07:22:13'))
console.log(Date.parse('2018-07-22 07:22:13'))
console.log(Date.parse('2018-07-22T07:22:13'))
```

// 注: 如果传入`Date.parse()`方法的字符串不能表示日期, 那么它会返回NaN;

根据parse()返回值创建Date对象;

```
var d = new Date(Date.parse("May 25, 2020"));
console.log(d)

// Mon May 25 2020 00:00:00 GMT+0800 (中国标准时间)
```

实际上, 如果直接将表示日期的字符串传递给Date构造函数, 也会在后台调用Date.parse(), 两者是等价的, 如:

```
var d = new Date("May 25, 2020");
```

注意: 月份前面有0和没有0是不一样的 (中间连接符是‘-’的时候才会有区别) 其他都是GTM时间

```
var time = Date.parse('2019-04-03'); // +8区时间
var time1 = Date.parse('2019-4-03'); // 标准UTC时间
var d = new Date(time);
var d1 = new Date(time1);
console.log(d);
console.log(d1);
```

Wed Apr 03 2019 08:00:00 GMT+0800 (中国标准时间)

Wed Apr 03 2019 00:00:00 GMT+0800 (中国标准时间)

注：日期对象在不同浏览器实现的并不统一，比如，传入了超出范围的值：

(负数可以直接取正，前面的0可以省略)

```
var d = new Date("January 33,2020");
console.log(d) // Invalid Date
```

// 在解析January 33,2020，有些浏览器返回：Invalid Date；IE返回：Sun Feb 02 2020（把超出的时间往后自动推算）；

// UNIX 时间戳的原因以秒（seconds）为单位。JavaScript 以毫秒（milliseconds）为单位记录时间。

可在使用UNIX 时间戳去实例化Date 对象；

```
var timestamp = 1591866649;
var d = new Date(timestamp * 1000);
console.log(d);
```

10-3 Date.UTC()

Date.UTC方法：//（参数不用引号，而且需要逗号隔开）月份是基于0的

语法：

// date.UTC(year,month,[date,hrs,min,sec,ms])

必需参数：// year, month

// 其参数为日期中的年,月（基于0）,日,小时（0到23）,分,秒,毫秒，其中年月必选；如果没有提供日，默认为1，如果省略其他参数，则统统默认为0；

// 至少应该是3个参数，但是大多数 JavaScript 引擎都能解析 2 个或 1 个参数；

```
var d = Date.UTC(2020);
var d1 = Date.UTC(2020,6); // 毫秒数1593561600000
var d2 = new Date(Date.UTC(2020,6));
var d3 = new Date(Date.UTC(2020,6,6,17,55,55)); // 自动添加时区，返回当地日期和时间
var d4 = new Date(2020,6,10); //月份从0开始,6即是7月
```


1577836800000

1593561600000

Wed Jul 01 2020 08:00:00 GMT+0800 (中国标准时间)

Tue Jul 07 2020 01:55:55 GMT+0800 (中国标准时间)

Fri Jul 10 2020 00:00:00 GMT+0800 (中国标准时间)

// 如果没有任何关于时区的信息，会将日期视为 UTC，并自动执行到当前计算机时区的转换；

// 可以直接把UTC参数传递给Date()构造函数，如：

```
var d=new Date(2020,6); // wed Jul 01 2020 00:00:00 GMT+0800
var d = new Date(2020,6,6,17,55,55); // 即为GMT时间
console.log(d);
```

// 当初始化一个 Date 对象时可以选择时区，可以通过添加 +HOURS 的格式，或者通过一个被圆括号包裹的时区名来描述一个时区：

注：兼容性有点问题

```
console.log(new Date('Jun 7,2020 13:51:01 +0700'));
console.log(new Date('Jun 7,2020 13:51:01 (CET)')); // CET欧洲中部时间
```

补充：

```
var time2 = Date.parse('2022-4-03')
console.log(new Date(time2))
var time3 = Date.parse('2022-04-03')
console.log(new Date(time3))
```

```
var time2 = Date.parse('2022/4/03')
console.log(new Date(time2))
var time3 = Date.parse('2022/04/03')
console.log(new Date(time3))
```

```
var time2 = Date.parse('2022,4,03')
console.log(new Date(time2))
var time3 = Date.parse('2022,04,03')
console.log(new Date(time3))
```

```
var time2 = Date.parse('2022-4-03 10:56:36')
console.log(new Date(time2))
var time3 = Date.parse('2022-04-03T10:56:36')
console.log(new Date(time3))
```

```
Sun Apr 03 2022 00:00:00 GMT+0800 (中国标准时间)
Sun Apr 03 2022 08:00:00 GMT+0800 (中国标准时间)
Sun Apr 03 2022 00:00:00 GMT+0800 (中国标准时间)
Sun Apr 03 2022 00:00:00 GMT+0800 (中国标准时间)
Sun Apr 03 2022 00:00:00 GMT+0800 (中国标准时间)
Sun Apr 03 2022 00:00:00 GMT+0800 (中国标准时间)
Sun Apr 03 2022 10:56:36 GMT+0800 (中国标准时间)
Sun Apr 03 2022 10:56:36 GMT+0800 (中国标准时间)
```

10-3 总结

(1) // `Date.parse()` 当只有年月日時，且连接符是 `-` 的时候月份前面有0和没有0是有区别的，没有0则是UTC时间，有0的话加8小时；

```
var time2 = Date.parse('2022-4-03')
console.log(new Date(time2))
var time3 = Date.parse('2022-04-03')
console.log(new Date(time3))
```

```
Sun Apr 03 2022 00:00:00 GMT+0800 (中国标准时间)
Sun Apr 03 2022 08:00:00 GMT+0800 (中国标准时间)
```

(2) // 当连接符不是 `-` 的时候，月份前面加不加0都一样，都返回UTC时间
(3) // 当年月份后面加上 时间以后，月份前面有没有0也都一样，都返回正确的 时间
(4) // 只有当 `-` 充当连接符，而且月份前面有0是，年月份跟时间之间才可以加T，当使用其他连接符或者用 `-` 当连接符，但是月份前面没有0 时就会报错

```
var time2 = Date.parse('2022-4-03 10:56:36')
var time3 = Date.parse('2022-04-03T10:56:36')
```

```
var time2 = Date.parse('2022,4,03 10:35:56')
var time3 = Date.parse('2022,04,03 10:35:56')
```

```
var time2 = Date.parse('2022-4-03 10:56:36')
var time3 = Date.parse('2022-04-03T10:56:36')
```

没有 0，加 T 报错

```
Sun Apr 03 2022 10:56:36 GMT+0800 (中国标准时间)
Sun Apr 03 2022 10:56:36 GMT+0800 (中国标准时间)
```

```
var time2 = Date.parse('2022,4,03 10:35:56')
var time3 = Date.parse('2022,04,03 10:35:56')
```

除了- 其他连接符加 T 就报错

```
Sun Apr 03 2022 10:35:56 GMT+0800 (中国标准时间)
Sun Apr 03 2022 10:35:56 GMT+0800 (中国标准时间)
```

10-4 Date 对象的方法

```
1.Date(): // 返回当日的日期和时间。
2.getDate(): // 从 Date 对象返回一个月中的某一天 (1 ~ 31)。
3.getDay(): // 从 Date 对象返回一周中的某一天 (0 ~ 6)。
4.getMonth(): // 从 Date 对象返回月份 (0 ~ 11)。
5.getFullYear(): // 从 Date 对象以四位数字返回年份。
6.getYear(): // 请使用 getFullYear() 方法代替。
7.getHours(): // 返回 Date 对象的小时 (0 ~ 23)。
8.getMinutes(): // 返回 Date 对象的分钟 (0 ~ 59)。
9.getSeconds(): // 返回 Date 对象的秒数 (0 ~ 59)。
10.getMilliseconds(): // 返回 Date 对象的毫秒(0 ~ 999)。
11.getTime(): // 返回 1970 年 1 月 1 日至今的毫秒数，与valueOf()返回值相同。
12.getTimezoneOffset(): // 返回本地时间与格林威治标准时间 (GMT) 的分钟差。
13.getUTCDate(): // 根据世界时从 Date 对象返回月中的一天 (1 ~ 31)。
14.getUTCDay(): // 根据世界时从 Date 对象返回周中的一天 (0 ~ 6)。
15.getUTCMonth(): // 根据世界时从 Date 对象返回月份 (0 ~ 11)。
16.getUTCFullYear(): // 根据世界时从 Date 对象返回四位数的年份。
17.getUTCHours(): // 根据世界时返回 Date 对象的小时 (0 ~ 23)。
18.getUTCMinutes(): // 根据世界时返回 Date 对象的分钟 (0 ~ 59)。
19.getUTCSeconds(): // 根据世界时返回 Date 对象的秒钟 (0 ~ 59)。
20.getUTCMilliseconds(): // 根据世界时返回 Date 对象的毫秒(0 ~ 999)。
21.parse(): // 返回1970年1月1日午夜到指定日期 (字符串) 的毫秒数。
22.setDate(): // 设置 Date 对象中月的某一天 (1 ~ 31)。
23.setMonth(): // 设置 Date 对象中月份 (0 ~ 11)。
24.setFullYear(): // 设置 Date 对象中的年份 (四位数字)。
25.setYear(): // 请使用 setFullYear() 方法代替。
26.setHours(): // 设置 Date 对象中的小时 (0 ~ 23)。
27.setMinutes(): // 设置 Date 对象中的分钟 (0 ~ 59)。
28.setSeconds(): // 设置 Date 对象中的秒钟 (0 ~ 59)。
29.setMilliseconds(): // 设置 Date 对象中的毫秒 (0 ~ 999)。
30.setTime(): // 以毫秒设置 Date 对象。
31.setUTCDate(): // 根据世界时设置 Date 对象中月份的一天 (1 ~ 31)。
32.setUTCMonth(): // 根据世界时设置 Date 对象中的月份 (0 ~ 11)。
33.setUTCFullYear(): // 根据世界时设置 Date 对象中的年份 (四位数字)。
34.setUTCHours(): // 根据世界时设置 Date 对象中的小时 (0 ~ 23)。
35.setUTCMinutes(): // 根据世界时设置 Date 对象中的分钟 (0 ~ 59)。
36.setUTCSeconds(): // 根据世界时设置 Date 对象中的秒钟 (0 ~ 59)。
37.setUTCMilliseconds(): // 根据世界时设置 Date 对象中的毫秒 (0 ~ 999)。
toSource(): // 返回该对象的源代码。
toString(): // 把 Date 对象转换为字符串。
toTimeString(): // 把 Date 对象的时间部分转换为字符串。
toDateString(): // 把 Date 对象的日期部分转换为字符串。
toGMTString(): // 请使用 toUTCString() 方法代替。
toUTCString(): // 根据世界时，把 Date 对象转换为字符串。
toLocaleString(): // 根据本地时间格式，把 Date 对象转换为字符串。
toLocaleTimeString(): // 根据本地时间格式，把 Date 对象的时间部分转换为字符串。
toLocaleDateString(): // 根据本地时间格式，把 Date 对象的日期部分转换为字符串。
toISOString(): // 返回对应的UTC时间的 ISO8601 写法，如2012-12-31T16:00:00.000Z，
toJSON(): // 返回值同toISOString()
UTC(): // 根据世界时返回 1970 年 1 月 1 日 到指定日期的毫秒数。
valueOf(): // 返回 Date 对象的原始值。
```

以上方法大概分为三种：to方法、get方法和set方法。

10-4-1 to方法

(1) to方法-日期格式化方法:

// date()类型还有一些专门用于将日期格式化为字符串的方法, 如:

```
toString():  
toDateStrig(): // 以特定于实现的格式显示星期几、月、日和年;  
toTimeString(): // 以特定于实现的格式显示时、分、秒和时区;  
toLocaleDateString(): // 以特定于地区的格式显示星期几、月、日和年;  
toLocaleTimeString(): // 在特定于地区的格式显示 时、分、秒;  
toUTCString(): // 以特定于实现的格式显示UTC日期;  
toISOString(): // 返回ISO表示的日期;  
toGMTString()方法, // 这是一个与toUTCString()等价的方法, 其存在的目的在于确保向后兼容; 不过ECMAScript推荐使用 toUTCString()方法;
```

继承的方法

与其他引用类型一样, Date类型也重写了toLocaleString()、toString()和valueOf()方法; 但这些方法的返回值与其他类型中的方法不同。

valueOf()方法: // 返回日期的毫秒数;

toString()方法: // 通常返回带有时区信息的日期和时间; 其中时间一般以军用时间(即小时从0到23);

```
var d = new Date();  
console.log(d); // 默认就是调用toString()方法  
console.log(d.valueOf()); // 返回毫秒数  
console.log(Date.parse(new Date())); //返回毫秒数  
console.log(d.toString());
```

Sat Dec 17 2022 13:14:39 GMT+0800 (中国标准时间)

1671254079063

1671254079000

Sat Dec 17 2022 13:14:39 GMT+0800 (中国标准时间)

toLocaleString(): //会按照与浏览器设置的地区相适应的格式返回日期和时间; 即时间格式中会包含AM或PM, 但不会包含时区信息;

```
var d = new Date();  
console.log(d.toLocaleString()); // 2022/12/17 13:16:37
```

注: 真实场景中, toString()和toLocaleString()没有什么用, 仅在调试代码时使用;

至于valueOf()方法, 返回的是毫秒数, 因此, 可以方便的使用比较操作来比较日期, 如:

```
var d1 = new Date(2019,0,1);  
var d2 = new Date(2020,2,1);  
console.log(d1);  
console.log(d2);
```

Tue Jan 01 2019 00:00:00 GMT+0800 (中国标准时间)

Sun Mar 01 2020 00:00:00 GMT+0800 (中国标准时间)

true

```
var d1 = new Date(2019,0,1);
var d2 = new Date(2020,2,1);
console.log(d2-d1); // 毫秒数相减
console.log(d2.valueOf()-d1.valueOf());
```

Tue Jan 01 2019 00:00:00 GMT+0800 (中国标准时间)

Sun Mar 01 2020 00:00:00 GMT+0800 (中国标准时间)

true

相减默认调用的就是valueOf方法

+默认调用toString()方法，进行字符串拼接

```
var d1 = new Date(2019,0,1);
var d2 = new Date(2020,2,1);
console.log(d2+d1); // 字符串拼接，默认调用toString()方法
```

Sun Mar 01 2020 00:00:00 GMT+0800 (中国标准时间)Tue Jan 01 2019 00:00:00 GMT+0800 (中国标准时间)

注意日期比较的惯性思维，如2019.1.1早于2020.2.1日，但后者返回的毫秒数大。

10-4-2 get 方法

```
var d = new Date();
console.log(d.getDate()); //18
console.log(d.getDay()); //4
console.log(d.getFullYear()); //2020
console.log(d.getMonth()); //5 (starts from 0)
console.log(d.getHours()); //17
console.log(d.getMinutes()); //30
console.log(d.getSeconds()) //13
console.log(d.getMilliseconds()); //765
console.log(d.getTime()) //1591868420160
```

10-4-3 set 方法

```
var d = new Date();
d.setDate(6);
d.setFullYear(2022);
d.setMonth(4);
d.setHours(4);
d.setMinutes(4);
d.setSeconds(4);
d.setMilliseconds(123);
d.setTime(1598765678999);
console.log(d);
```

```
// 注: setDate 和 setMonth 从 0 开始编号;  
// 这些方法基本是跟getter方法一一对应的, 但是没有setDay方法, 因为星期几是计算出来的, 而不是设置的;  
set方法中的参数如果超出它的范围, 会进位, 称为冒泡, 如: date.setHours(48), 这也会将日期数变大;
```

```
var date = new Date();  
date.setFullYear(2022,1,18);  
// date.setMonth(24);  
date.setMonth(2,8);  
date.setHours(16,18,28,208);  
console.log(date.toLocaleString());
```

```
// 如果参数是负数, 表示从上个月的最后一天开始减去相应的单位数  
// 以上的方法都有一个相对应的 UTC set方法:  
获取当前时间戳:
```

获取当前时间戳:

```
console.log(new Date().getTime());  
  
console.log(Date.now());
```

Date.now()方法返回表示调用这个方法时的日期和时间的毫秒数; 其简化了Date.getTime()方法, 如:

如果有些浏览器不支持Date.now(), 可以使用+操作符获取Date对象的时间戳, 如:

```
var start = +Date.now();  
num = 0;  
for(var i=0;i<100000000;i++){  
    num+=1;  
    num+=2;  
} // 模拟其他处理代码  
var stop = +Date.now();  
var result = stop-start;  
console.log(result)  
console.log(num)
```

10-5 日期计算

```
var d1 = new Date("2020-06-18");  
var d2 = new Date("2020-06-19");  
console.log(d1 - d2); // -86400000  
console.log(d1 + d2); // 返回两个日期的字符串拼接  
// 或  
var d1 = new Date('July 18,2020 14:10:18');  
var d2 = new Date('July 19,2020 14:10:18');  
var diff = d2.getTime() - d1.getTime();  
console.log(diff);
```

-86400000

Thu Jun 18 2020 08:00:00 GMT+0800 (中国标准时间)Fri Jun 19 2020
08:00:00 GMT+0800 (中国标准时间)

86400000

// getTime() 方法返回以毫秒计的数字，所以需要将当日时刻计入；如：July 18, 2020 14:14:14 不等于July 18, 2020。在这种情况下，可以使用 setHours(0, 0, 0, 0) 来重置当日时刻；

10-5-1 计算本年度还剩下多少天

```
function leftDays() {  
    var today = new Date();  
    var endYear = new Date(today.getFullYear(), 11, 31, 23, 59, 59, 999);  
    var msPerDay = 24 * 60 * 60 * 1000;  
    return Math.round((endYear.getTime() - today.getTime()) / msPerDay);  
}  
console.log(leftDays());
```

10-5-2 中文月份和日期

```
var d = new Date();  
var month = d.getMonth();  
var week = d.getDay();  
var monthArr = ['一月', '二月', '三月', '四月', '五月', '六月', '七月', '八月', '九月', '十月', '十一月', '十二月'];  
var weekArr = ['星期一', '星期二', '星期三', '星期四', '星期五', '星期六', '星期天'];  
console.log(monthArr[month], weekArr[week-1])
```

10-5-3 获取日期部分信息

```
var d = new Date()  
Date.prototype.dataPart = function(part){  
    if(!part) part = 'd';  
    else part = part.toLowerCase();  
    var monthArr = ['一月', '二月', '三月', '四月', '五月', '六月', '七月', '八月', '九月', '十月', '十一月', '十二月'];  
    var weekArr = ['星期一', '星期二', '星期三', '星期四', '星期五', '星期六', '星期天'];  
  
    switch(part){  
        case 'y':  
            return this.getFullYear();  
            break;  
        case 'm':  
            return monthArr[this.getMonth()];  
            break;  
        case 'd':  
            return this.getDate();  
            break;  
        case 'h':  
            return this.getHours();  
            break;  
        case 'm':  
            return this.getMinutes();
```



```

        break;
    case 's':
        return this.getSeconds();
        break;
    case 'w':
        return weekArr[this.getDay()];
        break;
    default:
        return this.getDate();
    }
    return this.getDate();
}
console.log(d.dataPart('d'))
console.log(d.dataPart('s'))
console.log(d.dataPart('h'))
console.log(d.dataPart('y'))

```

// 在date对象原型中定义方法，只要是用date对象创建的日期都能用这个方法

10-5-4 还有多长时间退休

```

function retireDays(birthday,age){
    var d1 = new Date(birthday).getFullYear();
    var d2 = new Date().getFullYear();
    var old = d2 - d1;
    console.log("现在你的年龄是: " + old,"，将于" + (d1 + age) + "退休");
    if(age - old > 0){
        console.log("还差" + (age - old) + "年退休")
    }else{
        console.log("你已经退休啦，好好享受老年生活吧");
    }
}
retireDays('2020.6.6',60);

```

10-5-5 网页时钟

```

function checkTime(time){
    if (time < 10){
        time = '0'+time
    }
    return time
}
function showTime(){
    var d = new Date();
    var h = checkTime(d.getHours());
    var m = checkTime(d.getMinutes());
    var s = checkTime(d.getSeconds());
    myP = document.getElementById("myP");
    myP.innerHTML = h + ':' + m + ':' + s;
    timer = setTimeout('showTime()',1000);
}
showTime()

```


10-5-6 倒计时

```
function getCountDown(d){
    var d1 = new Date();
    var d2 = new Date(d); //
    var diff = d2 - d1;    // 相差毫秒数
    var o = {};
    if(diff >= 0){
        var day = Math.floor(diff / 1000 / 60 / 60 / 24); // 剩下多少天
        var hour = Math.floor(diff / 1000 / 60 / 60 % 24); // 剩下多少小时
        var minute = Math.floor(diff / 1000 / 60 % 60); // 剩下多少分
        var second = Math.floor(diff / 1000 % 60); // 剩下多少秒
        o.stop = false;
        o.str = "距离"+d+" 还剩下"+day+"天"+hour+"小时"+minute+"分"+second+"秒";
    }else{
        o.stop = true;
        o.str = "已时结束";
    }
    return o;
}

var timer = setInterval(function(){
    var mydate = document.getElementById('mydate');
    mydate.innerHTML = getCountDown('2022.12.31 23:59:59').str;
    if(getCountDown('2022.12.31 23:59:59').stop) clearInterval(timer);
},1000);
```

10-5-7 计算某个日期加上天数

```
function addDate(date,days){
    var d = new Date(date);
    d.setDate(d.getDay() + days);
    var month = d.getMonth() + 1;
    var day = d.getDate();
    if(month < 10)
        month = "0" + month;
    if(day < 10)
        day = "0" + day;
    var value = d.getFullYear() + "-" + month + "-" + day;
    return value;
}

console.log(addDate('2020-6-6',50));
console.log(addDate('2020-6-6',-6));
```

10-5-8 判断闰年

四年一闰，百年不闰，四百年再闰

```
Date.prototype.isLeapYear = function(){
    return (this.getFullYear() % 4 == 0 && ((this.getFullYear() % 100 !=0) ||
    (this.getFullYear() % 400 == 0)));
}

var d = new Date();
console.log(d.isLeapYear());
d.setFullYear(2019);
console.log(d.isLeapYear());
```

10-5-9 计算两个日期相差的天数

```
function daysDiff(dateOne,dateTwo){
    var oneMonth = dateOne.substring(5, dateOne.lastIndexOf('-'));
    var oneDay = dateOne.substring(dateOne.length,dateOne.lastIndexOf('-') + 1);
    var oneYear = dateOne.substring(0, dateOne.indexOf('-'));
    var twoMonth = dateTwo.substring(5, dateTwo.lastIndexOf('-'));
    var twoDay = dateTwo.substring(dateTwo.length, dateTwo.lastIndexOf('-') + 1);
    var twoYear = dateTwo.substring(0, dateTwo.indexOf('-'));
    var diff = ((Date.parse(oneMonth+'/' +oneDay+'/' +oneYear) -
    Date.parse(twoMonth+'/' +twoDay+'/' +twoYear)) / 86400000);
    return diff;
}
console.log(daysDiff('2020-6-6','2020-5-30'));
```

10-5-10 格式化输出

```
Date.prototype.format = function(fmt){
    var o = {
        "M+" : this.getMonth() + 1,
        "d+" : this.getDate(),
        "h+" : this.getHours(),
        "m+" : this.getMinutes(),
        "s+" : this.getSeconds(),
        "q+" : Math.floor((this.getMonth() + 3) / 3),
        "S"  : this.getMilliseconds()
    };
    if(/(y+)/.test(fmt)){
        fmt = fmt.replace(RegExp.$1,
            (this.getFullYear() + "").substr(4 - RegExp.$1.length));
    }
    for(var k in o){
        if(new RegExp("(" + k + ")").test(fmt)){
            fmt = fmt.replace(RegExp.$1,
                RegExp.$1.length ===1
                ? o[k]
                : ("00" + o[k]).substr(("" + o[k]).length));
        }
    }
    return fmt;
};
var d = new Date(2020,6,6,0,0,0);
console.log(d);
console.log(d.format('yyyy年MM月dd日')); // 2020年07月06日
console.log(d.format('yyyy年MM月d日 hh:mm:ss')); // 2020年07月6日 00:00:00
```

10-5-11 时间控件

```
<body>
<input type="date" id="mydate">
<input type="datetime" id="mydatetime">
<input type="datetime-local" id="mylocaldate">
<input type="time" id="mytime">
<input type="button" value="提交" onclick="show()">
<script>
```

```
function show(){
    var mydate = document.getElementById('mydate');
    var mydatetime = document.getElementById('mydatetime');
    var mylocaldate = document.getElementById('mylocaldate');
    var mytime = document.getElementById('mytime');

    console.log(mydate.value);
    console.log(mydatetime.value);
    console.log(mylocaldate.value);
    console.log(mytime.value);
}
</script>
</body>
```

2022/12/30  2022年09月15日 2022/12/23 21:55  21:55 

2022-12-30

2022年09月15日

2022-12-23T21:55

21:55

10-5-12 制作日历

```
function getDays(y,m){
    var d = new Date(y,m);
    d.setMonth(m+1);
    d.setDate(0);
    return d.getDate();
}
function changeDay(target,d){
    var year = d.getFullYear();
    var month = d.getMonth();
    var date = d.getDate();
    var week = d.getDay();
    var days = getDays(year,month); //一个月有多少天

    var current = new Date();
    currentyear = current.getFullYear();
    currentmonth = current.getMonth();
    currentday = current.getDate();
    currentweek = current.getDay();
    var daylist = document.getElementById('daylist');
    for(var i=daylist.children.length-1;i>=0;i--){
        daylist.removeChild(daylist.childNodes[i]);
    }
    var d1 = d
    d1.setDate(1);
    var firstweek = d1.getDay(); //获取当月1号对应星期几
    for(var i=0;i<firstweek%7;i++){
        var li = document.createElement('li');
        daylist.appendChild(li)
    }

    for(var i=1;i<=days;i++){
```

```

        var li = document.createElement('li');
        li.innerHTML = i;
        if((i<currentday && month == currentmonth && year == currentyear) ||
(month<currentmonth && year==currentyear) || (year < currentyear)){
            li.className = "lightgray";
        }else if(i== currentday && month == currentmonth && year == currentyear)
        {
            li.className = 'currentbox'
        }else{
            li.className = 'darkgray'
        }

        daylist.appendChild(li)
    }
    document.getElementById(target+'-month').innerHTML = month + 1 + '月';
    document.getElementById(target+'-year').innerHTML = year;
}
var d = new Date();
changeDay('calender',d)
var prev = document.getElementById('prev');
var next = document.getElementById('next');
prev.addEventListener('click',function(){
    d.setMonth(d.getMonth()-1);
    changeDay('calender',d)
},false);
next.addEventListener('click',function(){
    d.setMonth(d.getMonth()+1);
    changeDay('calender',d)
},false);

```

12月

2022

上一月

下一月

日	一	二	三	四	五	六
				1	2	3
4	5	6	7	8	9	10
11	12	13	14	15	16	17
18	19	20	21	22	23	24
25	26	27	28	29	30	31

10-6 Datejs 日期库

// 官网: www.datejs.com

10-6-1 返回特定的日期

```
console.log(Date.today());  
console.log(Date.today().toString('yyyy-MM-d HH:m:s'))  
console.log(Date.today().next().friday().toString('yyyy-MM-d HH:m:s'))  
console.log(Date.today().last().friday().toString('yyyy-MM-d HH:m:s'))  
console.log(Date.last().week().toString('yyyy-MM-d HH:m:s'))
```

10-6-2 判断

```
console.log(Date.today().is().sunday());  
console.log(Date.today().is().saturday());  
console.log(Date.today().is().dec());  
console.log(Date.today().is().weekday()); //判断是不是工作日
```

10-6-3 返回加一天或减一天后的日期

可以是负数

```
console.log(Date.today().addDays(1));  
console.log(Date.today().add(1).day());  
console.log(Date.today().add(1).month());  
console.log(Date.today().add(1).year());  
console.log(Date.today().add(1).week());
```

10-6-4 返回某个月的某个日期

```
console.log(Date.monday().toString('yyyy-MM-d HH:m:s'));  
console.log(Date.next().monday().toString('yyyy-MM-d HH:m:s'));  
console.log(Date.april().toString('yyyy-MM-d HH:m:s'));  
console.log(Date.today().first().monday().toString('yyyy-MM-d HH:m:s')); //本月第一个星期一  
console.log(Date.today().second().monday().toString('yyyy-MM-d HH:m:s')); //本月第二个星期二  
console.log(Date.today().final().sunday().toString('yyyy-MM-d HH:m:s')); //当前月的最后一个星期天  
console.log(Date.april().final().monday().toString('yyyy-MM-d HH:m:s')); //返回四月的最后一个monday
```

10-6-5 返回今天的某个时刻

```
console.log(Date.today().at('4:18pm').toString('yyyy-MM-d HH:m:s'));
```

10-6-6 根据对象构件日期

```
var t = {hour:18,minute:30}  
console.log(Date.today().at(t).toString('yyyy-MM-d HH:m:s'));
```

10-6-7 日期解析转换

```
console.log(Date.parse('t'));  
console.log(Date.parse('tomorrow'));  
console.log(Date.parse('next friday'));  
console.log(Date.parse('yesterday'));  
console.log(Date.parse('last monday'));  
console.log(Date.parse('July 8th, 2020'));  
console.log(Date.parse('July-08-2020'));  
console.log(Date.parse('July/08/2020'));  
console.log(Date.parse('2020 6 16'));  
console.log(Date.parse('2020.6.16'));  
console.log(Date.parse('6.16.2016'));  
console.log(Date.parse('16:30:30'));  
console.log(Date.parse('4:30:30 pm'));
```

```
console.log(Date.parse('t + 5d')); //今天加上5天  
console.log(Date.parse('t + 5m')); //今天加上五个月  
console.log(Date.parse('t - 1m')); //今天减去5个月  
console.log(Date.parse('+')); //今天加上一天  
console.log(Date.parse('-y')); //今天加上一天
```

10-6-8 链式操作

```
//添加1个月零5天，然后检查该日期是否为星期五  
Date.today().add({ months: 1, days: 5 }).is().fri();  
//输入日期，然后移至下一个星期五，减去一个月  
Date.parse("10-July-2004").next().friday().add(-1).month();
```

10-6-9 日期比较

```
Date.today().equals( Date.parse("today")); // true  
Date.parse("last Tues").equals(Date.today()); // true|false  
  
Date.equals(Date.today(), Date.parse("today")); // true|false  
Date.compare(Date.today(), Date.parse("today")); // 1 = greater, -1 = less than,  
  
Date.today().compareTo(Date.parse("yesterday")); // 1 = greater, -1 = less than, 0 = equal  
Date.today().between(startDate, endDate); // true|false
```

10-6-10 转换为字符串

注意该format参数对于该.toString()功能是可选的。如果未提供format，.toString()则将调用本地JavaScript Date 函数。

```
s: // 分钟介于0到59之间的秒数，如： 0 to 59
```

ss: // 如果需要, 分钟的秒数, 前导零, 如: 00 to 59
m: // 每小时的分钟数, 介于0到59之间, 如: 0 or 59
mm: // 每小时的分钟, 前导零 (如果需要), 如: 00 to 59
h: // 1到12之间的一天中的小时, 如: 1 to 12
hh: // 如果需要, 一天中的小时数, 前导零, 如: 01 to 12
H: // 0-23之间的一天中的小时, 如: 0 to 23
HH: // 如果需要, 一天中的小时数, 前导零, 如: 00 to 23
d: // 每月的1到31之间的日期, 如: 1 to 31
dd: // 如果需要的话, 该月的某天前导零。如: 01 to 31
ddd: // 缩写的天名, 如: Mon to Sun
dddd: // 全日名称, 如: Monday to Sunday
M: // 一年中的1-12点之间的月份, 如: 1 to 12
MM: // 一年中的前导零 (如果需要), 如: 01 to 12
MMM: // 缩写的月份名称, 如: Jan to Dec
MMMM: // 完整的月份名称, 如: January to December
yy: // 将年份显示为两位数, 如: 99 or 07
yyyy: // 显示完整的四位数年份, 如: 1999 or 2007
t: // 显示AM / PM指示符的第一个字符, 如: A or P
tt: // 显示AM / PM指示符, 如: AM or PM
S: // 当日的序数后缀, 如: st, nd, rd, or th

自定义日期和时间格式说明符

d: // shortDate 格式模式, 如: M/d/yyyy
D: // longDate 格式模式, 如: dddd, MMMM dd, yyyy
F: // fullDateTime 格式模式, 如: dddd, MMMM dd, yyyy h:mm:ss tt
m: // monthDay 格式模式, 如: MMMM dd
r: // rfc1123 格式模式, 如: ddd, dd MMM yyyy HH:mm:ss GMT
s: // sortableDateTime 格式模式, 如: yyyy-MM-ddTHH:mm:ss
t: // shortTime 格式模式, 如: h:mm tt
T: // longTime 格式模式, 如: h:mm:ss tt
u: // universalSortableDateTime 格式模式, 如: yyyy-MM-dd HH:mm:ssZ
y: // yearMonth 格式模式, 如: MMMM, yyyy

分隔符

正斜杠、空格、- 连字号、逗号

```
console.log(Date.today().toString());  
console.log(Date.today().toString('M/d/yyyy'))  
console.log(Date.today().toString('d'));  
console.log(Date.today().toString('MMMM dS,yyyy'));
```

```
new Date().toString(); //星期三2007年10月31日格林尼治标准时间0700 (太平洋夏令时间)  
new Date().toString("M/d/yyyy"); //2007年10月31日  
  
Date.today().toString("d-MMM-yyyy"); //2007年10月31日  
new Date().toString("HH:mm"); // 16:18  
  
Date.today().toString("MMMM dS, yyyy"); // April 12th, 2008
```

```
Date.today().toShortDateString();// "10/31/2007". 根据
Date.CultureInfo.shortDatePattern特定于区域性
Date.today().toLongDateString();// "Wednesday, October 31, 2007". 根据
Date.CultureInfo.longDatePattern特定于区域性

new Date().toShortTimeString();// "4:18 PM". 根据
Date.CultureInfo.shortTimePattern特定于区域性
new Date().toLongTimeString();// "4:18:34 PM". 根据
Date.CultureInfo.longTimePattern特定于区域性
```

核心用法

```
//将日期设置为当前月份和年份的15号;
//其他对象值包括year|month|day|hour|minute|second。
Date.today().set({ day: 15 });
Date.today().set({ year: 2007, month: 1, day: 20 });
//将Date添加2天。其他对象值包括 year|month|day|hour|minute|second。
Date.today().add({ days: 2 });
Date.today().add({ years: -1, months: 6, hours: 3 });
Date.today().addYears(1); //增加1年
Date.today().addMonths(-2); //相减2个月
Date.today().addWeeks(1); //增加1周
Date.today().addDays(4); //增加4天
Date.today().addHours(6); //增加6小时
Date.today().addMinutes(-30); //相减30分钟
Date.today().addSeconds(15); //增加15秒
Date.today().addMilliseconds(200); //增加200毫秒

Date.today().moveToFirstDayOfMonth();//返回当前月份的第一天
Date.today().moveToLastDayOfMonth();//返回当前月份的最后一天

new Date().clearTime(); //将时间设置为00:00（一天的开始）
Date.today().setTimeToNow();//将时间重置为当前时间;与clearTime()的功能相反
```

其他用法

```
Date.getMonthNumberFromName("March");// 2-特定于CultureInfo。<static>
Date.getDayNumberFromName("sat");// 6-特定于CultureInfo。<静态>
Date.isLeapYear(2008) // true|false。 <static>
Date.getDaysInMonth(2007, 9) // 31 <static>
Date.today().getWeek();//返回一年中的第几周。根据年份Date 返回1到（52 | 53）
Date.today().setWeek(1); //将一年中的星期几设置为星期几
var test = new Date(); // Do something... like run a test...
test.getElapsed(); //返回距现在的毫秒数
Date.today().isDaylightSavingTime();// true|false。 在夏令时之内
Date.today().hasDaylightSavingTime();// true|false。 是否遵守夏令时
```

10-7 Momentjs日期库

```
// 官网: https://momentjs.com/
```


10-7-1 获取当前的时间

```
console.log(moment()); //返回一个对象
```

```
$ { _isAMomentObject: true, _isUTC: false, _pf: {...}, _locale: ne, _d: Sat Dec 31 2022 20:47:39 GMT+0800 (中国标准时间), ... }
  ▼ _d: Sat Dec 31 2022 20:47:39 GMT+0800 (中国标准时间)
    ▶ [[Prototype]]: Object
    _isAMomentObject: true
    _isUTC: false
    _isValid: true
  ▼ _locale: ne
    ▶ ordinal: f (e)
    _abbr: "en"
    ▶ _calendar: { sameDay: '[Today at] LT', nextDay: '[Tomorrow at] LT', nextWeek: 'dddd [at] LT', lastDay:
    ▶ _config: { calendar: {...}, longDateFormat: {...}, invalidDate: 'Invalid date', dayOfMonthOrdinalParse: /\d{1,2}(th|st|nd|rd)/
    ▶ _dayOfMonthOrdinalParse: /\d{1,2}(th|st|nd|rd)/
    ▶ _dayOfMonthOrdinalParseLenient: /\d{1,2}(th|st|nd|rd)|\d{1,2}/
    ▶ _eras: (2) [{...}, {...}]
    _invalidDate: "Invalid date"
    ▶ _longDateFormat: { LTS: 'h:mm:ss A', LT: 'h:mm A', L: 'MM/DD/YYYY', LL: 'MMM D, YYYY', LLL: 'MMM D, Y
    ▶ _meridiemParse: /[ap]\.?m?\.?/i
    ▶ _months: (12) ['January', 'February', 'March', 'April', 'May', 'June', 'July', 'August', 'September',
    ▶ _monthsShort: (12) ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']
    ▶ _relativeTime: { future: 'in %s', past: '%s ago', s: 'a few seconds', ss: '%d seconds', m: 'a minute',
    ▶ week: { dow: 0, doy: 6 }
```

```
console.log(moment()._d);
console.log(moment().format('yyyy-M-d')); //2022-12-6
console.log(moment(undefined).format('yyyy-M-d')); //2022-12-6
console.log(moment([]).format('yyyy-M-d'))
```

10-7-2 Format days

```
moment().format('MMMM Do YYYY, h:mm:ss a');
moment().format('dddd');
moment().format("MMM Do YY");
moment().format('YYYY [escaped] YYYY');
moment().format();
```

10-7-3 Relative Time

```
moment("20111031", "YYYYMMDD").fromNow();
moment("20120620", "YYYYMMDD").fromNow();
moment().startOf('day').fromNow();
moment().endOf('day').fromNow();
moment().startOf('hour').fromNow();
```

10-7-4 Calendar Time

```
console.log(moment().subtract(10, 'days').calendar());
console.log(moment().subtract(6, 'days').calendar());
console.log(moment().subtract(3, 'days').calendar());
console.log(moment().subtract(1, 'days').calendar());
console.log(moment().calendar());
console.log(moment().add(1, 'days').calendar());
console.log(moment().add(3, 'days').calendar());
console.log(moment().add(10, 'days').calendar());
```

12/21/2022
Last Sunday at 9:02 PM
Last Wednesday at 9:02 PM
Yesterday at 9:02 PM
Today at 9:02 PM
Tomorrow at 9:02 PM
Tuesday at 9:02 PM
01/10/2023

其余的去官网了解

11, RegExp 正则

11-1 正则创建

11-1-1 直接量

```
var pattern = /aini/;  
var str = 'aini is a good boy'  
console.log(pattern.test(str))
```

11-1-2 使用RegExp构造函数

```
var pattern = new RegExp('aini')  
var str = 'aini is a good boy'  
console.log(pattern.test(str))
```

11-2 修饰符

- 1, g :// 表示全局(global)模式, 即模式将被应用于所有字符串, 而非在发现第一个匹配项时立即停止;
- 2, i : // 表示不区分大小写(case-insensitive)模式;
- 3, m :// 表示多行(multiline), 即在到达一行文本末尾时还会继续查找下一行中是否存在匹配项;

一个正则表达式就是一个模式与上述 3 个标志的结合体, 不同组合产生不同结果, 如:

```
var pattern1 = /at/g; // 匹配字符串中所有 at 的实例  
var pattern2 = /[bc]at/i; // 匹配第一个 bat 或 cat, 不区分大小写  
var pattern3 = /.at/gi; // 匹配所有以 at 结尾的 3 个字符的组合, 不区分大小写
```

另一种创建正则表达式的方式是使用 RegExp 构造函数; 它接收两个参数: 一个是要匹配的字符串模式, 另一个是可选的修饰符字符串; 如:

```
var pattern1 = new RegExp("[bc]at", "i");
```

- 1, // 也可以不使用 new 操作符, 其等同于 new RegExp();
- 2, // 但如果 pattern 参数是一个正则表达式时有所不同, 它只是简单地返回 pattern, 而不会创建一个新的 RegExp 对象, 但如果使用了不同的修饰符, 就会返回新的 RegExp 对象:

```
var re = new RegExp(".at", "g");
newre = RegExp(re); // true
newre = RegExp(re, "i"); // false （模式改变了就不是同一个正则了）
console.log(re === newre);
```

RegExp构造函数最大的特点是可以动态的创建正则表达式，这种情况往往用在：没办法通过写死在正则直接量中；

```
var arr = ['坏蛋', '傻女人', '妈的', '脑子有病', '神经病', '疯子']
```

```
var arrStr = ['你真是个坏蛋', '你是傻女人', '他妈的，气死我了', '你脑子有病', '神经病啊你，干嘛打我', '狗疯子，我要打死你，你妈的，脑子真有病']
for(var i=0; i<arrStr.length; i++){
  for(var j=0; j<arr.length; j++){
    //var reg = /arr[j]/g; // 不能用直接量，这是错误的
    var reg = new RegExp(arr[j], 'img');
    arrStr[i] = arrStr[i].replace(reg, '*');
  }
}
console.log(arrStr)
```

11-3 转义符

- 1, // 如果模式中用到特殊字符(元字符)，包括非字母字符，必须使用转义 \ ，包括\自身：
- 2, // 如果使用 **RegExp** 构造函数，转换符必须使用双 \ ，即 \\：

```
var str = 'aini is [aini] ia ainsii';
var pattern = /[aini]/g ;
var pattern1 = new RegExp('\\[aini\\]', 'g'); 双斜杠转义
console.log(str.match(pattern))
console.log(str.match(pattern1))
```

11-4 精确匹配

11-4-1 元字符

// 元字符是正则表达式拥有特殊含义的字符，即是正则表达式语法的一部分，其包括：
 // ^ \$. * + ? = ! : | \ / () [] { } （15 个）
 // 这些元字符在正则表达式中都有一或多种特殊用途，某些符号只有在正则的某些上下文中才具有某种特殊含义，在其他
 // 上下文中则被当作直接量处理；然而，任何时候，如果要使用这些特殊字符进行匹配，都必须进行转义。

```
var pattern1 = /[bc]at/i; // 匹配第一个 bat 或 cat，不区分大小写
var pattern2 = /[bc\]at/i; // 匹配第一个[bc]at，不区分大小写
var pattern3 = /.at/gi; // 匹配所有以 at 结尾的 3 个字符的组合，不区分大小写
var pattern4 = /\.\at/gi; // 匹配所有.at，不区分大小写
```

// 注：其他标点符号没有特殊含义，可以直接当作字面量进行匹配；如果记不住这些特殊符号，可以为每个标点符号前都加上反斜杠；另外，许多字母和数字在有反斜杠做前缀时也有特殊含义

11-4-2 使用特殊字符

- 1, // 可以直接使用字符表示它们本身，但也可以使用它们的 **ASCII** 或者 **Unicode** 代码来指定字符；要使用 **ASCII** 来表示一个字符，则必须指定一个两位的十六进制代码，并在前面加上 \x
- 2, // 也可以使用八进制代替十六进制来指定字符；
- 3, // 如果要使用 **Unicode** 来表示，必须指定字符串的四位的十六进制 \uxxxx

```
var sColor = "blue";// b 的 ASCII 为 98，等于十六进制 62，因此 b 可以用 \x62
var re = /\x62/; // 16 进制 相当于 /b/
var re = /\142/; // 8 进制 相当于 /b/
var re = /\u0062/; // Unicode 相当于 /b/
console.log(re.test(sColor));
```

11-4-3 其他特殊字符

\o NULL 字符 (\u0000)、
\t 制表符 (\u0009)、
\n 换行符 (\u000A)、
\v 垂直制表符 (\u000B)、
\f 换页符 (\u000C)、
\r 回车符 (\u000D)、
\b 回退字符、
\a alert 字符、
\e escape 字符、
\xnn 由 16 进制数 nn 指定的拉丁字符，
如：\x0A 等价于 \n、
uxxxx 由 16 进制数 xxxx 指定的 Unicode 字符，
如：\u0009 等价于 \t、 cX 与 X 相对应的控制字符，
如，\cJ 等价于换行符 \n。

```
<body>
<textarea id="txt"></textarea>
<textarea id="result"></textarea>
<p id="myp"></p>
<input type="button" onclick="show()" value="提交" />
<script>
function show(){
var str = document.getElementById("txt").value;
var re = /\n/g;
str = str.replace(re, "\n"); // 保存在数据中
document.getElementById("result").value = str;
str = str.replace(re, "<br/>"); // 显示在页面中
document.getElementById("myp").innerHTML = str;
}
show()
```

```
my name is
abuduaini
what is your
name
```

```
my name is
abuduaini
what is your
name
```

```
my name is
abuduaini
what is your
name
```

提交

11-4-4 预定义字符

也称为字符类；一个字符类可以匹配它所包含的任意字符；由于某些模式会反复用到，所以提供了一些预定义字符来提高匹配的效率；

```
. // 等同于 [^\n\r]，除了换行符和回车之外的任意字符
\d // 等同于 [0-9]，数字（ASCII 数字）
\D // 等同于 [^0-9]，非数字字符（除了 ASCII 数字之外的任何字符）
\s // 等同于 [ \t\n\0B\f\r]，空白字符（任何 Unicode 空白符）
\S // 等同于 [^\t\n\0B\f\r]，非空白字符（任何非 Unicode 空白符的字符），注：和 \w 不同
\w // 等同于 [a-zA-Z0-9_]，单词字符（即 ASCII 字符，包括所有字母，所有数字和下划线） □ \W 等同于 [^\a-zA-Z0-9_]，非单词字符（任何不是 ASCII 字符）
[\b]，// 退格直接量（特例）
```

注：以上的字符指的是 ASCII 字符，非 Unicode 字符

```
var str = "567 9838 zeronetwork 王唯";
var re = /.*/gi;
var re = /\d/gi;
var re = /\d\d\d/gi; // 匹配三个数字
var re = /\D/gi;
var re = /\s/gi;
var re = /\S/gi;
var re = /\w/gi;
var re = /\W/gi;
```

通过将一些字符放入方括号中，表示要匹配的范围；

```
简单范围：// 形如：/[acf]at/g
排除范围：// 使用^(脱字符号)，用来定义否定字符类，必须出现在[之后，匹配所有不包含在方括号内的字符，形如：/[^acf]at/
连续范围：// 使用 - 连字符表示一个连续范围，如：[a-z]，[0-9]，[^1-4]
组合范围：// 形如：[a-m1-4\n]
```

```

var str = "a bat, a Cat, a fAt baT, a faT, a faT cat";
var re = /[bcf]at/gi; //["bat", "Cat", "fAt", "baT", "faT", "faT", "cat"]
var re = /[\\u0062cf]at/gi; //["bat", "Cat", "fAt", "baT", "faT", "faT", "cat"]
var re = /^[bc]at/gi; //["fAt", "faT", "faT"]
console.log(str.match(re));
var str = "num1, num2, num3, num4, num5, num6, num7, num8, num9";
var re = /num[1-4]/gi;
console.log(str.match(re)); // ["num1", "num2", "num3", "num4"]
var str = "567 9838 abc";
var re = /[0-9][0-9][0-9]/gi; // ["567", "983"]
var re = /[0-9]{3}/gi; // ["567", "983"]
console.log(str.match(re));

```

// 注：有些字符类转义字符只能匹配 ASCII 字符，还没有扩展到可以处理 Unicode 字符，但可以通过十六进制表示法来显式定义 Unicode 字符类，如：/[\u0400-\u04FF]/ 用以匹配所有 Cyrillic 字符（斯拉夫语）

11-4-5 量词

非贪婪模式放在量词后面

```

{n} : // 匹配 n 次
{n, m} : // 匹配至少 n 次，但不超过 m 次
{n, } : // 匹配至少 n 次
? : // 匹配 0 次或 1 次，等价于{0, 1}
* : // 匹配 0 次或多次，等价于{0, }
+ : // 匹配 1 次或多次，等价于{1, }

```

```

var str = "wangwei age is 18, (birthday) is 1998 year. jing123 age is";
var re = /\d{2,4}/g; // ["18", "1998", '123']
var re = /\w{4}\d?/g; ['wang', 'birt', 'hday', '1998', 'year', 'jing1']
var re = /\s+age\s+/g; // [" age ", " age "]
var re = /[^()]*;/g; // 匹配非左括号或右括号的字符

```

11-4-6 贪婪与非贪婪

```

var str = "wwwwww";
var re = /w{2,4}/;
console.log(str.match(re)); // www
var str = "wangwei";
var re = /\w+/;
console.log(str.match(re)); // wangwei

```

以上匹配的特点是：尽可能多地匹配；这种匹配称为“贪婪”匹配；

贪婪量词的原理：// 先看整个字符串是否匹配，如果没有发现匹配，它去掉该字符串中的最后一个字符，并再次尝试，如果还没有发现匹配，那么再去掉最后一个字符，这个过程一直重复到发现一个匹配或者字符串不剩下任何字符；

// 与贪婪对应的就是非贪婪，称为惰性方式，只需要在量词的后面跟随一个? 问号即可，如：??、+?、*?或{1,5}?；如：修改上例；

惰性量词的原理：先看字符串中的第一字母是否匹配，如果不匹配，再读出下一字符，一直继续，直到发现匹配或者整个字符串都检查过出没有匹配，与贪婪工作方式相反；

支配量词：只尝试匹配整个字符串；如果整个字符串不匹配，就不做进一步尝试；（已不被支持）

```
贪婪：// ? * + {n} {n,m} {n, }  
惰性：// ?? *? +? {n}? {n,m}? {n, }?  
支配：// ?+ *+ ++ {n}+ {n,m}+ {n, }+
```

使用非贪婪模式所得到的结果可能和期望并不一致：

```
var str = "aaab";  
var re = /a+b/; // aaab  
var re = /a+?b/; // aaab  
console.log(str.match(re));
```

由于惰性匹配是从左往右匹配；

11-4-7 复杂模式

7-1 候选

// 候选就是用“|”来表示的模式或关系，它表示的是在匹配时可以匹配“|”的左边或右边。这个“|”相当于“或”

```
var str = "i like colors:red black";  
var re = /red|black|green/; // red  
console.log(str.match(re));  
var re = /jpg|png|gif/;  
console.log(re.test("xxx.jpg"));  
var str = "wang is 18 age."  
var re = /\d{2}|[a-z]{4}/; // wang  
console.log(str.match(re));
```

候选项的尝试匹配次序是从左到右，直到发现了匹配项；如果左边的选择项匹配，就忽略右边的匹配项，即使它会产生更好的匹配，如：

```
var str = "abcde";  
var re = /a|ab/; // a 只会匹配一个 a  
console.log(str.match(re));
```

候选结合 replace() 方法 主要用在从用户输入删除不合适的单词；

```
var str = "你妈的他妈的不是东西，都是坏蛋!";  
var re = /坏蛋|你妈的|他妈的/gi;  
var newStr = str.replace(re, "*****");  
console.log(newStr);  
var newStr = str.replace(re, function(sMatch){  
    return sMatch.replace(/./g, "*");  
});
```

7-2 分组

通过用一对圆括号，可以把单独的项组合成子表达式；

```
var str = "wangwei1998";
var re = /[a-z]+(\d+)/;
console.log(str.match(re));
```

```
▶ (2) ['wangwei1998', '1998', index: 0, input: 'wangwei1998', groups: undefined]
```

也会把圆括号里匹配的内容单独提取出来

为什么需要分组？：

- 1, // 它是一个组合项或子匹配，可作为一个单元，统一操作；如可以统一使用|、*、+等进行处理。
- 2, // 可以把分组匹配的结果单独抽取出来以备后用；

```
var str = "javascript";
var re = /java(script)?/; // true script 可有可无
console.log(re.test(str));
var str = "dogdogdog";
var re = /(dog){3}/;
console.log(str.match(re));
// 只关心匹配尾部的数字，把它单独提取出来
var str = "京 A88888";
var re = /.{2}(\d+)/;
var arr = re.exec(str);
console.log(arr);
console.log(arr[1]);
```

还可以嵌套分组：

```
var str = "zeronetwork";
var re = /(zero(net(work))) /;
console.log(str.match(re));
```

```
▶ (4) ['zeronetwork', 'zeronetwork', 'network', 'work', index: 0, input: 'zeronetwork', groups: undefined]
```

反向引用：

// 每个分组都被存放在一个特殊的地方以备将来使用，这此分组也称为捕获组，这些存储在分组中的特殊值，称之为反向引用；即允许在同一正则表达式的后部引用前面的子表达式；其是通过在字符“\”后加一位或多数数字实现的，该数字指定了分组的子表达式的在正则中的位置：

```
var str = "god godod gododod godododod";
var re = /g(od)\1*/g;
console.log(str.match(re));
```

```
▶ (4) ['god', 'godod', 'gododod', 'godododod']
```

由于分组可以嵌套，所以反向引用是按照从左到右遇到的左括号的顺序进行创建和编号的，如 (A?(B?(C?)))

1.(A?(B?(C?))) 2.(B?(C?)) 3. (C?):


```
var str = "aaabbcbb aaabbcbb";
var re = /(a+(b+))(c)\3\2\s\1/;
console.log(str.match(re));
```

[biaogeaini.htm](#)

► (4) ['aaabbcbb aaabb', 'aaabb', 'bb', 'c', index: 0, input: 'aaabbcbb aaabbcbb', groups: undefined]

对分组的引用，并不是对分组表达式的引用，而是对分组模式相匹配的文本的引用；再如：

```
// 匹配单引号与双引号之间的字符
var str = 'wangwei \'is" "18 old", he is \'good\' man';
var re = /['"](?:[^\s"]*"|'[^']*')/g; // 不要求引号的匹配
var re = /(?:['"](?:[^\s"]*"|'[^']*'))/g; // 不要求引号的匹配
console.log(str.match(re));
```

► (2) ['" ', "'good'"]

反向引用的情景：通常用来处理相同连续的内容，如：

```
// 匹配连续相同的三个数字
console.log("111a222b333c123d".match(/(\d)\1\1/ig)); // ["111", "222", "333"]
console.log("111a222b333c123d".match(/(\d)\1{2}/ig)); // ["111", "222", "333"]
// 不同点，这是匹配 3 个数字，而不是相同的数字
console.log("111a222b333c123d".match(/(\d){3}/ig)); // ["111", "222", "333", "123"]
// 匹配 ABAB 格式的数字，如：1212 或 3434
console.log("1212a3434b4545c123d".match(/(\d)(\d)\1\2/g)); // ["1212", "3434", "4545"]
// 匹配 ABBA 格式的数字，如：1221 或 3443
console.log("1221a3443b4554c123d".match(/(\d)(\d)\2\1/g)); // ["1221", "3443", "4554"]
// 检索 html 标记及内容
var html = '请访问: <a href="https://www.zeronetwork.cn">zeronetwrok</a>网站';
var reg = /<(\w+)[\s]*.+?>(.*?)<\/\1>/ig;
console.log(html.match(reg));
```

['zeronetwrok']

反向引用几种使用方法：

```
// 使用正则表达对象的 test(), match(), search() 方法后，反向引用的值可以从 RegExp 构造函数中获得；
```

```

var str = "#123456789";
var re = /#(\d+)/;
re.test(str);
console.log(RegExp.$1); // 123456789
// 去重
var str = "aaaabbbbbbbccccc";
var re = /(\w)\1*/g;
console.log(str.replace(re, "$1")); // abc
// 格式化输出
var str = "1234 5678";
var re=/(\d{4}) (\d{4})/;
var newStr = str.replace(re, "$2 $1"); // 5678 1234
console.log(newStr);

```

7-3 非捕获性分组

- 1, // 反向引用，称为捕获性分组；如果只需要组合，不需要反向引用，则可以使用非捕获性分组；
- 2, // 在较长的正则表达式中，存储反向引用会降低匹配速度；
- 3, // 非捕获性分组：在左括号的后面加一个问号和一个紧跟的冒号，如：(?:)；此时，使用\1 就访问不了捕获组了。

```

var str = "zeronetwork";
var re = /(?:net)/;
console.log(str.match(re));
console.log(RegExp.$1); // 空
// 删除 HTML 标识
String.prototype.stripHTML = function(){
var re = /<(?:.|\s)*?>/g;
return this.replace(re,"");
};
var str="<a href=#><b>零点程序员</b></a>";
document.write(str + "<br>");
document.write(str.stripHTML());

```

7-4 边界

边界(boundary)：用于正则表达式中表示模式的位置；也称为匹配表达式的锚

^ : // 匹配字符串的开头，在多行中，匹配行开头；
 \$: // 匹配字符串的结尾，在多行中，匹配行结尾；
 \b : // 匹配单词的边界，即位于字符\w 和\w 之间的位置，或位于字符\w 和字符串的开头或者结尾之间；
 \B : // 匹配非单词的边界的位置；

```

var str = "JavaScript"; // JavaScript
var str = "JavaScript Code"; // null
var re = /^JavaScript$/; // 如果不使用$,使用\s 也可以，但是包含了空格
console.log(str.match(re));
var str = "Study Javascript Code. JavaScripter is good";
var re = /Java[ss]cript/g; // ["Javascript", "JavaScript"]
var re = /\bJava[ss]cript\b/g; // ["Javascript"]
var re = /\bJava[ss]cript\B/g; // ["JavaScript"]
var re = /\B[ss]cript/g; // 不会匹配单独的 script 或 Script
console.log(str.match(re));
var str = "wangwei is good man";

```

```

var re = /(w+)$/; // man
re.test(str);
console.log(RegExp.$1);
var re = /^(w+)/; // wangwei
re.test(str);
console.log(RegExp.$1);
var re = /^(.+?)\b/; // wangwei
re.test(str);
console.log(RegExp.$1);
var str = "First second third fourth fifth sixth";
var re = /\b(\S+)\b/g;
var re = /\b(w+)\b/g;
var arr = str.match(re);
console.log(arr);

```

7-5 前瞻 (先行断言)

() 不是分组

前瞻(lookahead) : // 是指检查接下来出现的是不是位于某个特定字符之前; 分为正向和负向;
 // 正向前瞻要将模式放在(?= 和)之间, 如: (?=p); 要求接下来的字符都与 p 匹配, 但不能包括匹配 p 的那些字符; 也称为正向先行断言;
 // 负向前瞻: 将模式放到(?! 或)之间, 如: (?!p); 要求接下来的字符不与 p 匹配; 也称为负向先行断言;

注: 虽然用到括号, 但这不是分组; JS 不支持后瞻, 后瞻可以匹配, 如: 匹配 b 且仅当它前面没有 a;

```

var str1 = "bedroom Bedding";
var re = /([bB]ed(?:=room))/; // bed
var re = /([bB]ed(?:!room))/; // Bed
console.log(str1.match(re));
console.log(RegExp.$1);
var str = "JavaScript: function is simple javascript.";
var re = /[jJ]avascript(?:=:)/g; // Javascript 后面有冒号才匹配
var re = /[jJ]avascript(?:!:)/g; // javascript 后面没有冒号才匹配
console.log(str.match(re));
var str = "JavaScript Javascript JavaBeans javascripter";
var re = /[jJ]ava(?:=[ss]cript)/g; // ["Java", "Java", "java"]
var re = /[jJ]ava(?:![ss]cript)/g; // Java
var re = /[jJ]ava(?:![ss]cript)\w+/g; // JavaBeans
console.log(str.match(re));
// 添加千分位
var str = "钱: 1234567890123";
var re = /(?:=(\B)(\d{3})+)$/g;
console.log(str.match(re));
console.log(str.replace(re, ","));

```

7-6 多行模式

// 要指定多行模式, 需要指定 m 选项, 如果待匹配字符串包含多行, 那么^与\$锚字符除了匹配整个字符串的开始和结尾之外, 还能匹配每行的开始和结尾;

```

var str = "wangwei is\ntechnical director\nof zeronetwork";
var re = /(\\w+)$/g; // zeronetwork
var re = /(\\w+)$/gm; // ["is", "director", "zeronetwork"]
console.log(str.match(re));
var re = /^(\\w+)/g; // wangwei
var re = /^(\\w+)/gm; // ["wangwei", "technical", "of"]
console.log(str.match(re));
var re = /^(.+)$/g; // null 如果待匹配字符串中有换行，如果不使用 m,则结果为 null
console.log(str.match(re));

```

RegExp 实例属性:

```

global: // 只读, 布尔值, 表示是否设置了 g 标志
ignoreCase: // 只读, 布尔值, 是否设置了 i 标志
multiline: // 只读, 布尔值, 是否设置了 m 标志
lastIndex: // 可读写, 整数, 如果模式带有 g, 表示开始搜索下一个匹配项的字符位置, 从 0 起; 该属性一般会在 exec()和 test() . 中用到;
source: // 只读, 正则表达式的字符串表示, 按照字面量形式而非传入构造函数中的字符串模式返回; 但不包含修饰符;

```

```

var str = "wangwei name is wangwei \r\n WANGWEI age is 18 wangwei";
var reg = /wangwei/img;
console.log(reg.global); //true
console.log(reg.ignoreCase); //true
console.log(reg.multiline); //true

```

```

var str = "wangwei name is wangwei \r\n WANGWEI age is 18 wangwei";
var reg = /wangwei/img;
reg.test(str);
console.log(reg.lastIndex); //7

```

```
// 是匹配完以后最后一个字符后面的位置
```

匹配多次以后, 可以依次找到匹配的位置

```

var str = "wangwei name is wangwei \r\n WANGWEI age is 18 wangwei";
var reg = /wangwei/img;
reg.test(str);
console.log(reg.lastIndex); //7
reg.test(str);
console.log(reg.lastIndex); //23
reg.test(str);
console.log(reg.lastIndex); //34

```

可以通过设置 lastIndex 的值来控制搜索匹配的位置

```
var str = "wangwei name is wangwei \r\n WANGWEI age is 18 wangwei";
var reg = /wangwei/img;
reg.test(str);
console.log(reg.lastIndex); //7
reg.test(str);
console.log(reg.lastIndex); //23
reg.lastIndex = 0; //从头开始匹配
reg.test(str);
console.log(reg.lastIndex); //7
```

11-5 RegExp 实例方法

`toLocaleString()`和 `toString()`方法，是 `RegExp` 实例继承的方法，其都会返回正则表达式的字面量，与正则表达式的方式无关，如

```
var pattern = new RegExp("\\[bc\\]at","gi");
alert(pattern.toString()); // /\[bc\\]at/gi
alert(pattern.toLocaleString()); // /\[bc\\]at/gi
```

说明：即使是通过 `RegExp` 构造函数创建的，但 `toLocaleString()`和 `toString()`方法仍然会像它以字面量的形式返回；

正则表达式的 `valueOf()`方法返回正则表达式本身；`instanceof` 判断的话是 `RegExp`

```
var pattern = new RegExp("\\[bc\\]at","gi");
alert(pattern.valueOf()); // /\[bc\\]at/gi
```

`RegExp` 对象定义了两个用于执行模式匹配操作的方法；它们的行为与 `String` 类中的与模式相关的方法很类似；

11-5-1 test() 方法

测试目标字符串与某个模式是否匹配，接受一个字符串参数，如果匹配返回 `true`，否则返回 `false`，该方法使用非常方便，一般被用在 `if` 语句主，如：

```
var pattern = /zero/i;
console.log(pattern.test("Zeronetwork")); // true
var text = "000-00-0000";
var pattern = /\d{3}-\d{2}-\d{4}/;
if (pattern.test(text)){
    alert("匹配");
}
```

11-5-2 exec()方法

// 是 `RegExp` 对象最主要的方法，该方法是专门为捕获组而设计的；

// 其会接受一个参数，即要应用模式的字符，会返回第一个匹配项信息的数组（就像 `String` 类的 `match()`方法为非全局检索返回的数组一样）；如果没有匹配成功，返回 `null`；在数组中，第一项是与整个模式匹配的字符串，其他项是与模式中的捕获组匹配的字符串（如果模式中没有捕获组，该数组只包含一项）；

// 返回的虽然为 `Array` 实例，但包含两个额外的属性：`index` 和 `input`；`index` 表示匹配项在字符串中的位置，而 `input` 表示应用正则表达式的字符串；

```

var text = "mom and dad and baby";
var pattern = /mom( and dad( and baby)?)?/gi;
var arr = pattern.exec(text);
console.log(arr); // mom and dad and baby, and dad and baby, and baby
console.log(arr.index); // 0
console.log(arr.input); // mom and dad and baby
console.log(arr[0]); // mom and dad and baby
console.log(arr[1]); // and dad and baby
console.log(arr[2]); // and baby

```

// 对于 `exec()`，不管有没有设置全局标志，其每次也只会返回一个匹配项；
 // 在不设置全局标志情况下，在同一个字符串上多次调用 `exec()` 将始终返回第一个匹配项的信息。
 // 而设置全局标志的情况下，每次调用 `exec()` 则都会在字符串中继续查找新匹配项，会将当前正则对象的 `lastIndex` 属性设置为紧挨着匹配子串的字符位置，为下一次执行 `exec()` 时指定检索位置；如果 `exec()` 没有匹配结果，`lastIndex` 被重置为 0；

```

var text = "cat, bat, sat, fat";
var pattern = /.at/; // 没有使用 g 全局
var pattern = /.at/g; // 使用 g 全局
var matches = pattern.exec(text);
console.log(matches);
console.log("match:"+matches[0]+",index:" + matches.index + ",lastIndex:" +
pattern.lastIndex);
// 再一次执行，使不使用 g，结果不一样
matches = pattern.exec(text);
console.log(matches);
console.log("match:"+matches[0]+",index:" + matches.index + ",lastIndex:" +
pattern.lastIndex);
// 再一次执行，使不使用 g，结果不一样
matches = pattern.exec(text);
console.log(matches);
console.log("match:"+matches[0]+",index:" + matches.index + ",lastIndex:" +
pattern.lastIndex);

```

```

▶ ['cat', index: 0, input: 'cat, bat, sat, fat', groups: undefined]

```

```

match:cat,index:0,lastIndex:3

```

```

▶ ['bat', index: 5, input: 'cat, bat, sat, fat', groups: undefined]

```

```

match:bat,index:5,lastIndex:8

```

```

▶ ['sat', index: 10, input: 'cat, bat, sat, fat', groups: undefined]

```

```

match:sat,index:10,lastIndex:13

```

使用 `test()` 与 `exec()` 是等价的，当 `exec()` 的结果不是 `null`，`test()` 返回 `true` 时，它们都会影响模式的 `lastIndex()` 属性，当然是在全局下；如此，也可以使用 `test()` 遍历字符串，就跟 `exec()` 一样，如

```

var text = "cat, bat, sat, fat";
var pattern = /.at/g;
console.log(pattern.test(text));
console.log(pattern.lastIndex);
console.log(pattern.test(text));
console.log(pattern.lastIndex);
// 一直调用 4 次, test()就会返回 false
// 遍历
var text = "cat, bat, sat, fat";
var pattern = /.at/g;
while(pattern.test(text)){
  console.log(pattern.lastIndex);
}

```

```

true
3
true
8
3
8
13
18

```

// `lastIndex` 属性是可读写的，可以在任何时候设置它，以便定义下一次搜索在目标字符串中的开始位置；`exec()`和`test()`在没有找到匹配项时会自动将 `lastIndex` 设置为 0；如果在一次成功的匹配之后搜索一个新的字符串，一般需要显式地把这个属性设置为 0；

```

var text = "cat, bat, sat, fat";
var pattern = /.at/g;
console.log(pattern.exec(text));
console.log(pattern.lastIndex);
// 匹配新的字符串
var str = "gat,good,pat";
pattern.lastIndex = 0; // 3 如果没有此句，返回 9，即 pat 的位置
console.log(pattern.exec(str));
console.log(pattern.lastIndex);

```

11-6 RegExp 构造函数属性

// `RegExp` 构造函数包括一些属性，这些属性适用于作用域中的所有正则表达式，并且基于所执行的最近一次操作而变化，另外这些属性名都有别名，即可以通过两种方式访问它们，可以称为长属性和短属性；

1. `input` -- `$_` : // 最近一次要匹配的字符串；
2. `lastMatch` -- `$&` : // 最近一次的匹配项；
3. `lastParen` -- `$+` : // 最近一次匹配组；
4. `leftContext` -- `$`` : // `input` 字符串中 `lastMatch` 之前（左边）的文本
5. `rightContext` -- `$'` : // `input` 字符串中 `lastMatch` 之后（右边）的文本
6. `multiline` -- `$*` : // 布尔值，表示是否所有表达式都使用多行模式，部分浏览器未实现

使用这些属性都可以从 `exec()`或 `test()`执行的操作中提取出更具体的信息；

```
var text = "this has been a short summer";
var pattern = /(.)hort/gm;
if(pattern.test(text)){
    console.log(RegExp.input); // this has been a short summer
    console.log(RegExp.lastMatch); // short
    console.log(RegExp.lastParen); // s
    console.log(RegExp.leftContext); // this has been a
    console.log(RegExp.rightContext); // summer
    console.log(RegExp.multiline); // undefined
}
```

由于短属性大都不是有效 JS 标识符，所以必须通过方括号访问；

```
var text = "this has been a short summer";
var pattern = /(.)hort/gm;
if(pattern.test(text)){
    console.log(RegExp.$_); // this has been a short summer
    console.log(RegExp["$&"]); // short
    console.log(RegExp["$+"]); // s
    console.log(RegExp["$`"]); // this has been a
    console.log(RegExp["$'"]); // summer
    console.log(RegExp["$*"]); // false
}
```

// 除了以上，还有 9 个用于存储捕获组的构造函数属性，语法为： `RegExp.$1` . `RegExp.$2`...`RegExp.$9`，分别用于匹配第一，第二...第九个捕获组，在调用 `exec()`或 `test()`方法时，这些属性会被自动填充；

```
var text = "this has been a short summer";
var pattern = /(.)or(.)g;
if(pattern.test(text)){
    console.log(RegExp.$1); // sh
    console.log(RegExp.$2); // t
    console.log(RegExp.$3); // ""
}
```

说明：包含了两个捕获组；本质上就是反向引用。

11-7 字符串属性

7-1 search()方法

接受的参数就一个正则表达式，其返回字符串中第一个匹配项的索引；如果没有找到匹配项，则返回-1；如：

```
var text = "cat, bat, sat, fat";
// 以下的.at 可以改成 at
var pattern = /.at/; // 0
var pos = text.search(/.at/); // 0
var pos = text.search(/.at/g); // 0
console.log(pos);
```



```
// search()不支持全局检索，它会忽略正则表达式参数中的修饰符 g；也会忽略 RegExp 的 lastIndex 属性，总是从String 的开始位置搜索，即它总是返回 String 中第一个匹配子串的位置；
// 如果 search()参数不是正则表达式，则首先会通过 RegExp 构造函数将它转换成正则表达式；
```

```
var text = "cat, bat, sat, fat";
var pos = text.search('at'); // 1
console.log(pos);
```

注：也可以理解为就是查找普通的字符串的索引位置；

7-2 match()方法

本质上与 RegExp 的 exec()方法相同；其只接受一个参数，一个正则表达式或是一个 RegExp 对象，如果参数不是 RegExp，则会被 RegExp()转换为 RegExp 对象，如

```
var text = "cat, bat, sat, fat";
var pattern = /.at/; //[ "cat", index: 0, input: "cat, bat, sat, fat", ...]
var pattern = ".at"; //会被转换成 RegExp 对象
var pattern = /.at/g; //[ "cat", "bat", "sat", "fat"]
var matches = text.match(pattern);
console.log(matches);
```

返回一个包含匹配结果的数组，如果没有匹配结果，返回 null；

```
// 如果没有使用修饰符 g，match()不会进行全局检索，它只会检索第一个匹配，但其依然会返回一个数组，此时，数组唯一的一项就是匹配的字符串；
```

```
// 如果使用了分组，第一项是匹配的整个字符串，之后的每一项（如果有）保存着与正则表达式中的捕获组匹配的字符串；
```

```
var str = "zeronetwork is good zeronetwork";
var re1 = /zero(net(work))/;
var re2 = /zero(net(work))/g;
var result1 = str.match(re1);
var result2 = str.match(re2);
console.log(result1);
console.log(result2);
```

```
▶ (3) ['zeronetwork', 'network', 'work', index: 0, input: 'zeronetwork is good zeronetwork', groups: d] biaogeain
▶ (2) ['zeronetwork', 'zeronetwork'] biaogeain
```

全局与非全局返回的数组不一样

如果使用了全局检索，数组会保存所有匹配结果；而不会显示分组里面匹配的结果，数组没有 input，index属性

```
// match()方法如果使用了一个非全局的正则，实际上和给 RegExp 的 exec()方法传入的字符串是一样的，它返回的数组带有两个属性：index 和 input，index 指明了匹配文本在 String 中的开始位置，input 则是对该 String本身的引用。
```

```
// 匹配 URL
var str = "访问零点网络官网：https://www.zeronetwork.cn/index.html";
var re = /(w+):\/\/([w.]+)\.([s*])/;
```

```

var result = str.match(re);
console.log(result);
var o={};
if(result != null){
o.fullurl = result[0];
o.protocol = result[1];
o.host = result[2];
o.path = result[3];
}
for(var p in o)
console.log(p + ":" + o[p]);

```

```

(4) ['https://www.zeronetwork.cn/index.html', 'https', 'www.zeronetwork.cn', 'index.html', i
访问零点网络官网: https://www.zeronetwork.cn/index.html', groups: undefined]
fullurl:https://www.zeronetwork.cn/index.html
protocol:https
host:www.zeronetwork.cn
path:index.html

```

如果使用了全局，返回的是每个匹配的子串数组，没有 index 和 input 属性；如果希望在全局搜索时取得这此信息，可以使用 `RegExp.exec()`方法；

7-3 replace()方法

// 该方法接受两个参数，第一个参数可以是一个 **RegExp** 对象或一个字符串（这个字符串不会被转换成正则表达式），第二个参数是要替换的字符串，可以是一个字符串或一个函数；如果第一个参数是字符串，那么只会替换第一个子字符串，要想替换所有子字符串，唯一的办法就是提供一个正则表达式，而且要指定全局 **g** 标志，如

```

var text = "cat, bat, sat, fat";
console.log(text.replace("at","ond")); // cond, bat, sat, fat
console.log(text.replace(/at/g,"ond")); // cond,

```

// 在第二个参数中使用捕获组：使用\$加索引数字，`replace()`将用与指定的子表达相匹配的文本来替换字符；这是一个非常实用的特性，如

```

var str = "cat, bat, sat, fat";
console.log(str.replace(/(at)/g,"$1er")); //cater, bater, sater, fater
// 将英文引号替换为中文引号
var str = 'zero"net"work';
var quote = /"([\^"]*)"/g;
console.log(str.replace(quote,"“$1”"));

```

如果第二个参数是字符串，还可以使用一些特殊的字符序列，将正则表达式操作得到的值插入到结果字符串中，如：

`$$` : // 美元符号
`&` : // 匹配整个模式的子字符串, 与 `RegExp.lastMatch` 的值相同
``` : // 匹配的子字符串之前 (左边) 的子字符串, 与 `RegExp.leftContext` 的值相同  
`'` : // 匹配的子字符串之后 (右边) 的子字符串, 与 `RegExp.rightContext` 的值相同  
`$n` : // 匹配第 `n` 个捕获组的子字符串, 其中 `n` 等于 `0-9`, 如, `$1` 是匹配第一个捕获组的子字符串, `$2` 是第二个捕获组的子字符串, 以此类推; 如果正则表达式中没有定义捕获组, 则使用空字符串  
`$nn` : // 匹配第 `nn` 个捕获组的子字符串, 其中 `nn` 等于 `00-99`, 如`$01` 是匹配第一个捕获组的子字符串, `$02` 是匹配第二个捕获组的子字符串, 以此类推; 如果正则表达式中没有定义捕获组, 则使用空字符串

通过这些特殊的字符序列, 可以使用最近一次匹配结果中的内容, 如:

```
var text = "my name is wangwei zero";
var re = /(\\w+)g(\\w+)/g;
console.log(text.replace(re,"$$"));
console.log(text.replace(re,"&"));
console.log(text.replace(re,"$`"));
console.log(text.replace(re,"$'"));
console.log(text.replace(re,"$1"));
console.log(text.replace(re,"$01"));
var text = "cat, bat, sat, fat";
var result = text.replace(/(.at)/g,"word ($1)");
console.log(result); // word (cat), word (bat), word (sat), word (fat)
// 把名字格式颠倒
var name = "Wei, wang";
console.log(name.replace(/(\\w+)\\s*,\\s*(\\w+)/, "$2 $1")); // Wang Wei
```

// `replace()`方法的第二个参数也可以是一个函数: 该回调函数将在每个匹配结果上调用, 其返回的字符串则将作为替换文本;

// 该函数最多可传递 3 个参数: 模式的匹配项、模式匹配项在字符串中的位置和原始字符串;

// 在正则表达式中定义了多个捕获组的情况下, 传递给函数的参数依次是模式的匹配项、第一个捕获组的匹配项、第二个捕获组的匹配项....., 但最后两个参数仍然分别是模式的匹配项在字符串中的位置和原始字符串; 此种方式, 可以实现更加精细、动态的替换操作, 如:

```
var str = "zero net work";
var result = str.replace(/\\b\\w+\\b/g, function(match,pos,text){
return 'match:'+match + ',pos:'+pos + ',text:'+text + '\\n'
// word.substring(0,1).toUpperCase() + word.substring(1);
});
console.log(result);
```

```
match:zero,pos:0,text:zero net work
match:net,pos:5,text:zero net work
match:work,pos:9,text:zero net work
```

```
// 所有单词的首字母大写
var str = "zero net work";
var result = str.replace(/\\b\\w+\\b/g, function(word){
return word.substring(0,1).toUpperCase() + word.substring(1);
});
console.log(result); // Zero Net Work
```

// 为 HTML 字符转换为实体

```
function htmlEscape(text){
return text.replace(/["<>"]/g, function(match, pos, originalText){
 switch(match){
 case "<":
 return "<";
 case ">":
 return ">";
 case "&":
 return "&";
 case """:
 return """;
 case "\'":
 return "'";
 }
});
}

var text = "<p class=\"greeting\">zero network!</p>";
console.log(text);
console.log(htmlEscape(text)); // <p class="greeting">zero network!</p>
document.write(text);
document.write(htmlEscape(text));
```

<p class="greeting">zero network!</p>

[bi.](#)

&lt;p class=&amp;greeting&amp;gt;zero network!&lt;/p&gt;

[bi.](#)

zero network!

<p class=&greeting&gt;zero network!</p>

还可以在 replace 中回调函数中使用捕获组:

```
var str = "aabb";
var reg = /(\w)\1(\w)\2/g;
console.log(str.replace(reg, function($, $1, $2){
//return $ + ", " + $1 + ", " + $2; aabb a b
return "提取了两个字母: "+"$1+"和"+"$2;
}));
// 替换成驼峰写法
```

提取了两个字母: a和b

```
var str = "zero-net-work";
var reg = /-(\w)/g;
console.log(str.replace(reg, function($, $1){
return $1.toUpperCase();
}));
```

zeroNetwork

注: 与 exec()和 test()不同, String 类的方法 search()、replace()和 match()并不会用到 lastIndex 属性; 实际上, String 类的这些方法只是简单的将 lastIndex 属性值重置为 0;

## 7-4 split 方法

可以基于指定的分隔符将一个字符串分割成多个子字符串，并将结果放在一个数组中；分隔符可以是字符串，也可以是 RegExp 对象；其可以接受可选的第二个参数，用于指定数组的大小，以便确保返回的数组不会超过既定大小，如：

```
console.log("1, 2, 3, 4, 5".split(/\s*,\s*/)); // ["1", "2", "3", "4", "5"]
var colorText = "red,blue,green,yellow";
var colors = colorText.split(","); // red,blue,green,yellow
var colors = colorText.split(",",2); // red,blue
var colors = colorText.split(/[^\,]+/); // ["", ",", ",", ",", ",", "", ""]
console.log(colors);
```

## 11-8 ES 正则的局限性

尽管 ECMAScript 的正则表达式的功能比较完备，但仍缺少某些语言所支持的高级正则表达式特性，如下：

```
// 匹配字符串开始和结尾的\A 和\Z 锚（但支持^和$）；
// 向后查找（lookbehind）（但完全支持向前查找 lookahead）；
// 并集和交集类；
// 原子组（atomic grouping）；
// unicode 支持（单个字符除外，如\rFFFF）；
// 命名的捕获组（但支持编号的捕获组）；
// s(single，单行)和 x(free-spacing，无间隔)匹配模式；
// 条件匹配；
// 正则表达式注释；
```

即使存在这些限制，ECMAScript 正则表达式仍然是非常强大的，能够完成绝大多数模式匹配任务。

## 12，变量，作用域，预编译

### 12-1 变量的作用域

```
// 变量的作用域是指一个变量在哪个范围内可以使用，可以分为两种：
```

全局变量：// 在所有函数之外定义的变量，其作用范围是整个变量定义之后的所有语句，包括其后定义的函数及其后的<script>中的代码；

局部变量：// 定义在函数之内的变量，只有在该函数中才可使用；

```
// 如果函数中定义了与全局变量同名的局部变量，会覆盖全局变量；
```

```
var msg = "这是全局变量的值";
function show(){
 var str = "局部变量";
 console.log(msg);
 console.log(str);
}
show();
console.log(str); // Error str is not defined
```

如果函数中使用隐式声明变量，即没有使用var声明，则该变量自动变成全局变量；使用var声明的变量会自动被添加到最接近的环境中；在函数内部，最接近的环境就是函数的局部环境；如：

```
function add(num1,num2){
var sum = num1+num2;
// sum = num1+num2;
 return sum;
}
var result = add(10,20);
alert(sum); // 由于sum不是有效的变量，因此会导致错误
```

// 注：在JavaScript中，不声明而直接初始化变量是一个常见的错误做法，因为这样可能会导致意外；建议在初始化变量之前，一定要先声明；并且，在严格模式下，初始化未经声明的变量会导致错误；

// ES的变量与其他语言的变量有很大区别：ES变量是松散类型的，这个特点决定了它只是在特定时间用于保存特定值的一个名字而已；由于不存在定义某个变量必须要保存何种数据类型值的规则，所以，变量的值及其数据类型可以在脚本的生命周期内可以被改变；尽管从某种角度看，这可能是一个灵活强大的特性，但同时也是容易出问题的特性；

// 在实际应用中，ES变量还是比较复杂的；比如：函数的参数，由于参数的数据类型不一致，导致的结果也不致；嵌套函数：

// 也称为私有函数：是指处于局部作用域中的函数；当函数嵌套定义时，子级函数就是父级函数的私有函数；外界不能调用私有函数，私有函数只能被拥有该函数的函数代码调用；子级函数可以使用父级函数定义的变量，父级函数不能使用子级函数定义的变量；其他函数不能直接访问子级函数，如此，就实现了信息的隐藏；如：

```
function funA(){
 var strA = "funA定义的变量strA";
 funB();
 function funB(){
 var strB = "funB定义的变量strB";
 console.log(strA);
 console.log(strB);
 }
}
funA();
```

## 12-2 预编译

// ES是一种具有函数优先的轻量级解释型或即时编译型的编程语言，其可以不经过编译而直接运行，但是ES存在一个预编译的机制，这也是Java等一些语言中没有的特性，也就正是因为这个预编译的机制，导致了ES中变量提升的一些问题；

// JavaScript运行三部曲：

// 脚本执行期间JS引擎按照以下步骤进行处理：

- 1. 语法分析；
- 2. 预编译；
- 3. 解释执行；

// 即在执行代码前，还需要两个步骤：

// 语法分析，就是引擎检查你的代码有没有什么低级的语法错误；

// 解释执行：就是执行代码；

// 预编译简单理解就是在内存中开辟一些空间，存放一些变量与函数；

// JS预编译发生时刻：

// 预编译是在脚本执行前就发生了，更确切的说是在函数执行前发生的，也就是说函数执行时，预编译已经结束；

## 12-2-1 预编译前奏

mply global暗示全局变量：任何变量，如果未经声明就赋值，这些变量就为全局对象(window)所有(即为window对象的属性);一切声明的全局变量，也是window所有；如：

```
var a = 123;
window.a = 123;
function test(){
 // 这里的b是未经声明的变量，所以是归window所有的；连等的操作也视为无var
 var a = b = 110;
}
```

## 12-2-2 变量声明提升

在JavaScript函数里的所有声明(只是声明，不涉及赋值)都被提前到函数体的顶部，预编译时并不会对变量进行赋值(即不会进行初始化),变量赋值是在脚本执行阶段进行的；如：

```
console.log('before:' + a); // before:undefined
var a = 1;
console.log('after:' + a); // after:1
```

## 12-2-3 函数声明整体提升

函数声明语句将会被提升到外部脚本或者外部函数作用域的顶部，如：

```
a(); // function
console.log(a); // f a(){...}
function a(){
 console.log("function");
}
console.log(a);
a();
```

在预编译时，function的优先级比var高，如：

```
// var a=1; // 异常，会导致下行的a()异常
a();
var a=1;
function a(){console.log("function");}
var a;
console.log(typeof a);
```

此时a的类型是function,而不是number;

函数表达式用的是变量，函数并不会提升：

```
b();// b is not a function
var b = function(){
 console.log('function b');
};
b();
```

声明同名的函数会覆盖掉之前声明的函数：

```
function c(){
 console.log('function c1');
}
c(); // function c2
function c(){
 console.log('function c2');
}
```

要理解预编译，只要弄清两点：变量/函数声明与变量赋值；在预编译阶段，只进行变量/函数声明，不会进行变量的初始化(即变量赋值，所有变量的值都是undefined);变量赋值是在执行阶段才进行的；

## 12-3 预编译步骤

// 首先JavaScript的执行过程会先扫描一下整体语法语句，如果存在逻辑错误或者语法错误，那么直接报错，程序停止执行，没有错误的话，开始从上到下解释一行执行一行。

执行器上下文，// 英文名Activation Object,简称AO,也称为活动对象；

全局对象，// 英文名Global Object,简称GO；

// 函数执行前会进行预编译，产生AO；

// 全局变量在执行前也会有预编译，产生GO；

// 局部预编译的4个步骤：

创建AO；

找形参和变量声明，将变量和形参名作为AO属性名，值为undefined

将实参值和形参统一；

在函数体里面找函数声明，值赋予函数体；

// 由于全局中没有参数的概念，所以省去了实参形参相统一这一步；

// 注：GO对象是全局预编译，所以它优先于AO对象所创建和执行；

**AO对象示例：**

```
function fn(a) {
 console.log(a); // f a(){}
 // 变量声明+变量赋值，但只提升变量声明，不提升变量赋值
 var a = 123;
 console.log(a); // 123
 // 函数声明
 function a() {}
 console.log(a); // 123
 // 函数表达式
 var b = function() {}
 console.log(b); // f (){}
 // 函数
 function c() {}
}
fn(1); // 调用
```

在进行完预编译后，执行函数则会以AO为基础对函数中的变量进行赋值，函数执行完毕，销毁AO对象。

**GO对象的示例：**



```
global = 100;
function test() {
 console.log(global); // undefined
 var global = 200;
 console.log(global); // 200
 var global = 300;
}
test();
var global;
```

// 注：关于GO对象和AO对象，它们俩是一个种链式关系，如上例，如果在函数体的内部没有定义global变量，这也意味着AO对象中将有这个global这个属性；如果没有，会去GO对象中寻找，即是就近原则；

// 另外需要注意的是JS不是全文编译完成再执行，而是块编译，即一个script块中预编译然后执行，再按顺序预编译下一个script块再执行，但是此时上一个script块中的数据都是可用的了，而下一个块中的函数和变量则是不可用的。

## 12-4 执行环境

// 执行环境(execution context)是ES中最为重要的一个概念，也称为执行上下文；

// 执行环境定义了变量或函数有权访问的其他数据，决定了它们各自的行为；每个执行环境都有一个与之关联的变量对象，环境中定义的所有变量和函数都保存在这个对象中；但无法访问这个对象，只有解析器在处理数据时会在后台使用它；

// 执行环境中有个全局执行环境的概念；

// 全局执行环境是最外围的一个执行环境；根据ES实现所在的宿主环境不同，表示执行环境的对象也不一样；在web浏览器中，全局执行环境被认为是window对象，因此所有全局变量和函数都是作为window对象的属性和方法创建的；某个执行环境中的所有代码执行完毕后，该环境被销毁，保存在其中的所有变量和函数定义也随之销毁。每个函数都有自己的执行环境；当执行流进入一个函数时，函数的环境就会被推入一个环境栈中；而在函数执行之后，栈将其环境弹出，把控制权返回给之前的执行环境；ES程序中的执行流就是由这个的机制控制着；

## 12-5 作用域链

// 当代码在一个环境中执行时，会创建变量对象的一个作用域链(scope chain)；作用域链的用途，能够保证对执行环境有权访问的所有变量和函数的有序访问；作用域的前端，始终都是当前执行的代码所在环境的变量对象；如果这个环境是函数，则将其活动对象作为变量对象；活动对象在最开始时只包含一个对象，即arguments对象；作用域链中的下一个变量对象来自包含(外部)环境，而再下一个变量对象则来自下一个包含环境；这样，一直延续到全局执行环境；全局执行环境的变量对象始终都是作用域中的最后一个对象；

```
function f(){
 var a = 10;
 function b(){};
}
var g = 100;
f();
```

// 查询标识符(在作用域链中查找):

// 当在某个环境中为了读取或写入而引用一个标识符时, 必须通过搜索作用域链来确定. 该标识符实际代表什么; 搜索过程从作用域链的前端开始, 向上逐级查询; 如果在局部环境中找到了该标识符, 搜索过程停止, 变量就绪; 如果在局部环境中没有找到该变量, 则继续沿作用域链向上搜索; 搜索过程一直追溯到全局环境的变量对象; 如果在全局环境中也没有找到这个标识符, 则意味着该变量尚未声明, 从而会导致错误发生, 如:

```
var color="blue";
function getColor(){
 // var color = "red";
 return color;
}
console.log(getColor());
```

// 内部环境可以通过作用域链访问所有的外部环境, 但外部环境不能访问内部环境中的任何变量和函数; 这些环境之间的联系是线性的、有次序的; 每个环境都可以向上搜索作用域链, 以查询变量和函数名; 但任何环境都不能通过向下搜索作用域链而进入另一个执行环境, 如:

```
var color="blue";
function changeColor(){
 var anotherColor="red";
 function swapColors(){
 // 这里可以访问color、anotherColor和tempColor
 var tempColor = anotherColor;
 anotherColor = color;
 color = tempColor;
 }
 swapColors(); // 这里可以访问color和anotherColor, 但不能访问tempColor
}
changeColor(); // 这里只能访问color
console.log("color is " + color); // red
```

**注: 函数参数也被当作变量来对待, 因此其访问规则与执行环境中的其他变量相同;**

## 12-6 延长作用域链

// 虽然执行环境的类型总共只有两种: 全局和局部(函数), 但还是有其他办法来延长作用域链; 因为有些语句可以在作用域链的前端临时增加一个变量对象, 该变量对象在代码执行时, 作用域就会加长, 在代码执行后被移除:

// try-catch语句的catch块及with语句; 这两个语句都会作用域链的前端添加一个变量对象; 对with语句来说, 会将指定的对象添加到作用域链中; 对catch语句来说, 会创建一个新的变量对象, 其中包含的是被抛出的错误对象的声明; 如:

```
function buildurl(){
 var qs = "?name=wangwei";
 with(location){
 var url = href+qs;
 }
 return url;
}
console.log(buildurl());
```

## 12-7 作用域中的this对象

// this引用的是函数执行的环境对象，即是调用函数的对象，或者也可以说是this值(当在网页的全局作用域中调用函数时，this对象引用的就是window)。

```
window.color = "red";
var o = {color:"blue"};
function sayColor(){
 console.log(this.color);
}
sayColor(); // red
o.sayColor = sayColor;
o.sayColor(); // blue
```

## 12-8 没有块级作用域

ES没有块级作用域；在其他类C的语言中，由花括号封闭的代码块都有自己的作用域(如果用ES的角度来讲，就是它们自己的执行环境),因而支持根据条件来定义变量，如，下面的代码在ES并不会得到想象中的结果：

```
if(true){
 var color="blue";
}
console.log(color); // blue;
```

在ES中，if语句中的变量声明会将变量添加到当前的执行环境(在这里是全局环境)中；在使用for语句时表现的最为明显，如：

```
function outputNumber(count){
 for(var i = 0; i<count; i++){
 console.log(i);
 }
 // var i;
 console.log("最后的值: " + i) // 5
}
outputNumber(5);
```

可以模拟块级作用域，使用立即执行函数进行模拟；

## 12-9 立即执行函数

立即执行函数也称为自运行函数，其没有声明，本质上就是匿名函数；其在一次执行后立即释放；

适合做一些初始化的工作或者模拟块级作用域；

```
(function(){
 console.log("这里是立即执行函数");
})();
```

立即执行函数不允许使用函数声明方式，但是如果在function前加一个+号即可，同时在控制台中，该函数名也会被忽略，如：

```
+function myFun(){
 console.log("这里是立即执行函数");
}();
```

在function前加上+、!、一、~等一元操作符，也是立即执行函数的写法，等同上面的立即执行函数，如果没有这些符号，解析器会把function认为为一个函数声明；

同理，只要在function前加上其他的表达式语句，都可以，如：

```
true && function myFun(){
 console.log("这里是立即执行函数");
}();
// 或
0,function(){
 console.log("ok");
}();
```

立即执行函数可以传值，也可以有返回值，如：

```
// 传值
(function(x,y,z){
 console.log(x+y+z)
})(1,2,3); // 6
// 返回值
var result = (function(x,y,z){
 var sum = x+y+z;
 return sum;
})(1,2,3);
console.log(result); // 6//传值
(function(x,y,z){
 console.log(x+y+z)
})(1,2,3);// 6
/返回值
var result =(function(x,y,z){
 var sum =x+y+z;
 return sum;
})(1,2,3);
console.log(result);// 6
```

## 特例

```
function myFun(){
 console.log("这里是立即执行函数");
}(1,2,3);
// 此时，不会报错，函数也存在，但不会立即执行，原因是解析器会把它拆分成两条语句；如：
function myFun(){console.log("这里是立即执行函数");};
(1,2,3);
```

// 这种技术经常在全局作用域中被用在函数外部，从而限制向全局作用域中添加过多的变量和函数；

// 一般来说，应该尽量少向全局作用域中添加变量和函数；在一个由很多开发人员共同参与的大型应用程序中，过多的全局变量和函数很容易导致命名冲突，而通过创建私有作用域，每个开发人员都可以使用自己的变量，而不必担心搞乱全局作用域；

```
(function(){
 var now = new Date();
 if(now.getMonth() == 0 && now.getDate() == 1){
 console.log("Happy new Year");
 }
})();
```

说明：变量now是匿名函数的局部变量；

立即执行函数也是后面要讲的闭包的基本形式，但闭包有个问题，就是内存占用的问题，此种方法可以减少闭包占用的内存问题，因为没有指向匿名函数的引用，只要函数执行完毕，就立即销毁其作用域链了；

**思考两个小示例：**

```
var foo = (
 function f(){return "1";},
 function g(){return 2;}
)();
console.log(typeof foo); // number

var x = 1;
// (function fun(){})) 因为在括号中，所以是一个表达式，
// 运行完后就消失了，所以typeof fun就是undefined
if(function fun(){}){
 x += typeof fun;
 console.log(typeof fun); // undefined
}
console.log(x); // 1undefined
```

## 13，闭包

### 13-1 闭包

闭包是指有权访问另一个函数作用域中的变量的函数；其本质就是在一个函数内部创建另一个内部函数；并且此内部函数被暴露在外包，其会导致原有作用域链不能被释放，造成内存泄露；

其有3个特性

- 函数嵌套函数；
- 读取函数内部变量；
- 持久性，即让局部变量始终保存在内存中；

**读取函数内部变量：**

```
function a(){
 var name = "wangwei";
 return function(){
 return name;
 }
}
var b = a();
console.log(b());
```

些变量的值始终保持在内存中：

其表现就是在一个函数内返回一个函数；即内部函数访问了外部函数的变量，通过把返回的函数赋值给一个外部全局变量，即使外部函数执行完毕，但其内部函数还一直

存在，并且其访问的外部函数中的变量也一直存在；

```
function a(){
 var i = 0;
 function b(){
 console.log(++i);
 }
 return b;
}
var c = a();
c(); // 1
c(); // 2
c(); // 3
// 或
function f1(){
 var n = 999;
 nAdd = function(){
 n++;
 };
 function f2(){
 console.log(n);
 }
 return f2;
}
var result = f1();
result(); // 999
nAdd();
result(); // 1000
```

会产生内存消耗的问题：

```
function fn(){
 var n = 2;
 return function(){
 var m = 0;
 return "n:" + (++n) + ",m:" + (++m);
 }
}
var f = fn();
console.log(f()); // n:3,m:1
console.log(f()); // n:4,m:1
```

经典示例：定时器与闭包：

```
for(var i=0;i<5;i++){
 setTimeout(function(){
 console.log(i + " ");
 },500);
} // 5个5
修改：
```

使用闭包来保存变量*i*，将*setTimeout*放入立即执行函数中，将*for*循环中的循环值*i*作为参数传递；但此时，是500毫秒后，同时打印出1-5；如何实现每隔500毫秒依次输出1-4？修改：

```

for(var i=0;i<5;i++){
 (function(i){
 setTimeout(function(){
 console.log(i + " ");
 },i*500);
 })(i);
}

```

## 13-2 闭包的作用：

```

// 保护函数内的变量，实现封装，防止变量流入其他环境发生命名冲突；
// 在内存中维持一个变量，可以做缓存；
// 实现公有变量，如函数累加器；

```

```

// 累加器
function add(){
 var count = 0;
 function done(){
 count++;
 console.log(count);
 }
 return done;
}
var counter = add();
counter(); // 1
counter(); // 2
counter(); // 3

function func(){
 var num = 100;
 function a(){
 num++;
 console.log(num);
 }
 function b(){
 num--;
 console.log(num);
 }
 return [a,b];
}
var arr = func();
arr[0](); // 101
arr[1](); // 100

// 封装，私有变量
function playing(){
 var item = "";
 var obj = {
 play: function(){
 console.log("playing:" + item);
 item = "";
 },
 push: function(myItem){
 item = myItem;
 }
 }
}

```

```

 };
 return obj;
}
var player = playing();
player.push('football');
player.play(); // playing:football

```

### 13-3 闭包的副作用：

闭包只能取得包含函数中任何变量的最后一个值，所以多次调用，只能取相同的一个值；

```

function createFun(){
 var result = new Array();
 for(var i = 0; i<10; i++){
 result[i] = function(){
 return i;
 }
 }
 return result;
}
var arr = createFun();
for(var i=0;i<arr.length;i++){
 console.log(arr[i]());
}

```

可以使用立即执行函数强制让闭包的行为符合预期；

```

function createFun(){
 var result = new Array();
 for(var i = 0; i<10; i++){
 result[i] = function(num){
 return function(){
 return num;
 }
 }(i);
 }
 return result;
}
var arr = createFun();
for(var i=0;i<arr.length;i++){
 console.log(arr[i]());
}

```

一个经典的应用，在若干个DOM对象绑定事件，分别输出不同的内容，如：

```

window.onload = function(){
 var ul = document.getElementsByTagName('ul')[0];
 var lis = ul.getElementsByTagName('li');
 for(var i=0; i<lis.length; i++){
 lis[i].addEventListener('click',function(e){
 // console.log(this.innerText); // 不同
 console.log(i); // 全是 4
 },false);
 }
}

```



```
// 改成:
window.onload = function(){
 var ul = document.getElementsByTagName('ul')[0];
 var lis = ul.getElementsByTagName('li');
 for(var i=0; i<lis.length; i++){
 (function(j){
 lis[j].addEventListener('click',function(e){
 console.log(j); // 达到预期, 值不同
 },false)
 })(i);
 }
}
```

## 13-4 闭包中的this

// 在闭包中使用**this**对象也可能导致一些问题：**this**对象是在运行时基于函数的执行环境绑定的，即**this**对象本身就指调用函数的对象；在全局环境中，**this**对象通常指向**window**，而当函数被作为某个对象的方法调用时，**this**就等于那个对象；不过，匿名函数的执行环境具有全局性，因此**this**对象通常指向**window**，但有时候，由于编写闭包的方式不同，这一点可能不会那么明显；

```
var name = "The window";
// var object = {
// name: 'My object',
// getNameFunc: function(){
// return function(){
// return this.name;
// }
// }
// };
// 把object改成:
var object = {
 name: 'My object',
 getNameFunc: function(){
 var that = this;
 return function(){
 return that.name;
 }
 }
}
console.log(object.getNameFunc()()); // The window或myobject
```

**arguments**存在着同样的问题；如果想访问作用域中的**arguments**对象，必须将该对象的引用保存到另一个闭包能够访问的变量中；有几种特殊情况下，**this**的值可能会意外的改变，如：

```
var name = "The window";
var object = {
 name: "My Object",
 getName: function(){
 return this.name;
 }
};
alert(object.getName()); // My Object
alert((object.getName)()); // My Object
alert((object.getName=object.getName)()); // The window
```

## 13-5 闭包中的内存泄露

由于闭包会携带包含它的函数的作用域，即使函数内变量被保存在内存中，所以内存消耗很大；因此在退出函数前，将不用的变量删除；

```
function handler(){
 var element = document.getElementById("someElement");
 element.onclick = function(){
 console.log(element.id);
 }
}
// 改成
function handler(){
 var element = document.getElementById("someElement");
 var id = element.id;
 element.onclick = function(){
 console.log(id);
 };
 element = null;
}
```

## 13-6 闭包中的作用域链

```
function createComparison(propertyName){
 return function(object1,object2){
 var value1 = object1[propertyName];
 var value2 = object2[propertyName];
 if(value1<value2){
 return -1;
 }else if(value1>value2){
 return 1;
 }else{
 return 0;
 }
 }
}
```

当某个函数被调用时，会创建一个执行环境及相应的作用域链；然后，使用arguments和其他命名参数的值来初始化函数的活动对象；但在作用域链中，外部函

数的活动对象始终处于第二位，外部函数的函数的活动对象处于第三位...直到作为作用域链终点的全局执行环境；

在函数执行过程中，为读取和写入变量的值，就需要在作用域中查找变量，如：

```
function compare(value1,value2){
 if(value1<value2){
 return -1;
 }else if(value1>value2){
 return 1;
 }else{
 return 0;
 }
}
var result = compare(3,7);
```

在另一个函数内部定义的函数会将包含函数的活动对象添加到它的作用域链中；因此，在 createComparison() 函数内部定义的匿名函数的作用域中，实际上将会包含外部函数 createComparison() 的活动对象；

```
var compare = createCompare("name");
var result = compare({name:"Nicholas"},{name:"Greg"});
compare=null;
```

说明：解除对匿名函数的引用，以便释放内容，即通知垃圾回收将其清除；

## 13-7 闭包的几个应用

通常在使用只有一个方法的对象的地方，都可以使用闭包；在实际场景中，这种情况特别常见，比如，有很多代码都是基于事件的：定义某种行为，然后将其添加到用户触发的事件之上，通常称为回调，这个回调就是为响应事件而执行的函数，它们其实绝大部分都是闭包；

```
<style>
body{font-size: 14px;} h1{font-size: 1.5em;} h2{font-size: 1.2em;}
p{font-size: 1em;}
</style>
<h1>Web前端开发</h1>
<h2>JavaScript</h2>
<p>零点程序员</p>
<p>14 16
 18</p>
<script>
function makeSize(size){
 return function(){
 document.body.style.fontSize = size + 'px';
 };
}
var size14 = makeSize(14);
var size16 = makeSize(16);
var size18 = makeSize(18);
document.getElementById('size-14').onclick = size14;
document.getElementById('size-16').onclick = size16;
document.getElementById('size-18').onclick = size18;
</script>
```

### 13-7-1 函数工厂

```
function add(x){
 return function(y){
 return x + y;
 }
}
var add5 = add(5);
var add10 = add(10);
console.log(add5(3)); // 8
console.log(add10(8)); // 18
console.log(add(1)(2)); // 3
```

## 13-7-2 表单控件的提示

```
<p id="tips">有关提示</p>
<p>邮箱: <input type="text" id="email" /></p>
<p>用户名: <input type="text" id="name" /></p>
<p>年龄: <input type="text" id="age" /></p>
<script>
function showTip(tip){
 document.getElementById('tips').innerHTML = tip;
}
function makeTip(tip){
 return function(){
 showTip(tip);
 };
}
function setupTips(){
 var tipText = [
 {'id':'email', 'tip': '邮箱地址'},
 {'id':'name', 'tip': '你的用户名'},
 {'id':'age', 'tip': '你的年龄'},
];
 for(var i=0;i<tipText.length; i++){
 var item = tipText[i];
 document.getElementById(item.id).onfocus = makeTip(item.tip);
 }
}
setupTips();
</script>
```

## 13-7-3 使用匿名闭包

```
for(var i=0;i<tipText.length; i++){
 (function(){
 var item = tipText[i];
 document.getElementById(item.id).onfocus = function(){
 showTip(item.tip);
 };
 })();
}
```

### 用ES6的let关键词

还可以使用forEach遍历数组，为每个元素添加一个监听器

```
tipText.forEach(function(item){
 document.getElementById(item.id).onfocus = function(){
 showTip(item.tip);
 }
});
```

## 14, 函数, 作用域, 垃圾回收

## 14-1 私有变量

严格来讲，Javascript中没有私有成员的概念；所有对象属性都是公有的，但是有一个私有变量的概念；任何在函数中定义的变量，都可能被认为是私有变量，因为不能在函数的外部访问这些变量；

私有变量包括函数的参数、局部变量和在函数内部定义的其他函数；如：

```
function add(num1,num2){
 var sum = num1+num2;
 return sum;
}
```

可以把有权访问私有变量和私有函数的公有的方法称为特权方法(privileged method),利用私有和特权成员，可以隐藏那些不应该被直接修改的数据；有两种在对象上创建特权方法的方式；第一种是在构造函数中定义特权方法，如：

```
function Person(name,age){
 // 此处不使用this的原因，是想隐藏内部数据
 // this.myName = name;
 // this.myAge = age;
 var myName = name;
 var myAge = age;
 this.getName = function(){
 return myName;
 };
 this.setName = function(value){
 myName = value;
 };
 this.getAge = function(){
 return myAge;
 };
 this.setAge = function(value){
 myAge = value;
 };
}

var person = new Person("wangwei",18);
console.log(person.getName());
person.setName("WuJing");
console.log(person.getName());
person.setAge(person.getAge()+1);
console.log(person.getAge());
```

这种方式，因为每次调用构造函数都会重新创建其中的所有方法，这显然不是必须的，也是一个缺点，使用静态私有变量来实现特权方法就可以避免这个问题；

## 14-2 静态私有变量

通过在私有作用域中定义私有变量或函数，同样可以创建特权方法；

这个模式与在构造函数中定义特权方法的主要区别，就是在于构造函数中的私有变量和函数是由实例共享的；而特权方法是在原型上定义的，因此所有实例都使用同一个函数；而这个特权方法，作为一个闭包，总是保存着对包含作用域的引用，如：

```
(function(){
 var site, domain;
 myObject = function(s,d){
```

```

 site = s;
 domain = d;
 };
 MyObject.prototype.getSite = function(){
 return site;
 };
 MyObject.prototype.setSite = function(value){
 site = value;
 };
 // 再添加getDomain及setDomain方法
})();
var website = new MyObject("零点网络","www.zeronetwork.cn");
console.log(website.getSite());
website.setSite("zeronetwork");
console.log(website.getSite());
var p = new MyObject("王唯个人网站","www.lingdian.com");
console.log(website.getSite());
console.log(p.getSite());

```

以这种方式创建静态私有变量会让每个实例都没有自己的私有变量；到底是使用实例变量，还是静态私有变量，最终还是看具体的需求；

## 14-3 函数的属性和方法

因为函数是对象，所以函数也有属性和方法；如length属性；name属性，非标准，通过这个属性可以访问到函数的名字；

```

function show(a,b,c){console.log(arguments.length);}
console.log(show.name); // show

```

如果是使用new Function()定义的，会返回anonymous；如：

```

var show = new Function();
console.log(show.name); // anonymous

```

使用函数表达式也可以返回函数名字；

```

var show = function(){console.log("func")};
console.log(show.name); // show

```

### 14-3-1 caller属性

该属性保存着调用当前函数的函数的引用；如果是在全局作用域中调用当前函数，它的值为null；

```

function outer(){inner();}
function inner(){console.log(inner.caller);}
outer();

```

为了实现更松散的耦合，也可以通过arguments.callee.caller来访问相同的信息，如：

```

function inner(){console.log(arguments.callee.caller);}

```

```
// 注：当在严格模式下运行时，arguments.callee会导致错误；
// 注：在严格模式下，还有一个限制：不能为函数的caller属性赋值，否则导致错误；
```

## 14-3-2 prototype属性

// 在ES核心所定义的全部属性中，最有意思的就是prototype属性了，其表示函数的原型；对于ES中的引用类型来说，prototype是保存它们所有实例方法的真正所在；换句话说，诸如toString()和valueOf()等方法实际上都保存在prototype属性中，只不过是各自通过各自对象的实例访问罢了；在创建自定义引用类型以及实现继承时，prototype属性的作用是极为重要的；在ES中，prototype属性是不可枚举的，因此使用for-in无法发现；

## 14-3-3 apply()和call()

每个函数都包含这两个方法；这两个方法的用途都是在特定的作用域中调用函数，实际上等于设置函数体内this对象的值；

apply()方法接收两个参数：一个是在其中运行该函数的作用域，另一个是参数数组；其中第二个参数可以是Array的实例，也可以是arguments对象；如：

```
function sum(num1,num2){return num1+num2;}
function callSum1(num1,num2){
 return sum.apply(this,arguments); //传入arguments对象
}
function callSum2(num1,num2){
 return sum.apply(this,[num1,num2]); //传入数组
}
console.log(callSum1(10,20));
console.log(callSum2(10,20));
```

// 注：在严格模式下，未指定环境对象而调用函数，则this值不会指向window；

call()方法也接受两个以上参数，第一个参数是与apply()的第一个参数相同，但其余参数都直接传递给函数；换句话说，在使用call()时，传递给函数的参数必须逐个列举出来，如：

```
function sum(num1,num2){return num1+num2;}
function callSum(num1,num2){
 return sum.call(this,num1,num2);
}
alert(callSum(10,20));
```

其真正的apply()和call()作用是能够扩充函数赖以运行的作用域；如：

```
window.color="red";
var o={color:"blue"};
function sayColor(){console.log(this.color);}
sayColor(); // red
sayColor.call(this); // red
sayColor.call(window); // red
sayColor.call(o); // blue
```

```
// 使用call()或apply()来扩充作用域的最大好处，就是对象不需要与方法有任何耦合关系；

// bind()方法：主要作用就是将函数绑定至某个对象；

// 语法：fun.bind(this,arg1,arg2,...);
```

该方法会创建一个新的函数，称为绑定函数，其可传入两个参数，第一个参数作为this，第二个及以后的参数则作为函数的参数调用；即调用新的函数会把原始的函数当作对象的方法来调用；如：

```
window.color="red";
var o={color:"blue"};
function sayColor(){console.log(this.color);}
var objectSayColor = sayColor.bind(o);
objectSayColor(); // blue
var x = 10;
function fun(y){return this.x + y;}
var o = {x:1};
var g = fun.bind(o);
console.log(fun(5)); // 15
console.log(g(5)); // 6
```

有些浏览器可能不支持bind方法，兼容性的做法：

```
function bind(f,o){
 if(f.bind) return f.bind(o);
 else return function(){
 return f.apply(o,arguments);
 }
}
```

为bind()方法传入参数，该参数也会绑定至this；这种应用是一种常见的函数式编程技术，也被称为“柯里化”（currying），如：

```
var sum = function(x,y){return x + y;};
// 创建一个类似sum的新函数，但this的值绑定到null
// 并且第一个参数绑定到1，这个新的函数期望只传入一个实参
var succ = sum.bind(null,1);
console.log(succ(2)); // 3 x绑定到1，并传入2作为实参y
// 又如
function f(y,z){return this.x + y + z}; // 累加计算
var g = f.bind({x:1},2); // 绑定this和y
console.log(g(3)); // 6, this.x绑定到1, y绑定到2, z绑定到3
```

bind()方法返回的新函数，该函数对象的length属性是绑定函数的形参个数减去绑定实参的个数，即调用新函数时所期望的实参的个数，如：

```
var sum = function(x,y,z){return x + y + z;};
var o = {};
var fun = sum.bind(o,1); // 3 - 1 = 2
console.log(fun(2,3)); // 6
console.log(fun.length); // 2
```

使用bind()方法也可以用做构造函数，当bind()返回的函数用做构造函数时，将忽略传入的bind()的this；如：



```

var sum = function(x,y,z){
 this.x = x;
 this.y = y;
 this.z = z;
 this.getNum = function(){
 return this.x + this.y + this.z + this.a;
 }
};
var o = {a:1};
var fun = sum.bind(o,1);
var myFun = new fun(8,9,10);
console.log(myFun);
console.log(myFun.getNum()); // NAN

```

## 14-4 高阶函数

所谓高阶函数 (higher-order function) 就是操作函数的函数，它接收一个或多个函数作为参数，或者返回一个函数；如：

```

var powFun = function(x){
 return Math.pow(x,2);
};
function add(f,x,y){
 return f(x) + f(y);
}
console.log(add(powFun,3,4)); // 25

```

其实数组中有关迭代的方法全是高阶函数；比如，典型的一个应用，数组对象的map()方法，如：

```

function pow(x){
 return Math.pow(x,2);
}
var arr = [1,2,3,4,5];
var result = arr.map(pow);
console.log(result);

// 所返回的函数的参数应当是一个实参数组，并对每个数组元素执行函数f()
// 并返回所有计算结果组成的数组
function mapper(f){
 return function(a) {return a.map(f);};
}
var increment = function(x){return x + 1;}
var incrementer = mapper(increment);
console.log(incrementer([1,2,3]));

```

更常见的应用：

```

function not(f){
 return function(){ // 返回新的函数
 var result = f.apply(this,arguments); // 调用f()
 return !result; // 结果求反
 };
}
var even = function(x){ // 判断是否为偶数
 return x % 2 === 0;
}

```

```

};
var odd = not(even);
console.log([1,1,3,5,5].every(odd)); // true 每个元素都是奇数

// 返回一个新的可以计算f(g(...))的函数
// 返回的函数h()将它所有的实参传入g(),然后将g()的返回值传入f()
// 调用f()和g()时的this值和调用h()时的this值是同一个this
function compose(f,g){
 return function(){
 // 需要给f()传入一个参数,所以使用f()的call()方法
 // 需要给g()传入很多参数,所以使用g()的apply()方法
 return f.call(this, g.apply(this, arguments));
 };
}
var square = function(x){return x*x;};
var sum = function(x,y){return x + y;};
var squareofsum = compose(square, sum);
console.log(squareofsum(2,3)); // 25

```

## 递归:

```

// 递归是指函数调用自己;
// 语法:
function f1(){
 ...
 f1();
 ...
}

// 隐含递归:
function f1(){...; f2(); ...}
function f2(){...; f1(); ...}

```

## 通过递归打印出1-9的数值, 如:

```

function printNum(n){
 if(n>=1){
 printNum(n - 1);
 }
 console.log(n);
}
printNum(9);

```

```

// 递归函数效率低,但有利于理解和解决现实问题;
// 递归函数的执行过程:第一阶段"回推",第二阶段"递推";
// 函数在适当的时候能结束递归,否则会进入死循环;

```

```
function test(n){
 console.log("a" + n);
 n++;
 if(n<=5){
 test(n);
 }
 console.log("b" + n);
}
test(1); // 12345665432
```

又如:

```
// 5个人, 第5个人比第4个人大2岁,... 第一个人10岁, 第5个人几岁?
function age(n){
 if(n == 1){
 return 10;
 }else{
 return age(n - 1) + 2;
 }
}
console.log("第5个人的年龄为: " + age(5));
```

阶乘:

```
function factorial(num){
 if(num<=1){
 return 1;
 }else{
 return num*factorial(num-1);
 }
}
```

注: 但是如果类似于以下的代码, 就会出错:

```
var anotherFactorial = factorial;
factorial = null;
alert(anotherFactorial(4));
```

在这种情况下, 如果函数内部可以使用`arguments.callee`就可以解决问题; 其指向正在执行的函数的指针, 因此可以用它来实现对函数的递归调用:

```
return num*arguments.callee(num-1);
```

但在严格模式下, 不能通过访问`arguments.callee`, 访问这个属性会导致错误; 不过, 可以通过使用命名函数表达式来达到相同的效果; 如:

```
var factorial = (function f(num){
 if(num<=1){
 return 1;
 }else{
 return num*f(num-1);
 }
})();
var anotherFactorial = factorial;
factorial = null;
alert(anotherFactorial(4));
```

## 14-5 垃圾回收

// JS实现了垃圾自动回收处理机制，即，执行环境会负责管理代码执行过程中使用的内存，会自动分配、释放内存；在其他语言中，一般是手工跟踪内存的使用情况，比如C语言，开发人员可以显式的分配和释放系统的内存；但在JavaScript中，开发人员不用关心内存使用问题，所需内存的分配以及无用内存的回收完全实现了自动管理；其实现的原理是：找出那些不再继续使用的变量，然后释放其占用的内存；为此垃圾回收器会按照固定的时间间隔或在某个预定的收集时间，周期性地执行；

```
var a = "zero";
var b = "network";
a = b; // "zero" 所占空间被释放
```

### 14-5-1 变量的生命周期

// 无论哪种开发语言，其内存的生命周期几乎是一样的：分配内存空间-使用内存空间-释放空间；

// 函数中局部变量的正常生命周期：只在函数执行的过程中存在；而在这个过程中，会为局部变量在栈或堆内存上分配相应的空间，以便存储它们的值；当函数执行结束，局部变量就没有存在的必要了，因此可以释放它们的内存以供将来使用；在这种情况下，很容易判断变量是否还有存在的必要；但并非所有情况下都这么容易判断；垃圾收集器必须跟踪哪个变量有用哪个变量没有用，对于不再有用的变量打上标记，以备将来收回其占用的内存；用于标识无用变量的策略可能会因实现而异，但具体到浏览器中的实现，则通常有两个策略：

#### 1-1 标记清除

// JavaScript中最常用的垃圾收集方式是标记清除（mark-and-sweep）；当变量进入环境时，就将这个变量标记为“进入环境”；

// 垃圾收集器在运行的时候会给存储在内存中的所有变量都加上标记。然后，它会去掉环境中的变量以及被环境中的变量引用的标记。而在此之后再被加上标记的变量将被视为准备删除的变量，原因是环境中的变量已经无法访问到这些变量了。最后。垃圾收集器完成内存清除工作，销毁那些带标记的值，并回收他们所占用的内存空间。

// 目前，各浏览器使用的都是标记清除的策略，只不过垃圾收集的时间间隔互相不同。

#### 1-2 引用计数

// 另一种不太常见的垃圾收集策略叫做引用计数（reference counting）；引用计数的含义是跟踪记录每个值被用的次数；当声明了一个变量并将一个引用类型值赋给该变量时，则这个值的引用次数就是1，如果同一个值又被赋给另一个变量，则该值的引用次数加1；相反，如果包含对这个值引用的变量又取得了另外一个值，则这个值的引用次数减1；当这个值的引用次数变成0时，则说明没有办法再访问这个值了；因而就可以将其占用的内存空间回收回来；这样，当垃圾收集器下次再运行时，它就会释放那些引用次数为零的值所占用的空间了。

Navigator3是最早使用引用计数策略的浏览器，但遇到了一个严重的问题：循环引用；即对象A中包含指向对象B的指针，而对象B中也包含一个指向对象A的引用，如：

```
function problem(){
 var objectA = new Object();
 var objectB = new Object();
 objectA.other = objectB;
 objectB.another = objectA;
}
```

为此，Navigator4中放弃了引用计数方式，转而采用标记清除来实现其垃圾收集机制；

但是，IE中某些对象还在采用引用计数方式，这些对象不是原生的JavaScript对象，如BOM和DOM中的对象就是使用C++以COM对象的形式实现的，而COM对象的垃圾收集机制采用的就是计数策略；因此，即使IE的JavaScript引擎是使用标记清除策略来实现的，但JavaScript访问的COM对象依然是基于引用计数策略的；换句话说，只要在IE中涉及COM对象，就会存在循环引用的问题；如：

```
var element = document.getElementById("some_element");
var myObject = new Object();
myObject.element = element;
element.someObject = myObject;
```

由于存在这个循环引用，即使将示例中的DOM从页面中移除，其也永远不会被回收；

```
// 为了避免类似这样的循环引用问题，最好是在不使用它们的时候手工断开原生JavaScript对象与DOM元素
// 之间的连接，如：
myObject.element = null;
element.someObject = null;
```

目前，IE早已把BOM和DOM对象都转换成了真正的JavaScript对象；这样，就避免了两种垃圾收集算法并存导致的问题，也消除了常见的内存泄漏现象；

### 1-3 管理内存

// 使用具备垃圾收集机制的语言编写程序，开发人员一般不必要操心内存管理的问题；但是，JavaScript在进行内存管理及垃圾收集时面临的问题还是与众不同；其中最主要的一个问题，就是分配给web浏览器的可用内存数量通常要比分配给桌面应用程序的要少；这样做的目的主要是出于安全方面的考虑，目的是防止运行JavaScript的网页耗尽全部系统内存而导致系统崩溃；内存限制问题不仅会影响给变量分配内存，同时还会影响调用栈以及在一个线程中能够同时执行的语句数量。

因此，确保占用最少的内存可以让页面获得更好的性能；而优化内存占用的最佳方式，就是为执行中的代码只保存必要的数据；一旦数据不再有用，最好通过将其值设置为null来释放其引用，即解除引用(dereferencing)，其适用于大多数全局变量和全局对象的属性；如：

```
function createPerson(name){
 var localPerson = new Object();
 localPerson.name = name;
 return localPerson;
}
var globalPerson = createPerson("wangwei");
globalPerson=null; // 手工解除globalPerson的引用
```

```
// 注：解除一个值的引用并不意味着自动回收该值所占用的内存；解除引用的值作用是让值脱离执行环境，以便垃圾收集器下次运行时将其回收。
```

```
// JS的自动内存管理存在一些问题，例如垃圾回收实现可能存在缺陷或者不足，因此，需要找到一个合适的解决方法；
```

## 14-6 内存泄露

Javascript的几种内存泄露：

### 14-6-1 全局变量

```
// 一个没有声明的变量，成为了一个全局变量；因此，要避免这种情况出现，或者使用严格模式；
```

### 14-6-2 循环引用

```
// 即：A引用B，B引用A，如此，其引用计数都不为0，所以不会被回收；
// 解决：手工将它们设为null；
```

### 14-6-3 闭包

```
// 闭包会造成对象引用的生命周期脱离当前函数的作用域，使用不当，会造成内存泄露；
```

### 14-6-4 延时器、定时器

```
// setInterval / setTimeout中的this指向的是window对象，所以内部定义的变量也挂载在了全局，
if引用了someResource变，如果没有清除setInterval/setTimeout的话，someResource得不到释放；
```

```
var someResource = getData();
setInterval(function(){
 var node = document.getElementById('Node');
 if(node){
 node.innerHTML = JSON.stringify(someResource);
 }
},1000);
```

### 14-6-5 DOM引用的内存泄露

未清除DOM的引用：

```
var refA = document.getElementById('refA');
document.body.removeChild(refA);
// refA不能回收，因此存在变量refA对它的引用，虽然移除了refA节点，但依然无法回收
// 解决方案
refA = null;
```

DOM对象添加的属性是一个对象的引用：

```
var myObj = {};
document.getElementById('myDiv').myPro = myObj;
// 解决方案, 在页面onunload事件中释放
document.getElementById('myDiv').myPro = null;
```

给DOM对象绑定事件:

```
var btn = document.getElementById("myBtn");
btn.onclick = function(){
 // 虽然最后把btn这个DOM移除, 但是绑定的事件没有被移除, 也会引起内存泄露, 需要清除事件
 // btn.onclick = null;
 document.getElementById("mydiv").innerHTML = "zeronetwork";
}
// 其他
document.body.removeChild(btn);
btn = null
```

## 15, 对象和构造函数

### 15-1 理解对象

可以创建一个最简单的自定义对象, 就是使用Object, 然后再为它添加属性和方法, 如:

```
var person = new Object();
person.name = "wangwei";
person.age = 18;
person.job = "Engineer";
person.sayName = function(){
 alert(this.name);
}
```

在多种场景中, 常用对象字面量创建对象, 如:

```
var person = {
 name:"wangwei",
 age:18,
 job:"Engineer",
 sayName:function(){
 alert(this.name);
 }
};
```

### 15-2 对象中的this

// 当一个函数作为对象的属性存在时, 并且通过对象调用这个方法, 那么函数中的this就指向调用函数的对象;

// this的好处在于, 可以更加方便的访问对象内部成员;

## 15-3 早绑定和晚绑定

绑定：`//` 把对象的成员与对象实例结合在一起的方法。

早绑定：

`//` 指在实例化对象之前定义它的属性和方法，这样编译器或解释程序就能够提前转换机器代码。**ES**不是强类型语言，所以不支持早绑定；

晚绑定：

`//` 编译器或解释程序在运行前，不知道对象的类型。使用晚绑定，无需检查对象的类型，只需检查对象是否支持属性和方法即可；**ES**中的所有变量都采用晚绑定方法；这样就允许执行大量的对象操作；

## 15-4 属性访问错误

属性访问并不总是返回或设置一个值，如果访问一个不存在的属性并不会报错，会返回`undefined`；但如果试图访问一个不存在的对象的属性就会报错；`null`和`undefined`值是没有属性的，因此，访问这两个值的属性就会报错，如：

```
var book = {};
console.log(book.subtitle); // undefined
// console.log(book.subtitle.length); // 异常
// 解决方案
var len = book && book.subtitle && book.subtitle.length;
console.log(len); // undefined, 不会报错
```

有些属性是只读的，不能重新赋值，有一些对象不允许新增属性，但如果操作这些属性，也不会报错，如：

```
// 内置构造函数的原型是只读的
// 赋值失败，但没有报错，Object.prototype没有修改
Object.prototype = 0;
```

这是历史遗留问题，但在严格模式下会抛出异常；

## 15-5 删除属性

`delete`删除对象的属性，但只是断开属性和宿主对象的联系，而不会去操作属性中的属性；

```
var a = {p:{x:1}};
var b = a.p;
delete a.p;
console.log(b.x); // 1
```

删除的属性的引用还存在，因此在某些实现中，有可能会造成内存泄漏；因此，在销毁对象时，要遍历属性中的属性，依次删除；

`delete`删除成功或者没有任何副作用时，它返回`true`；或者删除的不是一个属性访问表达式，同样返回`true`，如：



```
var o = {x:1};
delete o.x;
delete o.x;
console.log(delete o.toString); // 什么也没做，toString是继承来的
console.log(delete 1) // 无意义

// delete不能删除那些可置性为false的属性，
```

某些内置对象的属性是不可配置的，比如通过变量声明和函数声明创建的全局对象的属性；在严格模式下，删除一个不可配置属性会报一个类型错误，在非严格模式中，这些操作会返回false，如：

```
console.log(delete Object.prototype); // 不能删除，属性是不可配置的
var x = 1;
console.log(delete this.x); // 不能删除
function f(){}
console.log(delete this.f); // 不能删除
```

在非严格模式中，删除全局对象的可配置属性时，可以省略对全局对象的引用，但在严格模式下会报错，如：

```
"use strict";
this.x = 1;
console.log(delete this.x);
console.log(delete x); // 严格模式下异常
```

因此，必须显式指定对象及其属性；

虽然Object构造函数或对象字面量都可以用来创建单个对象，但这些方法有个明显的缺点：使用同一个接口创建很多对象，会产生大量的重复代码；为解决这个问题，可以使用工厂模式的方式创建对象；

## 15-6 工厂模式

工厂模式是软件工程领域一种广泛使用的设计模式，其抽象了创建具体对象的过程（还有其他设计模式）；在ES中无法创建类，所以就发明了一种函数，用该函数来封装特定接口创建对象的细节，如：

```
function createPerson(name,age,job){
 var o = new Object();
 o.name = name;
 o.age = age;
 o.job = job;
 o.sayName = function(){
 alert(o.name);
 };
 return o;
}
var p1 = createPerson("wangwei",18,"Engineer");
var p2 = createPerson("wujiang",28,"doctor");
alert(p1.name);
alert(p2.name);
```

在工厂函数外定义对象方法，再通过属性指向该方法；

```
// 在上面的代码中改
function sayName(){
 alert(this.name);
}
// 在原来的o.sayName = function(){...}改成如下
o.sayName = sayName;
```

## 15-7 构造函数

// 可以使用构造函数来创建特定类型的对象，如：**Object**和**Array**这种原生构造函数，在运行时会自动出现在执行环境中；

// 构造函数内能初始化对象，并返回对象；

// 此外，也可以创建自定义的构造函数，从而自定义对象类型的属性和方法；使用此种方式的目的：更加类似真正的面向对象创建(类)对象方法，也就是首先创建类；如：

```
function Person(name,age,job){
 this.name = name;
 this.age = age;
 this.job = job;
 this.sayName = function(){
 alert(this.name);
 };
}
var p1 = new Person("wangwei",18,"Engineer");
var p2 = new Person("wujing",28,"Doctor");
alert(p1.name);
alert(p2.name);

// 这里的Person本身就是函数，只不过可以用来创建对象而已；
```

要创建实例对象，必须使用new实例化对象；以这种方式调用函数实际上会历经下4个步骤：

```
// 创建一个新对象；
// 将构造函数的作用域赋给新对象(因此this就指向了这个新对象)
// 执行构造函数中的代码(为这个新对象添加属性)；
// 返回新对象，即隐式的返回了this；
```

### 15-7-1 关于构造函数的返回值

```
// 先使用this，再使用o
function Person(name,age){
 var o = {};
 o.name = name;
 o.age = age;
 return o;
}
var p = new Person("wangwei",18);
console.log(p);
```

但如果返回是一个原始值，如：return 100，此时无任何影响，说明构造函数内返回的一定是一个对象；

在构造函数内还可以使用闭包：

```
function Person(name, age){
 var money = 100;
 this.name = name;
 this.age = age;
 function show(){
 money ++;
 console.log(money);
 }
 this.say = show;
}
var p1 = new Person();
p1.say();
p1.say();
var p2 = new Person();
p2.say();
```

### 15-7-2 constructor（构造函数）属性

实例都有一个constructor（构造函数）属性，该属性指向Person；

即：构造函数方式创建的实例有constructor(构造函数)属性，该属性指向类函数，如：

```
alert(p1.constructor == Person);
alert(p2.constructor == Person);
```

对象的constructor属性最初是用来标识对象类型的。但检测对象类型，instanceof操作符更可靠；

```
alert(p1 instanceof Object);
alert(p1 instanceof Person);
```

### 15-7-3 构造函数的特点

构造函数与其他函数的唯一区别：就在于调用它们的方式不同；构造函数也是函数，不存在定义构造函数的特殊语法；

任何函数，只要通过new操作符来调用，那它就可以作为构造函数；而任何函数，如果不通过new操作符调用，就是一个普通函数，如：

```
// 当作构造函数使用
var p1 = new Person("wangwei", 18, "Engineer");
p1.sayName();
// 当作普通函数调用
Person("wujing", 28, "Doctor");
window.sayName();
// 在另一个对象的作用域中调用
var o = new Object();
Person.call(o, "Hello", 38, "worker");
o.sayName();
```

## 15-7-4 构造函数的缺点

这种方式虽然比较方便好用，但也并非没有缺点；缺点是：每个方法都要在每个实例上重新创建一遍，如sayName()方法，每个实例拥有的sayName()，但都不是同一个Function实例，如：

```
alert(p1.sayName == p2.sayName); // false
```

在ES中的函数是对象，因此每定义一个函数，也就实例化了一个对象，从逻辑上说，相当于：

```
this.sayName = new Function("alert(this.name)");
```

以这种方式创建函数，会导致不同的作用域链和标识符解析；但创建Function新实例的机制仍然是相同的；

可以把函数定义在构造函数外部；如：

```
function sayName(){
 alert(this.name);
}
function Person(name,age,job){
 this.name = name;
 this.age = age;
 this.job = job;
 this.sayName = sayName;
}
// 当作构造函数使用
var p1 = new Person("wangwei",18,"Engineer");
var p2 = new Person("wangwei",18,"Engineer");
alert(p1.sayName == p2.sayName); // true
```

## 16, 原型prototype

### 16-1 原型prototype

// 创建的每个函数都有一个**prototype**(原型)属性，这个属性是一个指针，其指向一个原型对象，该对象的用途是：包含由特定类型创建的所有实例共享的属性和方法；

// 每个对象都从原型继承属性，也就是说，如果一个函数是一个类的话，这个类的所有实例对象都是从同一个原型对象上继承成员；因此，原型是类的核心；

**优点：**可以让所有对象实例共享它所包含的属性和方法；如：

```
function Person(){
 Person.prototype.name = "wangwei";
 Person.prototype.age = 18;
 Person.prototype.job = "Engineer";
 Person.prototype.sayName = function(){
 alert(this.name);
 };
}
var p1 = new Person();
p1.sayName();
var p2 = new Person();
p2.sayName();
alert(p1.sayName == p2.sayName);
```

```
// 所有通过对象字面量创建的对象都具有同一个原型对象，并可以通过Object.prototype获取对原型对象的引用；
// 通过new构造函数创建的对象，其原型就是构造函数的prototype属性；
// 同样，通过new Array()创建的对象的原型就是array.prototype，通过new Date()创建的对象的原型就是Date.prototype；
// 并不是所有的对象都具有原型，比如：Object.prototype本身就是一个对象，它就没有原型，并且也不继承任何属性；
```

对于一个实例对象来说，该实例的内部将包含一个指针，指向构造函数的原型对象；ES把这个指针称为[[Prototype]]，但在脚本中，没有标准的方式访问[[Prototype]]，但在很多实现中，每个对象上都支持一个属性**proto**，其就指向原型对象，并且可以通过脚本访问到；

```
function Person(name,age){}
console.log(Person.prototype);
var p = new Person("wangwei",18);
console.log(p.__proto__);
console.log(Person.prototype === p.__proto__);
```

构造函数的prototype属性被用作新对象的原型；这意味着通过同一个构造函数创建的所有对象，都继承自一个相同的对象，因此它们都是同一个类的成员；

## 16-2 构造函数和类的标识

```
// 原型对象是类的唯一标识，当且仅当两个对象继承自同一个原型对象时，它们才是属于同一个类的实例；
// 而初始化对象的状态的构造函数则不能作为类的标识，两个构造函数的prototype属性可能指向同一个原型对象，那么这两个构造函数创建的实例是属于同一个类的；
```

## 16-3 constructor属性

任何JavaScript函数都可以用作构造函数，并且调用构造函数是需要用到prototype属性的；

在默认情况下，所有原型对象都会自动获得一个constructor属性，该属性是一个指向prototype属性所在函数的指针，其也是prototype属性中的唯一不可枚举属性，它的值就是一个函数对象；如Person.prototype.constructor指向Person；如：

```
var F = function(){}; // F是函数对象
var p = F.prototype; // 这是F相关联的原型对象
var c = p.constructor; // 这是与原型相关联的函数
console.log(c === F); //true
```

可以看到构造函数的原型中存在预先定义好的constructor属性，该属性指向对象的构造函数；由于构造函数是类的“公共标识”，因此constructor属性为对象提供了类；

```
var o = new F();
console.log(o.constructor === F); // true
```

创建了自定义的构造函数之后，其原型对象默认只会取得constructor属性，至于其他方法，则都是从Object继承而来的；

上面的Range()构造函数，使用它自身的一个新对象重写预定义的Range.prototype对象，这个新定义的原型对象不含有constructor属性，因此Range类的实例也不含有constructor属性，可以显式的为原型添加一个构造函数，如：

```
// 在Range.prototype中添加
 constructor: Range, // 显式设置构造函数反向引用
// 设置了constructor，可以继续为原型对象添加其他属性和方法。
// 修改原例
Range.prototype.includes = function(x){return this.from <=x && x <= this.to;};
Range.prototype.foreach = function(f){
 for(var x = Math.ceil(this.from); x <= this.to; x++) f(x);
};
Range.prototype.toString = function(){return "(" + this.from + "... " + this.to +
 ")";}
```

isPrototypeOf()方法：虽然在所有实现中都无法访问到[[Prototype]]，但可以通过isPrototypeOf()方法来确定对象之间是否存在这种关系；从本质上讲，如果[[Prototype]]指向调用isPrototypeOf()方法的对象，那么这个方法就返回true，如：

```
var p = {x:1};
var o = Object.create(p);
console.log(p.isPrototypeOf(o));
console.log(Person.prototype.isPrototypeOf(p1));
```

Object.getPrototypeOf()：该方法返回[[Prototype]]的值，即查询一个对象的原型，如：

```
alert(Object.getPrototypeOf(p1) == Person.prototype); // true
alert(Object.getPrototypeOf(p1).name); // wangwei
alert(Object.getPrototypeOf(p1));
```

// 当代码读取某个对象的某个属性时，会执行一次搜索，目标是具有给定名字的属性；

// 搜索首先从对象实例本身开始，再到原型对象；如果在实例中找到了具有给定名字的属性，则返回该属性值，如果没找到，则继续搜索指针指向的原型对象，如果找到，就返回该属性值；也就是说，前面调用p1.sayName()时，会先后执行两次搜索；其次，p2也是如此；正因为如此，多个对象实例才能共享原型中所保存的属性和方法。

虽然可以通过对象实例访问保存在原型中的值，但不能通过对象实例重写原型中的值；如果在实例中添加一个属性，而该属性与原型中的一个属性同名，那么就会在该实例中创建该属性，该属性会屏蔽原型中的同名属性；

```
// 在以上的示例中添加
var p1 = new Person();
var p2 = new Person();
p1.name = "wujing";
alert(p1.name); // wujing 来自实例
alert(p2.name); // wangwei 来自原型
```

如果想恢复访问原型中的属性，默认情况下是恢复不了的，即使将实例属性设置为null，也不会恢复其指向原型的连接，但可以使用delete完全删除实例属性，从而能够重新访问原型中的属性，如：

```
// p1.name = null;
delete p1.name;
alert(p1.name); // wangwei 来自原型
```

## 16-4 重写整个原型

为了减少不必要的输入，也为了从视觉上更好的封装原型的功能，常见的做法是用一个包含所有属性和方法的对象字面量来重写整个原型对象；

```
function Person(){}
Person.prototype = {
 name:"wangwei",
 age:18,
 sayName:function(){
 alert(this.name);
 }
};
```

此时，instanceof操作还能返回正确的结果，但通过constructor已经无法确定对象的类型了。

```
var p = new Person();
alert(p instanceof Person); // true
alert(p instanceof Object); // true
alert(p.constructor == Person); // false
alert(p.constructor == Object); // true
```

如果需要constructor属性，可以在代码块中显式声明；

```
Person.prototype = {
 constructor:Person,
 name:"wangwei",
 age:18,
 sayName:function(){
 alert(this.name);
 }
};
var p = new Person();
alert(p.constructor);
alert(p.constructor == Person); // true
```

但是，以这种方式重设constructor属性会导致它的[[Enumerable]]特性被设置为true；默认情况下，原生的constructor属性是不可枚举的；可以通过Object.defineProperty()方法重设constructor，如：

```
Object.defineProperty(Person.prototype, "constructor", {
 enumerable: false,
 value: Person
});
var p = new Person();
alert(p.constructor);
alert(p.constructor == Person); // true
```

## 16-5 Object.create()

Object.create()方法规范了原型式继承，这个方法接收两个参数：一个用作新对象原型的对象和一个为新对象定义额外属性的对象（可选的）；如：

```
var person = {
 name:"wangwei",
```

```

 friends: ["wujing", "lishi"]
 };
 var p1 = Object.create(person); // person作为原型对象传入,p1继承了person属性
 console.log(p1);
 console.log(person);
 p1.name = "wujing";
 p1.friends.push("adu");
 console.log(p1);
 var p2 = Object.create(person);
 console.log(p2);
 p2.name = "juanzi";
 p2.friends.push("van");
 console.log(p1.friends); //wujing,lishi,adu,van
 console.log(person.friends); //wujing,lishi,adu,van

```

如果传入参数null，就会创建一个没有原型的新对象，其也不会继承任何成员，可以对它直接使用in运算符，而无需使用hasOwnProperty()方法，如：

```

var o = Object.create(null);
console.log(o); // No properties

```

如果想创建一个普通的空对象，比如通过{}或new Object()创建的对象，需要传入Object.prototype，如：

```

var o = Object.create(Object.prototype);
console.log(o); // 与{}和new Object()一样

```

可以通过任意原型创建新对象，即可以使任意对象可继承，这是一个强大的特性；

Object.create()方法的第二个参数与Object.defineProperties()方法的第二个参数格式相同，每个属性都是通过自己的描述符定义的；以这种方式指定的任何属性都会覆盖原型对象上的同名属性；如：

```

var person = {
 name: "wangwei",
 friends: ["wujing", "lishi"]
};
var p1 = Object.create(person, {
 name: {
 value: "wujing"
 }
});
console.log(p1.name);

```

## 16-6 原型的动态性

ES中基于原型的继承机制是动态的：对象从其原型继承属性，如果创建对象之后原型的属性发生改变，也会影响到继承这个原型的所有实例对象；这意味着可以通过给原型对象添加新方法来扩充ES类，即使先创建了实例后再修改原型也如此；如：



```
function Person(){}
Person.prototype = {
 name:"wangwei",
 age:18,
 sayName:function(){
 alert(this.name);
 }
};
```

尽管可以随时为原型添加属性和方法，并且修改能够立即在所有对象实例中反映出来，但如果重写整个原型对象，本质上就不一样了；如：

```
var p = new Person();
alert(p instanceof Person); // true
alert(p instanceof Object); // true
alert(p.constructor == Person); // false
alert(p.constructor == Object); // true
```

重写原型对象切断了现有原型与任何之前已经存在的对象实例之间的联系；它们引用的仍然是最初的原型；

## 16-7 原生对象的原型

原型模式的重要性不仅体现在创建自定义类型方面，就连所有原生的引用类型，也是采用这种模式创建的，所有原生引用类型(Array, Object, String等)，都在其构造函数的原型上定义了方法，如：Array.prototype中可以找到sort方法，在String.prototype中找到substring() 方法；如：

```
Person.prototype = {
 constructor:Person,
 name:"wangwei",
 age:18,
 sayName:function(){
 alert(this.name);
 }
};
var p = new Person();
alert(p.constructor);
alert(p.constructor == Person); // true
```

通过原生对象的原型，不仅可以取得所有默认方法的引用，而且也可以定义新方法，可以像修改自定义对象的原型一样修改原生对象的原型，因此可以随时添加方法；

```
Object.defineProperty(Person.prototype, "constructor", {
 enumerable: false,
 value: Person
});
var p = new Person();
alert(p.constructor);
alert(p.constructor == Person); // true
```

可以给Object.prototype添加方法，从而使所有的对象都可以调用这些方法；尽管可以这么做，但不建议修改原生对象的原型，因为当在另一个支持该方法的实现中运行代码时，就可能会导致命名冲突，另外，这样做也可能会无意地重写原生方法，如果有必要添加的话，最好使用Object.defineProperty()方法进行属性的特性的定义；

## 16-8 原型对象的问题

原型缺点：首先其忽略了为构造函数传递初始化参数这一环节，所以在默认情况下所有实例将取得相同的属性值；其次，由于其属性是所有实例共享，对于基本类型的属性没有影响，毕竟通过在实例上添加一个同名属性，可以隐藏原型中的对应属性；但对引用类型的属性有影响；如：

```
function Person(){}
Person.prototype = {
 constructor: Person,
 name: "wangwei",
 age: 18,
 friends: ["she", "who"],
 sayName: function(){
 alert(this.name);
 }
};
var p1 = new Person();
var p2 = new Person();
p1.friends.push("van"); // 但是如果p1.friends = []就不一样了，相当于新建了数组
alert(p1.friends); // she,who,van
alert(p2.friends); // she,who,van
alert(p1.friends === p2.friends); // true
```

## 16-9 组合使用构造函数模式和原型模式

创建自定义类型的最常见的方式，就是组合使用构造函数模式和原型模式；即用构造函数定义对象的实例属性，而用原型方式定义对象的方法和共享的属性；另外，这种混合模式还支持向构造函数传递参数；如：

```
function Person(name, age, job){
 this.name = name;
 this.age = age;
 this.job = job;
 this.friends = ["she", "who"];
}
Person.prototype = {
 constructor: Person,
 sayName: function(){
 alert(this.name);
 }
};
var p1 = new Person("wangwei", 18, "Engineer");
var p2 = new Person("wujing", 28, "Doctor");
p1.friends.push("van");
alert(p1.friends);
alert(p2.friends);
alert(p1.friends === p2.friends);
alert(p1.sayName === p2.sayName);
```

总结：这种构造函数模式与原型模式的混合，是目前使用最广泛，认同度最高的一种创建自定义类型的方法；可以说，这是用来定义引用类型的一种默认模式。

## 16-10 动态原型模式

原型模式还不是像其他语言一样把所有属性和方法都封装起来;

动态原型模式的基本思想是把所有信息封装到构造函数中, 而通过构造函数初始化原型, 又保持了同时使用构造函数和原型的优点; 换句话说, 可以通过检查某个应该存在的方法是否有效, 来决定是否需要初始化原型; 如:

```
function Person(name,age,job){
 // 属性
 this.name = name;
 this.age = age;
 this.job = job;
 this.friends = ["she","who"];
 // 方法
 if (typeof this.sayName != "function"){
 Person.prototype.sayName = function(){
 alert(this.name);
 };
 }
}
var p = new Person("wangwei",18,"Engineer");
p.sayName();
```

这段代码只会在初次调用构造函数时才会执行; 此时, 原型已经完成初始化, 不需要再做什么修改了; 注意: 这里对原型的所做的修改, 能够立即在所有实例中得到反映; 因此, 这种方式比较完美; 其中, if 语句检查的可以是初始化之后应该存在的任何属性或方法, 不必用很多的if语句检查每个属性和方法, 只要检查其中一个即可; 对于采用这种模式创建的对象, 还可以使用instanceof操作符确定它的类型。

```
function Car(sColor,iDoors,iMpg){
 this.color = sColor;
 this.doors = iDoors;
 this.mpg = iMpg;
 this.drivers = new Array("Mike","Sue");
 if(typeof Car._init == "undefined"){
 Car.prototype.showColor = function(){
 alert(this.color);
 };
 Car._init = true;
 alert("ready");
 }
}
var oCar1 = new Car("red",4,23); // ready
var oCar2 = new Car("blue",3,24); // 无, 只执行一次
alert(oCar1 instanceof Car);
```

## 16-11 寄生构造函数模式(混合工厂)

基本思想: 创建一个函数, 该函数的作用仅仅是封装创建对象的代码, 然后再返回新创建的对象; 但从表面看, 这个函数又很像是典型的构造函数, 如:

```
function Person(name,age,job){
 var o = new Object();
 o.name = name;
 o.age = age;
 o.jog = job;
 o.sayName = function(){
 alert(this.name);
 };
 return o;
}
var p = new Person("wangwei",18,"Engineer");
p.sayName();
```

这种模式可以在特殊的情况下用来为对象创建构造函数，如创建一个具有额外方法的特殊数组，由于不能直接修改Array构造函数，因此可以使用这个模式，如：

```
function SpecialArray(){
 var values = new Array(); // 创建数组
 values.push.apply(values, arguments); // 添加值
 values.toPipedString = function(){ // 添加方法
 return this.join("|");
 };
 return values; // 返回数组
}
var colors = new SpecialArray("red","blue","green");
alert(colors.toPipedString()); // red|blue|green
```

## 16-12 稳妥构造函数模式(混合工厂)

```
// 道格拉斯Douglas Crockford发明了JS中的稳妥对象(durable objects)这个概念；

// 稳妥对象：指的是没有公共属性，而且其方法也不引用this的对象；

// 这种模式最适合在一些安全的环境中(会禁止使用this和new)，或者在防止数据被其他程序修改时使用；
```

稳妥构造函数模式遵循与寄生构造函数类似的模式，但有两点不同：一是新创建的对象实例方法不引用this；二是不使用new操作符调用构造函数；如：

```
function Person(name,age,jog){
 var o = new Object(); // 创建要返回的对象
 // 可以在这里定义私有变量和函数
 o.sayName = function(){
 alert(name);
 };
 return o;
}
var p = Person("wangwei",18,"Engineer");
p.sayName();
```

## 16-13 小示例

字符串操作(提高其性能);

```
var str="hello";
str += "world";
var arr=new Array();
arr[0]="hello";
arr[1]="world";
var str=arr.join("");
```

可以把其封装, 可复用:

```
function StringBuffer(){
 this._strings = new Array();
}
StringBuffer.prototype.append=function(str){
 this._strings.push(str);
}
StringBuffer.prototype.toString=function(){
 return this._strings.join("");
}
var buffer=new StringBuffer();
buffer.append("hello");
buffer.append("world");
var str=buffer.toString();
document.write(str);
```

测试性能:

```
function StringBuffer(){
 this._strings = new Array(); }
StringBuffer.prototype.append=function(str){
 this._strings.push(str); }
StringBuffer.prototype.toString=function(){
 return this._strings.join(""); }
var d1=new Date();
var str="";
for(var i=0; i<10000; i++){
 str += "text"; }
var d2=new Date();
document.write("页面执行时间为: " + (d2.getTime() - d1.getTime()) + "
");
d1=new Date();
var buffer=new StringBuffer();
for(var i=0; i<10000; i++)
 buffer.append("text");
str=buffer.toString();
d2=new Date();
document.write("页面执行时间为: " + (d2.getTime() - d1.getTime()) + "
");
```

利用prototype属性为所有对象自定义属性和方法;

```
//为Number对象添加输出16进制的方法;
Number.prototype.toHexString=function(){
 return this.toString(16);
```

```

}
var iNum=15;
console.log(iNum.toHexString());
//为数组添加队列方法;
Array.prototype.enqueue=function(vItem){
 this.push(vItem);
}
Array.prototype.dequeue=function(){
 return this.shift();
}
var arr=new Array("red","blue","white");
arr.enqueue("green");
console.log(arr);
arr.dequeue();
console.log(arr);
//如果想给所有内置对象添加新方法，必须在Object对象的prototype属性上定义;
Object.prototype.showValue=function(){
 console.log(this.valueOf());
};
var str="hello";
var iNum=23;
str.showValue();
iNum.showValue();
// 函数名只是指向函数的指针,因此可以使它指向其他函数
Function.prototype.toString=function(){
 return "函数内部代码隐藏";
}
function say(){
 console.log("hi");
}
console.log(say.toString());
//此方法会覆盖原始方法，所以可以在使用前存储它的指针，以便以后使用;
Function.prototype.oriToString=Function.prototype.toString;
Function.prototype.toString=function(){
 if(this.oriToString().length>100){
 return "内容过长，部分隐藏";
 }else{
 return this.oriToString();
 }
}
}

```

## 16-14 遍历和枚举属性

### 16-14-1 in操作符

in操作符会通过对象能够访问给定属性时返回true，无论属性存在于实例中还是原型中：

```

alert(p1.hasOwnProperty("name")); // false
alert("name" in p1); // true

```

除了in操作符，更为便捷的方式是使用“!==”判断一个属性是否是undefined，如：

```

var o = {x:1};
o.x !== undefined; // true
o.y !== undefined; // false
o.toString !== undefined; // true, o继承了toString

```

然而有一种场景只能使用in而不能使用上述属性访问的方式，in可以区分该属性存在但值为undefined的情景，如：

```
var o = {x:undefined};
o.x !== undefined //false,属性存在, 但值为undefined
o.y !== undefined // false, 属性不存在
"x" in o; // true,属性存在
"y" in o; // false,属性不存在
delete o.x;
"x" in o; // false
```

在使用“!==”时，要注意其与“!=”不同点，“!==”可以区分undefined和null，如：

```
var o={x:2};
// 如果o中存在属性x,且x的值不是null或undefined, 则o.x乘以2
if(o.x !== null) o.x *= 2;
// 如果o中存在属性x, 且x的值不能转换为false, o.x乘以2
// 如果x是undefined、null、false、“ ”、0或NaN, 则它保持不变
if(o.x) o.x *=2;
```

同时使用hasOwnProperty()方法和in操作符，可以确定该属性是存在于对象还是原型中；

```
// 判断是否为原型
var p1 = new Person();
// p1.name = "wujing"; // 添加此名就会返回false
function hasPrototypeProperty(object, name){
 return !object.hasOwnProperty(name) && (name in object);
}
alert(hasPrototypeProperty(p1,"name")); // true
// 或者:
var obj = {
 name:"wang",
 age:100,
 sex:'male',
 __proto__: {
 lastName:"wei"
 }
}
Object.prototype.height = '178CM';
for(var p in obj){
 if(obj.hasOwnProperty(p)){
 console.log(obj[p]);
 }
}
```

除了检测对象的属性是否存在，还会经常遍历对象的属性；通常使用for/in循环遍历，但ES还提供了两个更好的方案；

for-in：可以遍历所有能够通过对象访问的、可枚举的(enumerated)属性，其中既包括存在于实例中的属性，也包括存在原型中的属性；

```

var o = {x:1,y:2,z:3};
Object.defineProperty(o, "z", {value:4, enumerable:false});
console.log(o.z);
for(p in o)
 console.log(p);

function Person(){this.name="wangwei";this.age=10;}
Person.prototype.toString = function(){return "wangwei";}
var p1 = new Person();
Object.defineProperty(p1, "age", {value:18, enumerable:false});
for(prop in p1)
 console.log(prop + ":" + p1[prop]);

```

有时需要过滤遍历，如：

```

var obj={a:1,b:2};
var o = Object.create(obj);
o.x=1,o.y=2,o.z=3;
o.sayName = function(){}
for(p in o){
 if(!o.hasOwnProperty(p)) continue; // 跳过继承的属性
 console.log(p);
}
for(p in o){
 if(typeof o[p] === "function") continue;
 console.log(p); // 跳过方法
}

```

用来枚举属性的工具函数：

```

// 把p中的可枚举属性复制到o中，并返回o，如果o和p中含有同名属性，则覆盖
function extend(o,p){
 for(prop in p)
 o[prop] = p[prop];
 return o;
}

// 如果o和p中有同名属性，则o中的属性不受影响
function merge(o,p){
 for(prop in p){
 if(o.hasOwnProperty(prop)) continue;
 o[prop] = p[prop];
 }
 return o;
}

// 如果o和p中没有同名属性，则从o中删除这个属性
function restrict(o,p){
 for(prop in o){
 if(!(prop in p)) delete o[prop];
 }
 return o;
}

// 如果o和p中有同名属性，则从o中删除这个属性
function subtract(o,p){
 for(prop in p){

```



```

 delete o[prop]; // 删除一个不存在的属性也不会报错
 }
 return o;
}
// 返回一个新对象，这个对象同时拥有o和p的属性，如果o和p有重名属性，则用p的属性
function union(o,p){
 return extend(extend({},o), p);
}
// 返回一个新对象，这个对象同时拥有o和p的属性，交集，但p中属性的值被忽略
function intersection(o,p){
 return restrict(extend({},o), p);
}
// 返回一个数组，这个数组包含的是o中可枚举的自有属性的名字
function keys(o){
 if(typeof o !== "object") throw TypeError; // o必须为对象
 var result = [];
 for(var prop in o){ // 所有可枚举的属性
 if(o.hasOwnProperty(prop)) // 判断是否是自有属性
 result.push(prop);
 }
 return result;
}

```

## 16-15 Object.keys()方法

取得对象上所有可枚举的实例属性，该方法接受一个对象作为参数，返回一个包含所有可枚举属性的字符串数组，如：

```

var keys = Object.keys(Person.prototype);
alert(keys); // name,age,job,sayName,toString
var p1 = new Person();
p1.name = "wujing";
p1.age = 28;
keys = Object.keys(p1);
alert(keys); // name,age

```

## 16-16 Object.getOwnPropertyNames()方法

与Object.keys()类型，但获得是所有实例属性，无论它是否可枚举如：

```

var keys = Object.getOwnPropertyNames(Person.prototype);
alert(keys); // constructor,name,age,job,sayName,toString

```

分析说明：结果中包含了不可枚举的constructor属性；

注：Object.keys()和Object.getOwnPropertyNames()方法都可以用来替代for-in循环。

## 17，类和模块化

## 17-1 Java式的类和对象

Java类和对象有以下成员：

实例字段：// 基于实例的属性或变量，用以保存独立对象的状态；  
实例方法：// 是所有实例共享的方法，由每个独立的对象调用；  
类字段：// 属于类的属性或变量，不是属于某个实例的；  
类方法：// 属于类的方法，不属于某个实例的；

// Javascript与Java的一个不同之处在于，ES中的函数都是以值的形式出现的，方法和字段之间并没有太大的区别；如果属性值是函数，那么这个属性就定义了一个方法，否则，它只是一个普通的属性或字段；虽然有这些区别，但依然可以模拟出Java中的这四种成员类型；

// ES中的类牵扯三种不同的对象，这三种对象的属性的行为可以在一定程序上模拟Java的成员：

构造函数：

// 构造函数为ES的类定义了名字，任何添加到这个构造函数对象中的属性都是类字段和类方法（如果属性值是函数的话就是类方法）；

原型：

// 原型对象的属性被所有实例所共享和继承，如果原型对象的属性值是函数的话，这个函数就作为实例的方法来调用；

实例：

// 类的每个实例都是一个独立的对象，直接给这个实例定义的属性是它专属的，如果该属性是函数，那它就是该实例的方法；

在ES中，定义类的步骤大概分为三步：

- 第一步，先定义一个构造函数，并设置初始化新对象的实例属性；
- 第二步，给构造函数的prototype对象定义实例的方法；
- 第三步，给构造函数定义类字段和类属性；

可以将这三步封装进一个defineClass()函数中，如：

```
function extend(o,p){
 for(prop in p)
 o[prop] = p[prop];
 return o;
}
// 封装一个用以定义简单类的函数
function defineClass(constructor, // 用以设置实例的属性的函数
 methods, // 实例的方法，复制到原型中
 statics){ // 类属性，复制到构造函数中
 if(methods) extend(constructor.prototype, methods);
 if(statics) extend(constructor, statics);
 return constructor;
}
// Range类的一个实现
var SimpleRange = defineClass(function(f,t){ this.f = f; this.t = t;},
 {
 includes: function(x) {return this.f <= x && x<=this.t;},
 toString: function(){ return this.f + "... " + this.t;}}
 ,{upto: function(t){ return new SimpleRange(0, t);}});
```

封装一个表示复数的类，此示例体现了ES模拟Java式的类成员：

```
// Complex.js:表示复数的类
// 复数是实数和虚数的和，并且虚数i是-1的平方根
// 构造函数内的r和i,分别保存复数的实部和虚部，它们是对应的状态
function Complex(real, imaginary){
 if(isNaN(real) || isNaN(imaginary)) throw new TypeError();
 this.r = real;
 this.i = imaginary;
}
// 当前复数对象加上另外一个复数，并返回一个新的计算和值后的复数对象
Complex.prototype.add = function(that){
 return new Complex(this.r + that.r, this.i + that.i);
};
// 当前复数乘以另外一个复数，并返回一个新的计算乘积之后的复数对象
Complex.prototype.mul = function(that){
 return new Complex(this.r*that.r - this.i*that.i, this.r*that.i +
this.i*that.r);
};
// 计算复数的模，复数的模定义为原点(0,0)到复平面的距离
Complex.prototype.mag = function(){
 return Math.sqrt(this.r*this.r + this.i * this.i);
}
// 复数的求负运算
Complex.prototype.neg = function(){
 return new Complex(-this.r, -this.i);
};
// 将复数对象转换为一个字符串
Complex.prototype.toString = function(){
 return "{" + this.r + "," + this.i + "}";
};
// 检测当前复数对象是否和另外一个复数值相等
Complex.prototype.equals = function(that){
 return that != null &&
 that.constructor === Complex &&
 this.r === that.r && this.i === that.i;
};

// 定义静态类属性和方法，直接定义为构造函数的属性
// 它们只对其参数进行操作
// 先定义一些常量，以用在复数运算中，当然也可把它们设为只读的
Complex.ZERO = new Complex(0,0);
Complex.ONE = new Complex(1,0);
Complex.I = new Complex(0,1);
// 这个类方法将实例对象的toString方法返回的字符串解析为一个Complex对象
Complex.parse = function(s){
 try{
 var m = Complex._format.exec(s);
 return new Complex(parseFloat(m[1]), parseFloat(m[2]));
 }catch(x){
 throw new TypeError("Can't parse '" + s + "' as a complex number.");
 }
}
// 定义私有属性，下划线表明它是类内部使用的，不属于类的公有API部分
Complex._format = /^{\s*([^\s,]+),\s*([^\s,]+)\s*}$/;
// 应用
var c = new Complex(2,3);
```

```
var d = new Complex(c.i,c.r);
console.log(c.add(d).toString()); // {5,5}
var result = Complex.parse(c.toString()).// 将c转换为字符串，再转换为Complex对象
 add(c.neg()). // 加上它的负数
 equals(Complex.ZERO); // 结果应当永远是零
console.log(result); // true
```

尽管ES可以模拟出Java式的类成员，但Java中有很多重要的特性是无法在ES中模拟的；比如，对于Java类的实例方法来说，实例字段可以用做局部变量，并不需要使用this来引用它们，但ES是没有办法模拟这个特性的；

## 17-2 模块化

**单例** (singleton)：确保某一个类只有一个实例，而且自行实例化并向整个系统提供这个实例；这样的类称为单例类；

单例其实有点类似于C# /C++里面的静态类；在ES中，是以对象字面量的方式来创建单例对象的；这样定义的对象，不能使用new的方式来生成另外的对象（因为不存在prototype和constructor属性）。

```
var singleton = {
 name: value,
 method: function(){
 // 这里是方法的代码
 }
};
```

**增强模块模式(即包含私有成员的单例模式)：**

为单例创建私有变量和特权方法（公有方法），从而能增强单例的可访问性；

以模块模式定义的私有变量和私有函数只有单例对象本身的特权（公有）方法可以访问到，其他外部的任何对象都不可以；

其本质上就是使用闭包及匿名函数；

```
var singleton = function(){
 var privateVar = 10; // 私有变量
 function privateFun(){ // 私有函数
 return false;
 }
 return { // 特权/公有方法和属性
 publicProperty: true,
 publicMethod: function(){
 privateVar++;
 return privateFun();
 }
 }
}();
```

从本质上来讲，这个对象字面量定义的是单例的公共接口；这种模式在需要对单例进行某些初始化，同时又需要维护其私有变量时是非常有用的，如：

```
function BaseComponent(){}
function OtherComponent(){}
var application = function(){
 var components = new Array();
```

```

components.push(new BaseComponent());
return{
 getComponentCount : function(){
 return components.length;
 },
 registerComponent : function(component){
 if(typeof component == "object"){
 components.push(component);
 }
 }
};
}();
application.registerComponent(new OtherComponent());
alert(application.getComponentCount());

```

### 增强的模块模式：

可以对模块模式进行增强，即在返回对象之前加入对其增强的代码；这种增强的模块模式适合那些单例必须是某种类型的实例，同时还必须添加某些属性和方法对其加以增强的情况，如：

```

function CustomType(){}
var singleton = function(){
 var privateVar = 10;
 function privateFun(){
 return false;
 }
 var object = new CustomType();
 object.publicProperty = true;
 object.publicMethod = function(){
 privateVar++;
 return privateFun();
 };
 return object;
}();
singleton.publicMethod();

```

如，前例中的application对象必须是BaseComponent的实例，修改如下：

```

function BaseComponent(){}
function OtherComponent(){}
var application = function(){
 var components = new Array();
 components.push(new BaseComponent());
 var app = new BaseComponent();
 app.getComponentCount = function(){
 return components.length;
 };
 app.registerComponent = function(component){
 if(component.constructor == BaseComponent){
 components.push(component);
 }else throw new Error("must be BaseComponent!");
 };
 return app;
}();
application.registerComponent(new OtherComponent()); // 异常
console.log(application.getComponentCount());

```

单例模式定义方式都是在定义时创建的单例，这样很浪费内存，可以使用惰性加载(lazy loading，更多的用于图片的延迟加载)，即先定义，再在首次调用才创建对象；

```
var singleton = function() {
 var unique;
 return {
 getInstance : function(){
 if(!unique){
 unique = new constructor();
 return unique;
 }
 }
 };
 function constructor(){
 var private_member = 10;
 function private_method(){
 console.log(private_member);
 }
 return { //这里才是真正的单例
 public_member : "",
 public_method : function(){
 private_member++;
 private_method();
 }
 };
 }
}();
singleton.getInstance().public_method();
```

## 17-3 命名空间

// 将代码组织到类中的一个重要原因是，让代码显得更加“模块化”，可以在很多不同场景中复用代码；但类不是唯一的模块化的方式，一般来说，模块是一个独立的JS文件；模块文件可以包含一个类定义、一组相关的类、一个实用函数库或者是一些待执行的代码；只要以模块的形式编写代码，任何JS代码段就可以当做一个模块；

// ES中并没有定义用以支持模块的语言结构，这也意味着在ES中编写模块化的代码更多的是遵循某一种编码约定；

// 很多JS库和客户端编程框架都包含一些模块系统，比如，Dojo工具包（包含了 web 开发中的常用JavaScript 工具）和Google的Closure库定义了provide()和require()函数，用以声明和加载模块；并且，CommonJS服务器端JavaScript标准规范创建了一个模块规范，其同样使用了require()函数；这种模块系统通常用来处理模块加载和依赖性管理；如果使用这些框架，则必须按照框架提供的模块编写约定来定义模块；

// 模块化的目标是支持大规模的程序开发，处理分散源中代码的组装，并且能让代码正确运行，哪怕包含了所不期望出现的模块代码，也可以正确执行代码；为了做到这一点，不同的模块必须避免修改全局执行上下文，因此后续模块应当在它们所期望运行的原始（或接近）上下文中执行；这实际上意味着模块应当尽可能少地定义全局标识；理想的状况是，所有模块都不应当定义超过一个）；

// 在模块创建过程中避免污染全局变量的一种方法是使用一个对象作为命名空间；它将函数和值作为命名空间对象属性存储起来（可以通过全局变量引用），而不是定义全局函数和变量；即体现了“保持干净的全局命名空间”的观点；

一个更好的方法是将类定义为一个单独的全局对象，如var sets = {}，这个sets对象是模块的命名空间，并且将每个“集合”类都定义为这个对象的属性，如：

```
sets.SingletonSet = sets.AbstractEnumerableSet.extend(...);
```

如果想使用这样定义的类，需要通过命名空间来调用所需的构造函数，

如：

```
var s = new sets.SingletonSet(1);
```

// 模块与模块之间往往需要配合工作，因此，需要注意这种命名空间的用法带来的命名冲突；此时，可以使用所谓的“导入”功能，如：

```
var Set = sets.Set;
```

// 就会将Set导入到全局命名空间中，如：var s = new Set(1,2,3); 就可以这样使用了，而不必每次都加sets；

有时，会使用更深层嵌套的命名空间，如果sets模块是另外一组更强大的模块集合的话，比如它的命名空间可能是collections.sets，其代码会这样写：

```
var collections; // 声明（或重新声明）这个全局变量
if(!collections) // 如果它原本不存在
 collections = {}; // 创建一个顶层的命名空间对象
collections.sets = {}; // 将sets命名空间创建在它的内部
// 在collections.sets内定义set类
collections.sets.AbstractSet = function(){//...}
```

最顶部的命名空间往往用来标识创建模块的作者或组织，并避免命名空间的命名冲突；比如，Google的Closure库在它的命名空间goog.structs中定义了Set类；每个开发者都会反转域名的组成部分，这样创建的命名空间前缀是全局唯一的，一般不会被其他模块作者采用；

如：

```
cn.zeronetwork.colloctions.sets
```

使用很长的命名空间来导入模块的方式很麻烦，可以将整个模块导入全局命名空间，而不是导入单独的类；如：

```
var sets = cn.zeronetwork.collections.sets;
```

按照约定，模块的文件名应当和命名空间匹配，sets模块应当保存文件sets.js中；如果这个模块使用命名空间collections.sets，那么这个文件应当保存在目录collections/下，比如：使用命名空间cn.zeronetwork.collections.sets的模块应当在文件cn/zeronetwork/collections/sets.js中；

作为私有命名空间的函数：

模块对外导出一些公用API，这些API是提供给其他程序员使用的，它包括函数、类、属性和方法；但模块的实现往往需要一些额外的辅助函数和方法，这些函数和方法并不需要在模块外部可见；比如，前面的\_v2s()函数，并不希望Set类的用户在某时刻调用这个函数，因此这个方法最好在类的外部是不可以访问的；

可以通过将模块定义在某个函数的内部来实现，即将这个函数作用域用做模块的私有命名空间（有时也称为模块函数），如下面的示例就是模块函数，如：

```
// 声明全局变量Set，使用一个函数的返回值给它赋值
// 需要立即执行，它的返回值将赋值给Set；
var Set = (function invocation(){
 function Set(){ // 这个构造函数是局部变量
 this.values = {};
 }
})();
```

```

 this.n = 0;
 this.add.apply(this, arguments);
 }
 // 给Set.prototype定义实例方法
 Set.prototype.contains = function(value){
 // 调用了v2s(), 而不是调用带有前缀的set._v2s()
 return this.values.hasOwnProperty(v2s(value));
 };
 Set.prototype.size = function(){return this.n;};
 Set.prototype.add = function(){/** */ };
 Set.prototype.remove = function(){/** */};
 Set.prototype.foreach = function(f,c){/** */};
 // 以下是上面的方法用到的一些辅助函数和变量
 // 它们不属于模块的共有的API, 但它们都隐藏在这个函数作用域内
 // 因此不必将它们定义为Set的属性或使用下划线作为其前缀
 function v2s(val){/** */};
 function objectId(o){/** */};
 var nextId = 1;
 // 这个模块的共有API是Set()构造函数
 // 需要把这个函数从私有空间中导出来, 以便在外部使用它
 return Set;
}());

```

其中函数名字invocation, 用以强调这个函数应当在定义后立即执行; namespace用来强调这个函数被用作命名空间;

一旦将模块代码封装进一个函数, 就需要一些方法导出其公用API, 以便在模块函数的外部调用它们; 模块函数返回构造函数, 这个构造函数随后赋值给一个全局变量; 将值返回已经清楚地表明API已经导出在函数作用域之外, 如果模块API包含多个单元, 则它可以返回命名空间对象, 如:

```

function AbstractSet(){}
function NotSet(){}
function AbstractEnumerableSet(){}
function SingletonSet(){}
function AbstractWritableSet(){}
function ArraySet(){}
// 创建一个全局变量用来存放集合相关的模块
var collections;
if(!collections) collections = {};
// 定义sets模块
collections.sets = (function namespace(){
 // 在这里定义多种集合类, 使用局部变量和函数
 // ...
 // 通过返回命名空间对象将API导出
 return {
 // 导出的属性名: 局部变量名字
 AbstractSet: AbstractSet,
 NotSet: NotSet,
 AbstractEnumerableSet: AbstractEnumerableSet,
 SingletonSet: SingletonSet,
 AbstractWritableSet: AbstractWritableSet,
 ArraySet: ArraySet
 };
}());
var a = new collections.sets.AbstractSet();
console.log(a);

```



另外一种类似的方式是将模块函数当做构造函数，通过new来调用，通过将它的实例赋值给this来将其导出：

```
// 创建一个全局变量用来存放集合相关的模块
var collections;
if(!collections) collections = {};
// 定义sets模块
collections.sets = (new function namespace(){
 // 在这里定义多种集合类，使用局部变量和函数
 // ...
 // 将API导出到this对象
 this.AbstractSet = AbstractSet;
 this.NotSet = NotSet
 //...
 // 注，这里没有返回值
})();
console.log(new collections.sets.AbstractSet());
```

作为一种替代方案，如果已经定义了全局命名空间对象，这个模块函数可以直接设置成那个对象的属性，不用返回任何内容，如：

```
var collections;
if(!collections) collections = {};
collections.sets = {};
(function namespace(){
 // ...
 // 将API导出到上面创建的命名空间对象上
 collections.sets.AbstractSet = AbstractSet;
 collections.sets.NotSet = NotSet
 //...
 // 导出的操作已经执行了，这里不需要再有返回值
})();
```

有些框架实现了模块加载功能，其中包括其他一些导出模块API的方法；比如，使用provides()函数来注册其API，提供exports对象用以存储模块API；由于JS目前还不具备模块管理的能力，因此应当根据所使用的框架和工具包来选择合适的模块创建和导出API的方式。

## 18，类型和对象

### 18-1 类属性

对象的类属性（class attribute）是一个字符串，用以表示对象的类型信息；ES5并没有提供设置这个属性的方法，并只有一种间接的方法可以查询它，即默认的toString()，比如，打印Object类型的对象时，返回[object Object]，因此，想要获得对象的类，可以调用对象的toString方法，然后提取返回字符串的第8个到第二个位置之间的字符；

但是，很多对象继承的toString()方法重写了，为了能调用正确的toString()版本，必须间接地调用Function.call()方法，如：

```
function classof(o){
 if(o===null) return "Null";
 if(o===undefined) return "Undefined";
 return Object.prototype.toString.call(o).slice(8,-1);
}
console.log(classof(null));
```

```
console.log(classof(1));
console.log(classof(""));
console.log(classof(false));
console.log(classof({}));
console.log(classof([]));
console.log(classof(/.\/));
console.log(classof(new Date()));
console.log(classof(window));
function F() {}
console.log(classof(new F()));
```

classof()函数可以传入任何类型的参数；但是对自定义类型，返回的也是Object，也就是说通过类属性没办法区分自定义类型；

## 18-2 检测对象的类型

### 18-2-1 instanceof运算符

```
// 如果o继承自c.prototype，那表达式o instanceof c值为true；这里的继承可以不是直接继承，如果o所继承的对象继承自另一个对象，后一个对象继承自c.prototype，这个表达式的运算结果也是true；
```

```
// 构造函数是类的公共标识，但原型是唯一 的标识；尽管instanceof右操作数是构造函数，但计算过程实际上是检测了对象的继承关系，而不是检测创建对象的构造函数；
```

如果想检测对象的原型链上是否存在某个特定的原型对象，可以使用isPrototypeOf()方法，如：

```
rang.methods.isPrototypeOf(r); // rang.methods是原型对象
```

instanceof和isPrototypeOf()方法的缺点时，无法通过对象来获得类名，只能检测对象是否属于指定的类名；在客户端javascript中有一个比较严重的问题，就是在多窗口和多框架子页面的Web应用中兼容性不好；每个窗口和框架子页面都具有单独的执行上下文，每个上下文都包含独有的全局变量和一组构造函数；在两个不同框架页面中创建的两个数组继承自两个相同但相互独立的原型对象，其中一个框架页面中的数组不是另一个框架页面的Array()构造函数的实例，instanceof运算结果是false；

构造函数和类的标识：

原型对象是类的唯一标识，当且仅当两个对象继承自同一个原型对象时，它们才是属于同一个类的实例；

而初始化对象的状态的构造函数则不能作为类的标识，两个构造函数的prototype属性可能指向同一个原型对象，那么这两个构造函数创建的实例是属于同一个类的；

尽管构造函数不像原型那样基础，但构造函数是类的“外在表现”；构造函数的名字通常用做类名，比如Person()构造函数创建Person对象；然而，更根本地讲，当使用instance运算符来检测对象是否属于某个类时会用到构造函数，如 p instanceof Person，但实际上instanceof运算符并不会检查p是否是由Person()构造函数初始化而来，而会检查p是否继承Person.prototype；

### 18-2-2 constructor属性

另一种识别对象是否属于某个类的方法是使用constructor属性，因为构造函数是类的公共标识，所以最直接的方法就是使用constructor属性，如：

```
function typeAndValue(x) {
 if(x==null) return ""; // Null和undefined没有构造函数
 switch(x.constructor){
 case Number: return "Number: " + x;
 case String: return "String: '" + x + "'";
 case Date: return "Date: " + x;
 case RegExp: return "RegExp: " + x;
 case Complex: return "Complex: " + x;
 }
}
```

使用constructor属性检测对象属于某个类的技术的不足之处和instanceof一样；在多个执行上下文的场景中它是无法正常工作的；

另外，在Javascript中也并非所有的对象都包含constructor属性；在每个新创建的函数原型上默认会有constructor属性，但我们经常会忽略原型上的constructor属性，比如前面的示例代码中所定义的两个类，它们的实例都没有constructor属性；

### 18-2-3 构造函数的名称

**另一种检测的可能的方式 是使用构造函数的名字而不是构造函数本身作为类标识符；**

两个不同窗口的Array构造函数是不相等的，但它们的名称是一样的；在一些Javascript的实现中为函数对象提供了一个非标准的属性name，用来表示函数的名称；对于那些没有name属性的Javascript实现来说，可以将函数转换为字符串，再从中提取出函数名，比如，下面的函数的getName()方法，就是使用这种方式取得函数名；如：

```
// 可以判断值的类型的type()函数
function type(o) {
 var t, c, n; // type,class,name
 if(o === null) return "null"; // 处理null值
 if(o !== o) return "nan"; // NaN和它自身不相等
 // 如果typeof的值不是object，则使用这个值，即可以识别出原始值和函数
 if((t = typeof o) !== "object") return t;
 // 返回对象的类型，除非值为Object，可以识别出大多数的内置对象
 if((c = classof(o)) !== "object") return c;
 // 如果对象构造函数的名字存在的话，则返回它
 if(o.constructor && typeof o.constructor === "function" &&
 (n = o.constructor.getName())) return n;
 // 其他的类型都无法推断，一律返回Object
 return "Object";
}

// 返回对象的类
function classof(o) {
 return Object.prototype.toString.call(o).slice(8,-1);
}

// 返回函数的名字（可能是空字符串），不是函数的话，返回null
Function.prototype.getName = function () {
 if("name" in this) return this.name;
 return this.name = this.toString().match(/function\s*([^\(]*)\s*\(/)[1];
}
```

## 18-2-4 鸭式辩型

上面所描述的检测对象的类的各种技术多少都会有问题，至少在客户端javascript中是如此；解决办法就是规避掉这些问题：不要关注“对象的类是什么”，而是关注“对象能做什么”；这种思考问题的方式在Python和Ruby中非常普通，称为“鸭式辩型”；

鸭式辩型，就是说检测对象是否实现了一个或多个方法；一个强类型的函数需要的参数必须是某种类型，而“鸭式辩型”，只要对象包含某些方法，就可以作为参数传入；

鸭式辩型在使用时，需要对输入对象进行检查，但不是检查它们的类，而是用适当的名字来检查它们所实现的方法；

示例：quacks()函数用以检查一个对象（第一个参数）是否实现了剩下的参数所表示的方法：

```
// 利用鸭式辩型实现的函数
// 如果o实现了除第一个参数之外的参数所表示的方法，则返回true
function quacks(o /*, ...*/) {
 for(var i=1; i<arguments.length; i++){ // 遍历o之后所有参数
 var arg = arguments[i];
 switch(typeof arg){
 case 'string':
 if(typeof o[arg] !== "function")
 return false;
 continue;
 case 'function':
 // 如果实参是函数，则使用它的原型
 arg = arg.prototype; // 进行下一个case
 case 'object':
 for(var m in arg){ // 遍历对象的每个属性
 // 跳过不是方法的属性
 if(typeof arg[m] !== "function") continue;
 if(typeof o[m] !== "function") return false;
 }
 }
 }
 // 如果程序能执行到这里，说明o实现了所有的方法
 return true;
}
```

## 18-2-5 集合类

集合（set）是一种数据结构，用以表示非重复值的无序集合；集合的基础方法包括添加值、检测值是否在集合中，这种集合需要一种通用的实现，以保证操作效率；

一个更加通用的Set类，它实现了从javascript值到唯一字符串的映射，将字符串用做属性名；对象和函数具备可靠的唯一字符串表示；因此集合类必须给集合中的每个对象或函数定义一个唯一的属性标识：

```
// Set.js 值的任意集合
function Set(){
 this.values = {}; //集合数据保存在对象的属性里
 this.n = 0; // 集合中值的个数
 this.add.apply(this, arguments); // 把所有参数都添加进这个集合
}
// 把所有参数都添加进这个集合
Set.prototype.add = function(){
 for(var i=0; i<arguments.length; i++){
 var val = arguments[i];
```

```

 var str = Set._v2s(val); // 把它转换为字符串
 if(!this.values.hasOwnProperty(str)){ // 如果不在集合里
 this.values[str] = val;
 this.n++;
 }
 }
 return this; // 支持链式调用
};

// 从集合删除元素，这些元素由参数指定
Set.prototype.remove = function(){
 for(var i=0;i<arguments.length; i++){
 var str = Set._v2s(arguments[i]);
 if(this.values.hasOwnProperty(str)){
 delete this.values[str];
 this.n--;
 }
 }
};

// 如果集合包含该值，返回true，反之返回false
Set.prototype.contains = function(value){
 return this.values.hasOwnProperty(Set._v2s(value));
};

// 返回集合的大小
Set.prototype.size = function(){
 return this.n;
};

// 遍历集合中的所有元素，在指定的上下文中调用f
Set.prototype.foreach = function(f, context){
 for(var s in this.values){
 if(this.values.hasOwnProperty(s)) // 忽略继承的属性
 f.call(context, this.values[s]);
 }
};

// 内部函数，用以将任意Javascript值和唯一的字符串对应起来
Set._v2s = function(val){
 switch(val){
 case undefined: return 'u'; // 特殊的原始值
 case null: return 'n'; // 只有一个字母代码
 case true: return 't';
 case false: return 'f';
 default: switch(typeof val){
 case 'number': return '#' + val; // 数字带#前缀
 case 'string': return '"' + val; // 字符串带"前缀
 default: return '@' + objectId(val); // 对象和函数带有@
 }
 }
}

function objectId(o){
 var prop = "|**objectId**|"; // 私有属性，用于存放id
 if(!o.hasOwnProperty(prop)){ // 如果对象没有id
 o[prop] = Set._v2s.next++; // 将下一个值赋给它
 }
 return o[prop];
}

Set._v2s.next = 100; // 设置初始id的值
// 应用
var persons = new Set("a","b");
persons.add("c",1,null,{name:"wangwei"});

```

```

console.log(persons);
console.log(persons.contains("b"));
console.log(persons.size());
function show(v){
 console.log("元素: " + v);
}
persons.foreach(show,window);

```

## 18-2-6 枚举类型-示例

枚举类型是一种类型，它是值的有限集合，如果值定义为这个类型则该值是可列出（可枚举）的；

以下的示例包含一个单独函数enumeration()，但它不是构造函数，它没有定义一个名叫enumeration的类，相反，它是一个工厂方法，每次调用它创建一个新的类，如：

```

// 使用4个值创建新的Coin类
var Coin = enumeration({Penny:1, Nickel:5, Dime:10, Quarter:25});
var c = Coin.Dime; // 这是新类的实例
console.log(c instanceof Coin); // true
console.log(c.constructor == Coin); // true
console.log(Coin.Quarter + 3 * Coin.Nickel); // 40,将值转换为数字
Coin.Dime == 10; // true, 更多转换为数字的例子
Coin.Dime > Coin.Nickel; // true
String(Coin.Dime) + ":" + Coin.Dime;
// 枚举类型
// 实参对象表示类的每个实例的名/值
// 返回一个构造函数，它标识这个新类
// 注：这个构造函数也会抛出异常，不能使用它来创建该类型的新实例
// 返回的构造函数包含名/值对的映射表
// 包括由值组成的数组，以及一个foreach()迭代器
function enumeration(namesToValues){
 // 这个虚拟的构造函数是返回值
 var enumeration = function(){throw "Can't Instantiate Enumerations";};
 // 枚举值继承自这个对象
 var proto = enumeration.prototype = {
 constructor: enumeration, // 标识类型
 toString: function(){ return this.name;}, // 返回名字
 valueOf: function(){ return this.value;}, // 返回值
 toJSON: function(){ return this.name;} // 转换为JSON
 };
 // 用以存放枚举对象的数组
 enumeration.values = [];
 // 创建新类型的实例
 for(name in namesToValues){
 var e = Object.create(proto); // 创建一个代表它的对象
 e.name = name; // 给它一个名字
 e.value = namesToValues[name]; // 给它一个值
 enumeration[name] = e; // 将它设置为构造函数的属性
 enumeration.values.push(e);
 }
 // 一个类方法，用来对类的实例进行迭代
 enumeration.foreach = function(f,c){
 for(var i=0; i<this.values.length; i++){
 f.call(c, this.values[i]);
 }
 };
 // 返回标识这个新类型的构造函数
 return enumeration;
}

```

```
}
```

用以上定义的枚举类型实现一副扑克牌的类:

// 定义一个表示玩牌的类

```
function Card(suit, rank){
 this.suit = suit; // 每张牌都有花色
 this.rank = rank; // 点数
}
// 使用枚举类型定义花色和点数
Card.Suit = enumeration({Clubs:1, Diamonds: 2, Hearts: 3, Spades: 4});
Card.Rank = enumeration({Two:2, Three:3, Four:4, Five:5, Six:6, Seven:7, Eight:8,
 Nine:9, Ten:10, Jack:11, Queen:12, King:13, Ace:14});
// 定义用以描述牌面的文本
Card.prototype.toString = function(){
 return this.rank.toString() + " of " + this.suit.toString();
};
// 比较扑克牌中两张牌的大小
Card.prototype.compareTo = function(that){
 if(this.rank < that.rank) return -1;
 if(this.rank > that.rank) return 1;
 return 0;
};
// 以扑克牌的玩法规则对牌进行排序的函数
Card.orderByRank = function(a, b){ return a.compareTo(b);};
// 以桥牌的玩法规则对牌进行排序的函数
Card.orderBySuit = function(a,b){
 if(a.suit < b.suit) return -1;
 if(a.suit > b.suit) return 1;
 if(a.rank < b.rank) return -1;
 if(a.rank > b.rank) return 1;
 return 0;
};
// 定义用以表示一副标准扑克牌的类
function Deck(){
 var cards = this.cards = []; // 一副牌就是由牌组成的数组
 Card.Suit.forEach(function(s){ // 初始化这个数组
 Card.Rank.forEach(function(r){
 cards.push(new Card(s, r));
 });
 })
}
// 洗牌的方法: 重新洗牌并返回洗好的牌
Deck.prototype.shuffle = function(){
 // 遍历数组中的每个元素, 随机找出牌面最小的元素, 并与之(当前遍历的元素)交换
 var deck = this.cards, len = deck.length;
 for(var i = len-1; i>0; i--){
 var r = Math.floor(Math.random()*(i+1)), temp; // 随机数
 temp = deck[i], deck[i] = deck[r], deck[r] = temp; // 交换
 }
 return this;
}
// 发牌的方法: 返回牌的数组
Deck.prototype.deal = function(n){
 if(this.cards.length < n) throw "Out of cards";
 return this.cards.splice(this.cards.length - n, n);
};
// 创建一副新扑克牌, 洗牌并发牌
var deck = (new Deck()).shuffle();
```



```
var hand = deck.deal(13).sort(Card.orderBySuit);
console.log(deck);
console.log(hand);
```

## 18-2-7 标准转换方法

对象类型转换所用到的方法，其中有一些在进行转换时由JavaScript解释器自动调用；

不需要为定义的每个类都实现这些方法，但这些方法的确非常重要，如果没有为自定义的类实现这些方法，也应当是有意为之，而不应该忽略掉它。

// **toString()**方法：这个方法的作用是返回一个可以表示这个对象的字符串；在希望使用字符串的地方用到对象的话，JavaScript会自动调用这个方法；如果没有实现这个方法，类会默认从**Object.prototype**中继承**toString()**方法，这个方法的运算结果是“**[object Object]**”，这个字符串用处不大；**toString()**方法应当返回一个可读的字符串，这样最终用户才能将这个输出值利用起来，然而有时候并不一定非要如此，不管怎样，可以返回可读字符串的**toString()**方法也会让后续的工作更加轻松；

// **toLocaleString()**和**toString()**极为类似：**toLocaleString()**是以本地敏感性的方式来将对象转换为字符串；默认情况下，对象所继承的**toLocaleString()**方法只是简单地调用**toString()**方法；有一些内置类型包含有用的**toLocaleString()**方法用以返回本地化相关的字符；如果需要为对象以字符的转换定义**toString()**方法，那么同样需要定义**toLocaleString()**方法用以处理本地化的对象到字符串的转换；

// **valueOf()**方法，它用来将对象转换为原始值；比如，当数学运算符（除了“+”）和关系运算符作用于数字文本表示的对象时，会自动调用 **valueOf()**方法；大多数对象都没有合适的原始值来表示它们，也没有定义这个方法；

// **toJSON()**方法，这个方法是由**JSON.stringify()**自动调用的；JSON格式用于序列化良好的数据结构，而且可以处理JavaScript原始值、数组和纯对象；它和类无关，当对一个对象执行序列化操作时，它会忽略对象的原型和构造函数；比如将**Range**对象或**Complex**对象作为参数传入**JSON.stringify()**，将会返回诸如“**{“from”:1, “to”:3}**”或**{...}**这种字符串；如果将这些字符串传入**JSON.parse()**，则会得到一个和**Range**对象或**Complex**对象具有相同属性的纯对象，但这个对象不会包含从**Range**和**Complex**继承来的方法；

// 这种序列化操作非常适用于诸如**Range**类和**Complex**这种类，但对于其他一些类则必须自定义**toJSON**方法来定制个性化的序列化格式；如果一个对象有**toJSON**方法，**JSON.stringify()**并不会对传入的对象做序列化操作，而会调用 **toJSON()**来执行序列化操作；比如，**Date**对象的**toJSON()**方法可以返回一个表示日期的字符；

对于一个集合，最接近JSON的表示方法就是数组；

```
// 将这些方法添加到Set类的原型对象中
extend(Set.prototype, {
 // 将集合转换为字符
 toString: function() {
 var s = "{", i=0;
 this.forEach(function(v) { s += ((i++>0) ? ", " : "") + v; });
 return s + "}";
 },
 // 类似toString，但是对于所有的值都将调用 toLocaleString()
 toLocaleString: function() {
 var s = "{", i=0;
 this.forEach(function(v) {
 if(i++>0) s += ", ";
 if(v == null) s += v; // null和undefined
 else s += v.toLocaleString(); // 其他情况
 });
 return s + "}";
 },
}
```



```
// 将集合转换为值数组
toArray: function(){
 var a = [];
 this.forEach(function(v){ a.push(v);});
}
});
// 对于要从JSON转换为字符串的集合都被当做数组来对待
Set.prototype.toJSON = Set.prototype.toArray;
```

## 18-3 比较方法

Javascript的相等运算符比较对象时，比较的是引用而不是值；也就是说，给定两个对象引用，如果要看它们是否指向同一个对象，不是检查这两个对象是否具有相同的属性名和相同的属性值，而是直接比较这两个单独的对象是否相等，或者比较它们的顺序；

如果定义一个类，并且希望比较类的实例，应该定义合适的方法来执行比较操作；

Java语言有很多用于对象比较的方法，Javascript可以模拟这些方法；为了能让自定义类的实例具备比较的功能，定义一个名为equals()实例方法；这个方法只能接收一个实参，如果这个实参和调用此方法的对象相等的话则返回true；当然，这个相等的含义是根据类的上下文来决定的；

对于简单的类，可以通过简单地比较它们的constructor属性来确保两个对象是相同类型，然后比较两个对象的实例属性以保证它们的值相等，如：

```
// 重写它的constructor属性
Range.prototype.constructor = Range;
// 一个Range对象和其他不是Range的对象均不相等
// 当且仅当两个范围的端点相等，它们才相等
Range.prototype.equals = function(that){
 if(that == null) return false;
 if(that.constructor != Range) return false; // 处理非Range对象
 return this.from == that.from && this.to == that.to;
};
```

给Set类定义equals()方法，如：

```
Set.prototype.equals = function(that){
 if(this === that) return true; // 一些闪要情况的快捷处理
 // 如果that对象不是一个集合，它和this不相等
 // 用到了instanceof，使得这个方法可以用于Set的任何子类
 // 如果希望采用鸭式辩型的方法，可以降低检查的严格程序
 // 或者可以通过this.constructor == that.constructor来加强检查的严格程序
 // 注，null和undefined两个值是无法用于instanceof运算的
 if(!(that instanceof Set)) return false;
 // 如果两个集合的大小不一样，则它们不相等
 if(this.size() != that.size()) return false;
 // 现在检查两个集合中的元素是否完全一样
 // 如果两个集合不相等，则通过抛出异常来终止foreach循环
 try{
 this.forEach(function(v){ if(!that.contains(v)) throw false;});
 return true;
 }catch(x){
 if(x===false) return false; // 如果集合中有元素在另外一个集合中不存在
 throw x; // 重新抛出异常
 }
}
```

如果将对象用于JavaScript的关系比较运算符，比如：<和<=，JavaScript会首先调用对象的valueOf方法，如果这个方法返回一个原始值，则直接比较原始值；但大多数类并没有valueOf()方法，为了按照显式定义的规则来比较这些类型的对象，可以定义一个compareTo()的方法；如：

```
Range.prototype.compareTo = function(that){
 if(!(that instanceof Range))
 throw new Error("Can't compare a Range with " + that);
 var diff = this.from - that.from; // 比较下边界
 if(diff == 0) diff = this.to - that.to; // 如果相等，再比较上边界
 return diff;
}
```

给类定义了compareTo()方法后，可以对类的实例组成的数组进行排序了；Array.sort()方法可以接收一个可选的参数，这个参数是一个函数，用来比较两个值的大小，这个函数返回值的约定和compareTo()方法保持一致，如：

```
ranges.sort(function(a,b){return a.compareTo(b)});
```

排序运算非常重要，如果已经为类定义了实例方法compareto()，还应当参照这个方法定义一个可传入这两个参数的比较函数；如：

```
Range.byLowerBound = function(a,b){
 return a.compareTo(b);
};
ranges.sort(Range.byLowerBound);
```

## 18-4 方法借用

多个类中的方法可以共用同一个单独的函数，比如，Array类通常定义了一些内置方法，如果定义了一个类，它的实例是类数组的对象，则可以从Array.prototype中将函数复制至所定义的类的原型对象中；

如果以经典的面向对象语言的视角来看JavaScript的话，把一个类的方法用到其他的类中的做法也称做：多重继承，然而，JavaScript并不是经典的面向对象语言，所以将这种方法重用称为“方法借用”；

不仅Array的方法可以借用，还可以自定义泛型方法，如：

```
var generic = {
 // 返回一个字符串，这个字符串包含构造函数的名字
 // 以及所有非继承来的、非函数属性的名字和值
 toString: function(){
 var s = '[';
 if(this.constructor && this.constructor.name)
 s += this.constructor.name + ": ";
 // 枚举所有非继承且非函数的属性
 var n = 0;
 for(var name in this){
 if(!this.hasOwnProperty(name)) continue; // 跳过继承的属性
 var value = this[name];
 if(typeof value === "function") continue // 跳过方法
 if(n++) s += ", ";
 s += name + "=" + value;
 }
 return s + ']';
 },
 // 这种方法适合于那些实例属性是原始值的情况
}
```

```
// 这里还处理一种特殊的情况，就是忽略由Set类添加的特殊属性
equals: function(that){
 if(that == null) return false;
 if(this.constructor !== that.constructor) return false;
 for(var name in this){
 if(name === "|**objectid**|") continue; // 跳过特殊属性
 if(!this.hasOwnProperty(name)) continue; // 跳过继承来的属性
 if(this[name] !== that[name]) return false; // 比较是否相等
 }
 return true; // 如果所有属性都匹配，两个对象相等
}
};
Range.prototype.equals = generic.equals;
```

## 18-5 私有状态

在经典的面向对象编程中，经常需要将对象的某个状态封装或隐藏在对象内，只有通过对象的方法才能访问这些状态，对外只暴露一些重要的状态变量可以直接读写；为了实现这个目的，类似Java的编程语言允许声明类的“私有”实例字段，这些私有实例字段只能被类的实例方法访问，且在类的外部是不可见的。

可以通过将变量（或参数）闭包在一个构造函数内来模拟实现私有实例字段，调用构造函数会创建一个实例；

```
// 对Range类的读取端点方法的简单封装
function Range(from, to){
 // 不要将端点保存为对象的属性，相反定义存取器函数来返回端点的值
 // 这些值都保存在闭包中
 this.from = function(){return from;};
 this.to = function(){return to;};
}
// 原型上的方法无法直接操作端点，它们必须调用存取器方法
Range.prototype = {
 constructor: Range,
 includes: function(x){return this.from() <= x && x <= this.to();},
 foreach: function(f){
 for(var x = Math.ceil(this.from()), max = this.to(); x <= max; x++)
 f(x);
 },
 toString: function(){return "(" + this.from() + "... " + this.to() + ")";}
}
```

这种封装技术造成了更多系统开销，使用闭包来封装类的状态的类一定会比不使用封装的状态变量的等价类运行速度更慢，并占用更多内存。

## 18-6 构造函数的重载和工厂方法

有时候，对象的初始化需要多种方式，可以通过重载这个构造函数让它根据传入参数的不同来执行不同的初始化方法，如，重载Set构造函数：

```
function Set(){
 this.values = {};
 this.n = 0;
 // 如果传入一个类数组的对象，将这个元素添至集合中
 // 否则，将所有的参数都添加至集合中
 if(arguments.length == 1 && isArrayLike(arguments[0]))
 this.add.apply(this, arguments[0]);
 else if(arguments.length > 0)
 this.add.apply(this, arguments);
}
```

通过工厂方法使用数组初始化Set对象：

```
Set.fromArray = function(a){
 s = new Set(); // 创建一个空集合
 s.add.apply(s, a); // 将数组a的成员作为参数传入add()方法
 return s;
}
```

在ES中是可以定义多个构造函数继承自一个原型对象的，由这些构造函数的任意一个所创建的对象都属于同一类型；：

```
// Set类的一个辅助构造函数
function SetFromArray(a){
 // 通过以函数的形式调用Set()来初始化这个新对象
 // 将a的元素作为参数传入
 Set.apply(this, a);
}
// 设置原型，以便SetFromArray能创建Set的实例
SetFromArray.prototype = Set.prototype;
var s = new SetFromArray([1,2,3]);
console.log(s instanceof Set); // true
```

## 19，继承

### 19-1 继承机制的实现

- 要实现继承，首先定义父类（基类）；基于父类，再定义子类；
- 有些父类不能直接使用，只是为了让子类继承，此类为抽象类；
- 创建的子类将继承父类的所有属性和方法，包括构造函数及方法的实现(不包括私有成员)；
- 子类可以添加父类中没有的新属性和方法，也可以覆盖父类中的属性和方法；

### 19-2 构造函数继承

在JS中实现继承的方式不止一种，因为JS中的继承机制并不是明确规定的，而是通过模仿实现的；

借用构造函数(对象冒充)：构造函数使用this关键字给所有属性和方法赋值，因为构造函数只是一个函数，所以可使classA的构造函数成为classB的方法，然后调用它classB就会得到classA的构造函数中定义的属性和方法：

```
function ClassA(sColor){
 this.color=sColor;
 this.sayColor=function(){
 console.log(this.color);
 }
}
function ClassB(sColor){
 this.newMethod=ClassA;
 this.newMethod(sColor);
 delete this.newMethod;
}
```

子类还可以添加自己的成员:

```
function ClassA(sColor){
 this.color=sColor;
 this.sayColor=function(){
 console.log(this.color);
 }
}
function ClassB(sColor,sName){
 this.newMethod=ClassA;
 this.newMethod(sColor);
 delete this.newMethod;
 this.name=sName;
 this.sayName=function(){
 console.log(this.name);}
}
var objA=new ClassA("red");
var objB=new ClassB("blue","wangwei");
objA.sayColor();
objB.sayColor();
objB.sayName();
```

多继承: 一个类可以继承多个父类:

```
function ClassX(){}
function ClassY(){}
function ClassZ(){
 this.newMethod=ClassX;
 this.newMethod();
 delete this.newMethod;

 this.newMethod=ClassY;
 this.newMethod();
 delete this.newMethod;
}
```

以上的实现继承的方法很流行, 因此JS专门为Function对象加入了两个新方法, 即call()和apply() 用于继承的实现;

此种方法与传统的方法一样, 只不过把新方法的赋值、调用和删除换成call方法或apply()即可; 如:

```
function ParentType(){
 this.colors = ["red", "blue", "green"];
```

```

}
function SubType(){
 ParentType.call(this);
}
var s1 = new SubType();
s1.colors.push("black");
console.log(s1.colors);
var s2 = new SubType();
console.log(s2.colors);
// 带参数
function ClassB(sColor){//}
function ClassB(sColor,sName){
 ClassA.call(this,sColor);
 this.name=sName;
 this.sayName=function(){
 console.log(this.name);
 }
}
//...
ParentType.sayCors = function(){
 console.log("show colors");
}
//...
s1.sayCors(); // s1.sayCors is not a function

```

## 19-3 原型继承

原型继承并没有使用严格意义上的构造函数，而是借助原型可以基于已有的对象创建新对象，同时也不必因此创建自定义类型；如：

```

function object(o){
 function F(){}
 F.prototype = o;
 return new F();
}
function object(o){
 function F(){}
 F.prototype = o;
 return new F();
}
var person = {
 name:"wangwei",
 friends:["wujing","lishi"]
};
var p1 = object(person);
p1.name = "wujing";
p1.friends.push("guanli");
var p2 = object(person);
p2.name = "toto";
p2.friends.push("adu");
console.log(p1.friends); // wujing,lishi,guanli,adu
console.log(person.friends); // wujing,lishi,guanli,adu

```

object()完全可以由Object.create()代替，如：

```

var p1 = Object.create(person);

```

## 19-4 原型链

// ES对象的属性有两种，一种是自有属性（own property），另一种是从原型对象继承而来的；

// ES将原型链作为实现继承的主要方法；

// 基本思想是利用原型让一个对象继承另一个对象的属性和方法；

// 原型链的基本概念原理：构造函数、原型和实例的关系：每个构造函数都有一个原型对象，原型对象都包含一个指向构造函数的指针，而实例都包含一个指向原型对象的内部指针；此时，可以让原型对象等于另一个类型的实例，即该原型对象将包含一个指向另一个原型的指针，相应地，另一个原型中也包含着一个指向另一个构造函数的指针；如果另一个原型又是另一个类型的实例，以上关系依然成立，如此层层递进，就构成了实例与原型的链条；这就是所谓的原型链的基本概念。

实现原型链有一种基本模式，如：

```
function SuperType(){
 this.supproperty = true;
}
SuperType.prototype.getSupervalue = function(){
 return this.supproperty;
};
function SubType(){
 this.subproperty = true;
}
SubType.prototype = new SuperType(); // 继承了SuperType
SubType.prototype.getSubTypeValue = function(){
 return this.subproperty;
};
var instance = new SubType();
alert(instance.getSupervalue()); // true
```

通过原型链，本质上扩展了之前所说的原型搜索机制；即，当以读取模式访问一个实例属性时，首先会在实例中搜索该属性，如果没有找到，则会继续搜索实例的原型；在通过原型链实现继承的情况下，搜索过程就得以沿着原型链继续向上。

### 19-4-1 默认的原型：

所有引用类型默认都继承了Object，而这个继承也是通过原型链实现的；当调用instance.toString()时，实际上调用的是保存在Object.prototype中的那个方法。

### 19-4-2 确定原型和实例的关系：

可以通过两种方式来确定原型和实例之间的关系；第一种方式使用instanceof操作符，测试实例与原型链中出现过的构造函数，均返回true；如：

```
alert(instance instanceof Object); // true
alert(instance instanceof SuperType); // true
alert(instance instanceof SubType);
```

第二种为isPrototypeOf方法，只要是原型链中出现过的原型，都可以说是该原型链所派生的实例的原型，均返回true，如：

```
alert(Object.prototype.isPrototypeOf(instance)); // true
alert(SuperType.prototype.isPrototypeOf(instance)); // true
alert(SubType.prototype.isPrototypeOf(instance)); // true
```

### 19-4-3 谨慎地定义方法：

子类型有时候需要覆盖超类型中的某个方法，或者需要添加超类中不存在的某个方法；但不管怎样，给原型添加方法的代码一定要放在替换原型的语句之后，如

```
// 接前例
SubType.prototype = new SuperType(); // 继承了SuperType
SubType.prototype.getSubTypeValue = function(){
 return this.subproperty;
};
SubType.prototype.getSuperValue = function(){ // 重写父类的方法
 return false;
}
var instance = new SubType();
alert(instance.getSuperValue());
```

通过原型链实现继承时，谨慎使用对象字面量添加原型方法，因为其本质是会重写原型链；如：

```
// 在前面的示例中改写
SubType.prototype = new SuperType(); // 继承了SuperType
SubType.prototype = { // 使用字面量添加新方法，会导致上一行代码无效，但注释这一段后就没有问题
 getSubValue:function(){
 return this.subproperty;
 },
 otherMethod:function(){
 return false;
 }
};
var instance = new SubType();
alert(instance.getSuperValue()); // error;
```

### 19-4-4 原型链的问题

原型链虽然很强大，可以用它来实现继承，但它也存在一些问题；其中，最主要的问题来自包含引用类型值的原型，因为原型链中的包含引用类型值的原型属性会被所有实例共享，这也正是为什么要在构造函数中，而不是在原型对象中定义属性的原因；如：

```
function SuperType(){
 this.colors = ["red","blue","green"];
}
function SubType(){}
SubType.prototype = new SuperType(); // 继承了SuperType
var instance = new SubType();
instance.colors.push("black");
console.log(instance.colors); // red,blue,green,black
var instance2 = new SubType();
console.log(instance2.colors); // red,blue,green,black
```



第二个问题，在创建子类的实例时，不能向超类的构造函数中传递参数；即没有办法在不影响所有对象实例的情况下，给超类的构造函数传递参数；

## 19-5 组合继承

组合继承（combination inheritance），有时候也叫做伪经典继承，指的是将原型链和借用构造函数的技术组合到一块，从而发挥二者之长的一种继承模式；原理是：用借用构造函数来实现对实例的属性的继承，用原型链继承prototype对象的属性和方法，这样，既通过在原型上定义方法实现了函数复用，又能保证每个实例都有它自己的属性，如：

```
function Super(name){
 this.name = name;
 this.colors = ["red","blue","green"];
}
Super.prototype.sayName = function(){
 console.log(this.name);
};
function Suber(name,age){
 Super.call(this,name); // 继承了SuperType
 this.age = age;
}
Suber.prototype = new Super(); // 继承属性
Suber.prototype.constructor = Suber;
Suber.prototype.sayAge = function(){
 console.log(this.age);
}
var instance = new Suber("wangwei",18);
instance.colors.push("black");
console.log(instance.colors); // red,blue,green,black
instance.sayName(); // wangwei
instance.sayAge(); // 18
var instance2 = new Suber("wujing",27);
console.log(instance2.colors); // red,blue,green
instance2.sayName(); // wujing
instance2.sayAge(); // 27
```

## 19-6 寄生式继承

寄生式（parasitic）继承同样是道格拉斯.克罗斯福提出，并与原型继承相关；其思路与寄生构造函数和工厂模式类似，即创建一个仅用于封装继承过程的函数，该函数在内部以某种方式来增强对象，最后再返回对象；如：

```
function object(o){
 function F(){}
 F.prototype = o;
 return new F();
}
function createAnother(original){
 var clone = object(original); // 通过调用函数创建一个新对象
 // var clone = Object.create(original);
 clone.sayHi = function(){ // 以某种方式增强这个对象
 console.log("Hi");
 };
 return clone;
}
// 使用createAnother()函数
```

```
var person = {
 name: "wangwei",
 friends: ["wujing", "lishi"]
};
var p1 = createAnother(person);
p1.sayHi(); // Hi
```

## 19-7 寄生组合

前文所说的组合继承是ES最常用的继承模式，不过，它也有不足；组合式最大的不足是调用了两次超类的构造函数：一次是在创建子类型原型时、另一次是在子类构造函数内部；如：

```
function Super(name){
 this.name = name;
 this.colors = ["red", "green", "blue"];
}
Super.prototype.sayName = function(){
 console.log(this.name);
};
function Suber(name, age){
 Super.call(this, name); // 第二次调用SuperType
 this.age = age;
}
Suber.prototype = new Super(); // 第一次调用SuperType()
Suber.prototype.constructor = Suber;
Suber.prototype.sayAge = function(){
 console.log(this.age);
}
var s = new Suber("wangwei", 18);
console.log(s);
console.log(Object.getPrototypeOf(s));
console.log(s.hasOwnProperty("name")); // true
console.log(s.hasOwnProperty("colors")); // true
```

可以采用寄生组合式继承解决这个问题；

寄生组合式继承，即通过借用构造函数来继承属性，通过原型链的混成形式来继承方法；

基本思路：不必为了指定子类的原型而调用超类的构造函数，所需要的无非就是超类原型的一个副本；本质上就是使用寄生式继承超类的原型，再将结果指定给子类的原型；

寄生组合式继承的基本模式如：

```
function inheritPrototype(subType, superType){
 var prototype = Object.create(superType.prototype); // 创建对象
 prototype.constructor = subType; // 指定构造函数为subType
 subType.prototype = prototype; // 指定subType原型为prototype
}
function SubType(name, age){
 SuperType.call(this, name);
 this.age = age;
}
inheritPrototype(SubType, SuperType); // 替换了原来的两行
SubType.prototype.sayAge = function(){
 console.log(this.age);
}
```

## 19-8 示例(形状)

```
一.创建基类: Polygon(['pɒlɪɡən] area: ['eəriə]):
function Polygon(iSides){
 this.sides=iSides;
}
Polygon.prototype.getArea=function(){
 return 0;
}
```

创建子类 Triangle: 确定为3条边, 所以覆盖Polygon类的sides属性, 设置为3; 使用三角形的面积公式:  $1/2 \text{底} \times \text{高}$  来覆盖getArea()方法, 因此需要新属性 base 和 height;

```
function Triangle(iBase,iHeight){
 Polygon.call(this,3);
 this.base=iBase;
 this.height=iHeight;
}
Triangle.prototype=new Polygon();
Triangle.prototype.getArea=function(){
 return 0.5*this.base*this.height;
}
```

创建子类 Rectangle: 确定为4条边, 所以覆盖Polygon类的sides属性, 设置为4; 使用矩面积公式: 长\*宽来覆盖getArea()方法, 需要新属性 length 和 width;

```
function Rectangle(iLength,iwidth){
 Polygon.call(this,4);
 this.length=iLength;
 this.width=iwidth;
}
Rectangle.prototype=new Polygon();
Rectangle.prototype.getArea=function(){
 return this.length*this.width;
}
var triangle=new Triangle(12,4);
var rectangle=new Rectangle(20,10);
console.log(triangle.sides);
console.log(triangle.getArea());
console.log(rectangle.sides);
console.log(rectangle.getArea());
```

## 19-9 深入继承与应用

扩展应用, 封装一个函数:

```
function defineSubClass(superclass, // 父类的构造函数
 constructor, // 子类的构造函数
 methods, // 实例方法: 复制至原型中
 statics) // 类属性: 复制到构造函数中
{
 // 建立子类的原型对象
 constructor.prototype = Object.create(superclass.prototype);
 constructor.prototype.constructor = constructor;
}
```

```

 // 像对常规类一样复制方法和类属性
 if(methods) extend(constructor.prototype, methods);
 if(statics) extend(constructor, statics);
 return constructor;
}
// 可以添加到Function的原型中
Function.prototype.extend = function(constructor, methods, statics){
 return defineSubClass(this, constructor, methods, statics);
};

```

定义一个Set类的子类SingletonSet，此类是一个特殊的集合，它是只读的，而且含有单独的常量成员：

```

// SingletonSet构造函数
function SingletonSet(member){
 this.member = member; // 集合中唯一的成员
}
// 创建一个原型对象，这个原型对象继承自Set的原型
SingletonSet.prototype = Object.create(Set.prototype);
// 给原型添加属性，如果有同名的就覆盖Set.prototype中同名属性
extend(SingletonSet.prototype, {
 constructor: SingletonSet,
 // 这个集合是只读的，调用add()或remove()都会报错
 add: function(){throw "read-only set";},
 remove: function(){throw "read-only set";},
 // SingletonSet的实例中永远只有一个元素
 size: function(){return 1;},
 // 这个方法只调用一次，传入这个集合的唯一成员
 foreach: function(f, context){f.call(context, this.member)},
 // 检查传入的值是否匹配这个集合唯一的成员
 contains: function(x){return x === this.member;}
});

```

也可以为SingletonSet类定义自己的equals()，会更高效一些，如：

```

SingletonSet.prototype.equals = function(that){
 return that instanceof Set && that.size() == 1 &&
 that.contains(this.member);
};

```

利用构造函数和原型链的示例，比如定义了Set类的子类NonNullSet，它不允许null和undefined作为它的成员；为了使用这种方式对成员做限制，NonNullSet需要在其add()方法中对null和undefined值做检测；但是，它需要完全重新实现一个add()方法：

```

// NonNullSet类是Set的子类，它的成员不能是null或undefined
function NonNullSet(){
 // 仅链接到父类
 // 作为普通函数调用父类的构造函数来初始化通过该构造函数调用创建的对象
 Set.apply(this, arguments);
}
// 将NonNullSet设置为Set的子类
NonNullSet.prototype = Object.create(Set.prototype);
NonNullSet.prototype.constructor = NonNullSet;
// 为了将null和undefined排除在外，只须重写add()方法
NonNullSet.prototype.add = function(){
 // 检查参数是不是null或undefined

```

```

 for(var i=0; i<arguments.length; i++){
 if(arguments[i] == null)
 throw new Error("Can't add null or undefined to a NonNullSet");
 }
 // 调用父类的add()方法以执行实际插入操作
 return Set.prototype.add.apply(this,arguments);
};

```

NonNullSet类只是在执行add()时，对参数进行了过滤；为此，可以定义一个类工厂函数，传入一个过滤函数，返回一个新的Set子类；：

```

// 类工厂和方法链
// 这个函数返回具体Set类的子类
// 关重写该类的add()方法用以对添加的元素做特殊处理
function filterSetSubClass(superclass, filter){
 // 子类构造函数
 var constructor = function(){
 superclass.apply(this, arguments); // 调用父类构造函数
 };
 var proto = constructor.prototype = Object.create(superclass.prototype);
 proto.constructor = constructor;
 proto.add = function(){
 // 在添加任何成员之前首先使用过滤器将所有参数进行过滤
 for(var i=0; i<arguments.length; i++){
 var v = arguments[i];
 if(!filter(v)) throw("value " + v + " rejected by filter");
 }
 // 调用父类的add()方法
 superclass.prototype.add.apply(this, arguments);
 };
 return constructor;
}
// 定义一个只能保存字符串的集合类
var StringSet = filterSetSubClass(Set, function(x){return typeof x ===
"string";});
// 定义一个成员不能是null或undefined或函数
var MySet = filterSetSubClass(Set, function(x){return typeof x !==
"function";});

```

类似这种创建类工厂的能力是ES语言动态特性的一个体现，类工厂是一种非常强大和有用的特性，比如，可以使用Function.prototype.extend()方法重写NonNullSet：

```

var NonNullSet = (function(){
 var superclass = Set; // 指定父类
 return superclass.extends(
 function(){superclass.apply(this, arguments)}, // 构造函数
 {
 add: function(){
 for(var i=0; i<arguments.length; i++){
 if(arguments[i] == null)
 throw new Error("Can't add null or undefined");
 }
 // 调用父类的add()方法以执行实际插入操作
 return superclass.prototype.add.apply(this,arguments);
 }
 }
);
});

```

```
}());
```

使用组合代替继承的集合的实现：

```
// 实现一个FilterSet，它包装某个指定的“集合”对象
// 并对传入add()方法的值应用了某种指定的过滤器
// “范围”类中其他所有的核心方法延续到包装后的实例中
var FilterSet = Set.extend(
 // 构造函数
 function FilterSet(set, filter){
 this.set = set;
 this.filter = filter;
 },
 { // 实例方法
 add: function(){
 // 如果已有过滤器，直接使用它
 if(this.filter){
 for(var i=0; i<arguments.length; i++){
 var v = arguments[i];
 if(!this.filter(v))
 throw new Error("FilterSet: value " + v + " rejected by filter");
 }
 }
 // 调用set中的add
 this.set.add.apply(this.set, arguments);
 return this;
 },
 // 剩下的方法保持不变
 remove: function(){
 this.set.remove.apply(this.set, arguments);
 },
 contains: function(v){return this.set.contains(v);},
 size: function(){return this.set.size();},
 foreach: function(f,c){this.set.foreach(f,c);}
 });
```

可以利用这个类的实例来创建任意带有成员限制的集合实例：

```
var s = new FilterSet(new Set(), function(x){return x !== null;});
// 还可以对已经过滤后的集合进行再过滤
var t = new FilterSet(s,function(x){return !(x instanceof Set);});
```

## 19-10 抽象类

抽象类用来描述一种类型应该具备的基本特征与功能，具体如何去完成这些行为由子类通过方法重写来完成；即抽象方法指只有功能声明，没有功能主体实现的方法；具有抽象方法的类一定为抽象类。

抽象类不能实例化对象，但类的其它功能依然存在，成员变量、成员方法和构造方法的访问方式和普通类一样；所以只能被子类继承后，创建子类对象；子类需要继承抽象父类并完成最终的方法实现细节(即重写方法，完成方法体)；而此时，方法重写不再是加强父类方法功能，而是父类没有具体实现，子类完成了具体实现，将这种方法重写也叫做实现方法。

由于抽象类不能实例化对象，所以抽象类必须被继承，才能被使用；

```
// 此函数可以做任何抽象方法
```

```

function abstractmethod(){throw new Error("abstract method;");}
// 定义了AbstractSet类，并定义抽象方法contains()
function AbstractSet(){throw new Error("Can't instantiate abstract classes");}
AbstractSet.prototype.contains = abstractmethod;
// NotSet是AbstractSet的一个非抽象子类
// 所有不在其他集合中的成员都在这个集合中
// 因为它是在其他集合中是不可写的条件下定义的
// 同时由于它的成员是无限个，因此它是不可枚举的
// 只能用它来检测元素成员的归属情况
// 使用了Function.prototype.extends()方法定义的
var NotSet = AbstractSet.extend(
 function NotSet(set) {this.set = set;},
 {
 contains: function(x) {return !this.set.contains(x);},
 toString: function(x){return "~" + this.set.toString();},
 equals: function(that){return that instanceof NotSet &&
this.set.equals(that.set);}
 }
);
// AbstractEnumerableSet是AbstractSet的一个抽象子类
// 它定义了抽象方法size()和foreach()
// 然后实现了非抽象方法isEmpty()、toArray()、to[Locale]String()和equals()
// 子类实现了contains()、size()和foreach，这三个方可以很轻易的调用这5个非抽象方法
var AbstractEnumerableSet = AbstractSet.extend(
 function(){throw new Error("Can't instatiate abstract classes");},
 {
 size: abstractmethod,
 foreach: abstractmethod,
 isEmpty: function(){return this.size() == 0;},
 toString: function(){
 var s = "{", i=0;
 this.foreach(function(v){
 if(i++>0) s += ", ";
 s += v;
 });
 return s + "}";
 },
 toLocalString: function(){
 var s = "{", i=0;
 this.foreach(function(v){
 if(i++>0) s += ", ";
 if(v == null) s += v; // null和undefined
 else s += v.toLocalString(); // 其他的情况
 });
 return s + "}";
 },
 toArray: function(){
 var a = [];
 this.foreach(function(v){ a.push(v);});
 return a;
 },
 equals: function(that){
 if(!(that instanceof AbstractEnumerableSet)) return false;
 // 如果它们的大小不同，则它们不相等
 if(this.size() != that.size()) return false;
 // 检查每一个元素是否也在that中
 try{
 this.foreach(function(v){if(!that.contains(v)) throw false;});
 }
 }
 }
);

```

```

 return true; // 所有的元素都匹配：集合相等
 }catch(x){
 if(x === false) return false; // 集合不相等
 throw x; // 发生了其他的异常：重新抛出异常
 }
}
});
// SingletonSet是AbstractEnumerableSet的非抽象子类
// SingletonSet集合是只读的，它只包含一个成员
var SingletonSet = AbstractEnumerableSet.extend(
 function SingletonSet(member){this.member = member;},
 {
 contains: function(x){return x === this.member;},
 size: function(){return 1;},
 foreach: function(f,ctx){f.call(ctx, this.member);}
 }
);
// AbstractWritableSet是AbstractEnumerableSet的抽象子类
// 它定义了抽象方法add()和remove()
// 然后实现了非抽象方法union()、intersection()和difference()
var AbstractWritableSet = AbstractEnumerableSet.extend(
 function(){throw new Error("Can't instantiate abstract classes");},
 {
 add: abstractmethod,
 remove: abstractmethod,
 union: function(that){
 var self = this;
 that.foreach(function(v){self.add(v);});
 return this;
 },
 intersection: function(that){
 var self = this;
 this.foreach(function(v){if(!that.contains(v)) self.remove(v);});
 return this;
 },
 difference: function(that){
 var self = this;
 that.foreach(function(v){self.remove(v);});
 return this;
 }
 }
);
// ArraySet是AbstractWritableSet的非抽象子类
// 它以数组的形式表示集合中的元素
// 对于它的contains()方法使用了数组的线性查找
// 因为contains()方法的算法复杂度是O(n)而不是O(1)
// 它非常适用于相对小型的集合
var ArraySet = AbstractWritableSet.extend(
 function ArraySet(){
 this.values = [];
 this.add.apply(this, arguments);
 },
 {
 contains:function(v){return this.values.indexOf(v) != -1;},
 size: function(){return this.values.length;},
 foreach: function(f,c){this.values.forEach(f,c);},
 add: function(){
 for(var i=0;i<arguments.length; i++){
 var arg = arguments[i];
 }
 }
 }
);

```



```

 if(!this.contains(arg)) this.values.push(arg);
 }
 return this;
},
remove: function(){
 for(var i=0;i<arguments.length; i++){
 var p = this.values.indexOf(arguments[i]);
 if(p == -1) continue;
 this.values.splice(p,1);
 }
 return this;
}
});

function StringSet(){
 this.set = Object.create(null); // 创建一个不包含原型的对象
 this.n = 0;
 this.add.apply(this, arguments);
}
// 在此指定了属性的特性
StringSet.prototype = Object.create(AbstractWritableSet.prototype, {
 constructor: {value: StringSet},
 contains: {value: function(x){return x in this.set;}},
 size: {value: function(x){return this.n;}},
 foreach: {value: function(f,c){ Object.keys(this.set).forEach(f,c);}},
 add: {
 value: function(){
 for(var i=0; i<arguments.length; i++){
 if(!(arguments[i] in this.set)){
 this.set[arguments[i]] = true;
 this.n++;
 }
 }
 return this;
 }
 },
 remove: {
 value: function(){
 for(var i=0; i<arguments.length; i++){
 if(arguments[i] in this.set){
 delete this.set[arguments[i]];
 this.n--;
 }
 }
 return this;
 }
 }
});

```

可以为集合中某个成员添加一些类似于“对象id”属性，默认是可以通过for/in遍历的，但也可以让属性不可枚举：

```

// 定义不可枚举的属性，封装在一个匿名函数中
(function(){
 // 定义一个不可枚举的属性objectId，它可以被所有对象继承
 // 定义了getter，没定义setter，不可配置的
 Object.defineProperty(Object.prototype, "objectId", {

```

```

 get: idGetter, // 取值器
 enumerable: false,
 configurable: false
 });
 // 当读取objectId时调用这个getter函数
 function idGetter(){
 if(!(idprop in this)){ // 如果对象不存在id
 if(!Object.isExtensible(this)) // 如果不可扩展，即不能添加属性
 throw Error("Can't define id for noextensible objects");
 Object.defineProperty(this, idprop, {
 value: nextid++,
 writable: false,
 enumerable: false,
 configurable: false
 });
 }
 return this[idprop];
 }
 // idGetter用到这些变量，这些都属于私有变量
 var idprop = "|**objectId**|";
 var nextid = 1; // 给它设置初始值
}());

```

对象的属性可设置为只读的，同样，实例也可以定义不可变的：

```

// Range可以使用new，也可省略new，即可以用做构造函数也可以用作工厂函数
function Range(from,to){
 // 这些是对from和to只读属性的描述符
 var props = {
 from: {value: from, enumerable:true,writable:false,configurable:false},
 to:{value:to, enumerable:true,writable:false,configurable:false}
 };
 if(this instanceof Range)
 Object.defineProperties(this, props);
 else
 return Object.create(Range.prototype, props);
}
// 用同样的方法给Range.prototype对象添加属性（符）
Object.defineProperties(Range.prototype, {
 includes:{value: function(x){return this.from <= x && x <= this.to;}},
 foreach:{value: function(f){
 for(var x = Math.ceil(this.from); x <= this.to; x++) f(x);
 }},
 toString:{
 value: function(){return "(" + this.from + "... " + this.to + ")";}
 }
});

```

改进的做法：将修改这个已定义属性的特性的操作定义为一个工具函数，如：

```

// 属性描述符工具函数
// 将o的指定名字（或所有）的属性设置为不可写的和不可配置的
function freezeProps(o){
 var props = (arguments.length == 1) // 如果只有一个参数
 ? Object.getOwnPropertyNames(o) // 使用所有的属性
 : Array.prototype.splice.call(arguments, 1); // 否则传入了指定名字的属性
}

```

```

 props.forEach(function(n){ // 将它们设置为只读和不可变的
 // 忽略不可配置的属性
 if(!Object.getOwnPropertyDescriptor(o,n).configurable) return;
 Object.defineProperty(o,n,{writable:false, configurable:false});
 });
 return o;
}
// 将o的指定名字（或所有）的属性设置为不可枚举的和可配置的
function hideProps(o){
 var props = (arguments.length == 1) // 如果只有一个参数
 ? Object.getOwnPropertyNames(o)
 : Array.prototype.splice.call(arguments, 1);
 props.forEach(function(n){
 if(!Object.getOwnPropertyDescriptor(o,n).configurable) return;
 Object.defineProperty(o,n,{enumerable: false});
 });
 return o;
}
// 应用
function Range(from,to){
 this.from = from;
 this.to = to;
 freezeProps(this); // 将属性设置为不可变的
}
Range.prototype = hideProps({
 constructor: Range,
 includes: function(x){return this.from <=x && x <= this.to;},
 foreach: function(f){for(var x=Math.ceil(this.from); x<=this.to; x++)
f(x);},
 toString: function(){return "(" + this.from + "..." + this.to + "");}
});

```

可以通过定义属性getter和setter方法将状态变量更健壮地封装起来，如：

```

function Range(from,to){
 if(this.from > to) throw new Error("Range: from must be <= to");
 // 定义存取器方法以维持不变
 function getFrom(){return from;}
 function getTo(){return to;}
 function setFrom(f){ // 设置from的值时，不允许from大于to
 if(f <= to) from = f;
 else throw new Error("Range: from must be <= to");
 }
 function setTo(t){
 if(t >= from) to = t;
 else throw new Error("Range: to must be >= from");
 }
 // 将使用取值器的属性设置为可枚举的，不可配置的
 Object.defineProperties(this, {
 from:{
 get: getFrom,
 set: setFrom,
 enumerable: true, configurable:false
 },
 to:{
 get:getTo,
 set:setTo,

```

```

 enumerable: true, configurable: false
 }
});

}
Range.prototype = hideProps({
 constructor: Range,
 includes: function(x){return this.from <=x && x <= this.to;},
 foreach: function(f){for(var x=Math.ceil(this.from); x<=this.to; x++)
f(x);},
 toString: function(){return "(" + this.from + "..." + this.to + ")";}
});

```

## 19-11 防止类的扩展

```

Object.freeze(enumeration.values);
Object.freeze(enumeration);

```

## 19-12 原型中的属性描述符

```

// 如果不带参数调用它，就表示该对象的所有属性
// 将所有逻辑闭包在一个私有属性作用域中
(function namespace(){
 // properties构造函数，其表示一个对象的属性集合
 function Properties(o, names){
 this.o = o; // 属性所属的对象
 this.names = names; // 属性的名字，是一个数组
 }
 // 这个函数成为所有对象的方法
 function properties(){
 var names; // 属性名组成的数组
 if(arguments.length == 0) // 所有有自有属性
 names = Object.getOwnPropertyNames(this);
 else if(arguments.length == 1 && Array.isArray(arguments[0]))
 names = arguments[0]; // 参数本身就是一个数组
 else // 参数本身就是名字
 names = Array.prototype.splice.call(arguments,0);
 // 返回一个新的Properties对象，用以表示属性名字
 return new properties(this, names);
 }
 // 将properties设置为Object.prototype的新的不可枚举的属性（方法）
 // 这是从私有函数作用域导出的唯一的一个值
 Object.defineProperty(Object.prototype, "properties", {
 value: properties,
 enumerable: false,writable:true,configurable:true
 });
 // 将代表这些属性的对象设置为不可枚举的
 Properties.prototype.hide = function(){
 var o = this.o;
 var hidden = {enumerable: false};
 this.names.forEach(function(n){
 if(o.hasOwnProperty(n))
 Object.defineProperty(o, n, hidden);
 });
 return this;
 };
});

```

```

// 将这些属性设置为只读的和不可配置的
Properties.prototype.freeze = function(){
 var o = this.o;
 var frozen = {writable: false, configurable: false};
 this.names.forEach(function(n){
 if(o.hasOwnProperty(n))
 Object.defineProperty(o,n,frozen);
 });
 return this;
};
// 返回一个对象，这个对象是名字到属性描述符的映射表
// 使用它来复制属性，连同属性特性一起复制
// Object.defineProperties(dest, src.properties().descriptors());
Properties.prototype.descriptors = function(){
 var o = this.o;
 var desc = {};
 this.names.forEach(function(n){
 if(!o.hasOwnProperty(n)) return;
 desc[n] = Object.getOwnPropertyDescriptor(o,n);
 });
 return desc;
};
// 返回一个格式化良好的属性列表
// 列表中包含名字、值和属性特性，使用permanent表示不可配置
// 使用readonly表示不可写，使用hidden表示不可枚举
// 普通的可枚举、可写和可配置属性不包含特性列表
Properties.prototype.toString = function(){
 var o = this.o; // 在下面嵌套的函数中使用
 var lines = this.names.map(nameToString);
 return "{\n " + lines.join(",\n ") + "\n}";

 function nameToString(n){
 var s = "";
 var desc = Object.getOwnPropertyDescriptor(o, n);
 if(!desc) return "no exist " + n + ": undefined";
 if(!desc.configurable)
 s += "Permanent ";
 if((desc.get && !desc.set) || !desc.writable)
 s += "readonly ";
 if(!desc.enumerable)
 s += "hidden ";
 if(desc.get || desc.set)
 s += "accessor " + n;
 else
 s += n + ": " + ((typeof desc.value === "function") ? "function"
: desc.value);
 return s;
 }
};
// 最后，将原型对象中的实例方法设置为不可枚举的
Properties.prototype.properties().hide();
}());

```

## 20, Web浏览器中的Javascript

## 20-1 Web浏览器中的Javascript

通常也称为客户端的JavaScript，就是JavaScript运行在浏览器中；

从内容上来看，它是包括BOM和DOM；

从形式上可以分为Web文档和Web应用两种形式；

## 20-2 Web文档里的Javascript

Javascript可以通过document对象和它包含的element对象遍历和管理文档内容；它可以通过操纵CSS样式和类，修改文档内容的呈现；并且可以通过注册适当的事件处理程序来定义文档元素的行为；

Web文档里应当少量地应用javascript，因为javascript真正的作用是增强用户的浏览体验，使信息的获取和传递更容易；用户的体验不应依赖于javascript，但javascript可以增强体验，如：

- 创建动画和其他视觉效果，引导和帮助用户进行页面导航；
- 对表格的列进行分组，让用户更容易找到所需要的内容；
- 隐藏某些内容，当用户需要了解更详细内容时，再逐渐展示详细信息；

## 20-3 window对象

客户端javascript中最重要的对象之一是window对象，window对象是所有客户端javascript特性和API的主要接入点；它表示Web浏览器的一个窗口或窗体；Window对象定义了一些属性，比如，指定当前窗口中的URL的location属性，其还可以允许脚本在窗口中载入新的URL；

```
window.location = "https://www.zeronetwork.cn/";
```

window对象还定义了一些方法，如alert()，可以弹出一个对话框用来显示一些信息；比如：setTimeout()，可以注册一个函数，在给定的一段时间之后触发一个回调，如：

```
setTimeout(function(){
 alert("零点网络");
},2000);
```

在客户端javascript中，window对象也是全局对象，也就是window对象处于作用域链的最顶部，它的属性和方法实际上是全局变量和全局函数，所以，window.setTimeout()可以直接使用setTimeout()，也就是说，如果想引用全局窗口或全局对象的属性，通常并不需要用到window；

window还定义了很多其它重要的属性、方法和构造函数；其中最重要的属性是document，它引用Document对象，表示的是在窗口中的文档；Document对象有一些重要的方法，比如getElementById()，可以基于元素id的值返回单一的HTML元素，如：

```
var mydiv = document.getElementById("mydiv");
```

getElementById()方法返回的Element对象，也拥有一些重要的属性和方法，比如允许脚本获取它的内容、设置属性值等，如：

```
var mydiv = document.getElementById("mydiv");
// 如果元素为空，则往里面插入当前的日期和时间
if(mydiv.firstChild == null){
 mydiv.appendChild(document.createTextNode(new Date().toString()));
}
```

每个Element对象才有style和className属性，允许脚本指定文档元素的CSS样式，或修改应用到元素上的CSS的类名，如：

```
var mydiv = document.getElementById("mydiv");
mydiv.style.height = "200px";
mydiv.style.backgroundColor = "yellow";
mydiv.className = "mydiv";
```

window对象、document对象和element对象还有一个重要的属性集合是事件处理程序相关的属性；可以在脚本中为之绑定一个函数，这个函数会在某个事件发生时以异步的方式调用；事件处理程序可以让Javascript代码修改窗口、文档和组成文档的元素的行为；事件处理程序的属性名是以单词“on”开头的，如：

```
mydiv.onclick = function(){
 this.innerHTML = "<h2>零点网络</h2>";
}
```

window对象的onload处理程序是最重要的事件处理程序之一；当显示在窗口中的文档内容稳定并可以操作时可以触发它；Javascript代码通常封装在onload事件处理程序里；比如，可以在onload事件中，查询文档元素、修改CSS类和定义事件处理程序，如：

```
<style>
.newslst *{display: none;}
.newslst h1{display: block;}
.newslst_show *{display: block;}
</style>
<script>
window.onload = function(){
 var elements = document.getElementsByClassName("newslst");
 for(var i=0; i<elements.length; i++){
 var elt = elements[i];
 var title = elt.getElementsByTagName("h1")[0]
 showHandler(title,elt);
 }

 function showHandler(title,elt){
 title.onclick = function(){
 if(elt.className == "newslst")
 elt.className = "newslst_show";
 else
 elt.className = "newslst";
 }
 }
}
</script>
<div class="newslst">
 <h1>零点网络</h1>
 <p>零点网络是一家科技公司</p>
</div>
```

## 20-4 Web应用里的Javascript

在Web文档中使用的Javascript特性在Web应用中都会用到，对于Web应用来说，除了内容、呈现和操作API之外，还依赖Web浏览器环境提供的各种基础的服务；

现代浏览器，已经不仅仅是作为显示文档的工具了，而渐渐变成了一个简易的操作系统；

Web应用就是用Javascript访问浏览器提供的各种服务，这些服务有很多都是在HTML5中定义的，HTML5和相关的标准为Web应用定义了很多其他重要的API，这些API包括以上所说的网络、图像和数据存储，还包含地理位置信息、历史管理和后台线程等，这些都是典型的Web应用；例如XMLHttpRequest对象，其可以发出HTTP请求，可以从服务器端获取新信息，而不用重新载入整个页面，这样的Web应用称为Ajax应用；并且它们可以离线操作，以及保存数据到本地，以便再次访问时进行状态恢复；

Javascript在Web应用里会比在Web文档里显得更加重要；Javascript增强了Web文档，但是设计良好的文档需要在禁用Javascript后还能继续工作；Web应用本质上就是Javascript程序，只不过使用了Web浏览器提供的服务，如果禁用了Javascript，Web应用就运行不了；

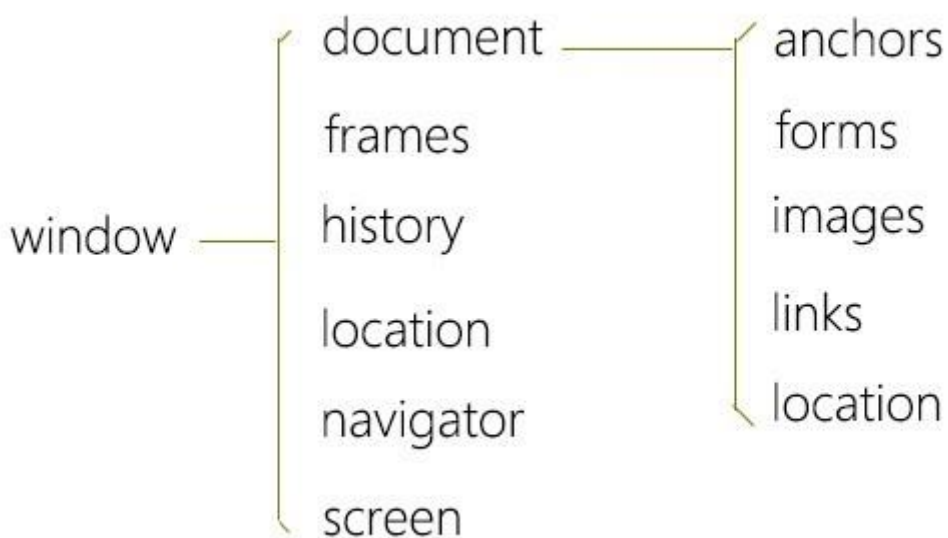
在真实的场景中，并不是完全分离Web文档和Web应用的这两种形式，而是结合了两者的特性；

## 21，浏览器对象模型BOM

BOM（Browser Object Model）浏览器对象模型；其提供了独立于内容而与浏览器窗口进行交互的对象；

// 没有BOM标准：不同的浏览器按照各自的实现及扩展BOM，于是，它们之间共有的对象成为了事实上的标准；近年来，W3C为了把浏览器中Javascript最基本的部分标准化，已经将BOM的主要方面纳入了HTML5的规范中。

// BOM由一系列相关的对象构成；DOM中各对象之间是层次关系；window对象是整个BOM的核心，所有对象和集合都以某种方式回接到window对象



头条 @零点程序员



## 21-1 window对象

// 其是BOM的核心对象，也是顶级对象，表示浏览器的一个实例；

// 浏览器窗口对文档提供一个显示的容器，是每一个加载文档的父对象；window对象表示整个浏览器窗口，但不表示其中所包含的内容；可以用于移动或调整浏览器的大小，或者产生其他的影响；

## 21-2 全局作用域

在浏览器中，window对象具有双重角色，即是通过JavaScript访问浏览器的一个接口，又是ES中的Global对象；

既然window是Global对象，所以window对象是所有其他对象的顶级对象，在网页中定义的任何对象、变量和函数，window对象都有权访问；即所有在全局作用域中声明的变量、函数都会变成window对象的属性和方法；因此window前缀可以省略；

```
var age = 18;
function sayAge(){
 alert(this.age);
}
alert(window.age);
sayAge();
window.sayAge();
```

定义全局变量与在window对象上直接定义属性还是有一点差别：全局变量不能通过delete操作符删除，而直接在window对象上定义的属性可以，如：

```
var age = 18;
window.color = "green";
console.log(delete window.age); // false
console.log(delete window.color); // true
console.log(window.age); // 18
console.log(window.color); // undefined
```

age属性的特性中的[[Configurable]]，值为false，因此其不能通过delete删除；

```
console.log(Object.getOwnPropertyDescriptor(window, "age"));
console.log(Object.getOwnPropertyDescriptor(window, "color"));
```

尝试访问未声明的变量会抛出错误，但通过查询window对象，可以知道某个可能未声明的变量是否存在，如：

```
var age = oldAge; // 抛出错误
var age = window.oldAge; // 不会抛出错误，因为这是一次属性查询
```

## 21-3 文档元素

如果在HTML文档中用id属性来元素命名，并且如果window对象没有此名字的属性，window对象会赋予一个属性，它的名字是id属性的值，而它们的值指向表示文档元素的HTMLElement对象，这称为全局变量的隐式应用；

在实际场景中，很少使用这种方式来访问HTML元素，它是Web浏览器发展过程中遗留的一个现象，是现代浏览器向后兼容性的考虑，如：

```

<button id="okay">按钮</button>
<input id="myinput" value="input" />
<div id="mydiv">mydiv</div>
<script>
var ui = ["okay", "myinput"];
ui.forEach(function(id){
 ui[id] = document.getElementById(id);
});
console.log(ui.okay);
console.log(ui.myinput);

// 定义一个更简单的方法
var $ = function(id){
 return document.getElementById(id);
};
$("#mydiv").innerHTML = "零点网络";
console.log($("#mydiv"));
</script>

```

## 21-4 窗口位置

获取窗口(视口)的位置(即相对于屏幕左边和上边的位置); 各浏览器都实现了screenLeft和screenTop属性表示窗口的位置; 但是之前的firefox并不支持, 现代firefox返回-8; 只有IE是文档区相对于主显示器屏幕的位置;

Firefox使用了screenX和screenY属性获取窗口位置信息; 各浏览器也实现了这两个属性, 但并不统一:

chrome与Opera实现了screenLeft、screenTop与screenX、screenY的统一; 且最大化时, 值为0; Firefox与IE实现了统一, 但最大化时, 值为-8; 且Firefox中的screenLeft、screenTop与这两个属性实现了对应; 但IE的screenLeft、screenTop与这两个属性并不对应; screenLeft、screenTop是文档区域的左上角的坐标, screenX、screenY是窗口的左上角坐标;

```

console.log(window.screenLeft);
console.log(window.screenTop);
console.log(window.screenX);
console.log(window.screenY);

// 为了兼容老的Firefox
var leftPos = (typeof window.screenLeft == "number") ? window.screenLeft :
window.screenX;
var topPos = (typeof window.screenTop == "number") ? window.screenTop :
window.screenY;
console.log("leftPos:", leftPos, ", topPos:", topPos);

```

pageXOffset及pageYOffset: 设置或返回当前页面相对于可视区域的左及上的位置; 但貌似只有Chrome支持, 同时, 如果是设置的话, 没有效果;

注: 目前, 无法在跨浏览器的条件下取得窗口左边和上边的精确坐标值; 如果在框架中使用, 除了IE, 其他浏览器的值都与以上统一;

## 21-5 移动浏览器窗口

- moveBy(dx,dy): 相对移动, dx,dy可以为负;
- moveTo(x,y): 移动, x,y可以为负;

```
window.moveTo(50,100);
window.moveBy(100,200);
window.moveBy(-50,0);
```

注: 一般很少用; 这两个方法有可能会被浏览器禁用; 这两个方法都不适用框架, 只能对最外层的window对象使用;

## 21-6 窗口大小

如果要获取浏览器窗口大小信息, 各浏览器并不统一; 但各浏览器都已实现了以下四个方法, 但返回值并不一定相同:

- innerWidth和innerHeight // 属性获取视口大小(注: 包括body的margin);
- outerWidth和outerHeight // 属性获取浏览器窗口大小(无论是从最外层的window对象还是从某个框架访问);

```
console.log(window.innerWidth);
console.log(window.innerHeight);
console.log(window.outerWidth);
console.log(window.outerHeight);
```

在各浏览器的实现中, 可以通过使用DOM提供的页面视口的相关信息:

document.documentElement.clientWidth和document.documentElement.clientHeight中保存了页面视口的信息; 其返回值与innerWidth和innerHeight一致;

在低版本的IE中, 必须通过document.body.clientWidth和document.body.clientHeight属性获取视口信息(实际上是实际内容的尺寸, 但不包括border值), 但不标准;

同时, document.body.offsetWidth和document.body.offsetHeight也能获取视口相关信息, 同clientWidth和clientHeight类似, 只不过其包括border的宽度;

跨浏览器取得页面视口的大小:

```
var pagewidth = window.innerWidth,
 pageHeight = window.innerHeight;
if(typeof pagewidth != "number"){
 // 判断页面是否处于标准模式
 if(document.compatMode == "CSS1Compat"){
 pagewidth = document.documentElement.clientWidth;
 pageHeight = document.documentElement.clientHeight;
 }else{
 pagewidth = document.body.clientWidth;
 pageHeight = document.body.clientHeight;
 }
}
console.log(pagewidth);
console.log(pageHeight);
```

## 21-7 调整浏览器窗口的大小:

- `resizeBy(dw,dh)`: 相对缩放, `dw,dh`可以为负;
- `resizeTo(w,h)`: 缩放到, `w,h`不能为负;

注: 一般很少用; 也有可能被禁用, 同移动窗口类似

```
window.resizeTo(400,300);
window.resizeBy(200,100);
```

## 21-8 滚动窗口 (如果有滚动条)

- `scrollBy(x, y)`: 按照指定的像素值相对来滚动的内容(第一个参数是滚动条向右滚动, 第二个参数是滚动条向下滚动, 方法执行重复执行, 值会累加); (可直接在控制台上演示)
- `scrollTo(x, y)`: 把内容滚动到指定的坐标;
- `scroll(x, y)`: 把内容滚动到指定的坐标;
- `scrollX`及`scrollY`:

## 21-9 对话框:

`window`对象通过`alert()`、`confirm()`和`prompt()`三个方法可以调用系统对话框向用户显示消息;

系统对话框与在浏览器中显示的网页没有关系, 也不包括HTML; 它们的外观由操作系统或浏览器设置决定的, 而不是由CSS决定的; 此外, 通过这几个方法打开的对话框都是同步和模态的, 也就是说, 显示这些对话框的时候代码会停止执行, 而关掉这些对话框后代码又会恢复执行;

一般来说, 不会使用, 都是自己实现一个效果;

警告对话框`window.alert(string)`:

警告对话框是一个带感叹图标的小窗口, 通常用来输出一些简单的文本信息; 该方法接受一个字符串并将其显示给用户, 并包含一个OK或确定的按钮等待用户关闭对话框;

通过`alert()`生成的警告对话框, 用户是无法控制的, 比如, 显示的消息内容, 用户只能看完消息后关闭对话框;

确认对话框`window.confirm(string)`:

确认对话框也是向用户显示消息的对话框, 但与警告对话框不同的是, 其具有OK与Cancel按钮, 根据用户的选择, 该方法返回不同的值(`true`或`false`); 程序可以根据不同的值予以不同的响应, 实现互动的效果; 通常放在网页中, 对用户进行询问并根据其选择而做不同的控制;

```
if(window.confirm("确定删除吗?")){
 alert("已经删除");
}else{
 alert("未删除");
}
```

输入对话框`window.prompt()`:

用于提示用户输入的对话框;

语法: `window.prompt(提示信息, 默认值)`; // 要显示的文本提示和文本输入域的默认值, 该默认值可以是一个空字符串;

```
prompt("请输入你的名字", "王唯");
```

如果用户单击了OK，则prompt()返回文本域中的值，如果单击了Cancel或使用其他方式关闭对话框，则该方法返回null，如：

```
var result = prompt("请输入密码: ", "");
if(result == "8888"){
 alert("登录成功");
}

var result = prompt("请输入你的名字", "王唯");
if(result != null){
 document.write("欢迎你: " + result);
}else{
 alert("你没有输入任何内容");
}
```

综上所述，这些系统对话框很适合向用户显示消息并由用户作出决定；由于不涉及HTML、CSS或JS，因此它们是增强Web应用程序的一种便捷方式；

```
do{
 var name = prompt("输入你的名字: ");
 var correct = confirm("你输入的是: " + name + ".\n" + "你可以单击确定或取消");
}while(!correct)
alert("你好, " + name);
```

在弹出对话框时，还有一个特性：如果当前脚本在执行过程中会打开两个或多个对话框，那么从第二个对话框开始，每个对话框中都会显示一个复选框，以使用户阻止后续的对话框显示，除非用户刷新页面；后续被阻止的对话框包括警告框、确认框和显示输入框；

该特性最初是由Chrome实现了，后续其他浏览器也实现该特性；其工作原理就是使用了一个叫对话框计数器，可以跟踪对话框；但是浏览器没有就对话框是否显示向开发人员提供任何信息；

在实际的场景中，这三个方法是很少用的，因为它们显示的文本是纯文本，不是HTML格式的文本，只能使用空格、换行符和各种标点符号，所以往往满足不了页面设计需要，并且有可能会破坏用户的浏览体验；常见的就是使用alert()方法进行调试，用来查看某个变量的输出结果；

这三个方法都会产生阻塞，也就是说，在用户关掉它们之前，它们不会返回，后面的代码不会执行；如果当前载入文档，也会停止载入，直到用户要求的输入进行响应为止；

可以自定义一个对话框：

```
<style>
 #alert_box{
 position: absolute; display: none; width: 400px; height: 300px; border-
radius: 3px;
 box-shadow: 0 0 5px rgba(0, 0, 0, .5);
 }
 #alert_box h1{
 margin: 0; padding: 0; font-size: 1.5em; line-height: 60px;
 height: 60px;
 text-align: center; background-color: rgba(255, 255, 255, 1);
 }
 #alert_box div{
 height: 240px;
 padding: 1em; line-height: 1.8em; background-color:
 rgba(255, 255, 255, .8);
 }
```

```

#alert_box span{
 position: absolute; width: 30px; height: 30px;
 top:-15px; right:-15px; background-color:#000; border-radius: 50%;;
}
</style>
<script>
window.alert = function(title,info){
 var box = document.createElement("div");
 box.id = "alert_box";
 box.style.left = ((window.innerWidth - 400) / 2) + "px";
 box.style.top = ((window.innerHeight - 300) / 2) + "px";

 var h1 = document.createElement("h1");
 h1.innerText = title;
 box.appendChild(h1);

 var innerBox = document.createElement("div");
 innerBox.innerHTML = info;
 box.appendChild(innerBox);

 var closeSpan = document.createElement('span');
 box.appendChild(closeSpan);
 closeSpan.addEventListener("click",function(e){
 document.body.removeChild(box);
 },false);

 box.style.display = "block";
 document.body.appendChild(box);
};
window.alert("零点网络","哪些是这样的? ");
</script>

```

Javascript还有两个工具性的对话框，即查找window.find()和打印window.print();

这两个对话框都是异步显示的，能够将控制权立即交还给脚本；其与浏览器菜单中的查找和打印命令是相同的；

这两个方法同样不会就用户对对话框中的操作给出任何信息，因此它们的用处有限；另外，既然这两个对话框是异步显示的，对话框计数器就不会将它们计算在内，所以它们也不会受用户禁用后续对话框显示的影响；

## 21-10 状态栏

// 浏览器的状态栏通常位于窗口的底部，用于显示一些任务状态信息等。在通常情况下，状态显示当前浏览器的工作状态或用户交互提示信息；具有status和defaultStatus属性；

// 默认状态栏信息：

// defaultStatus属性可以用来设置在状态栏中的默认文本，是一个可读写的字符串；

// 状态栏瞬间信息：

// status属性，在默认情况下，将鼠标放在一个超链接上时，状态栏会显示该超链接的URL，此时的状态栏信息就是瞬间信息；当鼠标离开超链接时，状态栏就会显示默认的状态栏信息，瞬间信息消失。

// 浏览器已经关闭了状态栏的功能：这是出于安全的考虑，防止隐藏了超链接真正目的的钓鱼攻击

## 22, 计时器setTimeout和setInterval

### 22-1 计时器

Javascript是单线程语言，但它允许通过设置超时和间歇时间值来调度代码在特定的时刻执行；其是通过setTimeout()和setInterval()两个window对象的全局函数实现的，用来注册在指定的时间之后单次或重复调用的函数；

#### 22-1-1 setTimeout()

延迟代码执行（也叫超时调用）：用来实现一段代码在指定的毫秒之后运行；

语法：window.setTimeout(code,delay)，code要执行的代码，可以是一个包含Javascript的代码字符串，也可以是一个函数，delay等待的毫秒数；

```
// 不建议传递字符串
setTimeout("alert('zeronetwork')",3000);
// 推荐的使用方式
setTimeout(function(){
 alert('zeronetwork');
},3000);
// 推荐的使用方式
setTimeout(show,3000);
function show(){
 alert('zeronetwork');
}
```

因为历史原因，第一个参数可以传递字符串，但有可能导致性能损失，因为这个字符串会在指定的超时时间之后进行求值，相当于执行eval()，因此不推荐使用字符串的形式；

```
// 能达到无限循环的目的
var n = 0;
function fun(){
 n++;
 console.log(n);
 setTimeout(fun, 1000);
}
fun(); // 直接执行
setTimeout(fun,3000);
```

第二个参数是一个表示等待多长时间的毫秒数，但经过该时间后指定的代码并不一定会执行；Javascript是一个单线程的解释器，因此一定时间内只能执行一段代码；为了控制要执行的代码，就有一个Javascript任务队列，这些任务会按照将它们添加到队列的顺序执行；这个参数实际上是告诉Javascript再过多长时间把当前任务添加到队列中，如果队列是空的，那么添加的代码会立即执行，如果队列不是空的，那么它就要等前面的代码执行完毕后再执行；

另外，如果该参数为0，也并不一定会立即执行，因为也需要将它放到队列中，等待前面的任务全部执行完后，才会“立即”执行；

setTimeout()方法会返回一个数字ID，ID本质上是要延迟进程的ID，是计划执行代码的唯一标识符；可以使用clearTimeout()方法，调用此ID，达到取消超时调用的目的；

```

var timeoutid = setTimeout(function(){
 alert("zeronetwork");
},3000);
console.log(timeoutid);
clearTimeout(timeoutid);

```

只要是在指定的时间尚未过去之前调用clearTimeout(), 就可以完全取消超时调用;

```

<input type="button" value="开始" onclick="showClock()" />
<input type="button" value="取消" onclick="window.clearTimeout(ident)" />
<div id="showtime">time</div>
<script>
function showClock(){
 var d = new Date();
 var showtime = document.getElementById("showtime");
 showtime.innerHTML = d.toLocaleString();
 // ident = setTimeout(showClock(), 1000);
 ident = setTimeout("showClock()", 1000);
}
</script>

```

示例: 可以利用 clearTimeout() 方法在特定条件下清除延迟处理代码。例如, 当鼠标指针移过某个元素, 停留半秒钟之后才会弹出提示信息, 一旦鼠标指针移出当前元素, 就立即清除前面定义的延迟处理函数, 避免干扰;

```

<h1>零点网络</h1>
<div>零点教育是从事IT教育</div>
<p>主讲: 零点网络</p>
<script>
var o = document.getElementsByTagName('body')[0].childNodes;
for(var i=0; i<o.length; i++){
 o[i].onmouseover = function(i){
 return function(){
 f(o[i]);
 }
 }(i);
 o[i].onmouseout = function(i){
 return function(){
 clearTimeout(o[i].out);
 }
 }(i);
}
function f(o){
 o.out = setTimeout(function(){
 console.log(o.tagName + ":" + o.innerHTML);
 },500);
}
</script>

```

除前两个参数之外, HTML5规范还允许setTimeout()传入额外的参数, 并在调用函数时把这些参数传递过去;



```
setTimeout(function(str,age){
 alert(str + "age:" + age);
},3000,"wangwei",18);
```

## 22-1-2 setInterval()

代码延迟执行机制在执行一次后就失效，而在应用中，有时希望某个程序能反复执行，比如说倒计时等，需要每秒执行一次；为此可以使用window方法的setInterval方法，其会按照指定的时间间隔重复执行代码，直到取消或页面被卸载；其与setTimeout()类似，参数也一致；

```
// 不建议使用字符串
setInterval("console.log('zero')", 3000);
// 推荐的方式
setInterval(function(){
 console.log('zero');
},3000);

function timer(){
 var d = new Date();
 document.getElementById("result").innerText = d.toLocaleTimeString();
}
setInterval(timer,1000);

// 输出的时间并不精确，并不是整1000毫秒
var firstTime = new Date().getTime();
setInterval(function(){
 var lastTime = new Date().getTime();
 console.log(lastTime - firstTime);
 // alert("ok"); // 会暂停
 firstTime = lastTime;
},1000);
同setTimeout()一样，setInterval()也支持第三个参数：
```

取消间隔性执行代码：

使用setInterval()方法同样会返回一个间隔调用ID，该ID可用于在将来某个时间取消间隔调用；可以使用clearInterval方法移除间隔调用，其接收一个计时器ID作为参数；

```
// 如果不使用它的返回值，可以直接使用数字1、2...
var i = 0;
setInterval(function(){
 console.log(i++);
 if(i>10)
 clearInterval(1);
},1000);
```

取消间隔调用的重要性要远远高于取消超时调用，因为在不取消的情况下，间隔调用将会一直执行到页面卸载；

```
var num = 0, max = 10;
var intervalId = null;
function incNum(){
 num++;
 console.log(num);
 // 如果执行次数达到了max设定的值，则取消后续的调用
```

```

 if(num == max){
 clearInterval(intervalId);
 alert("结束");
 }
 }
 intervalId = setInterval(incNum, 1000);

// 另外
var mInput = document.getElementsByTagName('input')[0];
var sInput = document.getElementsByTagName('input')[1];
var m = 4, s = 52;
var timer = setInterval(function(){
 s++;
 if(s == 60){
 s = 0;
 m++;
 }
 sInput.value = s;
 mInput.value = m;
 if(m == 5)
 clearInterval(timer);
}, 1000);

```

setTimeout()与setInterval()同时使用时，其返回的id也会按顺序返回；

在某些时候 setTimeout()与 setInterval() 可以实现同样的效果；

```

var num = 0, max = 10;
function incNum(){
 num++;
 console.log(num);
 // 如果执行次数达到了max设定的值，则取消后续的调用
 if(num < max){
 setTimeout(incNum, 1000)
 }else{
 alert("结束");
 }
}
setTimeout(incNum, 1000);

```

在使用超时调用时，没有必要使用超时调用ID，因为每次执行代码之后，如果不再设置另一次超时调用，调用就会自动停止；

一般认为，使用延迟代码来模拟时间间隔是一种最佳方式；在开发环境中，很少使用时间间隔，因为时间间隔可能会在前一个间隔调用结束之前启动，而延迟代码完全可以避免这一点；

```

<div id="loadBar" style="border: red 1px solid;"></div>
<script>
var num = 0;
var colors =
['#494949', '#646464', '#747474', '#888888', '#969696', '#A8A8A8', '#B6B6B6', '#C6C6C6',
'#D7D7D7', '#E1E1E1', '#F0F0F0', '#F9F9F9'];
function loading(){
 num++;
 var loadBar = document.getElementById("loadBar");
 loadBar.style.color = colors[num-1];
 loadBar.innerHTML = loadBar.innerHTML + "■";
}

```

```

 if(num < 12){
 setTimeout(loading, 1000);
 }else{
 loadBar.style.display = "none";
 window.open("https://www.zeronetwork.cn/", "new");
 }
}
window.onload = loading;
</script>

/*
定时器应用函数 invoke
如果只传递f,start, 则使用setTimeout
如果没有传递end, 则永久循环执行f, 否则在end后停止
*/
function invoke(f, start, interval, end){
 if(!start) start = 0; // 默认设置为0毫秒
 if(arguments.length <= 2) // 单次调用模式
 setTimeout(f, start);
 else // 多次调用模式
 setTimeout(repeat, start); // 若干秒后调用repeat()
 function repeat(){
 var h = setInterval(f, interval); // 循环调用f()
 // 在end毫秒后停止调用, 前提是end已经定义了
 if(end){
 setTimeout(function(){
 clearInterval(h);
 }, end);
 }
 }
}
invoke(function(){
 console.log("wangwei");
}, 1000, 2000, 5000);

```

## 23, location、history对象

### 23-1 location对象

// location对象, 是BOM最有用的对象之一了, 是window的属性(子对象), 它提供了与当前窗口中加载文档的URL, 还提供一些导航及载入文档的方法;

// location对象很特别, 即是window对象的属性, 也是document对象的属性, 即window.location与document.location引用的是同一个对象;

location对象的特点是, 它不仅保存着当前文档的信息, 它还将URL解析为独立的片段, 让开发人员可以通过不同的属性访问这些片段;

#### 23-1-1 属性

- **hash**: // 返回URL中的hash，即井号（#）后跟的多个字符，即URL中的锚，如果没有，则返回空字符串；如：**#name**；
- **host**: // 返回主机名和端口号(如果有)，如：**www.zeronetwork.cn:80**；
- **hostname**: // 返回当前主机名不包括端口号，如：**www.zeronetwork.cn**；
- **origin**: // 只读，返回当前协议、主机名和端口号，如：**https://www.zeronetwork.cn:80**；
- **href**: // 返回当前加载页面的完整URL，location对象的toString()也返回这个值，如：  
**https://www.zeronetwork.cn**；
- **pathname**: // 返回当前 URL 的路径部分，即目录和文件名，如：**/edu/index.html**；
- **port**: // 返回当前 URL 中的端口号，如果没有，则返回空字符串，如：**8080**；
- **protocol**: // 返回当前 URL 的协议，通常是http或https；
- **search**: // 返回从问号（?）开始的查询字符串，如：**?q=wangwei**

## 23-1-2 方法

- **assign()**: // 加载新的文档；
- **reload()**: // 重新加载当前文档；
- **replace()**: // 用新的文档替换当前文档；

## 23-1-3 解析URL

location对象的href属性是一个字符串，包含URL的完整文本；location对象的toString()方法返回href属性的值，因此在会隐匿调用toString()的情况下，可以使用location代替location.href；

```
console.log(location);
console.log(location.href);
console.log(location.toString());
console.log(location + ""); // 字符串
// 可以直接赋值，即为location.href = url
location = "https://www.zeronetwork.cn";

<!-- 页面跳转 -->
<p>页面会在秒后跳转到:https://www.zeronetwork.cn</p>
<script>
var n = 5;
result.innerText = n;
setInterval(function(){
 n--;
 if(n==0)
 location.href = "https://www.zeronetwork.cn/";
 else
 result.innerText = n;
},1000);
</script>
```

其他的属性都表示URL的各个部分，它们被称为“URL分解”属性，同时被Link对象（html中的[和元素](#)）支持；这些属性都是可写的；

```
// 获取页面名
var pathname = location.pathname;
var pagename = pathname.substring(pathname.lastIndexOf('/')+1);
console.log(pagename);
```

## 23-1-4 查询字符串参数

获取参数可以通过Location对象的search属性，获得从URL中传递过来的参数和参数值，但不能逐个访问参数，因此需要单独解析查询字符，用以处理需要获取的参数和参数值；

```
function getQueryString(){
 // 取得查询字符串并去掉开头的问
 var qs = location.search.length > 0 ? location.search.substring(1) : "";
 // 保存数据的对象
 var args = {};
 // 取得每一项
 var items = qs.length ? qs.split("&") : [];
 var item = null, name=null, value = null;
 var i=0, len=items.length;
 // 逐个将每一项添加到args对象中
 for(i=0; i<len; i++){
 item = items[i].split("=");
 name = decodeURIComponent(item[0]);
 value = decodeURIComponent(item[1]);
 if(name.length){
 args[name] = value;
 }
 }
 return args;
}
// 应用，假定为: ?q=zeronetwork&num=100
var args = getQueryString();
alert(args["q"]); //zeonetwork
alert(args["num"]); // 100
```

## 23-1-5 加载新文档

在实际场景中，时常会用到加载一个新的网页的情况；此时可以用Location对象的href属性就可以轻松完成这一功能，该属性返回值为当前文档的URL，如果将该属性值设置为新的URL，那么浏览器会自动加载该URL的内容，从而达到加载一个新的网页的目的；

```
console.log(location.href);
location.href = "https://www.zeronetwork.cn/";
// 或者直接为location赋新的URL值
window.location = "https://www.zeronetwork.cn/";
```

如此，就能立即打开新的URL并在浏览器的历史记录中生成一条记录；

修改location对象的其他属性也可以改变当前加载的页面，如：

```
// 注意先后顺序
location.pathname = "/edu/index.html";
location.hash = "#selection1";
location.search = "?q=wangwei";
location.port = "8080";
location.hostname = "www.zeronetwork.cn";

function gotoUrl(){
 window.location.href = "http://www.zeronetwork.cn";
}
// 传递参数
```

```
function gotoUrl(url,catalogid){
 var url = url + "?catalogid=" + catalogid;
 window.location = url;
}
function gotoUrl(url,catalogid){
 if(catalogid <= 0)
 location = url;
 else
 location = url + "?catalogid=" + catalogid;
}
```

每次修改location的属性（hash除外），页面都会以新的URL重新加载；

修改hash有可能不会在历史记录中生成一条记录；

```
<p id="info"></p>
<script>
var s = 5;
var info = document.getElementById("info");
function go(){
 if(s==0)
 window.location.href = "https://www.zeronetwrok.cn/";
 else
 info.innerHTML = "浏览器将在" + s + "后跳转";
 s--;
}
setInterval(go, 1000);
</script>
```

Document对象也有一个URL属性，是文档首次载入后保存该文档的URL的字符串；

```
console.log(location.href == document.URL);
```

## 23-1-6 装载新文档与重新装入当前文档

文档的装载在应用中也是比较常见的，有三个方法：assign、replace和reload；

```
// 使用按钮演示
var mybtn = document.getElementById("mybtn");
mybtn.addEventListener("click",function(){
 // ...
});
```

## 23-1-7 assign()方法

使窗口载入并显示指定的URL中的文档；

```
// window.location=URL与location.href=URL本质上都是调用了assign()方法；三者是等同的用法；
// location.assign("https://www.zeronetwork.cn/");
```

## 23-1-8 replace()方法

以上的方式修改URL后，在浏览器的历史记录中会生成一条新记录，因此用户通过单击“后退”按钮，都会导航到前一个页面；要禁用这种行为，可以使用replace()方法；该方法只接受一个URL参数，并不会在历史记录中生成新记录；

```
location.replace("https://www.zeronetwork.cn/");
// 如果浏览器不支持XMLHttpRequest对象，则重定向一个不需要Ajax的页面
if(!XMLHttpRequest) location.replace("noajax.html");
```

注：relace()的URL参数可以是绝对或相对的URL；

## 23-1-9 reload()方法

该方法用于根据浏览器reload按钮定义的策略重新加载当前窗口的文档；

reload()方法直接有可能从缓存中重新加载，如果加入参数true，会从服务器重新加载；

```
window.location=URL与location.href=URL本质上都是调用了assign()方法；三者是等同的用法；
location.assign("https://www.zeronetwork.cn/");
```

位于reload()方法调用之后的代码可能会也可能不会执行，这要取决于网络延迟或系统资源等因素，所以，reload()方法最好放到代码的最后一行；

## 23-2 history历史对象

history浏览历史对象并不常用，是window对象的属性，其保存着用户浏览的历史记录，每个窗口，标签页以及每个框架，都有自己的history对象与特定的window对象关联；

当页面的URL改变时，就会生成一条历史记录；

但通过使用history对象可以获知浏览器窗口近来访问过的网页个数，还可以实现从一个页面跳到另一个页面，在实际应用中，如涉及到页面的跳转问题，可以用这个对象来解决；

### 23-2-1 go(n)方法

表示前进或后退；参数是一个整数值，为正则前进，为负为后退；如：

```
history.go(-1); // 后退一页
history.go(1); // 前进一页
history.go(2); // 前进2页
也可以传历史记录中的字符串，如：
history.go("baidu.com");
```

此时浏览器会跳转到历史记录中包含该字符串的第一个记录，有可能后退，也可能前进，具体要看哪个位置最近；如果历史记录中不包含该字符串，此时该方法什么也不做；

### 23-2-2 back()和forward()方法

前进和后退，是go()的简写方法，如：

```
history.back(); // 后退一页
history.forward(); // 前进一页
```

也可以创建自定义的前进和后退功能；

## 23-2-3 length属性

获取历史记录中的数量；对于加载到窗口、标签页或框架中的第一个页面而言，其值为0；

```
alert(history.length);
// 判断用户是不是打开窗口后第一个加载此页面
if(history.length == 0){
 alert("第一个打开的页面");
}
```

如果窗口包含多个子窗口，子窗口的浏览历史会按时间顺序穿插在主窗口的历史中；这意味着在主窗口调用history.back()等方法可能会导致其中一个子窗口跳转，但主窗口保留当前状态不变；

## 23-2-4 history.pushState()和history.replaceState()

HTML5为history对象添加了两个新方法：history.pushState()和history.replaceState()，用来在浏览历史中添加和修改记录；

state属性用来保存记录对象，而popstate事件用来监听history对象的变化；

history.pushState()方法向浏览器历史添加了一个状态；其可以传三个参数：一个状态对象、一个标题（现在被忽略了）以及一个可选的URL地址，如：history.pushState(state, title, url)；

state：状态对象是一个由pushState()方法创建的、与历史记录相关的javascript对象；如果不需要这个对象，此处可以填null；

title：新页面的标题，但是所有浏览器目前都忽略这个值，因此这里可以填null；

URL：这个参数提供了新历史记录的地址；新URL必须和当前URL在同一个域，否则，pushState()将丢出异常。这个参数可选，如果它没有被特别标注，会被设置为文档的当前URL；

```
var stateObj = {foo:'bar'}
history.pushState(stateObj,"new page 1", 'one.html');
```

pushState方法不会触发页面刷新，只是导致history对象发生变化，地址栏的显示地址发生变化；

如果pushState的url参数，设置了一个新的锚点值(即hash)，并不会触发hashchange事件，即使新的URL和旧的只在hash上有区别；

如果设置了一个跨域网址，则会报错；这是由于同源策略限制；

history.replaceState方法的参数与pushState方法一模一样，不同之处在于replaceState()方法会修改当前历史记录条目而非创建新的条目；

```
history.pushState({page:1},"title 1", '?page=1');
history.pushState({page:2},"title 2", '?page=2');
history.replaceState({page:3},"title 3", '?page=3');
// 显示http://127.0.0.1:5500/sub/client.html?page=1
history.back();
// 显示http://127.0.0.1:5500/sub/client.html
history.back();
// 显示http://127.0.0.1:5500/sub/client.html?page=3
history.go(2);

history.state属性返回当前页面的state对象
history.pushState({page:3},'title 3','?page=3');
console.log(history.state);
```



## 24, navigator、screen对象

### 24-1 navigator对象

通过navigator对象，可以获取当前使用的是上面浏览器，浏览器的版本号，浏览器是否支持Java，当前浏览器有哪些插件（Plug-ins）可用等信息，根据这些信息可以定制一些自定义行为或进行相应的处理；

navigator对象是window对象的属性；

最早由Navigator2.0引入的navigator对象，现已成为识别客户端浏览器的事实标准；一般用来检测浏览器及操作系统；

其他浏览器也通过其他方式提供了相同或相似的信息，如，IE中的window.clientInformation和Opera的window.opera，但navigator对象是所有支持javascript的浏览器所共有的；但不同浏览器中的navigator对象都有一些不同的属性；

```
var sum=0;
for(var p in navigator){
 sum++;
 document.write(p + "
");
}
document.write(sum);
```

#### 24-1-1 属性或方法

- **appCodeName**：浏览器的名称；通常是Mozilla，即使在非Mozilla也是如此
- **appMinorVersion**：次版本信息
- **appName**：完整的浏览器名称；
- **appVersion**：浏览器的版本；一般不与实际的浏览器版本对应；
- **buildID**：浏览器编译版本
- **cookieEnabled**：表示cookie是否启用
- **cpuClass**：客户端计算机中使用的cpu类型（x86、68K、Alpha、PPC或其他）
- **javaEnabled()**：表示当前浏览器是否启用了java
- **language**：浏览器的主语言
- **mimeTypes**：在浏览器中注册的MIME类型数组
- **online**：表示浏览器是否连接到了因特网
- **opsProfile**：已不使用
- **oscpu**：客户端计算机的操作系统或使用的CPU
- **platform**：浏览器所在的系统平台
- **plugins**：浏览器中安装的插件信息的数组
- **preference()**：设置用户的首选项
- **product**：产品名称，如Gecko
- **productSub**：关于产品的次要信息，如Gecko的版本
- **register-ContentHandler()**：已经废弃；针对特定的MIME类型将一个站点注册为处理程序
- **register-ProtocolHandler()**：针对特定的协议将一个站点注册为处理程序
- **securityPolicy**：已经废弃；安全策略的名称
- **systemLanguage**：操作系统的名称
- **taintEnabled()**：已经废弃；表示是否允许变量被修改
- **userAgent**：浏览器的用户代理字符串
- **userLanguage**：操作系统的默认语言
- **userProfile**：借以访问用户个人信息的对象
- **vendor**：浏览器的品牌
- **verdorSub**：有关供应商的次要信息，这些属性通常用于检测显示网页的浏览器类型；

浏览器嗅探在有些时候还是必要的，比如，当需要解决存在于某个特定的浏览器的特定版本中的特殊的bug时，此时就可以使用navigator某些属性来获取当前浏览器的版本信息，比如：

appName：Web浏览器的全称；在低版本的IE中，返回“Microsoft Internet Explorer”；在Firefox等其它浏览器中，返回“Netscape”；

appVersion：此属性通常以数字开始，并跟着包含浏览器厂商和版本信息字符串；字符串前面的数字通常是4.0或5.0，表示它是第4或第5代兼容的浏览器；appVersion字符串没有标准的格式，所以，没有办法直接用它来判断浏览器的类型；

userAgent：浏览器在HTTP头部中发送的字符串，该属性通常包含appVersion中的所有信息，并且常常也可能包含其他的细节；它也没有标准格式；但由于它包含浏览器的绝大部分信息，因此浏览器嗅探通常使用它；

platform：操作系统平台的字符串；

## 24-1-2 检测浏览器内核及版本号

```
// 检测浏览器内核及版本号
var browser = (function(){
 var s = navigator.userAgent.toLowerCase();
 var match = /(webkit)[\/]([\w.]+)/.exec(s) ||
 /(opera)(?:.*version)?[\/]([\w.]+)/.exec(s) ||
 /(msie) ([\w.]+)/.exec(s) ||
 !/compatible/.test(s) &&
 /(mozilla)(?:.*? rv:([\w.]+))?/.exec(s) ||
 [];
 return {name: match[1] || "", version: match[2] || "0"};
})();
console.log(navigator.userAgent);
console.log(browser);
```

## 24-1-3 检测操作系统

所有 Windows 版本的操作系统都会包含“Win”字符串，所有 Macintosh 版本的操作系统都会包含“Mac”字符串，所有 Unix 版本的操作系统都会包含有“X11”，而 Linux 操作系统会同时包含“X11”和“Linux”；

```
["win","mac","x11","linux"].forEach(function(t){
 (t == "x11") ? t = "unix" : t; // 处理Unix系统
 // 为navigator对象扩展专用系统检测方法
 navigator["is" + t] = function(){
 return navigator.userAgent.indexOf(t) != -1;
 };
});
console.log(navigator.iswin());
console.log(navigator.ismac());
console.log(navigator.islinux());
console.log(navigator.isunix());
```

## 24-1-4 特征检测法

特征检测法就是根据浏览器是否支持特定的功能来决定相应操作的方式；这是一种非精确判断法，但却是最安全的检测方法，因为准确检测浏览器的类型和型号是一件很困难的事情，而且很容易存在误差；如果不关心浏览器的身份，仅仅在意浏览器的执行能力，那么使用特征检测法就完全可以满足需要；

```
if(document.getElementById)
```

```

 var a = document.getElementsByName("test");
else if(document.getElementsByTagName)
 var a = document.getElementsByTagName("a");
console.log(a);

function getXMLHttpRequest(url){
 var xhr = null;
 if(window.XMLHttpRequest)
 xhr = new XMLHttpRequest();
 else
 xhr = new ActiveXObject("Microsoft.XMLHTTP");
 if(xhr != null){
 // 开始请求提交
 alert("提交成功");
 }else{
 alert("你的浏览器不支持XMLHTTP");
 }
}
getXMLHttpRequest();

```

当使用一个对象、方法或属性时，先判断它是否存在；如果存在，则说明浏览器支持该对象、方法或属性，那么就可以放心使用；这是目前最主流的一种检测方式；

除了浏览器厂商和版本信息的属性之外，navigator对象还包含一些杂项的属性和方法，如以下这些属性，已经得到广泛的应用但未标准化；

- onLine：表示浏览器当前是否连接到网络；应用程序可能希望在离线状态下把状态保存到本地；
- geolocation：用于确定用户地理位置信息的API；
- javaEnabled()：一个非标准的方法，当浏览可以运行Java小程序时返回ture；
- cookieEnable()：非标准的方法，如果浏览器可以保存永久的cookie时，返回true，其它返回false；

## 24-1-5 检测插件

检测浏览器中是否安装了特定的插件是一种最常见的检测例程；

对于非低版本的IE可以使用plugins数组来达到目标；该数组中的每一项包含以下属性：

- name：插件的名字；
- description：插件的描述；
- filename：插件的文件名；
- length：插件所要处理的MIME类型数量；

一般来说，name属性中会包含检测插件必需的所有信息，但有时也完全如此；在检测插件时，需要迭代plugins数组中的每个插件，如：

```

// 检测非低版本的IE插件
function hasPlugin(name){
 name = name.toLowerCase();
 for(var i=0, len=navigator.plugins.length; i<len; i++){
 if(navigator.plugins[i].name.toLowerCase().indexOf(name) > -1)
 return true;
 }
 return false;
}
// 检测Flash
alert(hasPlugin("Flash"));
// 检测QuickTime

```

```
alert(hasPlugin("QuickTime"));
// 检测Java
alert(hasPlugin("Java"));
```

每个插件本身也是一个MimeTypes对象的数组，这些对象可以通过方括号语法来访问，共有四个属性：MIME类型描述description、回指插件对象enabledPlugin、表示与MIME类型对应的文件扩展名的字符串suffixes、表示完整MIME类型字符串type；

检测IE中的插件比较麻烦，因为IE不支持Netscape式的插件；在IE中检测插件的唯一方式就是使用专有的ActiveXObject类型，并尝试创建一个特定插件的实例；IE是以COM对象的方式实现插件的，而COM对象使用唯一标识符来标识；因此，要想检查特定的插件，就必须知道其COM标识符，如：Flash的标识符是：

ShockwaveFlash.ShockwaveFlash；

```
// 检测IE中的插件
function hasIEPlugin(name){
 try{
 new ActiveXObject(name);
 return true;
 }catch(e){
 return false;
 }
}
// 检测Flash
alert(hasIEPlugin("ShockwaveFlash.ShockwaveFlash"));
// 检测QuickTime
alert(hasIEPlugin("QuickTime.QuickTime"));
```

使用了try、catch，因为创建未知的COM对象会导致抛出错误；

两种检测方法差别太大，典型的作法是针对每个插件分别创建测试函数；如：

```
// 检测所有浏览器中的Flash
function hasFlash(){
 var result = hasPlugin("Flash");
 if(!result)
 result = hasIEPlugin("ShockwaveFlash.ShockwaveFlash");
 return result;
}
// 检测所有浏览器中的QuickTime
function hasQuickTime(){
 var result = hasPlugin("QuickTime");
 if(!result)
 result = hasIEPlugin("QuickTime.QuickTime");
 return result;
}
alert(hasFlash()); // 检测Flash
alert(hasQuickTime()); // 检测QuickTime;
```

plugins集合中有个refresh()方法，用于刷新plugins以反映最新安装的插件，这个方法接收一个参数：表示是否应该重新加载页面的一个布尔值；如果为true，重新加载包含插件的所有页面，否则，只更新plugins集合，不重新加载页面；

## 24-1-6 注册处理程序

registerProtocolHandler()方法接收三个参数：要处理的协议(mailto或ftp)、处理该协议的页面的URL和应用程序名称；如：将一个应用程序注册为默认的邮件客户端：

```
navigator.registerProtocolHandler("mailto","http://127.0.0.1:5500/?cmd=%s",
 "Some Mail Client"); // %s表示原始的请示
```

IE不支持，并且在生产环境中，几乎没有什么用途；

## 24-2 screen对象

screen对象提供了获取显示器信息的功能，显示器信息的主要用途是确定网页在客户机是所能达到的最大显示空间；此对象的用处不大，其只是用来获取客户端的能力，其中包括显示器的信息，如宽和高或颜色数量的信息；每个浏览器中的screen对象都包含着各不相同的属性；

### 24-2-1 属性

- availHeight 和 availWidth：只读，屏幕减去系统部件（比如任务栏）的高度和屏幕减去系统部件的宽度；即实际可用的大小；
- colorDepth：只读，返回颜色位数，如24，多数为32位
- pixelDepth：只读，屏幕的位深(FF)
- width 和 height：屏幕的宽度和屏幕的高度
- left 和 top：当前屏幕距左边的距离和距顶边的距离(FF支持)
- availLeft 和 availTop：只读，未被系统占用的最左侧的和最上方的像素值 (FF)
- bufferDepth：读、写用于呈现屏外位图的位数 (IE)
- deviceXDPI与deviceYDPI：只读，实际的水平与垂直DPI(IE)
- logicalXDPI与logicalYDPI：只读，屏幕逻辑的水平与垂直DPI (IE)
- fontSmoothingEnabled：只读，是否启用字体平滑(IE)
- updateInterval：读、写，以毫秒表示的屏幕刷新时间间隔(IE)

这些信息经常出现在测定客户端能力的站点跟踪工具中，但通常不会用于影响功能；不过，有时候也可能用到其中的信息来调整浏览器窗口的大小，使其占据屏幕的可用空间，如：

```
// 网页全屏，非IE会禁用调整窗口的能力，因此是无效的
window.moveTo(0,0);
window.resizeTo(screen.availwidth, screen.availHeight);

// 弹出窗口居中
function center(url){
 var w = screen.availwidth / 2;
 var h = screen.availHeight / 2;
 // 计算居中显示时左侧坐标
 var l = (screen.availwidth - w) / 2;
 // 计算居中显示时顶部坐标
 var t = (screen.availHeight - h) / 2;
 // 计算坐标参数字符串
 var p = "top=" + t + ",left=" + l + ",width=" + w + ",height=" + h;
 var win = window.open(url, "newin", p);
 win.focus();
}
center("https://www.zeronetwork.cn");
```

涉及移动设备的屏幕大小时，情况有所不同；运行iOS的设备始终会返回设备竖着方向的尺寸768X1024，而Android会相应调用screen.width和screen.height的值；

```

// 网页开屏
var x=0, y = window.screen.availHeight, dx=5;
var newWin, intervalID;
function showPage(){
 if(x < screen.availwidth)
 x += dx;
 else
 clearInterval(intervalID);
 newWin.resizeTo(x, y);
}
function showwin(){
 newWin = window.open("", "newWin", "menubar=no, toolbar=no");
 newWin.moveTo(0,0);
 newWin.resizeTo(x,y);
 intervalID = window.setInterval(showPage, 100);
}
showwin();

// 网页布局
function loadCSS(){
 var iwidth = screen.availwidth;
 var sCSSUrl;
 switch(iwidth){
 case 1024:
 sCSSUrl = "style1.css";
 break;
 case 1280:
 sCSSUrl = "style2.css";
 break;
 default:
 sCSSUrl = "default.css";
 break;
 }
 var oCSS = document.createElement("link");
 oCSS.setAttribute("rel", "stylesheet");
 oCSS.setAttribute("type", "text/css");
 oCSS.setAttribute("href", sCSSUrl);
 document.getElementsByTagName("head")[0].appendChild(oCSS);
}
window.onload = loadCSS;

```

## 24-2-2 错误处理

window对象的onerror属性是一个事件处理程序，当未捕获的异常传递到调用栈上时就会调用它，并把错误的消息输出到浏览器的JavaScript控制台上；

window.onerror的第一个参数是描述错误的一条消息，第二个参数是一个字符串，它存放引发错误的JavaScript代码所在的文档的URL，第三个参数是文档中发生错误的行数；

onerror处理程序也有一个返回值，如果返回false，它通知浏览器事件处理程序已经处理了错误，不需要其他操作（换句话说，浏览器不应该显示它自己的错误消息；

onerror处理程序是早期的JavaScript的产物，那时语言核心不包括try/catch异常处理语句；现在实际开发中，虽然很少使用它，但有些项目还在使用它，如：

```

// 在一个对话中弹出错误消息，但不超过三次
window.onerror = function(msg,url,line){

```

```

 if(onerror.num++ < onerror.max){
 alert("ERROR: " + msg + "\nurl: " + url + "\nline: " + line);
 return true;
 }
 }
 onerror.max = 3;
 onerror.num = 0;

 function show(a,b){
 return sum(a,b);
 }
 console.log(show(3,0));

```

## 25，面向对象

### 25-1 ES6 中的类和对象

#### 25-1-1 类的创建

```

// 语法
class name {
 // class body
}

// 创建实例
var xx = new name();

```

#### 25-1-2 类 constructor 构造函数

constructor() 方法是类的构造函数(默认方法)，用于传递参数,返回实例对象，通过 new 命令生成对象实例时，自动调用该方法。如果没有显示定义, 类内部会自动给我们创建一个constructor()

```

class Person {
 constructor(name,age) { // constructor 构造方法或者构造函数
 this.name = name;
 this.age = age;
 }
}

var ldh = new Person('刘德华', 18);
console.log(ldh.name)

```

#### 25-1-3 给类添加方法

```

class Person {
 constructor(name,age) { // constructor 构造器或者构造函数
 this.name = name;
 this.age = age;
 }
 say() {
 console.log(this.name + '你好');
 }
}

var ldh = new Person('刘德华', 18);
ldh.say()
// 注意： 方法之间不能加逗号分隔，同时方法不需要添加 function 关键

```

## 25-2 类的继承

现实中的继承：子承父业，比如我们都继承了父亲的姓。

程序中的继承：子类可以继承父类的一些属性和方法

```

class Father{ // 父类
}
class Son extends Father { // 子类继承父类
}

```

### 25-2-1 继承

```

class Father {
 constructor(surname) {
 this.surname= surname;
 }
 say() {
 console.log('你的姓是' + this.surname);
 }
}
class Son extends Father{ // 这样子类就继承了父类的属性和方法
}
var damao= new Son('刘');
damao.say();

```

### 25-2-2 super 关键字

super 关键字用于访问和调用对象父类上的函数。可以调用父类的构造函数，也可以调用父类的普通函数



```

class Person { // 父类
 constructor(surname){
 this.surname = surname;
 }
}
class Student extends Person { // 子类继承父类
 constructor(surname,firstname){
 super(surname); // 调用父类的constructor(surname)
 this.firstname = firstname; // 定义子类独有的属性
 }
}

```

注意: 子类在构造函数中使用super, 必须放到 this 前面 (必须先调用父类的构造方法,在使用子类构造方法)

```

class Father {
 constructor(surname) {
 this.surname = surname;
 }
 saySurname() {
 console.log('我的姓是' + this.surname);
 }
}
class Son extends Father { // 这样子类就继承了父类的属性和方法
 constructor(surname, fristname) {
 super(surname); // 调用父类的constructor(surname)
 this.fristname = fristname;
 }
 sayFristname() {
 console.log("我的名字是: " + this.fristname);
 }
}
var damao = new Son('刘', "德华");
damao.saySurname();
damao.sayFristname();

```

super关键字 用于访问和调用对象父类上的函数。可以调用父类的构造函数，也可以调用父类的普通函数

```

class Father {
 say() {
 return '我是爸爸';
 }
}
class Son extends Father { // 这样子类就继承了父类的属性和方法
 say() {
 // super.say() super 调用父类的方法
 return super.say() + '的儿子';
 }
}
var damao = new Son();
console.log(damao.say());

```

## 25-3 Class类

```
// 以手机类为例
function Phone(brand,price){
 this.brand = brand
 this.price = price
}
// 静态成员
//相当于 python里的staticmethod
Phone.name = '手机'
Phone.change = function(){
 console.log('我可以改变世界')
}

// 相当于Python里的classmethod
Phone.prototype.size = "5.5英寸"
Phone.prototype.call = function() {
 console.log('我可以打电话')
}

let Huawei = new Phone('华为',5999)

// ES6语法
class Phone{
 // 构造函数方法
 constructor(brand,price){
 this.brand = brand
 this.call = call
 }

 // 语法必须使用这个方法，不能用ES5老语法
 call() {
 console.log('我可以打电话')
 }
}

let Huawei = new Phone('华为',5999)
```

### 25-3-1 构造函数继承

```
function Phone(brand,price){
 this.brand = brand
 this.price = price
}

Phone.prototype.call = function(){
 console.log('我可以打电话')
}

//智能手机
function SmartPhone(brand,price,color,size){
 Phone.call(this,brand,price){
 this.color = color
 this.size = size
 }
}
```

```

// 设置子级构造函数的原型
SmartPhone.prototype = new Phone
SmartPhone.prototype.constructor = SmartPhone

// 声明子类的方法
SmartPhone.prototype.photo = function(){
 console.log('我可以拍照')
}

SmartPhone.prototype.playGame = function() {
 console.log('我可以打游戏')
}

const chuzi = new SmartPhone("锤子",2499,'黑色','505')

```

## 25-3-2 ES6里类的继承

```

class Phone{
 constructor(brand,price){
 this.brand = brand
 this.price = price
 }

 call(){
 console.log('我可以打电话')
 }
}

class SmartPhone extends Phone{
 constructor(brand,price,color,size){
 super(brand,price)
 this.color = color
 this.size = size
 }
 photo() {
 console.log('我可以拍照')
 }
 playGame() {
 console.log('我可以打游戏')
 }
}

const xiaomi = new SmartPhone('xiaomi',2499,'黑色','505')

```

## 25-3-3 子类对父类方法的重写

```

class Phone{
 constructor(brand,price){
 this.brand = brand
 this.price = price
 }

 call(){
 console.log('我可以打电话')
 }
}

```

```

}

class SmartPhone extends Phone{
 constructor(brand,price,color,size){
 super(brand,price)
 this.color = color
 this.size = size
 }
 photo() {
 console.log('我可以拍照')
 }
 playGame() {
 console.log('我可以打游戏')
 }
}

// 可以对父类方法重写
call(){
 console.log('我可以进行视频通话')
}
}

const xiaomi = new SmartPhone('xiaomi',2499,'黑色','505')

```

## 25-3-4 class中的getter和setter

```

class Phone{
 get price() {
 console.log('我被读取了')
 return 'i love you '
 }

 set price(val){
 console.log('价格修改了')
 this.val = val
 }
}

let s = new Phone()
console.log(s.price)
// '我被读取了'
// 'i love you ' -----> 这才是s.price的返回值

s.price = 'free' // '价格修改了'

```

## 26, JavaScript操作DOM

### 26-1 获取元素

#### 26-1-1 根据ID获取

```
document.getElementById('id');
```

使用 console.dir() 可以打印我们获取的元素对象，更好的查看对象里面的属性和方法。

```
<!DOCTYPE html>
```

```

<html lang="en">

<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <meta http-equiv="X-UA-Compatible" content="ie=edge">
 <title>Document</title>
</head>

<body>
 <div id="time">2019-9-9</div>
 <script>
 // 1. 因为我们文档页面从上往下加载，所以先得有标签 所以我们script写到标签的下面
 // 2. get 获得 element 元素 by 通过 驼峰命名法
 // 3. 参数 id是大小写敏感的字符串
 // 4. 返回的是一个元素对象
 var timer = document.getElementById('time');
 console.log(timer);
 console.log(typeof timer);
 // 5. console.dir 打印我们返回的元素对象 更好的查看里面的属性和方法
 console.dir(timer);
 </script>
</body>

</html>

```

## 26-1-2 根据标签名获取

使用 `getElementsByName()` 方法可以返回带有指定标签名的对象的集合。

```
document.getElementsByTagName('标签名');
```

注意：

```

// 因为得到的是一个对象的集合，所以我们想要操作里面的元素就需要遍历。
// 得到元素对象是动态的
// 如果获取不到元素，则返回为空的伪数组(因为获取不到对象)

```

还可以获取某个元素(父元素)内部所有指定标签名的子元素。

```

element.getElementsByTagName('标签名');
// 注意：父元素必须是单个对象(必须指明是哪一个元素对象)。获取的时候不包括父元素自己。

```

```

<!DOCTYPE html>
<html lang="en">

<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <meta http-equiv="X-UA-Compatible" content="ie=edge">
 <title>Document</title>
</head>

<body>

```

```


 知否知否，应是等你好久11
 知否知否，应是等你好久11
 知否知否，应是等你好久11
 知否知否，应是等你好久11

<ol id="o1">
 生僻字
 生僻字
 生僻字
 生僻字

<script>
 // 1. 返回的是 获取过来元素对象的集合 以伪数组的形式存储的
 var lis = document.getElementsByTagName('li');
 console.log(lis);
 console.log(lis[0]);
 // 2. 我们想要依次打印里面的元素对象我们可以采取遍历的方式
 for (var i = 0; i < lis.length; i++) {
 console.log(lis[i]);
 }
 // 3. 如果页面中只有一个li 返回的还是伪数组的形式
 // 4. 如果页面中没有这个元素 返回的是空的伪数组的形式
 // 5. element.getElementsByTagName('标签名'); 父元素必须是指定的单个元素
 // var o1 = document.getElementsByTagName('o1'); // [o1]
 // console.log(o1[0].getElementsByTagName('li'));
 var o1 = document.getElementById('o1');
 console.log(o1.getElementsByTagName('li'));
</script>
</body>

</html>

```

### 26-1-3 通过 HTML5 新增的方法获取

```

document.getElementsByClassName('类名'); // 根据类名返回元素对象集合
document.querySelector('选择器'); // 根据指定选择器返回第一个元素对象
document.querySelectorAll('选择器'); // 根据指定选择器返回

```

注意: // `querySelector` 和 `querySelectorAll` 里面的选择器需要加符号, 比如: `document.querySelector('#nav')`;

```

<!DOCTYPE html>
<html lang="en">

<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">

```

```

<meta http-equiv="X-UA-Compatible" content="ie=edge">
<title>Document</title>
</head>

<body>
 <div class="box">盒子1</div>
 <div class="box">盒子2</div>
 <div id="nav">

 首页
 产品

 </div>
 <script>
 // 1. getElementsByClassName 根据类名获得某些元素集合
 var boxes = document.getElementsByClassName('box');
 console.log(boxes);
 // 2. querySelector 返回指定选择器的第一个元素对象 切记 里面的选择器需要加符号
 .box #nav
 var firstBox = document.querySelector('.box');
 console.log(firstBox);
 var nav = document.querySelector('#nav');
 console.log(nav);
 var li = document.querySelector('li');
 console.log(li);
 // 3. querySelectorAll() 返回指定选择器的所有元素对象集合
 var allBox = document.querySelectorAll('.box');
 console.log(allBox);
 var lis = document.querySelectorAll('li');
 console.log(lis);
 </script>
</body>

</html>

```

## 26-1-4 获取特殊元素 (body, html)

### 4-1 获取body元素

```
document.body // 返回body元素对象
```

### 4-2 获取html元素

```
document.documentElement // 返回html元素对象
```

## 26-2 事件基础

### 26-2-1 事件的三要素

1. 事件源 (谁)
2. 事件类型 (什么事件)
3. 事件处理程序 (做啥)

## 26-2-2 案例：点击按钮弹出警示框

页面中有一个按钮，当鼠标点击按钮的时候，弹出“你好”警示框。



```
// 获取事件源（按钮）
// 注册事件（绑定事件），使用 onclick
// 编写事件处理程序，写一个函数弹出 alert 警示框
```

### 代码实现

```
var btn = document.getElementById('btn');
btn.onclick = function() {
 alert('你好吗');
};
```

## 26-2-3 执行事件的步骤

1. // 获取事件源
2. // 注册事件（绑定事件）
3. // 添加事件处理程序（采取函数赋值形式）

## 26-2-4 常见的鼠标事件

*鼠标事件*	*触发条件*
onclick	鼠标点击左键触发
onmouseover	鼠标经过触发
onmouseout	鼠标离开触发
onfocus	获得鼠标焦点触发
onblur	失去鼠标焦点触发
onmousemove	鼠标移动触发
onmouseup	鼠标弹起触发
onmousedown	鼠标按下触发



## 26-3 操作元素

### 26-3-1 改变元素的内容

`element.innerText`

// 从起始位置到终止位置的内容，但它去除 `html` 标签，同时空格和换行也会去掉

`element.innerHTML`

// 起始位置到终止位置的全部内容，包括 `html` 标签，同时保留空格和换行

```
<!DOCTYPE html>
<html lang="en">

<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <meta http-equiv="X-UA-Compatible" content="ie=edge">
 <title>Document</title>
</head>

<body>
 <div></div>
 <p>
 我是文字
 123
 </p>
 <script>
 // innerText 和 innerHTML的区别
 // 1. innerText 不识别html标签 非标准 去除空格和换行
 var div = document.querySelector('div');
 // div.innerText = '今天是: 2019';
 // 2. innerHTML 识别html标签 W3C标准 保留空格和换行的
 div.innerHTML = '今天是: 2019';
 // 这两个属性是可读写的 可以获取元素里面的内容
 var p = document.querySelector('p');
 console.log(p.innerText);
 console.log(p.innerHTML);
 </script>
</body>

</html>
```

```
<!DOCTYPE html>
<html lang="en">

<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <meta http-equiv="X-UA-Compatible" content="ie=edge">
 <title>Document</title>
 <style>
 div,
 p {
 width: 300px;
 }
 </style>
</head>

<body>
 <div>
 <p>我是文字</p>
 </div>
</body>

</html>
```

```

 height: 30px;
 line-height: 30px;
 color: #fff;
 background-color: pink;
 }
</style>
</head>

<body>
 <button>显示当前系统时间</button>
 <div>某个时间</div>
 <p>1123</p>
 <script>
 // 当我们点击了按钮， div里面的文字会发生变化
 // 1. 获取元素
 var btn = document.querySelector('button');
 var div = document.querySelector('div');
 // 2.注册事件
 btn.onclick = function() {
 // div.innerText = '2019-6-6';
 div.innerHTML = getDate();
 }

 function getDate() {
 var date = new Date();
 // 我们写一个 2019年 5月 1日 星期三
 var year = date.getFullYear();
 var month = date.getMonth() + 1;
 var dates = date.getDate();
 var arr = ['星期日', '星期一', '星期二', '星期三', '星期四', '星期五', '星期六'];

 var day = date.getDay();
 return '今天是: ' + year + '年' + month + '月' + dates + '日 ' +
arr[day];
 }
 // 我们元素可以不用添加事件
 var p = document.querySelector('p');
 p.innerHTML = getDate();
 </script>
</body>

</html>

```

## 26-3-2 常用元素的属性操作

1. innerText、innerHTML 改变元素内容
2. src、href
3. id、alt、title

演示

```

<!DOCTYPE html>
<html lang="en">

<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <meta http-equiv="X-UA-Compatible" content="ie=edge">
 <title>Document</title>
 <style>
 img {
 width: 300px;
 }
 </style>
</head>

<body>
 <button id="ldh">刘德华</button>
 <button id="zxy">张学友</button>

 <script>
 // 修改元素属性 src
 // 1. 获取元素
 var ldh = document.getElementById('ldh');
 var zxy = document.getElementById('zxy');
 var img = document.querySelector('img');
 // 2. 注册事件 处理程序
 zxy.onclick = function() {
 img.src = 'images/zxy.jpg';
 img.title = '张学友思密达';
 }
 ldh.onclick = function() {
 img.src = 'images/ldh.jpg';
 img.title = '刘德华';
 }
 </script>
</body>

</html>

```

## 2-1 分时间显示不同图片案例

根据不同时间，页面显示不同图片，同时显示不同的问候语。

如果上午时间打开页面，显示上午好，显示上午的图片。

如果下午时间打开页面，显示下午好，显示下午的图片。

如果晚上时间打开页面，显示晚上好，显示晚上的图片。

```

// 根据系统不同时间来判断，所以需要用到日期内置对象
// 利用多分支语句来设置不同的图片
// 需要一个图片，并且根据时间修改图片，就需要用到操作元素src属性
// 需要一个div元素，显示不同问候语，修改元素内容即可

```

```

<!DOCTYPE html>
<html lang="en">

```

```

<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <meta http-equiv="X-UA-Compatible" content="ie=edge">
 <title>Document</title>
 <style>
 img {
 width: 300px;
 }
 </style>
</head>

<body>

 <div>上午好</div>
 <script>
 // 根据系统不同时间来判断，所以需要用到日期内置对象
 // 利用多分支语句来设置不同的图片
 // 需要一个图片，并且根据时间修改图片，就需要用到操作元素src属性
 // 需要一个div元素，显示不同问候语，修改元素内容即可
 // 1. 获取元素
 var img = document.querySelector('img');
 var div = document.querySelector('div');
 // 2. 得到当前的小时数
 var date = new Date();
 var h = date.getHours();
 // 3. 判断小时数改变图片和文字信息
 if (h < 12) {
 img.src = 'images/s.gif';
 div.innerHTML = '亲，上午好，好好写代码';
 } else if (h < 18) {
 img.src = 'images/x.gif';
 div.innerHTML = '亲，下午好，好好写代码';
 } else {
 img.src = 'images/w.gif';
 div.innerHTML = '亲，晚上好，好好写代码';
 }
 </script>
</body>

</html>

```

## 26-3-3 表单属性设置

type、value、checked、selected、disabled

### 3-1 案例：仿京东显示密码

点击按钮将密码框切换为文本框，并可以查看密码明文。

京东登录

用户名/邮箱/已验证手机

12312312

忘记密码

登录

短信验证码登录

手机快速注册

核心思路： // 点击眼睛按钮，把密码框类型改为文本框就可以看见里面的密码  
一个按钮两个状态，点击一次，切换为文本框，继续点击一次切换为密码框  
算法： // 利用一个flag变量，来判断flag的值，如果是1 就切换为文本框，flag 设置为0，如果是0 就切换为密码框，flag设置为1

```
<!DOCTYPE html>
<html lang="en">

<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <meta http-equiv="X-UA-Compatible" content="ie=edge">
 <title>Document</title>
 <style>
 .box {
 position: relative;
 width: 400px;
 border-bottom: 1px solid #ccc;
 margin: 100px auto;
 }

 .box input {
 width: 370px;
 height: 30px;
 border: 0;
 outline: none;
 }

 .box img {
 position: absolute;
 top: 2px;
 right: 2px;
 width: 24px;
 }
 </style>
</head>

<body>
 <div class="box">
 <label for="">
```

```


 </label>
 <input type="password" name="" id="pwd">
</div>
<script>
 // 1. 获取元素
 var eye = document.getElementById('eye');
 var pwd = document.getElementById('pwd');
 // 2. 注册事件 处理程序
 var flag = 0;
 eye.onclick = function() {
 // 点击一次之后， flag 一定要变化
 if (flag == 0) {
 pwd.type = 'text';
 eye.src = 'images/open.png';
 flag = 1; // 赋值操作
 } else {
 pwd.type = 'password';
 eye.src = 'images/close.png';
 flag = 0;
 }
 }
</script>
</body>

</html>

```

## 26-3-4 样式属性操作

我们可以通过JS 修改元素的大小、颜色、位置等样式。

```

element.style // 行内样式操作

element.className // 类名样式操作

```

注意：

1. // JS 里面的样式采取驼峰命名法 比如 `fontSize`、`backgroundColor`
2. // JS 修改 `style` 样式操作，产生的是行内样式，CSS 权重比较高

```

<!DOCTYPE html>
<html lang="en">

<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <meta http-equiv="X-UA-Compatible" content="ie=edge">
 <title>Document</title>
 <style>
 div {
 width: 200px;
 height: 200px;
 background-color: pink;
 }
 </style>

```

```

 </style>
</head>

<body>
 <div></div>
 <script>
 // 1. 获取元素
 var div = document.querySelector('div');
 // 2. 注册事件 处理程序
 div.onclick = function() {
 // div.style里面的属性 采取驼峰命名法
 this.style.backgroundColor = 'purple';
 this.style.width = '250px';
 }
 </script>
</body>

</html>

```

#### 4-1 案例： 淘宝点击关闭二维码

当鼠标点击二维码关闭按钮的时候，则关闭整个二维码。



核心思路： 利用样式的显示和隐藏完成， display:none 隐藏元素 display:block 显示元素  
点击按钮，就让这个二维码盒子隐藏起来即可

```

<!DOCTYPE html>
<html lang="en">

<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <meta http-equiv="X-UA-Compatible" content="ie=edge">
 <title>Document</title>
 <style>
 .box {
 position: relative;
 width: 74px;
 height: 88px;
 border: 1px solid #ccc;
 margin: 100px auto;

```

```

 font-size: 12px;
 text-align: center;
 color: #f40;
 /* display: block; */
 }

 .box img {
 width: 60px;
 margin-top: 5px;
 }

 .close-btn {
 position: absolute;
 top: -1px;
 left: -16px;
 width: 14px;
 height: 14px;
 border: 1px solid #ccc;
 line-height: 14px;
 font-family: Arial, Helvetica, sans-serif;
 cursor: pointer;
 }
</style>
</head>

<body>
 <div class="box">
 淘宝二维码

 <i class="close-btn">x</i>
 </div>
 <script>
 // 1. 获取元素
 var btn = document.querySelector('.close-btn');
 var box = document.querySelector('.box');
 // 2.注册事件 程序处理
 btn.onclick = function() {
 box.style.display = 'none';
 }
 </script>
</body>

</html>

```

#### 4-2 案例：循环精灵图背景





```

</style>
</head>

<body>
 <div class="box">

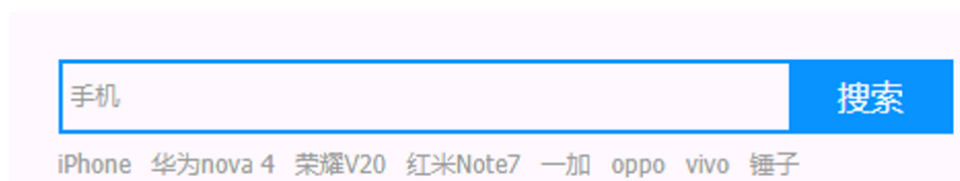
 </div>
 <script>
 // 1. 获取元素 所有的小li
 var lis = document.querySelectorAll('li');
 for (var i = 0; i < lis.length; i++) {
 // 让索引号 乘以 44 就是每个li 的背景y坐标 index就是我们的y坐标
 var index = i * 44;
 lis[i].style.backgroundColor = '0 -' + index + 'px';
 }
 </script>
</body>

</html>

```

#### 4-3 案例：显示隐藏文本框内容

当鼠标点击文本框时，里面的默认文字隐藏，当鼠标离开文本框时，里面的文字显示。



```

// 首先表单需要2个新事件，获得焦点 onfocus 失去焦点 onblur
// 如果获得焦点， 判断表单里面内容是否为默认文字，如果是默认文字，就清空表单内容
// 如果失去焦点， 判断表单内容是否为空，如果为空，则表单内容改为默认文字

```

我们可以通过 JS 修改元素的大小、颜色、位置等样式。

```
element.style // 行内样式操作
```

```
element.className // 类名样式操作
```

```
<!DOCTYPE html>
<html lang="en">

<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <meta http-equiv="X-UA-Compatible" content="ie=edge">
 <title>Document</title>
 <style>
 input {
 color: #999;
 }
 </style>
</head>

<body>
 <input type="text" value="手机">
 <script>
 // 1. 获取元素
 var text = document.querySelector('input');
 // 2. 注册事件 获得焦点事件 onfocus
 text.onfocus = function() {
 // console.log('得到了焦点');
 if (this.value === '手机') {
 this.value = '';
 }
 // 获得焦点需要把文本框里面的文字颜色变黑
 this.style.color = '#333';
 }
 // 3. 注册事件 失去焦点事件 onblur
 text.onblur = function() {
 // console.log('失去了焦点');
 if (this.value === '') {
 this.value = '手机';
 }
 // 失去焦点需要把文本框里面的文字颜色变浅色
 this.style.color = '#999';
 }
 </script>

</body>

</html>
```

#### 4-4 案例：密码框格式提示错误信息

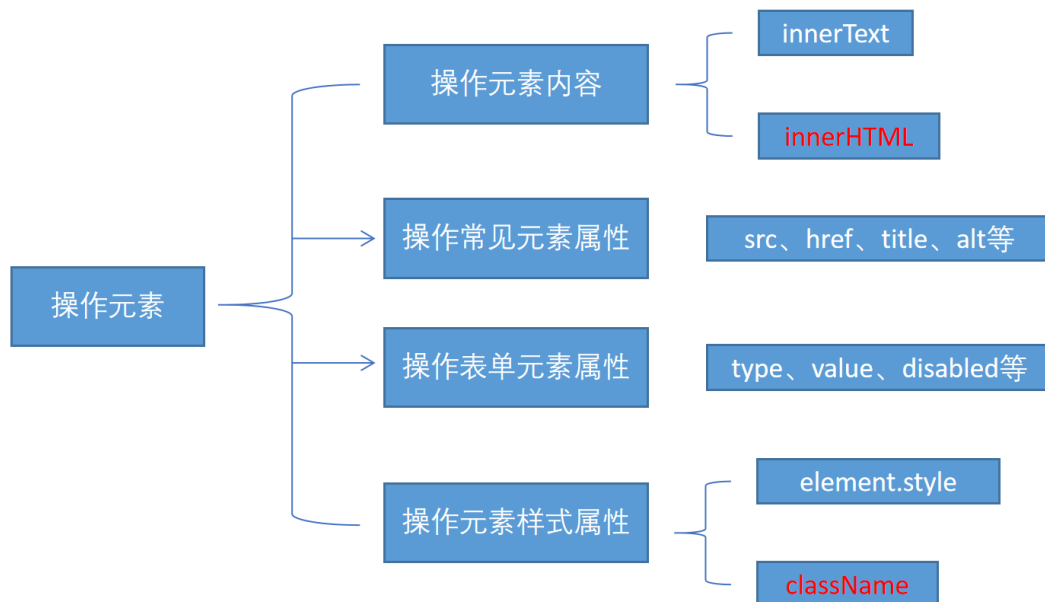
用户如果离开密码框，里面输入个数不是6~16，则提示错误信息，否则提示输入正确信息

\* 设置密码:  ✖ 请输入登录密码

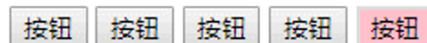
```
// 首先判断的事件是表单失去焦点 onblur
// 如果输入正确则提示正确的信息颜色为绿色小图标变化
// 如果输入不是6到16位，则提示错误信息颜色为红色 小图标变化
// 因为里面变化样式较多，我们采取className修改样式
```

## 26-3-5 操作元素总结

操作元素是 DOM 核心内容



## 26-3-6 排他思想



如果有同一组元素，我们想要某一个元素实现某种样式，需要用到循环的排他思想算法：

1. 所有元素全部清除样式（干掉其他人）
2. 给当前元素设置样式（留下我自己）
3. 注意顺序不能颠倒，首先干掉其他人，再设置自己

```
<!DOCTYPE html>
<html lang="en">

<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```

<meta http-equiv="X-UA-Compatible" content="ie=edge">
<title>Document</title>
</head>

<body>
 <button>按钮1</button>
 <button>按钮2</button>
 <button>按钮3</button>
 <button>按钮4</button>
 <button>按钮5</button>
 <script>
 // 1. 获取所有按钮元素
 var btns = document.getElementsByTagName('button');
 // btns得到的是伪数组 里面的每一个元素 btns[i]
 for (var i = 0; i < btns.length; i++) {
 btns[i].onclick = function() {
 // (1) 我们先把所有的按钮背景颜色去掉 干掉所有人
 for (var i = 0; i < btns.length; i++) {
 btns[i].style.backgroundColor = '';
 }
 // (2) 然后才让当前的元素背景颜色为pink 留下我自己
 this.style.backgroundColor = 'pink';
 }
 }
 //2. 首先排除其他人，然后才设置自己的样式 这种排除其他人的思想我们成为排他思想
 </script>
</body>

</html>

```

## 6-1 案例：百度换肤

这个案例练习的是给一组元素注册事件

给4个小图片利用循环注册点击事件

当我们点击了这个图片，让我们页面背景改为当前的图片

核心算法：把当前图片的src 路径取过来，给 body 做为背景即可

```

<!DOCTYPE html>
<html lang="en">

<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <meta http-equiv="X-UA-Compatible" content="ie=edge">
 <title>Document</title>
 <style>
 * {
 margin: 0;
 padding: 0;
 }

 body {
 background: url(images/1.jpg) no-repeat center top;
 }

 li {

```

```

 list-style: none;
 }

 .baidu {
 overflow: hidden;
 margin: 100px auto;
 background-color: #fff;
 width: 410px;
 padding-top: 3px;
 }

 .baidu li {
 float: left;
 margin: 0 1px;
 cursor: pointer;
 }

 .baidu img {
 width: 100px;
 }
</style>
</head>

<body>
 <ul class="baidu">

 <script>
 // 1. 获取元素
 var imgs = document.querySelector('.baidu').querySelectorAll('img');
 // console.log(imgs);
 // 2. 循环注册事件
 for (var i = 0; i < imgs.length; i++) {
 imgs[i].onclick = function() {
 // this.src 就是我们点击图片的路径 images/2.jpg
 // console.log(this.src);
 // 把这个路径 this.src 给body 就可以了
 document.body.style.backgroundImage = 'url(' + this.src + ')';
 }
 }
 </script>
</body>

</html>

```

## 6-2 案例：表格隔行变色

代码	名称	最新公布净值	累计净值	前单位净值	净值增长率	公布日期
003526	农银金穗3个月定期开放债券	1.075	1.079	1.074	+0.047%	2019-01-11
270047	广发理财30天债券B	0.903	3.386	0.000	0.000%	2019-01-16
163417	兴全合宜混合A	0.860	0.860	0.863	-0.382%	2019-01-16
003929	中银证券安进债券A	1.034	1.088	1.034	+0.077%	2019-01-16
360020	光大添天盈月度理财债券B	0.950	3.557	0.000	0.000%	2019-01-16

用到新的鼠标事件 鼠标经过 onmouseover 鼠标离开 onmouseout

核心思路：鼠标经过 tr 行，当前的行变背景颜色，鼠标离开去掉当前的背景颜色

注意：第一行（thead里面的行）不需要变换颜色，因此我们获取的是 tbody 里面的行

```
<!DOCTYPE html>
<html lang="en">

<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <meta http-equiv="X-UA-Compatible" content="ie=edge">
 <title>Document</title>
 <style>
 table {
 width: 800px;
 margin: 100px auto;
 text-align: center;
 border-collapse: collapse;
 font-size: 14px;
 }

 thead tr {
 height: 30px;
 background-color: skyblue;
 }

 tbody tr {
 height: 30px;
 }

 tbody td {
 border-bottom: 1px solid #d7d7d7;
 font-size: 12px;
 color: blue;
 }

 .bg {
 background-color: pink;
 }
 </style>
</head>

<body>
 <table>
 <thead>
 <tr>
 <th>代码</th>
```

[illegible]



```

<script>
 // 1. 获取元素 获取的是 tbody 里面所有的行
 var trs = document.querySelector('tbody').querySelectorAll('tr');
 // 2. 利用循环绑定注册事件
 for (var i = 0; i < trs.length; i++) {
 // 3. 鼠标经过事件 onmouseover
 trs[i].onmouseover = function() {
 // console.log(11);
 this.className = 'bg';
 }
 // 4. 鼠标离开事件 onmouseout
 trs[i].onmouseout = function() {
 this.className = '';
 }
 }
</script>
</body>

</html>

```

### 6-3 案例：表单全选取消全选案例

#### 业务需求：

1. 点击上面全选复选框，下面所有的复选框都选中（全选）
2. 再次点击全选复选框，下面所有的复选框都不选中（取消全选）
3. 如果下面复选框全部选中，上面全选按钮就自动选中
4. 如果下面复选框有一个没有选中，上面全选按钮就不选中
5. 所有复选框一开始默认都没选中状态

<input type="checkbox"/>	商品	价钱
<input type="checkbox"/>	iPhone8	8000
<input type="checkbox"/>	iPad Pro	5000
<input type="checkbox"/>	iPad Air	2000
<input type="checkbox"/>	Apple Watch	2000

- 全选和取消全选做法： 让下面所有复选框的checked属性（选中状态） 跟随 全选按钮即可
- 下面复选框需要全部选中， 上面全选才能选中做法： 给下面所有复选框绑定点击事件， 每次点击， 都要循环查看下面所有的复选框是否有没选中的， 如果有一个没选中的， 上面全选就不选中。
- 可以设置一个变量， 来控制全选是否选中

```

<!DOCTYPE html>
<html>

<head lang="en">
 <meta charset="UTF-8">
 <title></title>
 <style>
 * {
 padding: 0;
 margin: 0;
 }

 .wrap {
 width: 300px;
 margin: 100px auto 0;
 }

 table {
 border-collapse: collapse;
 border-spacing: 0;

```



```

 <td>5000</td>
 </tr>
 <tr>
 <td>
 <input type="checkbox" />
 </td>
 <td>iPad Air</td>
 <td>2000</td>
 </tr>
 <tr>
 <td>
 <input type="checkbox" />
 </td>
 <td>Apple watch</td>
 <td>2000</td>
 </tr>
 </tbody>
</table>
</div>
<script>

```

// 1. 全选和取消全选做法： 让下面所有复选框的checked属性（选中状态） 跟随 全选按钮  
即可

```

// 获取元素
var j_cbAll = document.getElementById('j_cbAll'); // 全选按钮
var j_tbs =
document.getElementById('j_tb').getElementsByTagName('input'); // 下面所有的复选框
// 注册事件
j_cbAll.onclick = function() {
 // this.checked 它可以得到当前复选框的选中状态如果是true 就是选中，如果是
false 就是未选中

```

```

 console.log(this.checked);
 for (var i = 0; i < j_tbs.length; i++) {
 j_tbs[i].checked = this.checked;
 }
}

```

// 2. 下面复选框需要全部选中， 上面全选才能选中做法： 给下面所有复选框绑定点击事件，每次点击，都要循环查看下面所有的复选框是否有没选中的，如果有一个没选中的， 上面全选就不选中。

```

for (var i = 0; i < j_tbs.length; i++) {
 j_tbs[i].onclick = function() {
 // flag 控制全选按钮是否选中
 var flag = true;
 // 每次点击下面的复选框都要循环检查者4个小按钮是否全被选中
 for (var i = 0; i < j_tbs.length; i++) {
 if (!j_tbs[i].checked) {
 flag = false;
 break; // 退出for循环 这样可以提高执行效率 因为只要有一个没有选中，剩下的就无需循环判断了
 }
 }
 j_cbAll.checked = flag;
 }
}

```

```

</script>
</body>

</html>

```

## 26-3-7 自定义属性

### 7-1 获取属性值

```
element.属性 // 获取属性值。
element.getAttribute('属性');
```

区别:

```
// element.属性 获取内置属性值（元素本身自带的属性）
// element.getAttribute('属性'); 主要获得自定义的属性（标准） 我们程序员自定义的属性
```

### 7-2 设置属性值

```
element.属性 = '值' // 设置内置属性值。
element.setAttribute('属性', '值');
```

区别:

```
// element.属性 设置内置属性值
// element.setAttribute('属性'); 主要设置自定义的属性（标准）
```

### 7-3 移除属性

```
element.removeAttribute('属性');
```

### 自定义属性演示

```
<!DOCTYPE html>
<html lang="en">

<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <meta http-equiv="X-UA-Compatible" content="ie=edge">
 <title>Document</title>
</head>

<body>
 <div id="demo" index="1" class="nav"></div>
 <script>
 var div = document.querySelector('div');
 // 1. 获取元素的属性值
 // (1) element.属性
 console.log(div.id);
 //(2) element.getAttribute('属性') get得到获取 attribute 属性的意思 我们程序员自己添加的属性我们称为自定义属性 index
 console.log(div.getAttribute('id'));
 console.log(div.getAttribute('index'));
 // 2. 设置元素属性值
 // (1) element.属性= '值'
 div.id = 'test';
 div.className = 'navs';
 // (2) element.setAttribute('属性', '值'); 主要针对于自定义属性
 div.setAttribute('index', 2);
```

```

 div.setAttribute('class', 'footer'); // class 特殊 这里面写的就是class 不是
 className
 // 3 移除属性 removeAttribute(属性)
 div.removeAttribute('index');
 </script>
</body>

</html>

```

#### 7-4 案例：tab 栏切换（重点案例）

##### 案例：tab 栏切换（重点案例）

当鼠标点击上面相应的选项卡（tab），下面内容跟随变化



- ① Tab栏切换有2个大的模块
- ② 上的模块选项卡，点击某一个，当前这一个底色会是红色，其余不变（排他思想）修改类名的方式
- ③ 下面的模块内容，会跟随上面的选项卡变化。所以下面模块变化写到点击事件里面。
- ④ 规律：下面的模块显示内容和上面的选项卡——对应，相匹配。
- ⑤ 核心思路：给上面的tab\_list 里面的所有小li 添加自定义属性，属性值从0开始编号。
- ⑥ 当我们点击tab\_list 里面的某个小li，让tab\_con 里面对应序号的 内容显示，其余隐藏（排他思想）

```

<!DOCTYPE html>
<html lang="en">

<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <meta http-equiv="X-UA-Compatible" content="ie=edge">
 <title>Document</title>
 <style>
 * {
 margin: 0;
 padding: 0;
 }

 li {
 list-style-type: none;
 }
 </style>

```

```

.tab {
 width: 978px;
 margin: 100px auto;
}

.tab_list {
 height: 39px;
 border: 1px solid #ccc;
 background-color: #f1f1f1;
}

.tab_list li {
 float: left;
 height: 39px;
 line-height: 39px;
 padding: 0 20px;
 text-align: center;
 cursor: pointer;
}

.tab_list .current {
 background-color: #c81623;
 color: #fff;
}

.item_info {
 padding: 20px 0 0 20px;
}

.item {
 display: none;
}
</style>
</head>

<body>
 <div class="tab">
 <div class="tab_list">

 <li class="current">商品介绍
 规格与包装
 售后保障
 商品评价（50000）
 手机社区

 </div>
 <div class="tab_con">
 <div class="item" style="display: block;">
 商品介绍模块内容
 </div>
 <div class="item">
 规格与包装模块内容
 </div>
 <div class="item">
 售后保障模块内容
 </div>
 <div class="item">

```

```

商品评价（50000）模块内容
</div>
<div class="item">
 手机社区模块内容
</div>

</div>
</div>
<script>
 // 获取元素
 var tab_list = document.querySelector('.tab_list');
 var lis = tab_list.querySelectorAll('li');
 var items = document.querySelectorAll('.item');
 // for循环绑定点击事件
 for (var i = 0; i < lis.length; i++) {
 // 开始给5个小li 设置索引号
 lis[i].setAttribute('index', i);
 lis[i].onclick = function() {
 // 1. 上的模块选项卡，点击某一个，当前这一个底色会是红色，其余不变（排他思想） 修改类名的方式

 // 干掉所有人 其余的li清除 class 这个类
 for (var i = 0; i < lis.length; i++) {
 lis[i].className = '';
 }
 // 留下我自己
 this.className = 'current';
 // 2. 下面的显示内容模块
 var index = this.getAttribute('index');
 console.log(index);
 // 干掉所有人 让其余的item 这些div 隐藏
 for (var i = 0; i < items.length; i++) {
 items[i].style.display = 'none';
 }
 // 留下我自己 让对应的item 显示出来
 items[index].style.display = 'block';
 }
 }
</script>
</body>

</html>

```

## 26-3-8 H5 自定义属性

```

// 自定义属性目的：是为了保存并使用数据。有些数据可以保存到页面中而不用保存到数据库中。
// 自定义属性获取是通过getAttribute(‘属性’) 获取。
// 但是有些自定义属性很容易引起歧义，不容易判断是元素的内置属性还是自定义属性。
// H5给我们新增了自定义属性：

```

### 8-1 设置H5自定义属性

```
// H5规定自定义属性data-开头做为属性名并且赋值。比如：
<div data-index="1"></div>
// 或者使用 JS 设置
element.setAttribute('data-index', 2)
```

## 8-2 获取H5自定义属性

```
// 兼容性获取
element.getAttribute('data-index');
// H5新增
element.dataset.index 或者 element.dataset['index'] ie 11才开始支持
```

```
<!DOCTYPE html>
<html lang="en">

<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <meta http-equiv="X-UA-Compatible" content="ie=edge">
 <title>Document</title>
</head>

<body>
 <div getTime="20" data-index="2" data-list-name="andy"></div>
 <script>
 var div = document.querySelector('div');
 // console.log(div.getTime);
 console.log(div.getAttribute('getTime'));
 div.setAttribute('data-time', 20);
 console.log(div.getAttribute('data-index'));
 console.log(div.getAttribute('data-list-name'));
 // h5新增的获取自定义属性的方法 它只能获取data-开头的
 // dataset 是一个集合里面存放了所有以data开头的自定义属性
 console.log(div.dataset);
 console.log(div.dataset.index);
 console.log(div.dataset['index']);
 // 如果自定义属性里面有多条-链接的单词，我们获取的时候采取 驼峰命名法
 console.log(div.dataset.listName);
 console.log(div.dataset['listName']);
 </script>
</body>

</html>
```

## 26-4 节点操作

### 26-4-1 为什么学节点操作



获取元素通常使用两种方式：

## 1. 利用 DOM 提供的方法获取元素

- document.getElementById()
- document.getElementsByTagName()
- document.querySelector 等
- 逻辑性不强、繁琐

## 2. 利用节点层级关系获取元素

- 利用父子兄节点关系获取元素
- 逻辑性强，但是兼容性稍差

这两种方式都可以获取元素节点，我们后面都会使用，但是节点操作更简单

## 26-4-2 节点概述

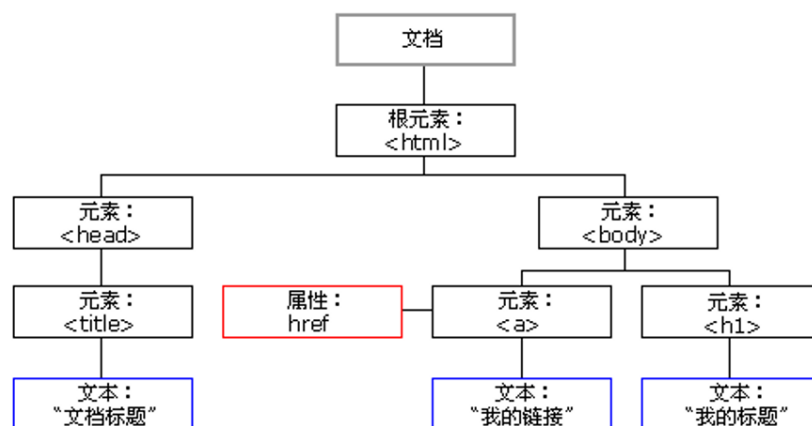
一般地，节点至少拥有nodeType（节点类型）、nodeName（节点名称）和nodeValue（节点值）这三个基本属性。

- 元素节点 nodeType 为 1
- 属性节点 nodeType 为 2
- 文本节点 nodeType 为 3（文本节点包含文字、空格、换行等）

我们在实际开发中，节点操作主要操作的是**元素节点**

## 26-4-3 节点层级

利用 DOM 树可以把节点划分为不同的层级关系，常见的是**父子兄层级关系**。



### 3-1 父级节点

node.parentNode

- parentNode 属性可返回某节点的父节点，注意是最近的一个父节点
- 如果指定的节点没有父节点则返回 null

```
<!DOCTYPE html>
<html lang="en">
```

```

<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <meta http-equiv="X-UA-Compatible" content="ie=edge">
 <title>Document</title>
</head>

<body>
 <!-- 节点的优点 -->
 <div>我是div</div>
 我是span

 我是li
 我是li
 我是li
 我是li

 <div class="demo">
 <div class="box">
 x
 </div>
 </div>

 <script>
 // 1. 父节点 parentNode
 var erweima = document.querySelector('.erweima');
 // var box = document.querySelector('.box');
 // 得到的是离元素最近的父级节点(亲爸爸) 如果找不到父节点就返回为 null
 console.log(erweima.parentNode);
 </script>
</body>

</html>

```

## 3-2 子节点

### 2-1 parentNode.childNodes (标准)

parentNode.childNodes (标准)

- parentNode.childNodes 返回包含指定节点的子节点的集合，该集合为即时更新的集合。
- 注意：返回值里面包含了所有的子节点，包括元素节点，文本节点等。
- 如果只想要获得里面的元素节点，则需要专门处理。所以我们一般不提倡使用childNodes

```

var ul = document.querySelector('ul');
for(var i = 0; i < ul.childNodes.length; i++) {
 if (ul.childNodes[i].nodeType == 1) { // ul.childNodes[i] 是元素节点
 console.log(ul.childNodes[i]);
 }
}

```

```

<!DOCTYPE html>
<html lang="en">

<head>

```

```

<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<meta http-equiv="X-UA-Compatible" content="ie=edge">
<title>Document</title>
</head>

<body>
 <!-- 节点的优点 -->
 <div>我是div</div>
 我是span

 我是li
 我是li
 我是li
 我是li

 我是li
 我是li
 我是li
 我是li

 <div class="demo">
 <div class="box">
 x
 </div>
 </div>

 <script>
 // DOM 提供的方法（API）获取
 var ul = document.querySelector('ul');
 var lis = ul.querySelectorAll('li');
 // 1. 子节点 childNodes 所有的子节点 包含 元素节点 文本节点等等
 console.log(ul.childNodes);
 console.log(ul.childNodes[0].nodeType);
 console.log(ul.childNodes[1].nodeType);
 // 2. children 获取所有的子元素节点 也是我们实际开发常用的
 console.log(ul.children);
 </script>
</body>

</html>

```

## 2-2 parentNode.children (非标准)

// **parentNode.children** 是一个只读属性，返回所有的子元素节点。它只返回子元素节点，其余节点不返回（这个是我们重点掌握的）。

// 虽然**children** 是一个非标准，但是得到了各个浏览器的支持，因此我们可以放心使用

## 2-3 parentNode.firstChild

`/ firstChild` 返回第一个子节点，找不到则返回`null`。同样，也是包含所有的节点。

## 2-4 parentNode.lastChild

`// lastChild` 返回最后一个子节点，找不到则返回`null`。同样，也是包含所有的节点。

## 2-5 parentNode.firstChildElementChild

`// firstElementChild` 返回第一个子元素节点，找不到则返回`null`。

## 2-6 parentNode.lastElementChild

`lastElementChild` 返回最后一个子元素节点，找不到则返回`null`。

**注意：这两个方法有兼容性问题，IE9 以上才支持。**

```
<!DOCTYPE html>
<html lang="en">

<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <meta http-equiv="X-UA-Compatible" content="ie=edge">
 <title>Document</title>
</head>

<body>

 我是li1
 我是li2
 我是li3
 我是li4
 我是li5

 <script>
 var ol = document.querySelector('ol');
 // 1. firstChild 第一个子节点 不管是文本节点还是元素节点
 console.log(ol.firstChild);
 console.log(ol.lastChild);
 // 2. firstElementChild 返回第一个子元素节点 ie9才支持
 console.log(ol.firstElementChild);
 console.log(ol.lastElementChild);
 // 3. 实际开发的写法 既没有兼容性问题又返回第一个子元素
 console.log(ol.children[0]);
 console.log(ol.children[ol.children.length - 1]);
 </script>
</body>

</html>
```

实际开发中，`firstChild` 和 `lastChild` 包含其他节点，操作不方便，而 `firstElementChild` 和 `lastElementChild` 又有兼容性问题，那么我们如何获取第一个子元素节点或最后一个子元素节点呢？

解决方案：

```
// 如果想要第一个子元素节点，可以使用 parentNode.children[0]
// 如果想要最后一个子元素节点，可以使用 parentNode.children[parentNode.children.length - 1]
```

## 2-7 案例：下拉菜单



- 导航栏里面的li 都要有鼠标经过效果，所以需要循环注册鼠标事件
- 核心原理：当鼠标经过li 里面的 第二个孩子 ul 显示，当鼠标离开，则ul 隐藏

```
var nav = document.querySelector('.nav');
var lis = nav.children; // 得到4个小li
for (var i = 0; i < lis.length; i++) {
 lis[i].onmouseover = function() {
 this.children[1].style.display = 'block';
 }
 lis[i].onmouseout = function() {
 this.children[1].style.display = 'none';
 }
}
```

完整代码

```
<!DOCTYPE html>
<html lang="en">

<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <meta http-equiv="X-UA-Compatible" content="ie=edge">
 <title>Document</title>
 <style>
 * {
 margin: 0;
 padding: 0;
 }

 li {
```

```

 list-style-type: none;
 }

 a {
 text-decoration: none;
 font-size: 14px;
 }

 .nav {
 margin: 100px;
 }

 .nav>li {
 position: relative;
 float: left;
 width: 80px;
 height: 41px;
 text-align: center;
 }

 .nav li a {
 display: block;
 width: 100%;
 height: 100%;
 line-height: 41px;
 color: #333;
 }

 .nav>li>a:hover {
 background-color: #eee;
 }

 .nav ul {
 display: none;
 position: absolute;
 top: 41px;
 left: 0;
 width: 100%;
 border-left: 1px solid #FECC5B;
 border-right: 1px solid #FECC5B;
 }

 .nav ul li {
 border-bottom: 1px solid #FECC5B;
 }

 .nav ul li a:hover {
 background-color: #FFF5DA;
 }
</style>
</head>

<body>
 <ul class="nav">

 微博


```

```

 私信

 评论

 @我

 微博

 私信

 评论

 @我

 微博

 私信

 评论

 @我

 微博

 私信

 评论

 @我

<script>
 // 1. 获取元素
 var nav = document.querySelector('.nav');
 var lis = nav.children; // 得到4个小li
 // 2. 循环注册事件

```

```

 for (var i = 0; i < lis.length; i++) {
 lis[i].onmouseover = function() {
 this.children[1].style.display = 'block';
 }
 lis[i].onmouseout = function() {
 this.children[1].style.display = 'none';
 }
 }
 </script>
</body>

</html>

```

### 3-3 兄弟节点

#### 3-1 node.nextSibling

// nextSibling 返回当前元素的下一个兄弟元素节点，找不到则返回null。同样，也是包含所有的节点

#### 3-2 node.previousSibling

// previousSibling 返回当前元素上一个兄弟元素节点，找不到则返回null。同样，也是包含所有的节点

#### 3-3 node.nextElementSibling

// nextElementSibling 返回当前元素下一个兄弟元素节点，找不到则返回null。

#### 3-4 node.previousElementSibling

// previousElementSibling 返回当前元素上一个兄弟节点，找不到则返回null。

**注意：这两个方法有兼容性问题，IE9 以上才支持。**

**问：如何解决兼容性问题？**

```

// 自己封装一个兼容性的函数
function getNextElementSibling(element) {
 var e1 = element;
 while (e1 = e1.nextSibling) {
 if (e1.nodeType === 1) {
 return e1;
 }
 }
 return null;
}

```

### 兄弟节点演示

```

<!DOCTYPE html>
<html lang="en">

<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">

```



```

<meta http-equiv="X-UA-Compatible" content="ie=edge">
<title>Document</title>
</head>

<body>
 <div>我是div</div>
 我是span
 <script>
 var div = document.querySelector('div');
 // 1.nextSibling 下一个兄弟节点 包含元素节点或者 文本节点等等
 console.log(div.nextSibling);
 console.log(div.previousSibling);
 // 2. nextElementSibling 得到下一个兄弟元素节点
 console.log(div.nextElementSibling);
 console.log(div.previousElementSibling);
 </script>
</body>

</html>

```

## 26-4-4 节点创建

```
document.createElement('tagName')
```

document.createElement() 方法创建由 tagName 指定的 HTML 元素。因为这些元素原先不存在，是根据我们的需求动态生成的，所以我们也称为动态创建元素节点。

## 26-4-5 添加节点

### 5-1 node.appendChild(child)

```
node.appendChild() // 方法将一个节点添加到指定父节点的子节点列表末尾。类似于 CSS 里面的 after 伪元素。
```

### 5-2 node.insertBefore(child, 指定元素)

```
node.insertBefore() // 方法将一个节点添加到父节点的指定子节点前面。类似于 CSS 里面的 before 伪元素。
```

## 创建和添加节点演示

```

<!DOCTYPE html>
<html lang="en">

<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <meta http-equiv="X-UA-Compatible" content="ie=edge">
 <title>Document</title>
</head>

<body>

 123


```

```

<script>
 // 1. 创建节点元素节点
 var li = document.createElement('li');
 // 2. 添加节点 node.appendChild(child) node 父级 child 是子级 后面追加元素
 类似于数组中的push
 var ul = document.querySelector('ul');
 ul.appendChild(li);
 // 3. 添加节点 node.insertBefore(child, 指定元素);
 var lili = document.createElement('li');
 ul.insertBefore(lili, ul.children[0]);
 // 4. 我们想要页面添加一个新的元素 : 1. 创建元素 2. 添加元素
</script>
</body>

</html>

```

### 5-3 案例：简单版发布留言案例

- 核心思路： 点击按钮之后，就动态创建一个li，添加到ul 里面。
- 创建li 的同时，把文本域里面的值通过li.innerHTML 赋值给 li
- 如果想要新的留言后面显示就用 appendChild 如果想要前面显示就用insertBefore

```

<!DOCTYPE html>
<html lang="en">

<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <meta http-equiv="X-UA-Compatible" content="ie=edge">
 <title>Document</title>
 <style>
 * {
 margin: 0;
 padding: 0;
 }

 body {
 padding: 100px;
 }

 textarea {
 width: 200px;
 height: 100px;
 border: 1px solid pink;
 outline: none;
 resize: none;
 }

 ul {
 margin-top: 50px;
 }

 li {
 width: 300px;
 padding: 5px;
 background-color: rgb(245, 209, 243);
 color: red;
 }
 </style>

```

```

 font-size: 14px;
 margin: 15px 0;
 }
</style>
</head>

<body>
 <textarea name="" id=""></textarea>
 <button>发布</button>

<script>
 // 1. 获取元素
 var btn = document.querySelector('button');
 var text = document.querySelector('textarea');
 var ul = document.querySelector('ul');
 // 2. 注册事件
 btn.onclick = function() {
 if (text.value == '') {
 alert('您没有输入内容');
 return false;
 } else {
 // console.log(text.value);
 // (1) 创建元素
 var li = document.createElement('li');
 // 先有li 才能赋值
 li.innerHTML = text.value;
 // (2) 添加元素
 // ul.appendChild(li);
 ul.insertBefore(li, ul.children[0]);
 }
 }
</script>
</body>

</html>

```

## 26-4-6 删除节点

```
node.removeChild(child)
```

**node.removeChild()** 方法从 DOM 中删除一个子节点，返回删除的节点。

**删除节点演示**

```

<!DOCTYPE html>
<html lang="en">

<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <meta http-equiv="X-UA-Compatible" content="ie=edge">
 <title>Document</title>
</head>

```

```

<body>
 <button>删除</button>

 熊大
 熊二
 光头强

 <script>
 // 1. 获取元素
 var ul = document.querySelector('ul');
 var btn = document.querySelector('button');
 // 2. 删除元素 node.removeChild(child)
 // ul.removeChild(ul.children[0]);
 // 3. 点击按钮依次删除里面的孩子
 btn.onclick = function() {
 if (ul.children.length == 0) {
 this.disabled = true;
 } else {
 ul.removeChild(ul.children[0]);
 }
 }
 </script>
</body>

</html>

```

## 6-1 案例：删除留言案例

- 当我们把文本域里面的值赋值给li 的时候，多添加一个删除的链接
- 需要把所有的链接获取过来，当我们点击当前的链接的时候，删除当前链接所在的li
- 阻止链接跳转需要添加 javascript:void(0); 或者 javascript:;

```

<!DOCTYPE html>
<html lang="en">

<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <meta http-equiv="X-UA-Compatible" content="ie=edge">
 <title>Document</title>
 <style>
 * {
 margin: 0;
 padding: 0;
 }

 body {
 padding: 100px;
 }

 textarea {
 width: 200px;
 height: 100px;
 border: 1px solid pink;
 outline: none;
 }
 </style>

```

```

 resize: none;
 }

 ul {
 margin-top: 50px;
 }

 li {
 width: 300px;
 padding: 5px;
 background-color: rgb(245, 209, 243);
 color: red;
 font-size: 14px;
 margin: 15px 0;
 }

 li a {
 float: right;
 }
}
</style>
</head>

<body>
 <textarea name="" id=""></textarea>
 <button>发布</button>

<script>
 // 1. 获取元素
 var btn = document.querySelector('button');
 var text = document.querySelector('textarea');
 var ul = document.querySelector('ul');
 // 2. 注册事件
 btn.onclick = function() {
 if (text.value == '') {
 alert('您没有输入内容');
 return false;
 } else {
 // console.log(text.value);
 // (1) 创建元素
 var li = document.createElement('li');
 // 先有li 才能赋值
 li.innerHTML = text.value + "删除";
 // (2) 添加元素
 // ul.appendChild(li);
 ul.insertBefore(li, ul.children[0]);
 // (3) 删除元素 删除的是当前链接的li 它的父亲
 var as = document.querySelectorAll('a');
 for (var i = 0; i < as.length; i++) {
 as[i].onclick = function() {
 // node.removeChild(child); 删除的是 li 当前a所在的li
 this.parentNode;

 ul.removeChild(this.parentNode);
 }
 }
 }
 }
}

```

```
</script>
</body>

</html>
```

## 26-4-7 复制节点

```
node.cloneNode()
// node.cloneNode() 方法返回调用该方法的节点的一个副本。 也称为克隆节点/拷贝节点
```

注意：

1. 如果括号参数为空或者为 false，则是浅拷贝，即只克隆复制节点本身，不克隆里面的子节点。
2. 如果括号参数为 true，则是深度拷贝，会复制节点本身以及里面所有的子节点。

### 复制节点演示

```
<!DOCTYPE html>
<html lang="en">

<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <meta http-equiv="X-UA-Compatible" content="ie=edge">
 <title>Document</title>
</head>

<body>

 1111
 2
 3

 <script>
 var ul = document.querySelector('ul');
 // 1. node.cloneNode(); 括号为空或者里面是false 浅拷贝 只复制标签不复制里面的内容
 // 2. node.cloneNode(true); 括号为true 深拷贝 复制标签复制里面的内容
 var lili = ul.children[0].cloneNode(true);
 ul.appendChild(lili);
 </script>
</body>

</html>
```

## 7-1 案例：动态生成表格

姓名	科目	成绩	操作
魏璿珞	JavaScript	100	<a href="#">删除</a>
弘历	JavaScript	90	<a href="#">删除</a>
傅恒	JavaScript	99	<a href="#">删除</a>
明玉	JavaScript	89	<a href="#">删除</a>

- ① 因为里面的学生数据都是动态的，我们需要js 动态生成。这里我们模拟数据，自己定义好数据。数据我们采取对象形式存储。
- ② 所有的数据都是放到tbody里面的行里面。
- ③ 因为行很多，我们需要循环创建多个行（对应多少人）
- ④ 每个行里面又有很多单元格（对应里面的数据），我们还继续使用循环创建多个单元格，并且把数据存入里面（双重for循环）
- ⑤ 最后一列单元格是删除，需要单独创建单元格。
- ⑥ 最后添加删除操作，单击删除，可以删除当前行。

```

<!DOCTYPE html>
<html lang="en">

<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <meta http-equiv="X-UA-Compatible" content="ie=edge">
 <title>Document</title>
 <style>
 table {
 width: 500px;
 margin: 100px auto;
 border-collapse: collapse;
 text-align: center;
 }

 td,
 th {
 border: 1px solid #333;
 }

 thead tr {
 height: 40px;
 background-color: #ccc;
 }
 </style>
</head>

<body>
 <table cellspacing="0">
 <thead>
 <tr>
 <th>姓名</th>
 <th>科目</th>

```

```

 <th>成绩</th>
 <th>操作</th>
 </tr>
</thead>
<tbody>

</tbody>
</table>
<script>
 // 1.先去准备好学生的数据
 var datas = [{
 name: '魏璿珞',
 subject: 'JavaScript',
 score: 100
 }, {
 name: '弘历',
 subject: 'JavaScript',
 score: 98
 }, {
 name: '傅恒',
 subject: 'JavaScript',
 score: 99
 }, {
 name: '明玉',
 subject: 'JavaScript',
 score: 88
 }, {
 name: '大猪蹄子',
 subject: 'JavaScript',
 score: 0
 }
];
 // 2. 往tbody 里面创建行: 有几个人(通过数组的长度)我们就创建几行
 var tbody = document.querySelector('tbody');
 for (var i = 0; i < datas.length; i++) { // 外面的for循环管行 tr
 // 1. 创建 tr行
 var tr = document.createElement('tr');
 tbody.appendChild(tr);
 // 2. 行里面创建单元格(跟数据有关系的3个单元格) td 单元格的数量取决于每个对象里
 面的属性个数 for循环遍历对象 datas[i]
 for (var k in datas[i]) { // 里面的for循环管列 td
 // 创建单元格
 var td = document.createElement('td');
 // 把对象里面的属性值 datas[i][k] 给 td
 // console.log(datas[i][k]);
 td.innerHTML = datas[i][k];
 tr.appendChild(td);
 }
 // 3. 创建有删除2个字的单元格
 var td = document.createElement('td');
 td.innerHTML = '删除 ';
 tr.appendChild(td);
 }
 // 4. 删除操作 开始
 var as = document.querySelectorAll('a');
 for (var i = 0; i < as.length; i++) {
 as[i].onclick = function() {

```



```

 // 点击a 删除 当前a 所在的行(链接的爸爸的爸爸) node.removeChild(child)

tbody.removeChild(this.parentNode.parentNode)
 }
}
// for(var k in obj) {
// k 得到的是属性名
// obj[k] 得到是属性值
// }
</script>
</body>

</html>

```

## 26-4-8 三种动态创建元素区别

- document.write()
- element.innerHTML
- document.createElement()

1. **document.write** // 是直接将内容写入页面的内容流，但是文档流执行完毕，则它会导致页面全部重绘
2. **innerHTML** // 是将内容写入某个 DOM 节点，不会导致页面全部重绘
3. **innerHTML** // 创建多个元素效率更高（不要拼接字符串，采取数组形式拼接），结构稍微复杂
4. **createElement()** // 创建多个元素效率稍低一点点，但是结构更清晰

// 总结：不同浏览器下，**innerHTML** 效率要比 **createElement** 高

## 26-5 事件高级

### 26-5-1 注册事件

#### 1-1 注册事件概述

给元素添加事件，称为**注册事件**或者**绑定事件**。

注册事件有两种方式：**传统方式**和**方法监听注册方式**



#### 传统注册方式

- 利用 on 开头的事件 onclick
- <button onclick="alert('hi~')"></button>
- btn.onclick = function() {}
- 特点：注册事件的**唯一性**
- 同一个元素同一个事件只能设置一个处理函数，最后注册的处理函数将会覆盖前面注册的处理函数

#### 方法监听注册方式

- w3c 标准 推荐方式
- addEventListener() 它是一个方法
- IE9 之前的 IE 不支持此方法，可使用 attachEvent() 代替
- 特点：同一个元素同一个事件可以注册多个监听器
- 按注册顺序依次执行

#### 1-2 addEventListener 事件监听方式

```
eventTarget.addEventListener(type, listener[, useCapture])
```

eventTarget.addEventListener()方法将指定的监听器注册到 eventTarget（目标对象）上，当该对象触发指定的事件时，就会执行事件处理函数。该方法接收三个参数：

- type：事件类型字符串，比如 click、mouseover，注意这里不要带 on

- listener: 事件处理函数, 事件发生时, 会调用该监听函数
- useCapture: 可选参数, 是一个布尔值, 默认是 false。学完 DOM 事件流后, 我们再进一步学习

### 1-3 attachEvent 事件监听方式

```
eventTarget.attachEvent(eventNameWithOn, callback)
```

`eventTarget.attachEvent()` 方法将指定的监听器注册到 `eventTarget` (目标对象) 上, 当该对象触发指定的事件时, 指定的回调函数就会被执行。

该方法接收两个参数:

- **eventNameWithOn**: 事件类型字符串, 比如 `onclick`、`onmouseover`, 这里要带 `on`
- **callback**: 事件处理函数, 当目标触发事件时回调函数被调用

**注意:** IE8 及早期版本支持

### 两种注册事件的演示

```
<!DOCTYPE html>
<html lang="en">

<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <meta http-equiv="X-UA-Compatible" content="ie=edge">
 <title>Document</title>
</head>

<body>
 <button>传统注册事件</button>
 <button>方法监听注册事件</button>
 <button>ie9 attachEvent</button>
 <script>
 var btns = document.querySelectorAll('button');
 // 1. 传统方式注册事件
 btns[0].onclick = function() {
 alert('hi');
 }
 btns[0].onclick = function() {
 alert('hao a u');
 }
 // 2. 事件侦听注册事件 addEventListener
 // (1) 里面的事件类型是字符串 必定加引号 而且不带on
 // (2) 同一个元素 同一个事件可以添加多个侦听器 (事件处理程序)
 btns[1].addEventListener('click', function() {
 alert(22);
 })
 btns[1].addEventListener('click', function() {
 alert(33);
 })
 // 3. attachEvent ie9以前的版本支持
 btns[2].attachEvent('onclick', function() {
 alert(11);
 })
 </script>
</body>
```

```
</html>
```

## 1-4 注册事件兼容性解决问题

```
function addEventListener(element, eventName, fn) {
 // 判断当前浏览器是否支持 addEventListener 方法
 if (element.addEventListener) {
 element.addEventListener(eventName, fn); // 第三个参数 默认是false
 } else if (element.attachEvent) {
 element.attachEvent('on' + eventName, fn);
 } else {
 // 相当于 element.onclick = fn;
 element['on' + eventName] = fn;
 }
}
```

兼容性处理的原则：首先照顾大多数浏览器，再处理特殊浏览器

## 26-1-2 删除事件

### 2-1 删除事件的方式

#### 1. 传统注册方式

```
eventTarget.onclick = null;
```

#### 2. 方法监听注册方式

- ① `eventTarget.removeEventListener(type, listener[, useCapture]);`
- ② `eventTarget.detachEvent(eventNameWithOn, callback);`

### 2-2 删除事件兼容性解决方案

```
function removeEventListener(element, eventName, fn) {
 // 判断当前浏览器是否支持 removeEventListener 方法
 if (element.removeEventListener) {
 element.removeEventListener(eventName, fn); // 第三个参数 默认是false
 } else if (element.detachEvent) {
 element.detachEvent('on' + eventName, fn);
 } else {
 element['on' + eventName] = null;
 }
}
```

## 2-3 删除事件代码演示

```
<!DOCTYPE html>
<html lang="en">

<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <meta http-equiv="X-UA-Compatible" content="ie=edge">
 <title>Document</title>
 <style>
 div {
 width: 100px;
 height: 100px;
 background-color: pink;
 }
 </style>
</head>

<body>
 <div>1</div>
 <div>2</div>
 <div>3</div>
 <script>
 var divs = document.querySelectorAll('div');
 divs[0].onclick = function() {
 alert(11);
 // 1. 传统方式删除事件
 divs[0].onclick = null;
 }
 // 2. removeEventListener 删除事件
 divs[1].addEventListener('click', fn) // 里面的fn 不需要调用加小括号

 function fn() {
 alert(22);
 divs[1].removeEventListener('click', fn);
 }
 // 3. detachEvent
 divs[2].attachEvent('onclick', fn1);

 function fn1() {
 alert(33);
 divs[2].detachEvent('onclick', fn1);
 }
 </script>
</body>

</html>
```

## 26-5-3 DOM事件流

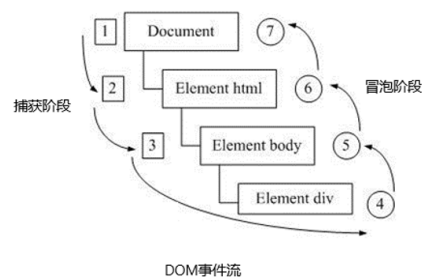
事件流描述的是从页面中接收事件的顺序。

事件发生时会在元素节点之间按照特定的顺序传播，这个传播过程即 DOM 事件流。

比如我们给一个div 注册了点击事件：

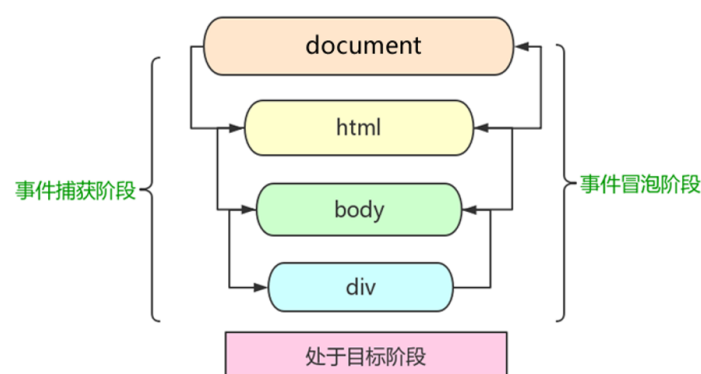
DOM 事件流分为3个阶段：

1. 捕获阶段
2. 当前目标阶段
3. 冒泡阶段



- 事件冒泡：IE 最早提出，事件开始时由最具体的元素接收，然后逐级向上传播到 DOM 最顶层节点的过程。
- 事件捕获：网景最早提出，由 DOM 最顶层节点开始，然后逐级向下传播到最具体的元素接收的过程。

我们向水里面扔一块石头，首先它会有一个下降的过程，这个过程就可以理解为从最顶层向事件发生的最具体元素（目标点）的捕获过程；之后会产生泡泡，会在最低点（最具体元素）之后漂浮到水面上，这个过程相当于事件冒泡。



事件发生时会在元素节点之间按照特定的顺序传播，这个传播过程即 DOM 事件流。

## 注意

1. JS 代码中只能执行捕获或者冒泡其中的一个阶段。
2. onclick 和 attachEvent 只能得到冒泡阶段。
3. addEventListener(type, listener[, useCapture]) 第三个参数如果是 true，表示在事件捕获阶段调用事件处理程序；如果是 false（不写默认就是false），表示在事件冒泡阶段调用事件处理程序。
4. 实际开发中我们很少使用事件捕获，我们更关注事件冒泡。
5. 有些事件是没有冒泡的，比如 onblur、onfocus、onmouseenter、onmouseleave
6. 事件冒泡有时候会带来麻烦，有时候又会帮助很巧妙的做某些事件，我们后面讲解。

```
<!DOCTYPE html>
<html lang="en">
```

```

<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <meta http-equiv="X-UA-Compatible" content="ie=edge">
 <title>Document</title>
 <style>
 .father {
 overflow: hidden;
 width: 300px;
 height: 300px;
 margin: 100px auto;
 background-color: pink;
 text-align: center;
 }

 .son {
 width: 200px;
 height: 200px;
 margin: 50px;
 background-color: purple;
 line-height: 200px;
 color: #fff;
 }
 </style>
</head>

<body>
 <div class="father">
 <div class="son">son盒子</div>
 </div>
 <script>
 // dom 事件流 三个阶段
 // 1. JS 代码中只能执行捕获或者冒泡其中的一个阶段。
 // 2. onclick 和 attachEvent (ie) 只能得到冒泡阶段。
 // 3. 捕获阶段 如果addEventListener 第三个参数是 true 那么则处于捕获阶段
 document -> html -> body -> father -> son
 // var son = document.querySelector('.son');
 // son.addEventListener('click', function() {
 // alert('son');
 // }, true);
 // var father = document.querySelector('.father');
 // father.addEventListener('click', function() {
 // alert('father');
 // }, true);
 // 4. 冒泡阶段 如果addEventListener 第三个参数是 false 或者 省略 那么则处于冒泡阶段
 段 son -> father -> body -> html -> document
 var son = document.querySelector('.son');
 son.addEventListener('click', function() {
 alert('son');
 }, false);
 var father = document.querySelector('.father');
 father.addEventListener('click', function() {
 alert('father');
 }, false);
 document.addEventListener('click', function() {
 alert('document');
 })
 </script>

```

```
</body>
```

```
</html>
```

## 26-5-4 事件对象

### 4-1 事件对象

```
eventTarget.onclick = function(event) {}
eventTarget.addEventListener('click', function(event) {})
// 这个 event 就是事件对象，我们还喜欢的写成 e 或者 evt
```

官方解释：event 对象代表事件的状态，比如键盘按键的状态、鼠标的位置、鼠标按钮的状态。

简单理解：事件发生后，跟事件相关的一系列信息数据的集合都放到这个对象里面，这个对象就是事件对象 event，它有很多属性和方法。

比如：

1. 谁绑定了这个事件。
2. 鼠标触发事件的话，会得到鼠标的相关信息，如鼠标位置。
3. 键盘触发事件的话，会得到键盘的相关信息，如按了哪个键。

### 4-2 事件对象的使用语法

```
eventTarget.onclick = function(event) {
 // 这个 event 就是事件对象，我们还喜欢的写成 e 或者 evt
}
eventTarget.addEventListener('click', function(event) {
 // 这个 event 就是事件对象，我们还喜欢的写成 e 或者 evt
})
```

- 这个 event 是个形参，系统帮我们设定为事件对象，不需要传递实参过去。
- 当我们注册事件时，event 对象就会被系统自动创建，并依次传递给事件监听器（事件处理函数）。

### 4-3 事件对象的兼容性方案

事件对象本身的获取存在兼容问题：

1. 标准浏览器中是浏览器给方法传递的参数，只需要定义形参 e 就可以获取到。
2. 在 IE6~8 中，浏览器不会给方法传递参数，如果需要的话，需要到 window.event 中获取查找。

**解决：**

```
e = e || window.event;
```

## 4-4 事件对象的常见属性和方法

```
// e.target 和 this 的区别:
// this 是事件绑定的元素, 这个函数的调用者 (绑定这个事件的元素)
// e.target 是事件触发的元素。
```

```
<!DOCTYPE html>
<html lang="en">

<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <meta http-equiv="X-UA-Compatible" content="ie=edge">
 <title>Document</title>
 <style>
 div {
 width: 100px;
 height: 100px;
 background-color: pink;
 }
 </style>
</head>

<body>
 <div>123</div>

 abc
 abc
 abc

 <script>
 // 常见事件对象的属性和方法
 // 1. e.target 返回的是触发事件的对象 (元素) this 返回的是绑定事件的对象 (元素)
 // 区别: e.target 点击了那个元素, 就返回那个元素 this 那个元素绑定了这个点击事件, 那么就返回谁
 var div = document.querySelector('div');
 div.addEventListener('click', function(e) {
 console.log(e.target);
 console.log(this);
 })
 var ul = document.querySelector('ul');
 ul.addEventListener('click', function(e) {
 // 我们给ul 绑定了事件 那么this 就指向ul
 console.log(this);
 console.log(e.currentTarget);

 // e.target 指向我们点击的那个对象 谁触发了这个事件 我们点击的是li
 e.target 指向的就是li
 console.log(e.target);
 })
 // 了解兼容性
 // div.onclick = function(e) {
 // e = e || window.event;
 // var target = e.target || e.srcElement;
```



```

 // console.log(target);

 // }
 // 2. 了解 跟 this 有个非常相似的属性 currentTarget ie678不认识
 </script>
</body>

</html>

```

#### 4-5 事件对象的常见属性和方法

*事件对象属性方法*	*说明*
e.target	返回触发事件的对象 标准
e.srcElement	返回触发事件的对象 非标准ie6-8使用
e.type	返回事件的类型比如click mouseover不带on
e.cancelBubble	该属性阻止冒泡非标准ie6-8使用
e.returnValue	该属性阻止默认事件(默认行为)非标准ie6-8使用比如不让链接跳转
e.preventDefault()	该方法阻止默认事件(默认行为)标准比如不让链接跳转
e.stopPropagation()	阻止冒泡标准

## 26-5-5 阻止事件冒泡

### 5-1 阻止事件冒泡的两种方式

- 事件冒泡：开始时由最具体的元素接收，然后逐级向上传播到 DOM 最顶层节点。
- 事件冒泡本身的特性，会带来的坏处，也会带来的好处，需要我们灵活掌握。

#### 阻止事件冒泡

```

// 标准写法：利用事件对象里面的 stopPropagation() 方法
e.stopPropagation()

// 非标准写法：IE 6-8 利用事件对象 cancelBubble 属性
e.cancelBubble = true;

```

### 5-2 阻止事件冒泡的兼容性解决方案

```

if(e && e.stopPropagation){
 e.stopPropagation();
}else{
 window.event.cancelBubble = true;
}

```

### 5-3 代码演示

```
<!DOCTYPE html>
<html lang="en">

<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <meta http-equiv="X-UA-Compatible" content="ie=edge">
 <title>Document</title>
 <style>
 .father {
 overflow: hidden;
 width: 300px;
 height: 300px;
 margin: 100px auto;
 background-color: pink;
 text-align: center;
 }

 .son {
 width: 200px;
 height: 200px;
 margin: 50px;
 background-color: purple;
 line-height: 200px;
 color: #fff;
 }
 </style>
</head>

<body>
 <div class="father">
 <div class="son">son儿子</div>
 </div>
 <script>
 // 常见事件对象的属性和方法
 // 阻止冒泡 dom 推荐的标准 stopPropagation()
 var son = document.querySelector('.son');
 son.addEventListener('click', function(e) {
 alert('son');
 e.stopPropagation(); // stop 停止 Propagation 传播
 e.cancelBubble = true; // 非标准 cancel 取消 bubble 泡泡
 }, false);

 var father = document.querySelector('.father');
 father.addEventListener('click', function() {
 alert('father');
 }, false);
 document.addEventListener('click', function() {
 alert('document');
 })
 </script>
</body>

</html>
```

## 26-5-6 事件委托

事件冒泡本身的特性，会带来的坏处，也会**带来的好处**，需要我们灵活掌握。生活中有如下场景：

咱们班有100个学生，快递员有100个快递，如果一个个的送花费时间较长。同时每个学生领取的时候，也需要排队领取，也花费时间较长，何如？

**解决方案：** 快递员把100个快递，**委托**给班主任，班主任把这些快递放到办公室，同学们下课自行领取即可。

**优势：** 快递员省事，委托给班主任就可以走了。同学们领取也方便，因为相信班主任。

事件冒泡本身的特性，会带来的坏处，也会带来的好处，需要我们灵活掌握。程序中也有如此场景：

```

 知否知否，应该有弹框在手
 知否知否，应该有弹框在手
 知否知否，应该有弹框在手
 知否知否，应该有弹框在手
 知否知否，应该有弹框在手

```

点击每个 li 都会弹出对话框，以前需要给每个 li 注册事件，是非常辛苦的，而且访问 DOM 的次数越多，这就会延长整个页面的交互就绪时间。

```
<!DOCTYPE html>
<html lang="en">

<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <meta http-equiv="X-UA-Compatible" content="ie=edge">
 <title>Document</title>
</head>

<body>

 知否知否，点我应有弹框在手！
 知否知否，点我应有弹框在手！
 知否知否，点我应有弹框在手！
 知否知否，点我应有弹框在手！
 知否知否，点我应有弹框在手！

 <script>
 // 事件委托的核心原理：给父节点添加侦听器， 利用事件冒泡影响每一个子节点
 var ul = document.querySelector('ul');
 ul.addEventListener('click', function(e) {
 // alert('知否知否，点我应有弹框在手！');
 // e.target 这个可以得到我们点击的对象
 e.target.style.backgroundColor = 'pink';
 });
 </script>
</body>
</html>
```

```
 })
 </script>
</body>

</html>
```

## 事件委托

事件委托也称为事件代理，在 jQuery 里面称为事件委派。

### 事件委托的原理

不是每个子节点单独设置事件监听器，而是事件监听器设置在其父节点上，然后利用冒泡原理影响设置每个子节点。

以上案例：给 ul 注册点击事件，然后利用事件对象的 target 来找到当前点击的 li，因为点击 li，事件会冒泡到 ul 上，ul 有注册事件，就会触发事件监听器。

### 事件委托的作用

我们只操作了一次 DOM，提高了程序的性能。

## 26-5-7 常用的鼠标事件

鼠标事件	触发条件
onclick	鼠标点击左键触发
onmouseover	鼠标经过触发
onmouseout	鼠标离开触发
onfocus	获得鼠标焦点触发
onblur	失去鼠标焦点触发
onmousemove	鼠标移动触发
onmouseup	鼠标弹起触发
onmousedown	鼠标按下触发

### 7-1 常用的鼠标事件

#### 1-1 禁止鼠标右键菜单

contextmenu 主要控制应该何时显示上下文菜单，主要用于程序员取消默认的上下文菜单

```
document.addEventListener('contextmenu', function(e) {
 e.preventDefault();
})
```

#### 1-2 禁止鼠标选中 (selectstart 开始选中)

```
document.addEventListener('selectstart', function(e) {
 e.preventDefault();
})
```

```

<!DOCTYPE html>
<html lang="en">

<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <meta http-equiv="X-UA-Compatible" content="ie=edge">
 <title>Document</title>
</head>

<body>
 我是一段不愿意分享的文字
 <script>
 // 1. contextmenu 我们可以禁用右键菜单
 document.addEventListener('contextmenu', function(e) {
 e.preventDefault();
 })
 // 2. 禁止选中文字 selectstart
 document.addEventListener('selectstart', function(e) {
 e.preventDefault();
 })
 </script>
</body>

</html>

```

## 7-2 鼠标事件对象

**event**对象代表事件的状态，跟事件相关的一系列信息的集合。现阶段我们主要是用鼠标事件对象 **MouseEvent** 和键盘事件对象 **KeyboardEvent**。

鼠标事件对象	说明
e.clientX	返回鼠标相对于浏览器窗口可视区的 X 坐标
e.clientY	返回鼠标相对于浏览器窗口可视区的 Y 坐标
e.pageX	返回鼠标相对于文档页面的 X 坐标 IE9+ 支持
e.pageY	返回鼠标相对于文档页面的 Y 坐标 IE9+ 支持
e.screenX	返回鼠标相对于电脑屏幕的 X 坐标
e.screenY	返回鼠标相对于电脑屏幕的 Y 坐标

```

<!DOCTYPE html>
<html lang="en">

<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <meta http-equiv="X-UA-Compatible" content="ie=edge">
 <title>Document</title>
 <style>
 body {
 height: 3000px;
 }
 </style>

```

```

</style>
</head>

<body>
 <script>
 // 鼠标事件对象 MouseEvent
 document.addEventListener('click', function(e) {
 // 1. client 鼠标在可视区的x和y坐标
 console.log(e.clientX);
 console.log(e.clientY);
 console.log('-----');

 // 2. page 鼠标在页面文档的x和y坐标
 console.log(e.pageX);
 console.log(e.pageY);
 console.log('-----');

 // 3. screen 鼠标在电脑屏幕的x和y坐标
 console.log(e.screenX);
 console.log(e.screenY);

 })
 </script>
</body>

</html>

```

### 7-3 跟随鼠标的天使

- ① 鼠标不断的移动，使用鼠标移动事件：mousemove
- ② 在页面中移动，给document注册事件
- ③ 图片要移动距离，而且不占位置，我们使用绝对定位即可
- ④ 核心原理：每次鼠标移动，我们都会获得最新的鼠标坐标，把这个x和y坐标做为图片的top和left 值就可以移动图片

```

var pic = document.querySelector('img');
document.addEventListener('mousemove', function(e) {
 var x = e.pageX;
 var y = e.pageY;
 pic.style.top = y - 40 + 'px';
 pic.style.left = x - 50 + 'px';
})

```

```

<!DOCTYPE html>
<html lang="en">

<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">

```

```

<meta http-equiv="X-UA-Compatible" content="ie=edge">
<title>Document</title>
<style>
 img {
 position: absolute;
 top: 2px;
 }
</style>
</head>

<body>

 <script>
 var pic = document.querySelector('img');
 document.addEventListener('mousemove', function(e) {
 // 1. mousemove 只要我们鼠标移动1px 就会触发这个事件
 // console.log(1);
 // 2. 核心原理: 每次鼠标移动, 我们都会获得最新的鼠标坐标, 把这个x和y坐标做为图片
 // 的top和left 值就可以移动图片
 var x = e.pageX;
 var y = e.pageY;
 console.log('x坐标是' + x, 'y坐标是' + y);
 // 3 . 千万不要忘记给 left 和 top 添加px 单位
 pic.style.left = x - 50 + 'px';
 pic.style.top = y - 40 + 'px';

 });
 </script>
</body>

</html>

```

## 26-5-8 常用的键盘事件

### 8-1 常见的键盘事件

键盘事件	触发条件
onkeyup	某个键盘按键被松开时触发
onkeydown	某个键盘按键被按下时触发
onkeypress	某个键盘按键被按下时 触发 但是它不识别功能键 比如 ctrl shift 箭头等

**注意:**

1. 如果使用addEventListener 不需要加 on
2. onkeypress 和前面2个的区别是, 它不识别功能键, 比如左右箭头, shift 等。
3. 三个事件的执行顺序是: keydown -- keypress --- keyup

```

<!DOCTYPE html>
<html lang="en">

<head>

```

```

<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<meta http-equiv="X-UA-Compatible" content="ie=edge">
<title>Document</title>
</head>

<body>
 <script>
 // 常用的键盘事件
 //1. keyup 按键弹起的时候触发
 // document.onkeyup = function() {
 // console.log('我弹起了');
 // }
 document.addEventListener('keyup', function() {
 console.log('我弹起了');
 })

 //3. keypress 按键按下的时候触发 不能识别功能键 比如 ctrl shift 左右箭头啊
 document.addEventListener('keypress', function() {
 console.log('我按下了press');
 })
 //2. keydown 按键按下的时候触发 能识别功能键 比如 ctrl shift 左右箭头啊
 document.addEventListener('keydown', function() {
 console.log('我按下了down');
 })
 // 4. 三个事件的执行顺序 keydown -- keypress -- keyup
 </script>
</body>
</html>

```

## 8-2 键盘事件对象

键盘事件对象 属性	说明
keyCode	返回该键的ASCII 值

**注意：** onkeydown 和 onkeyup 不区分字母大小写，onkeypress 区分字母大小写。

在实际开发中，我们更多的使用keydown和keyup，它能识别所有的键（包括功能键）

Keypress 不识别功能键，但是keyCode属性能区分大小写，返回不同的ASCII值

```

<!DOCTYPE html>
<html lang="en">

<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <meta http-equiv="X-UA-Compatible" content="ie=edge">
 <title>Document</title>
</head>

```



```

<body>
 <script>
 // 键盘事件对象中的keyCode属性可以得到相应键的ASCII码值
 // 1. 我们的keyup 和keydown事件不区分字母大小写 a 和 A 得到的都是65
 // 2. 我们的keypress 事件 区分字母大小写 a 97 和 A 得到的是65
 document.addEventListener('keyup', function(e) {
 // console.log(e);
 console.log('up:' + e.keyCode);
 // 我们可以利用keyCode返回的ASCII码值来判断用户按下了那个键
 if (e.keyCode === 65) {
 alert('您按下的a键');
 } else {
 alert('您没有按下a键')
 }
 })

 document.addEventListener('keypress', function(e) {
 // console.log(e);
 console.log('press:' + e.keyCode);
 })
 </script>
</body>

</html>

```

### 8-3 案例：模拟京东按键输入内容

- ① 核心思路：检测用户是否按下了s 键，如果按下s 键，就把光标定位到搜索框里面
- ② 使用键盘事件对象里面的keyCode 判断用户按下的是否是s键
- ③ 搜索框获得焦点：使用 js 里面的 focus() 方法

```

var search = document.querySelector('input');
document.addEventListener('keyup', function(e) {
 // console.log(e.keyCode);
 if (e.keyCode === 83) {
 search.focus();
 }
})

```

#### 完整代码

```

<!DOCTYPE html>
<html lang="en">

<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <meta http-equiv="X-UA-Compatible" content="ie=edge">
 <title>Document</title>
</head>

<body>

```

```

<input type="text">
<script>
 // 核心思路： 检测用户是否按下了s 键，如果按下s 键，就把光标定位到搜索框里面
 // 使用键盘事件对象里面的keyCode 判断用户按下的是否是s键
 // 搜索框获得焦点： 使用 js 里面的 focus() 方法
 var search = document.querySelector('input');
 document.addEventListener('keyup', function(e) {
 // console.log(e.keyCode);
 if (e.keyCode === 83) {
 search.focus();
 }
 })
</script>
</body>

</html>

```

#### 8-4 案例：模拟京东快递单号查询

要求：当我们在文本框中输入内容时，文本框上面自动显示大号号的内容。



- ① 快递单号输入内容时，上面的大号字体盒子（con）显示(这里面的文字)
- ② 同时把快递单号里面的值（value）获取过来赋值给 con盒子（innerText）做为内容
- ③ 如果快递单号里面内容为空，则隐藏大号字体盒子(con)盒子

```

<!DOCTYPE html>
<html lang="en">

<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <meta http-equiv="X-UA-Compatible" content="ie=edge">
 <title>Document</title>
 <style>
 * {
 margin: 0;
 padding: 0;
 }

 .search {

```



```
jd_input.addEventListener('focus', function() {
 if (this.value !== '') {
 con.style.display = 'block';
 }
})
</script>
</body>
```

## 27, JavaScript 操作BOM

### 27-1 BOM概述

#### 27-1-1 什么是 BOM

BOM (Browser Object Model) 即**浏览器对象模型**，它提供了独立于内容而与**浏览器窗口进行交互的对象**，其核心对象是 window。

BOM 由一系列相关的对象构成，并且每个对象都提供了很多方法与属性。

BOM 缺乏标准，JavaScript 语法的标准化组织是 ECMA，DOM 的标准化组织是 W3C，BOM 最初是 Netscape 浏览器标准的一部分。



#### DOM

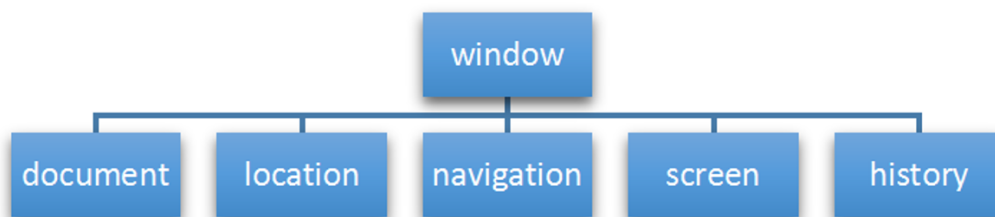
- 文档对象模型
- DOM 就是把「文档」当做一个「对象」来看待
- DOM 的顶级对象是 **document**
- DOM 主要学习的是操作页面元素
- DOM 是 W3C 标准规范

#### BOM

- 浏览器对象模型
- 把「浏览器」当做一个「对象」来看待
- BOM 的顶级对象是 **window**
- BOM 学习的是浏览器窗口交互的一些对象
- BOM 是浏览器厂商在各自浏览器上定义的，兼容性较差

#### 27-1-2 BOM 的构成

BOM 比 DOM 更大，它包含 DOM。



**window 对象是浏览器的顶级对象**，它具有双重角色。

1. 它是 JS 访问浏览器窗口的一个接口。
2. 它是一个全局对象。定义在全局作用域中的变量、函数都会变成 window 对象的属性和方法。

在调用的时候可以省略 window，前面学习的对话框都属于 window 对象方法，如 alert()、prompt() 等。

**注意：** window 下的一个特殊属性 **window.name**

```
<!DOCTYPE html>
<html lang="en">

<head>
 <meta charset="UTF-8">
```

```

<meta name="viewport" content="width=device-width, initial-scale=1.0">
<meta http-equiv="X-UA-Compatible" content="ie=edge">
<title>Document</title>
</head>

<body>
 <script>
 // window.document.querySelector()
 var num = 10;
 console.log(num);
 console.log(window.num);

 function fn() {
 console.log(11);
 }
 fn();
 window.fn();
 // alert(11);
 // window.alert(11)
 console.dir(window);
 // var name = 10;
 console.log(window.name);
 </script>
</body>

</html>

```

## 27-2 window对象的常见事件

```

<!DOCTYPE html>
<html lang="en">

<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <meta http-equiv="X-UA-Compatible" content="ie=edge">
 <title>Document</title>
 <script>
 // window.onload = function() {
 // var btn = document.querySelector('button');
 // btn.addEventListener('click', function() {
 // alert('点击我');
 // })
 // }
 // window.onload = function() {
 // alert(22);
 // }
 window.addEventListener('load', function() {
 var btn = document.querySelector('button');
 btn.addEventListener('click', function() {
 alert('点击我');
 })
 })
 window.addEventListener('load', function() {

 alert(22);
 })
 </script>

```

```

 })
 document.addEventListener('DOMContentLoaded', function() {
 alert(33);
 })
 // load 等页面内容全部加载完毕，包含页面dom元素 图片 flash css 等等
 // DOMContentLoaded 是DOM 加载完毕，不包含图片 flash css 等就可以执行 加载
 速度比 load更快一些
</script>
</head>

<body>

 <button>点击</button>

</body>

</html>

```

## 27-2-1 窗口加载事件

```

window.onload = function(){}
或者
window.addEventListener("load",function(){});

```

window.onload 是窗口 (页面) 加载事件,当文档内容完全加载完成会触发该事件(包括图像、脚本文件、CSS 文件等), 就调用的处理函数。

### 注意:

1. 有了 window.onload 就可以把 JS 代码写到页面元素的上方，因为 onload 是等页面内容全部加载完毕，再去执行处理函数。
2. window.onload 传统注册事件方式 只能写一次，如果有多个，会以最后一个 window.onload 为准。
3. 如果使用 addEventListener 则没有限制

```
document.addEventListener('DOMContentLoaded',function(){})
```

DOMContentLoaded 事件触发时，仅当DOM加载完成，不包括样式表，图片，flash等等。

ie9以上才支持

如果页面的图片很多的话，从用户访问到onload触发可能需要较长的时间，交互效果就不能实现，必然影响用户的体验，此时用 DOMContentLoaded 事件比较合适。



## 27-2-2 调整窗口大小事件

```

window.onresize = function(){}
window.addEventListener("resize",function(){});

```

// window.onresize 是调整窗口大小加载事件， 当触发时就调用的处理函数。

注意：

1. 只要窗口大小发生像素变化，就会触发这个事件。
2. 我们经常利用这个事件完成响应式布局。 window.innerWidth 当前屏幕的宽度

```
<!DOCTYPE html>
<html lang="en">

<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <meta http-equiv="X-UA-Compatible" content="ie=edge">
 <title>Document</title>
 <style>
 div {
 width: 200px;
 height: 200px;
 background-color: pink;
 }
 </style>
</head>

<body>
 <script>
 window.addEventListener('load', function() {
 var div = document.querySelector('div');
 window.addEventListener('resize', function() {
 console.log(window.innerWidth);

 console.log('变化了');
 if (window.innerWidth <= 800) {
 div.style.display = 'none';
 } else {
 div.style.display = 'block';
 }
 })
 })
 </script>
 <div></div>
</body>

</html>
```

## 27-3 定时器

### 27-3-1 两种定时器

window 对象给我们提供了 2 个非常好用的方法-定时器。

- setTimeout()
- setInterval()

## 27-3-2 setTimeout() 定时器

```
window.setTimeout(调用函数, [延迟的毫秒数]);
```

```
// setTimeout() 方法用于设置一个定时器，该定时器在定时器到期后执行调用函数
```

注意：

1. window 可以省略。
2. 这个调用函数可以直接写函数，或者写函数名或者采取字符串'函数名()'三种形式。第三种不推荐
3. 延迟的毫秒数省略默认是 0，如果写，必须是毫秒。
4. 因为定时器可能有很多，所以我们经常给定时器赋值一个标识符。

```
window.setTimeout(调用函数, [延迟的毫秒数]);
```

setTimeout() 这个调用函数我们也称为回调函数 callback

普通函数是按照代码顺序直接调用。

而这个函数，需要等待时间，时间到了才去调用这个函数，因此称为回调函数。

简单理解：回调，就是回头调用的意思。上一件事干完，再回头再调用这个函数。

以前我们讲的 `element.onclick = function(){} 或者 element.addEventListener("click", fn);` 里面的函数也是回调函数。

```
<!DOCTYPE html>
<html lang="en">

<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <meta http-equiv="X-UA-Compatible" content="ie=edge">
 <title>Document</title>
</head>

<body>
 <script>
 // 1. setTimeout
 // 语法规则： window.setTimeout(调用函数, 延时时间);
 // 1. 这个window在调用的时候可以省略
 // 2. 这个延时时间单位是毫秒 但是可以省略，如果省略默认的是0
 // 3. 这个调用函数可以直接写函数 还可以写 函数名 还有一个写法 '函数名()'
 // 4. 页面中可能有很多的定时器，我们经常给定时器加标识符 （名字）
 // setTimeout(function() {
 // console.log('时间到了');

 // }, 2000);
 function callback() {
 console.log('爆炸了');
 }
 var timer1 = setTimeout(callback, 3000);
 var timer2 = setTimeout(callback, 5000);
 // setTimeout('callback()', 3000); // 我们不提倡这个写法
 </script>
```



```
</body>
```

```
</html>
```

### 27-3-3 案例：5秒后自动关闭的广告

- ① 核心思路：5秒之后，就把这个广告隐藏起来
- ② 用定时器setTimeout

```
<!DOCTYPE html>
<html lang="en">

<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <meta http-equiv="X-UA-Compatible" content="ie=edge">
 <title>Document</title>
</head>

<body>

 <script>
 var ad = document.querySelector('.ad');
 setTimeout(function() {
 ad.style.display = 'none';
 }, 5000);
 </script>
</body>

</html>
```

### 27-3-4 停止 setTimeout() 定时器

```
window.clearTimeout(timeoutID)
```

// clearTimeout() 方法取消了先前通过调用 setTimeout() 建立的定时器。

注意：

- window 可以省略。
- 里面的参数就是定时器的标识符。

```
window.setInterval(回调函数, [间隔的毫秒数]);
```

setInterval() 方法重复调用一个函数，每隔这个时间，就去调用一次回调函数。

**注意：**

1. window 可以省略。
2. 这个调用函数可以**直接写函数**，或者**写函数名**或者采取字符串 '**函数名()**' 三种形式。
3. 间隔的毫秒数省略默认是 0，如果写，必须是毫秒，表示每隔多少毫秒就自动调用这个函数。
4. 因为定时器可能有很多，所以我们经常给定时器赋值一个标识符。
5. 第一次执行也是间隔毫秒数之后执行，之后每隔毫秒数就执行一次。

### 27-3-5 案例：倒计时



- ① 这个倒计时是不断变化的，因此需要定时器来自动变化 (setInterval)
- ② 三个黑色盒子里面分别存放时分秒
- ③ 三个黑色盒子利用innerHTML 放入计算的小时分钟秒数
- ④ 第一次执行也是间隔毫秒数，因此刚刷新页面会有空白
- ⑤ 最好采取封装函数的方式，这样可以先调用一次这个函数，防止刚开始刷新页面有空白问题

```
<!DOCTYPE html>
<html lang="en">

<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <meta http-equiv="X-UA-Compatible" content="ie=edge">
 <title>Document</title>
 <style>
 div {
 margin: 200px;
 }

 span {
 display: inline-block;
 width: 40px;
 height: 40px;
 background-color: #333;
 font-size: 20px;
 color: #fff;
 text-align: center;
 line-height: 40px;
 }
 </style>
</head>

<body>
 <div>
 01 20 39
 </div>
</body>
</html>
```

```

 }
 </style>
</head>

<body>
 <div>
 1
 2
 3
 </div>
 <script>
 // 1. 获取元素
 var hour = document.querySelector('.hour'); // 小时的黑色盒子
 var minute = document.querySelector('.minute'); // 分钟的黑色盒子
 var second = document.querySelector('.second'); // 秒数的黑色盒子
 var inputTime = +new Date('2019-5-1 18:00:00'); // 返回的是用户输入时间总的毫
秒数

 countDown(); // 我们先调用一次这个函数，防止第一次刷新页面有空白
 // 2. 开启定时器
 setInterval(countDown, 1000);

 function countDown() {
 var nowTime = +new Date(); // 返回的是当前时间总的毫秒数
 var times = (inputTime - nowTime) / 1000; // times是剩余时间总的秒数
 var h = parseInt(times / 60 / 60 % 24); //时
 h = h < 10 ? '0' + h : h;
 hour.innerHTML = h; // 把剩余的小时给 小时黑色盒子
 var m = parseInt(times / 60 % 60); // 分
 m = m < 10 ? '0' + m : m;
 minute.innerHTML = m;
 var s = parseInt(times % 60); // 当前的秒
 s = s < 10 ? '0' + s : s;
 second.innerHTML = s;
 }
 </script>
</body>

</html>

```

## 27-3-6 停止 setInterval() 定时器

```

window.clearInterval(intervalID);
// clearInterval()方法取消了先前通过调用 setInterval()建立的定时器。

```

注意：

- window 可以省略。
- 里面的参数就是定时器的标识符。

```

<!DOCTYPE html>
<html lang="en">

<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <meta http-equiv="X-UA-Compatible" content="ie=edge">

```

```

<title>Document</title>
</head>

<body>
 <button class="begin">开启定时器</button>
 <button class="stop">停止定时器</button>
 <script>
 var begin = document.querySelector('.begin');
 var stop = document.querySelector('.stop');
 var timer = null; // 全局变量 null是一个空对象
 begin.addEventListener('click', function() {
 timer = setInterval(function() {
 console.log('ni hao ma');

 }, 1000);
 })
 stop.addEventListener('click', function() {
 clearInterval(timer);
 })
 </script>
</body>

</html>

```

## 27-3-7 案例：发送短信

点击按钮后，该按钮60秒之内不能再次点击，防止重复发送短信

- ① 按钮点击之后，会禁用 disabled 为true
- ② 同时按钮里面的内容会变化，注意 button 里面的内容通过 innerHTML修改
- ③ 里面秒数是有变化的，因此需要用到定时器
- ④ 定义一个变量，在定时器里面，不断递减
- ⑤ 如果变量为0 说明到了时间，我们需要停止定时器，并且复原按钮初始状态。

```

<!DOCTYPE html>
<html lang="en">

<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <meta http-equiv="X-UA-Compatible" content="ie=edge">
 <title>Document</title>
</head>

<body>

```

```

手机号码: <input type="number"> <button>发送</button>
<script>
 // 按钮点击之后, 会禁用 disabled 为true
 // 同时按钮里面的内容会变化, 注意 button 里面的内容通过 innerHTML修改
 // 里面秒数是有变化的, 因此需要用到定时器
 // 定义一个变量, 在定时器里面, 不断递减
 // 如果变量为0 说明到了时间, 我们需要停止定时器, 并且复原按钮初始状态
 var btn = document.querySelector('button');
 var time = 3; // 定义剩下的秒数
 btn.addEventListener('click', function() {
 btn.disabled = true;
 var timer = setInterval(function() {
 if (time == 0) {
 // 清除定时器和复原按钮
 clearInterval(timer);
 btn.disabled = false;
 btn.innerHTML = '发送';
 } else {
 btn.innerHTML = '还剩下' + time + '秒';
 time--;
 }
 }, 1000);
 })
</script>
</body>

</html>

```

## 27-3-8 this

this的指向在函数定义的时候是确定不了的, 只有函数执行的时候才能确定this到底指向谁, 一般情况下this的最终指向的是那个调用它的对象

现阶段, 我们先了解一下几个this指向

1. 全局作用域或者普通函数中this指向全局对象window (注意定时器里面的this指向window)
2. 方法调用中谁调用this指向谁
3. 构造函数中this指向构造函数的实例

this指向问题案例

```

<!DOCTYPE html>
<html lang="en">

<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <meta http-equiv="X-UA-Compatible" content="ie=edge">
 <title>Document</title>
</head>

<body>
 <button>点击</button>
 <script>

```

```

// this 指向问题 一般情况下this的最终指向的是那个调用它的对象

// 1. 全局作用域或者普通函数中this指向全局对象window（ 注意定时器里面的this指向window）
console.log(this);

function fn() {
 console.log(this);
}

window.fn();
window.setTimeout(function() {
 console.log(this);

}, 1000);
// 2. 方法调用中谁调用this指向谁
var o = {
 sayHi: function() {
 console.log(this); // this指向的是 o 这个对象

 }
}

o.sayHi();
var btn = document.querySelector('button');
// btn.onclick = function() {
// console.log(this); // this指向的是btn这个按钮对象

// }

btn.addEventListener('click', function() {
 console.log(this); // this指向的是btn这个按钮对象

})

// 3. 构造函数中this指向构造函数的实例
function Fun() {
 console.log(this); // this 指向的是fun 实例对象

}

var fun = new Fun();
</script>
</body>

</html>

```

## 27-4 JS执行机制

### 27-4-1 JS 是单线程

JavaScript 语言的一大特点就是**单线程**，也就是说，**同一个时间只能做一件事**。这是因为 Javascript 这门脚本语言诞生的使命所致——JavaScript 是为处理页面中用户的交互，以及操作 DOM 而诞生的。比如我们对某个 DOM 元素进行添加和删除操作，不能同时进行。应该先进行添加，之后再删除。

单线程就意味着，所有任务需要排队，前一个任务结束，才会执行后一个任务。这样所导致的问题是：如果 JS 执行的时间过长，这样就会造成页面的渲染不连贯，导致页面渲染加载阻塞的感觉。

## 27-4-2 同步和异步

为了解决这个问题，利用多核 CPU 的计算能力，HTML5 提出 Web Worker 标准，允许 JavaScript 脚本创建多个线程。于是，JS 中出现了**同步**和**异步**。

### 同步

前一个任务结束后再执行后一个任务，程序的执行顺序与任务的排列顺序是一致的、同步的。比如做饭的同步做法：我们要烧水煮饭，等水开了（10分钟之后），再去切菜，炒菜。

### 异步

你在做一件事情时，因为这件事情会花费很长时间，在做这件事的同时，你还可以去处理其他事情。比如做饭的异步做法，我们在烧水的同时，利用这10分钟，去切菜，炒菜。

**他们的本质区别：这条流水线上各个流程的执行顺序不同。**

## 27-4-3 同步和异步任务

### 4.3 同步和异步

#### 同步任务

同步任务都在主线程上执行，形成一个**执行栈**。

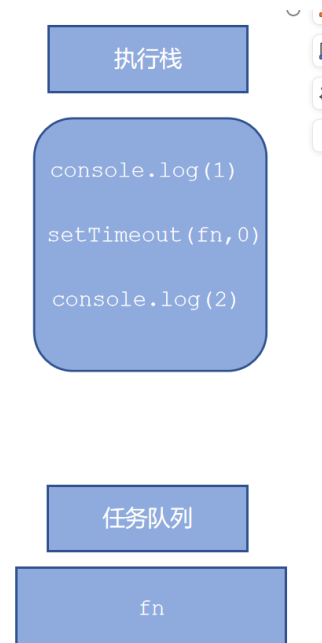
#### 异步任务

JS 的异步是通过回调函数实现的。

一般而言，异步任务有以下三种类型：

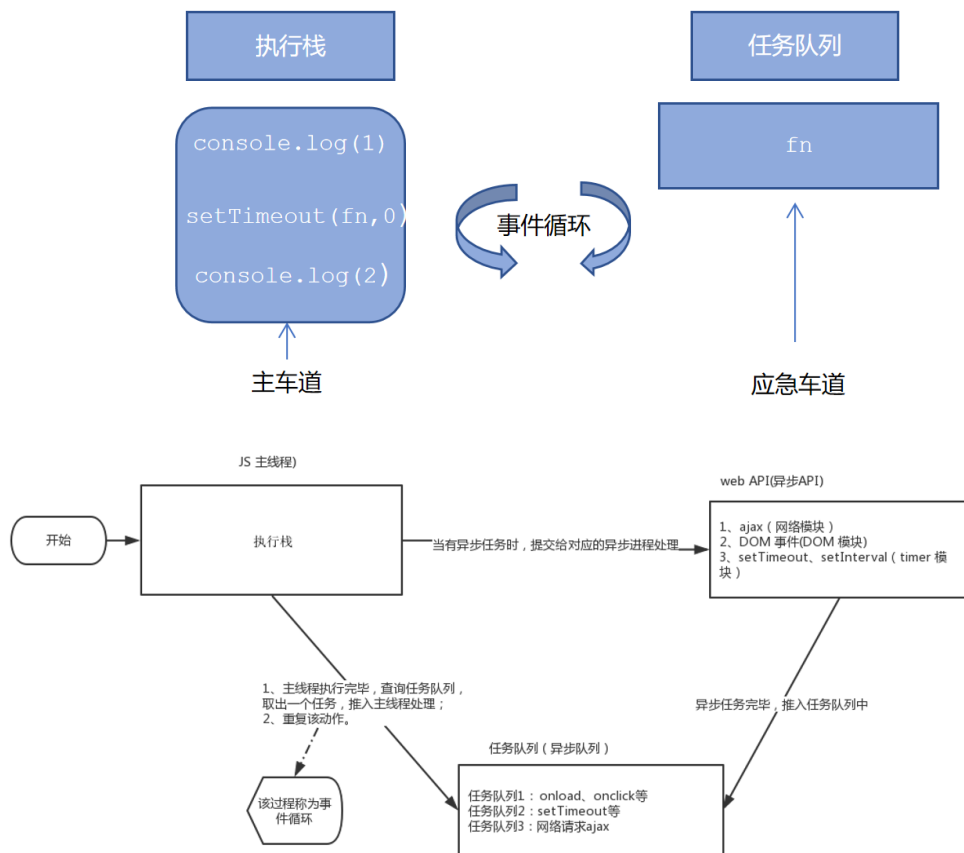
- 1、普通事件，如 click、resize 等
- 2、资源加载，如 load、error 等
- 3、定时器，包括 setInterval、setTimeout 等

异步任务相关**回调函数**添加到**任务队列**中（任务队列也称为消息队列）。



## 27-4-4 JS 执行机制

1. 先执行**执行栈中的同步任务**。
2. 异步任务（回调函数）放入**任务队列**中。
3. 一旦执行栈中的所有同步任务执行完毕，系统就会按次序读取**任务队列**中的异步任务，于是被读取的异步任务结束等待状态，进入执行栈，开始执行。



由于主线程不断的重复获得任务、执行任务、再获取任务、再执行，所以这种机制被称为**事件循环 (event loop)**。

## 27-5 location对象

### 27-5-1 什么是 location 对象

window 对象给我们提供了一个 location 属性用于获取或设置窗体的 URL，并且可以用于解析 URL。因为这个属性返回的是一个对象，所以我们将这个属性也称为 location 对象。

### 27-5-2 URL



统一资源定位符 (Uniform Resource Locator, URL) 是互联网上标准资源的地址。互联网上的每个文件都有一个唯一的 URL，它包含的信息指出文件的位置以及浏览器应该怎么处理它。

URL 的一般语法格式为：

```
protocol://host[:port]/path/[?query]#fragment

http://www.itcast.cn/index.html?name=andy&age=18#link
```

组成	说明
protocol	通信协议 常用的http,ftp,maito等
host	主机（域名） <a href="http://www.itheima.com">www.itheima.com</a>
port	端口号 可选，省略时使用方案的默认端口 如http的默认端口为80
path	路径 由 零或多个 '/' 符号隔开的字符串，一般用来表示主机上的一个目录或文件地址
query	参数 以键值对的形式,通过 & 符号分隔开来
fragment	片段 #后面内容 常见于链接 锚点

27-5-3 location 对象的属性

location对象属性	返回值
location.href	获取或者设置 整个URL
location.host	返回主机（域名） <a href="http://www.itheima.com">www.itheima.com</a>
location.port	返回端口号 如果未写返回 空字符串
location.pathname	返回路径
location.search	返回参数
location.hash	返回片段 #后面内容 常见于链接 锚点

重点记住： href 和 search

27-5-4 案例： 5秒钟之后自动跳转页面

- ① 利用定时器做倒计时效果
- ② 时间到了，就跳转页面。 使用 location.href

```
<!DOCTYPE html>
<html lang="en">

<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <meta http-equiv="X-UA-Compatible" content="ie=edge">
 <title>Document</title>
</head>

<body>
 <button>点击</button>
```

```

</div>
<script>
 var btn = document.querySelector('button');
 var div = document.querySelector('div');
 btn.addEventListener('click', function() {
 // console.log(location.href);
 location.href = 'http://www.itcast.cn';
 })
 var timer = 5;
 setInterval(function() {
 if (timer == 0) {
 location.href = 'http://www.itcast.cn';
 } else {
 div.innerHTML = '您将在' + timer + '秒钟之后跳转到首页';
 timer--;
 }
 }, 1000);
</script>
</body>

</html>

```

## 27-5-4 location 对象的方法

location对象方法	返回值
location.assign()	跟 href 一样，可以跳转页面（也称为重定向页面）
location.replace()	替换当前页面，因为不记录历史，所以不能后退页面
location.reload()	重新加载页面，相当于刷新按钮或者 f5 如果参数为true 强制刷新 ctrl+f5

```

<!DOCTYPE html>
<html lang="en">

<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <meta http-equiv="X-UA-Compatible" content="ie=edge">
 <title>Document</title>
</head>

<body>
 <button>点击</button>
 <script>
 var btn = document.querySelector('button');
 btn.addEventListener('click', function() {
 // 记录浏览历史，所以可以实现后退功能
 // location.assign('http://www.itcast.cn');
 // 不记录浏览历史，所以不可以实现后退功能
 // location.replace('http://www.itcast.cn');
 location.reload(true);
 })
 </script>
</body>

```

</html>

## 27-6 navigator 对象

navigator 对象包含有关浏览器的信息，它有很多属性，我们最常用的是 userAgent，该属性可以返回由客户机发送服务器的 user-agent 头部的值。

下面前端代码可以判断用户那个终端打开页面，实现跳转

```
if((navigator.userAgent.match(/(phone|pad|pod|iPhone|iPod|ios|iPad|Android|Mobile|BlackBerry|IEMobile|MQQBrowser|JUC|Fennec|wOSBrowser|BrowserNG|webOS|Symbian|Windows Phone)/i))) {
 window.location.href = ""; //手机
} else {
 window.location.href = ""; //电脑
}
```

## 27-7 history 对象

window 对象给我们提供了一个 history 对象，与浏览器历史记录进行交互。该对象包含用户（在浏览器窗口中）访问过的 URL。

history对象方法	作用
back()	可以后退功能
forward()	前进功能
go(参数)	前进后退功能 参数如果是 1 前进1个页面 如果是-1 后退1个页面